DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY

Modern Deep Convolutional Neural Networks with PyTorch

Modern CNN architectures, detection and segmentation

29
SECTIONS

178
FIGURES

154
PAGES

Compiled October 2026
github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 Modern Deep Convolutional Neural Networks 4
LECTURE 2 Computer Vision Problems 8
LECTURE 3 Classification Pipeline 15
LECTURE 4 Softmax Layer 23
LECTURE 5 Gradient Descent 30
LECTURE 6 Train Set and Batch Size 44
LECTURE 7 Linear Classifier and Output Neurons 45
LECTURE 8 Train Loader and Images Images 47
LECTURE 9 Linear Classifier and Linear Layers 48
LECTURE 10 Deep Learning and Descriptors 50
LECTURE 11 Linear Layer: Why Not for Images? 56
LECTURE 12 One Value 60
LECTURE 13 Zero Padding 66
LECTURE 14 Sigmoid Derivative 75
LECTURE 15 Max Pooling 80
LECTURE 16 Non Linearity and Build Deep 85
LECTURE 17 Output Size and Channels 88
LECTURE 18 Overfitting and Regularization 90
LECTURE 19 Dropout Regularization 95
LECTURE 20 Dropout DropBlock 102
LECTURE 21 Early Stopping 108
LECTURE 22 Batch Normalization 115
LECTURE 23 Data Augmentation 124
LECTURE 24 MNIST Dataset 130
LECTURE 25 Modern Convolutional Architectures 136
LECTURE 26 Transfer Learning 146
LECTURE 27 Data Sets and Dataset 151
LECTURE 1 Horizontal Flip and Random 153

LECTURE 29 Fully Connected 154

LECTURE 1

Modern Deep Convolutional Neural Networks

1. There exist a competition called ImageNet Large Scale Visual Recognition Challenge and Imagenet [https://image-net.org/index.php] which is a large-scale image recognition competition.

Hello my dear friend. My name is Dennis for Lansky. Welcome to the course. More than deep convolution all neural networks. Thank you very much for choosing this course. I'll do my best to share my knowledge in image processing with deep learning and convolution all neural networks with you. Let me begin with the motivation of learning evolution or neural networks.

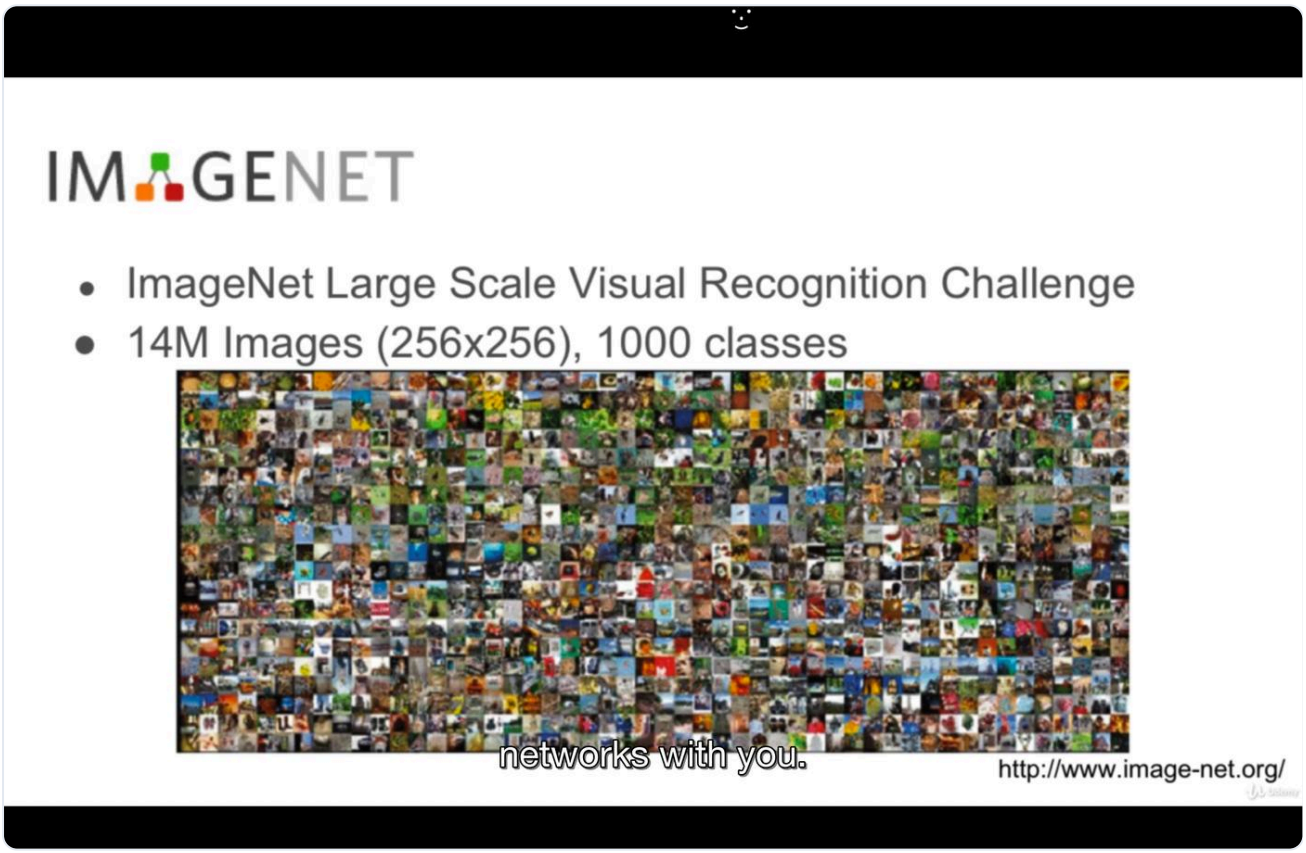


FIGURE 1.1

There exist at competition gold ImageNet large scale visual recognition challenge the task for their competitors is the following. You have 14 millions of images of size 256 to 256. Each image is assigned to one out of 1000 classes cats dogs frogs etc.. You have to build a classifier an algorithm that takes an image as an input. In return the class of this image this competition started in 2010 and the error on top five classes was 28 percent year after year people develop neural networks that are deeper than produce. And in 2012 they consisted of eight players then 19 then 22 and finally in 2015.

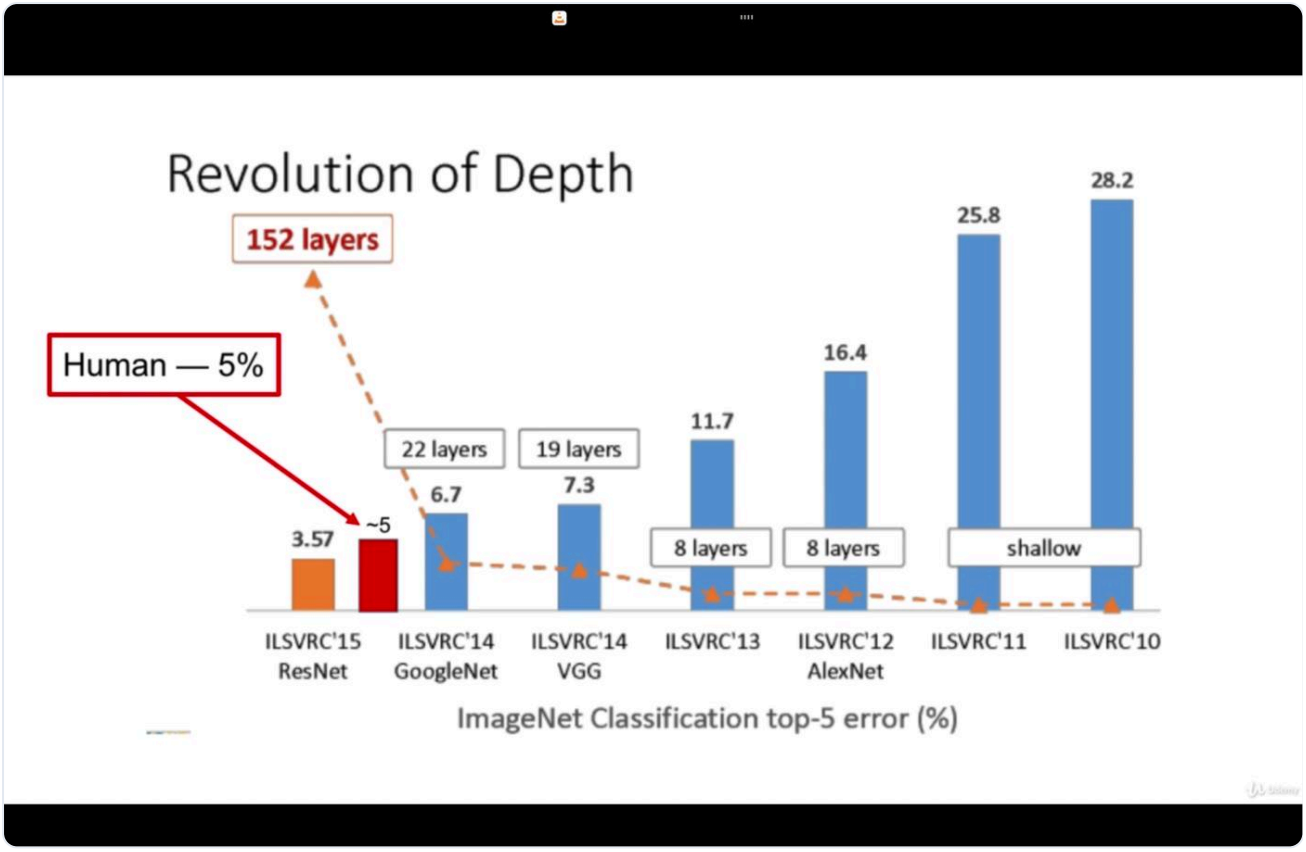


FIGURE 1.2

People develop or his net architecture which showed the error of three point five percent and what is amazing about it. It outperformed human. There are several reasons for it. First the network is really big. One hundred and fifty two. In fact learning of such a deep network not only Lon but also very hard and authors proposed how to overcome such limitations which we sure enough will discuss in our course.

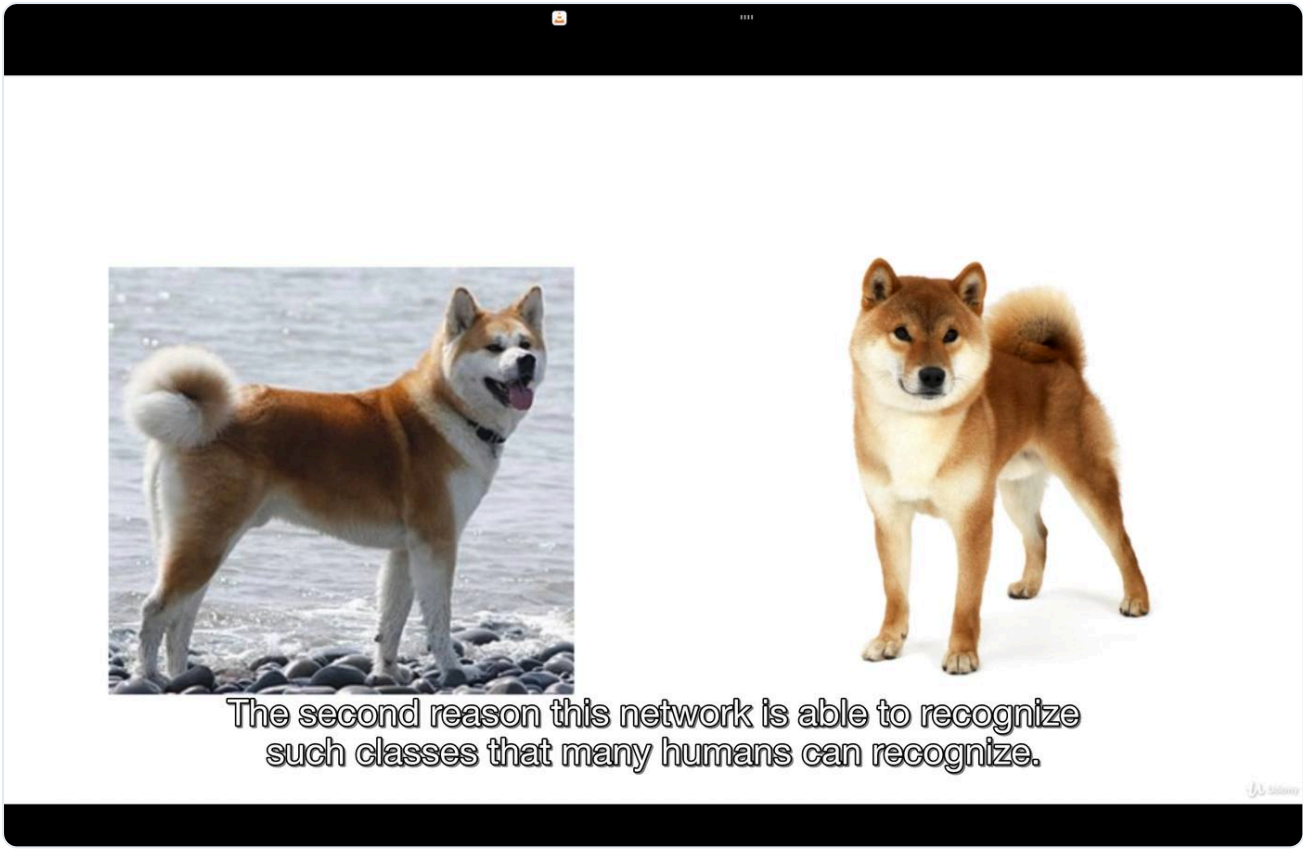


FIGURE 1.3

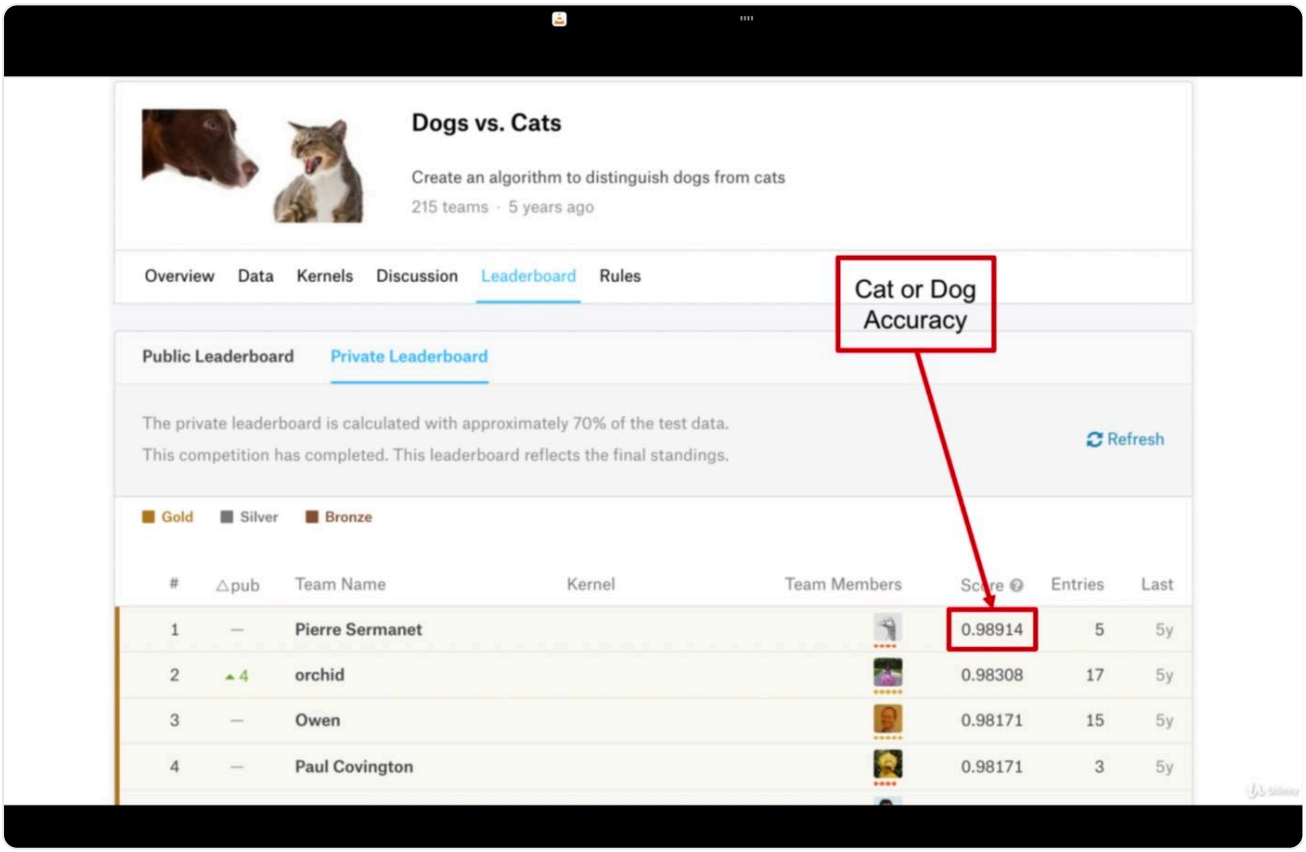


FIGURE 1.4 | Dogs vs. Cats

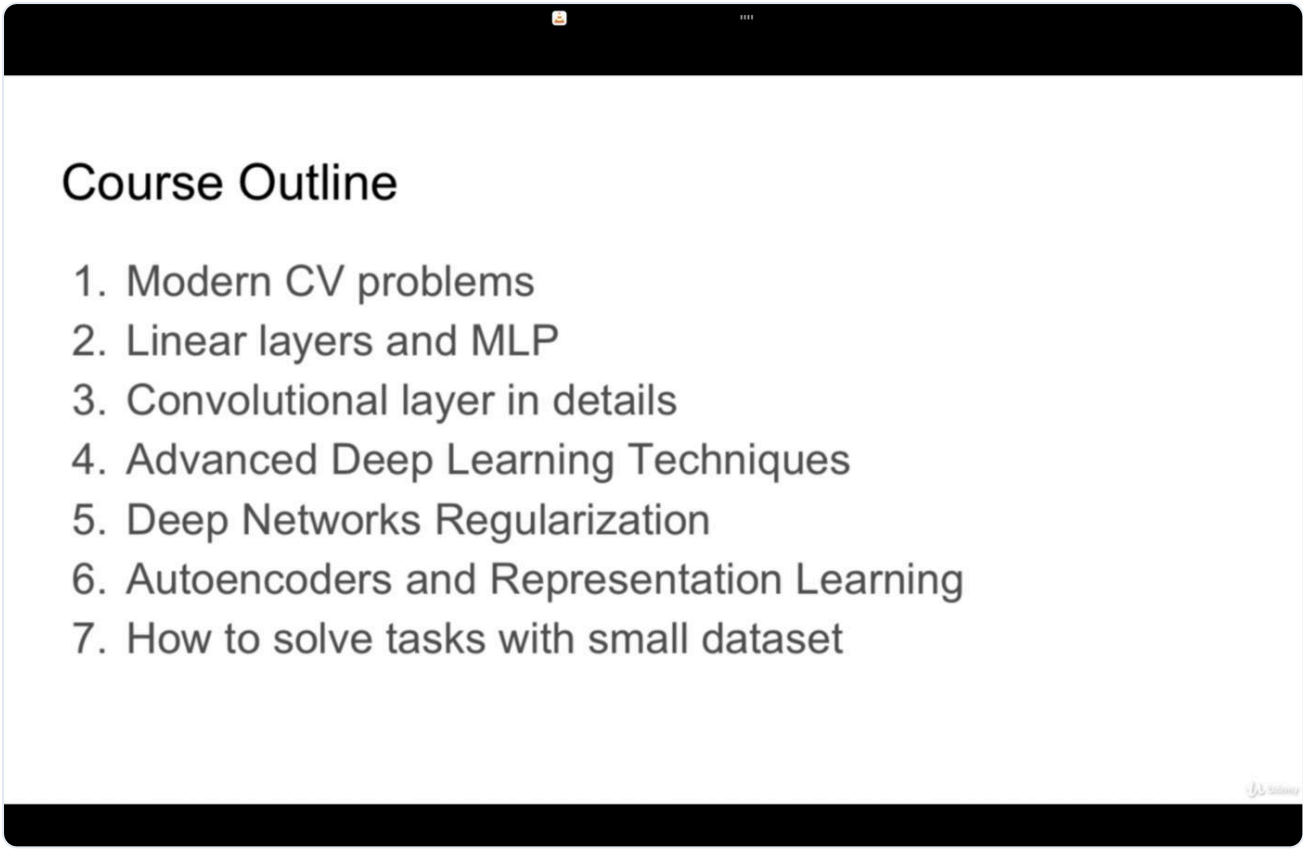


FIGURE 1.5 | Modern CV Problems

The second reason this network is able to recognize such classes that many humans can recognize. For example this network can distinguish different breeds of dogs and I am not sure that all of you can distinguish between these two different breeds of dogs and the slight but they are different neural networks show amazing performance in many tasks. In 2013 Kaggle the biggest machine learning competition platform hosted ad dogs was cat challenge. The task was to classify whether there is a cat or a dog in the photo and the accuracy of the winner's model was almost 99 percent during our course will move from the basics of convolutional neural networks to the recently developed advanced techniques for train them and achieving the high score.

Firstly we will consider which problems can be solved with convolutional neural networks. We will briefly remember Lee earlier and how to train multilayer perceptron on what is loss function and how gradient descent works. We will study convolution on neural networks what it is how to use it. What are the parameters etc.. Later we will move forward to study in advance deep learning techniques such as batch normalization is this normalization and others I will tell you. The most modern regularization techniques such as dropout drop connect and others. We will learn what is an auto encoder and how to apply it in practical business tasks. Finally we will study the most popular and relational models focus fixation task and study how to use them in order to solve problems with a small amount of data.

Thank you and see you in the next video.

LECTURE 2

Computer Vision Problems

Let us now consider the hottest topics of research in computer region. All of these problems are solved using an evolutionary neural networks. Each of the problems is a separate topic.

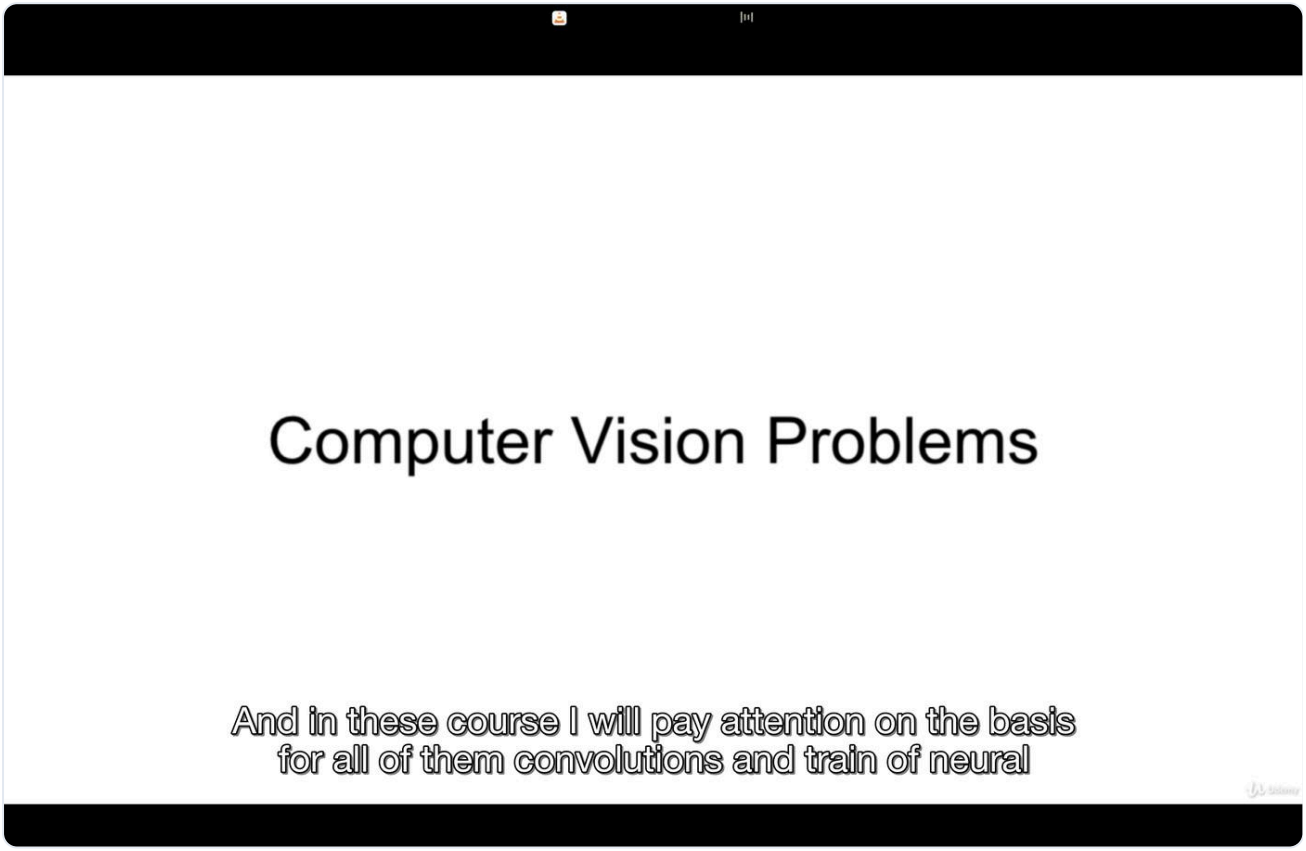


FIGURE 2.1 | Computer Vision Problems

And in these course I will pay attention on the basis for all of them convolutions and train of neural networks.

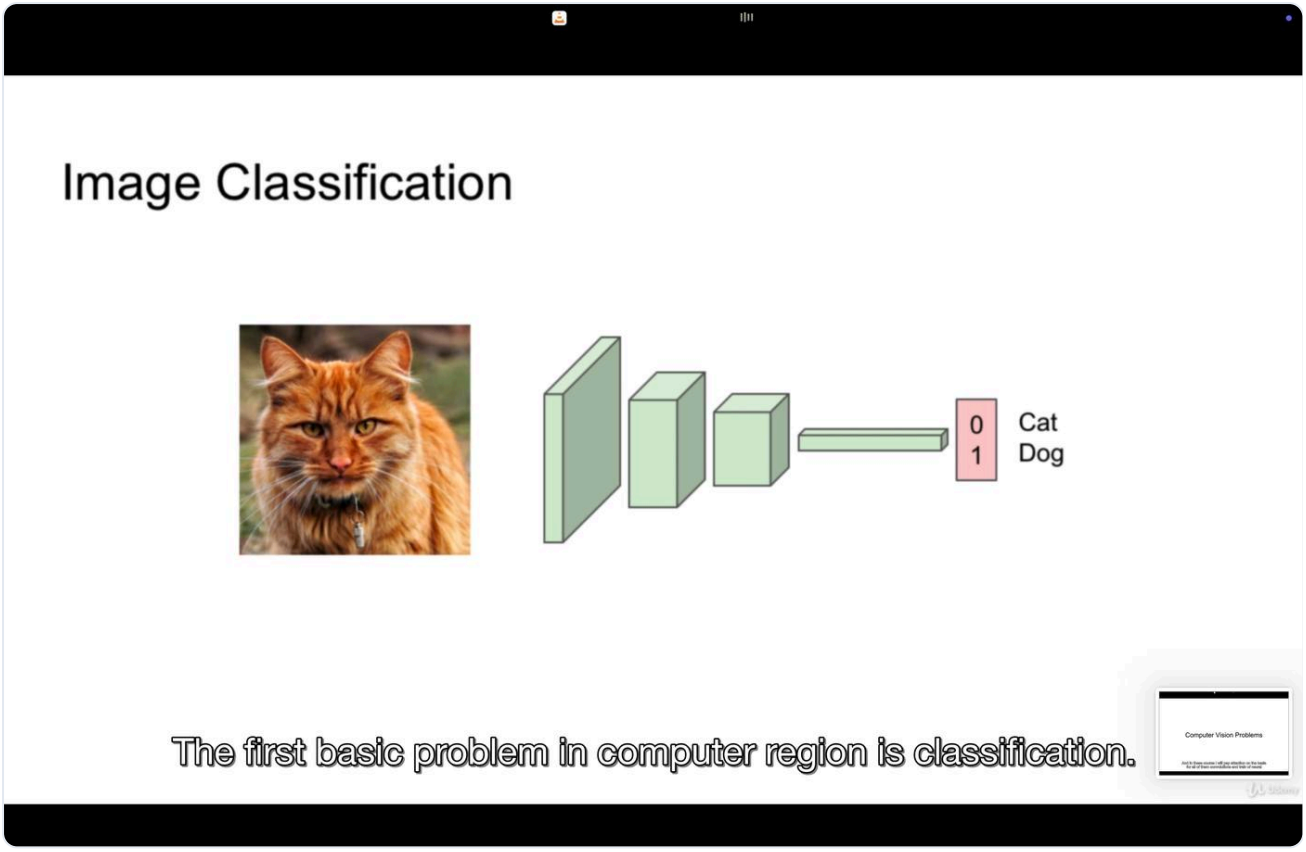


FIGURE 2.2 | Image Classification

The first basic problem in computer region is classification. Usually you have to be with a function that takes some image is an input and produce a label for this image. On the slide you can see a simple case of classification binary or two class problem.

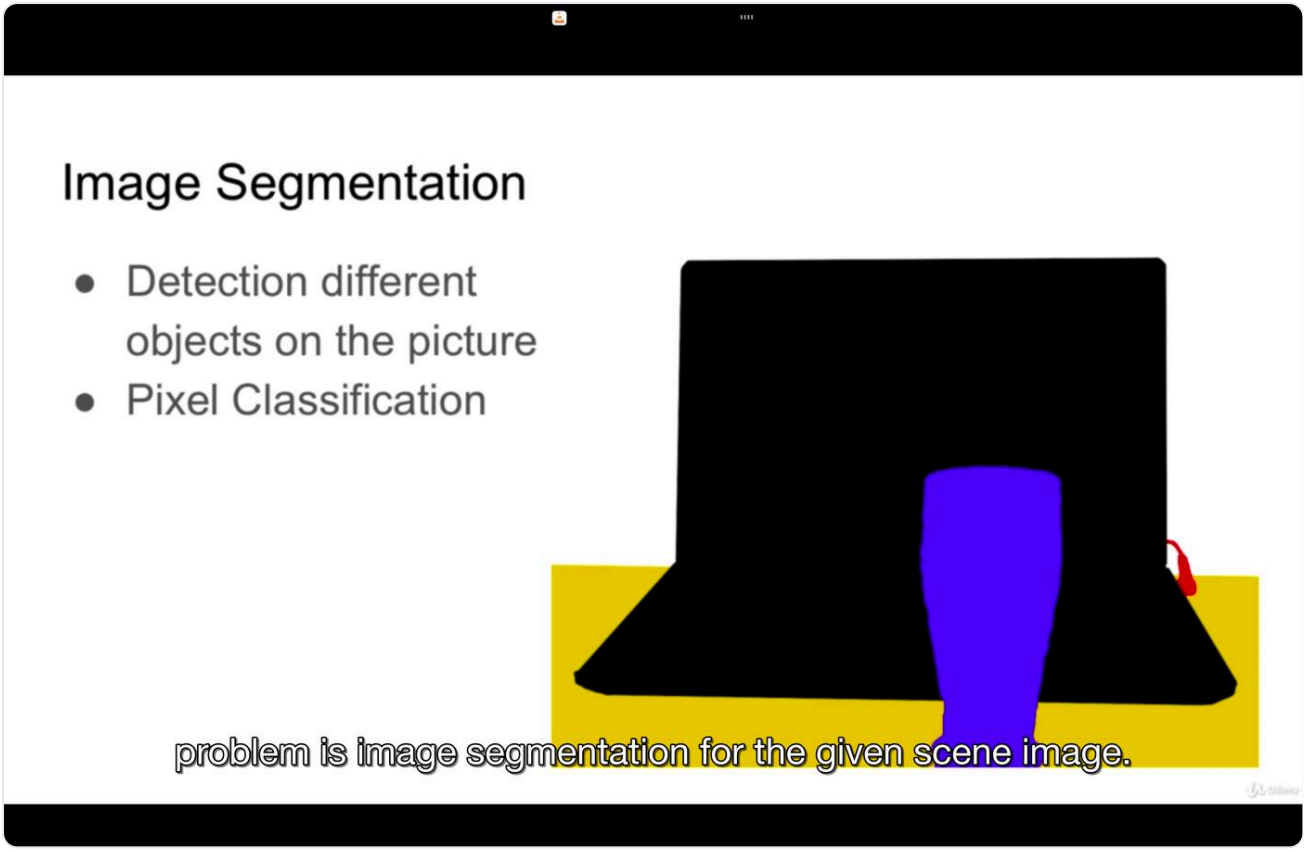


FIGURE 2.3 | Image Segmentation

Our neural network should decide whether there is a cat or a dog in the image second computer vision problem is image segmentation for the given scene image. We should find where each object is located and Mark pixels of different objects with different colors or classes. For example in the slide the glass is marked with the blue color. The laptop is black and the table is yellow.

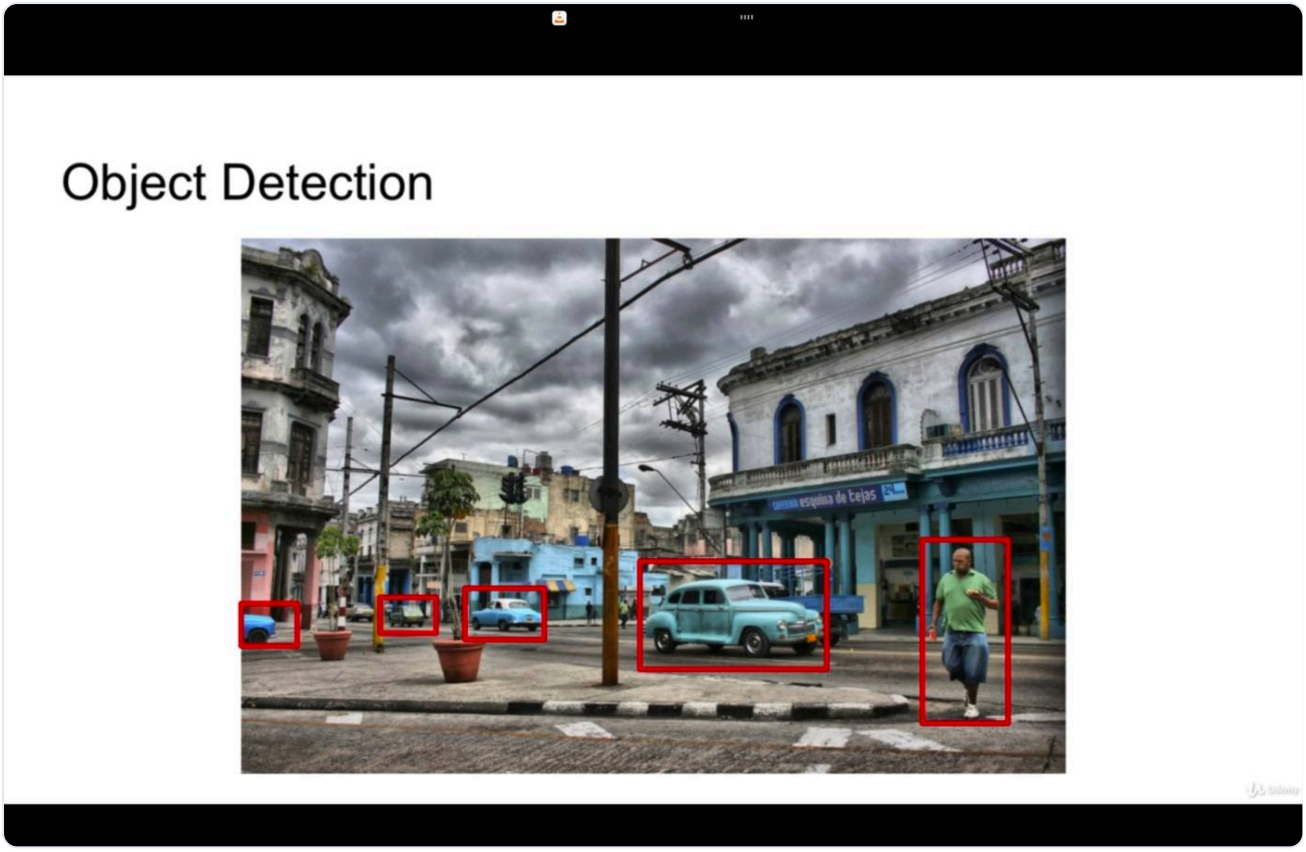


FIGURE 2.4 | Object Detection

Usually this task is sought with the classification of each pixel separately. Another example of computer vision task is object detection for the input in which we need to find all the objects in the image and highlight them with their tangle which is called the bounding box the next problem of computer region is. Image captioning for a given image.

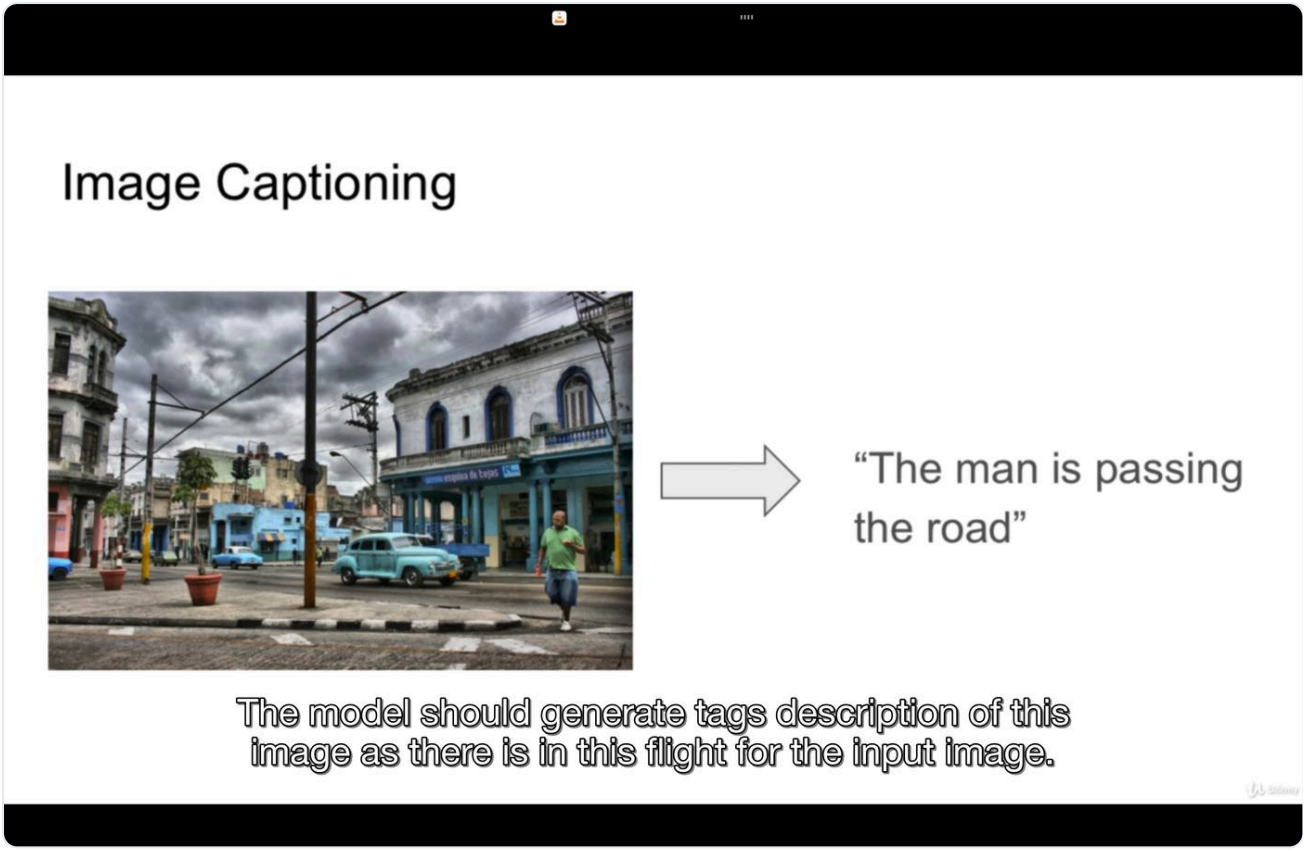


FIGURE 2.5 | Image Captioning

The model should generate tags description of this image as there is in this flight for the input image. The caption may be the man especially the road.

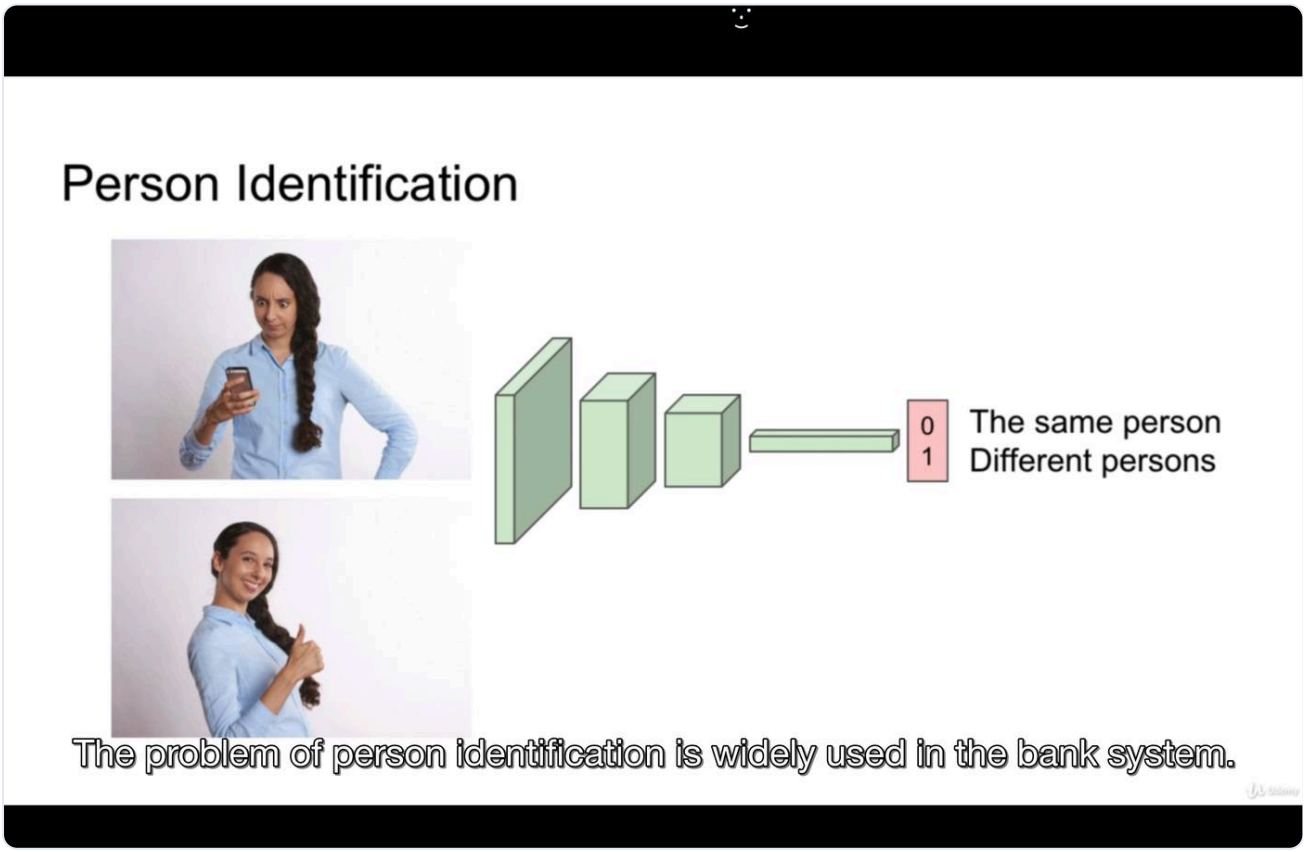


FIGURE 2.6 | Person Identification

The problem of person identification is widely used in the bank system. Usually banks would like to identify persons by the photo and see whether this is the same person or not. We will study this problem at the end of the course.

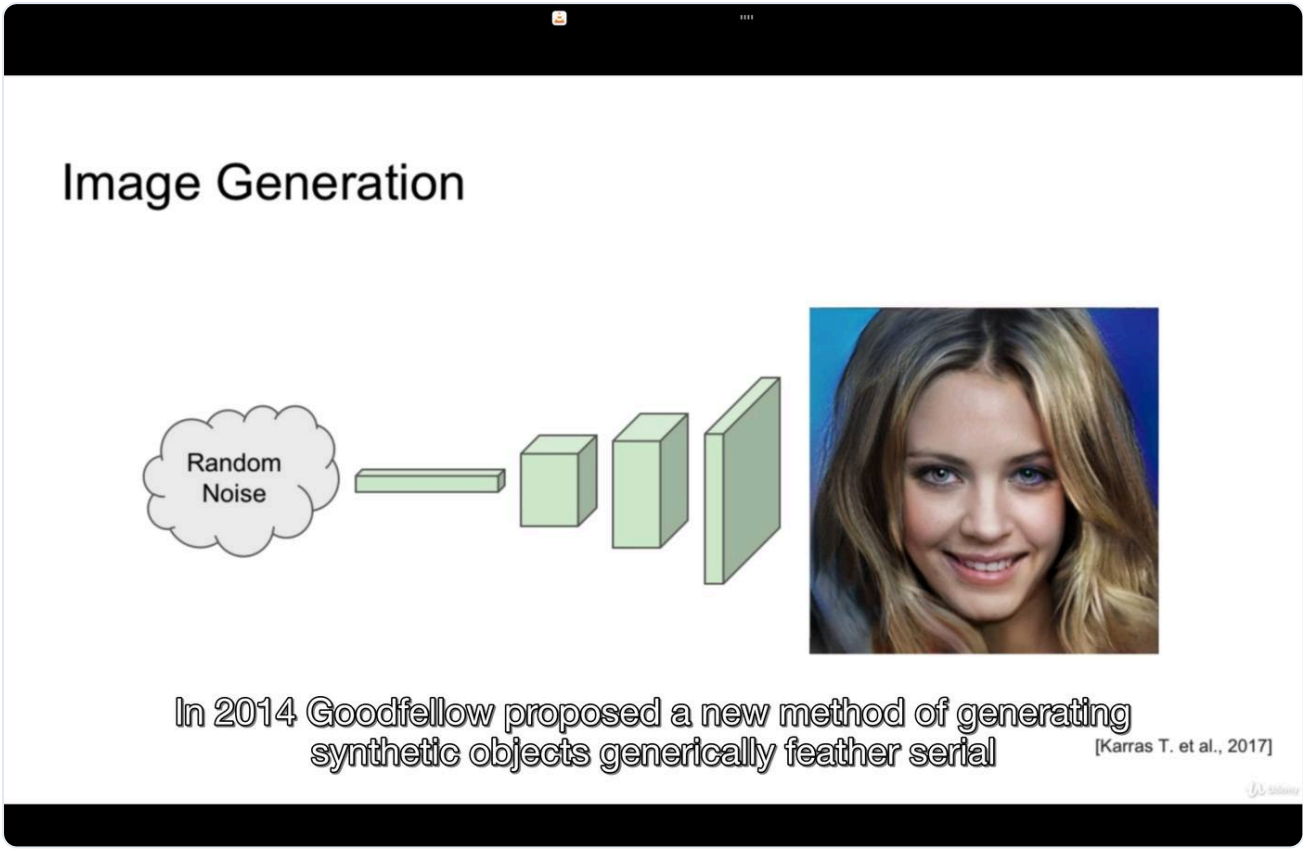


FIGURE 2.7

In 2014 Goodfellow proposed a new method of generating synthetic objects generically feather serial networks. They are able to learn the train data distribution and sample from the distribution.

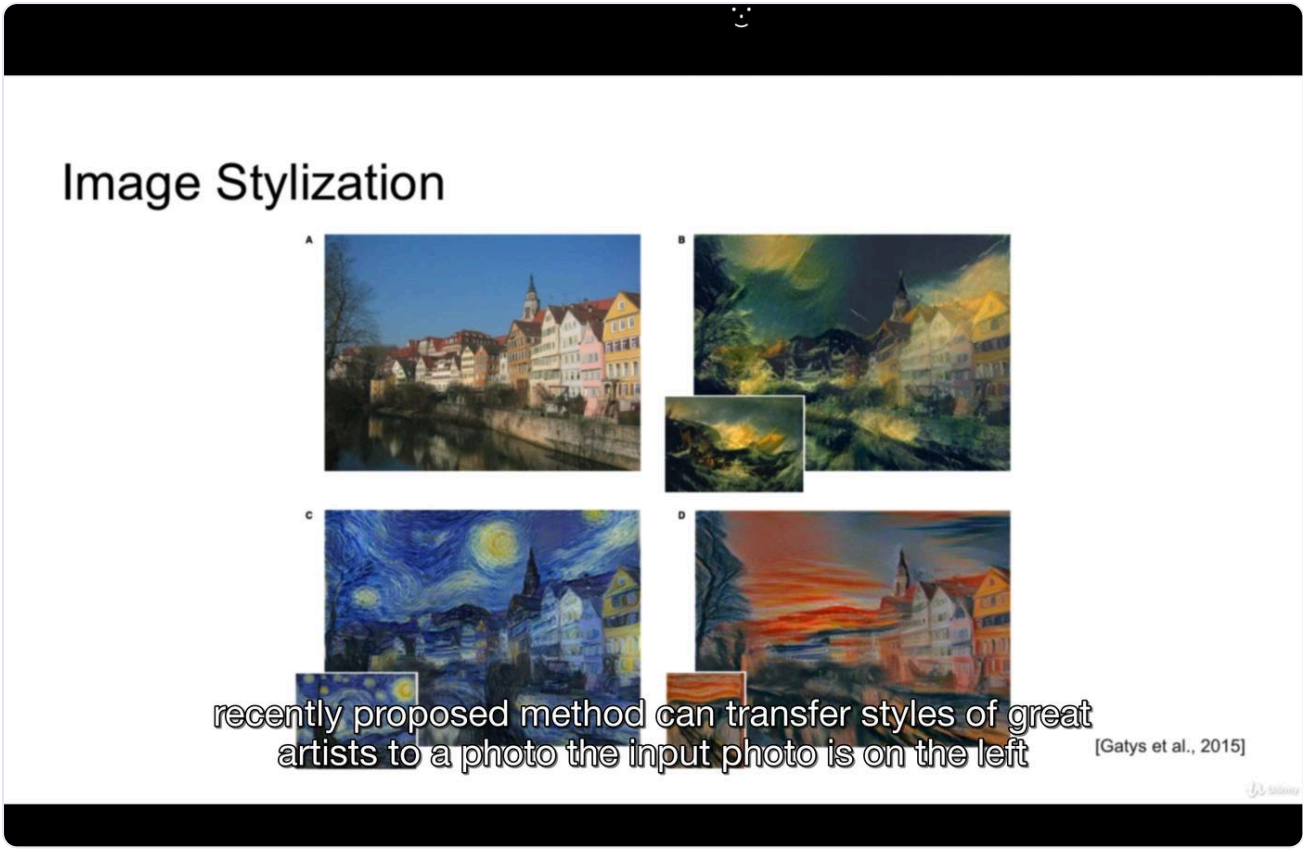


FIGURE 2.8

Now a generative adversarial networks can generate synthetic objects of big size and high quality another recently proposed method can transfer styles of great artists to a photo the input photo is on the left corner and other through photos are stylized to his different styles of famous drawings. Thank you for watching and see you in the next video.

LECTURE 3

Classification Pipeline

Now I am going to remind you the core classification and regression pipeline.

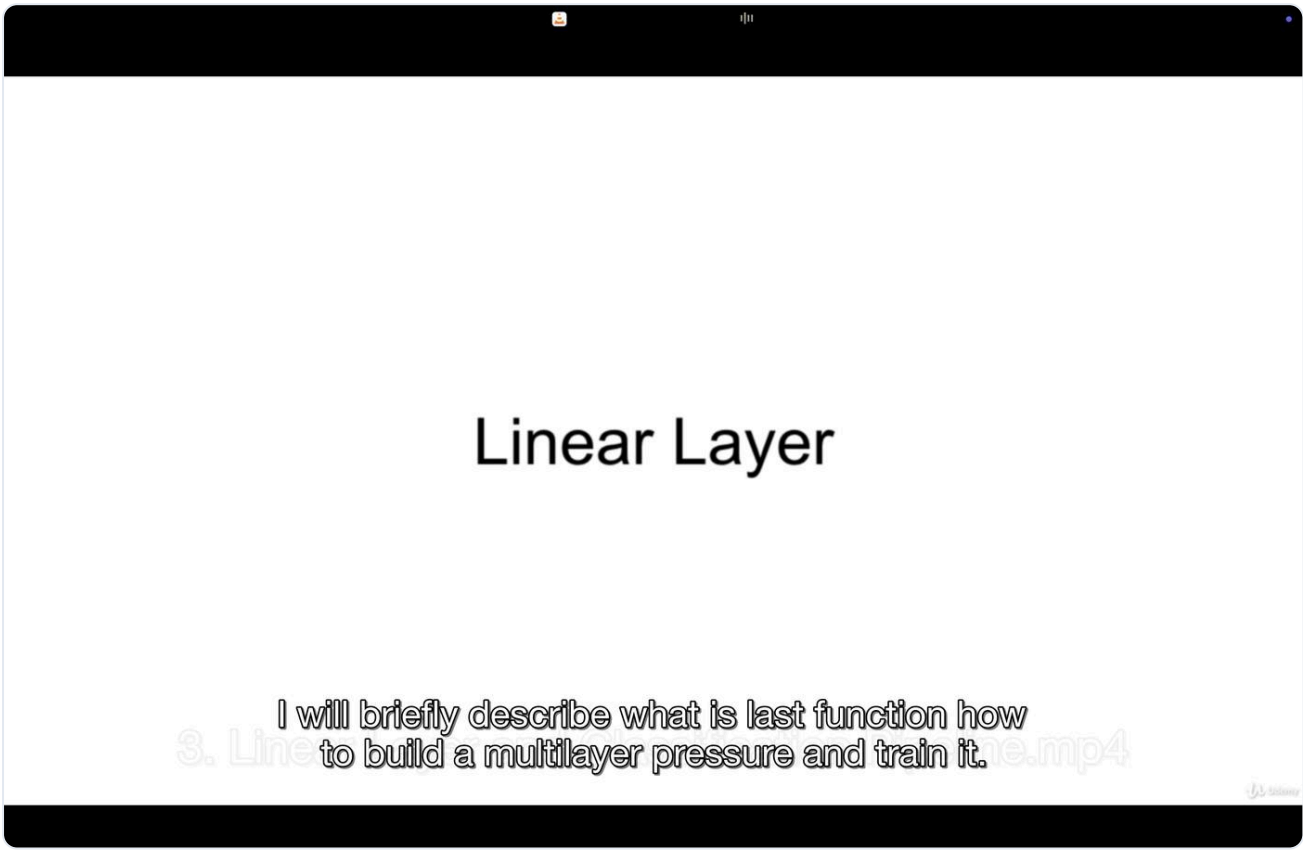


FIGURE 3.1 | Linear Layer

I will briefly describe what is loss function how to build a multilayer perceptron and train it. You can skip this video if you think you are good at these topics let's start with the your earlier recap. This in fact is a basis for every linear classifier such as logistic regression or SVM. As far as for linear regression many people like to explain linear layer using neurones which for me seems very complicated. I like to look at linear layer as the linear transformation from one linear space to another.

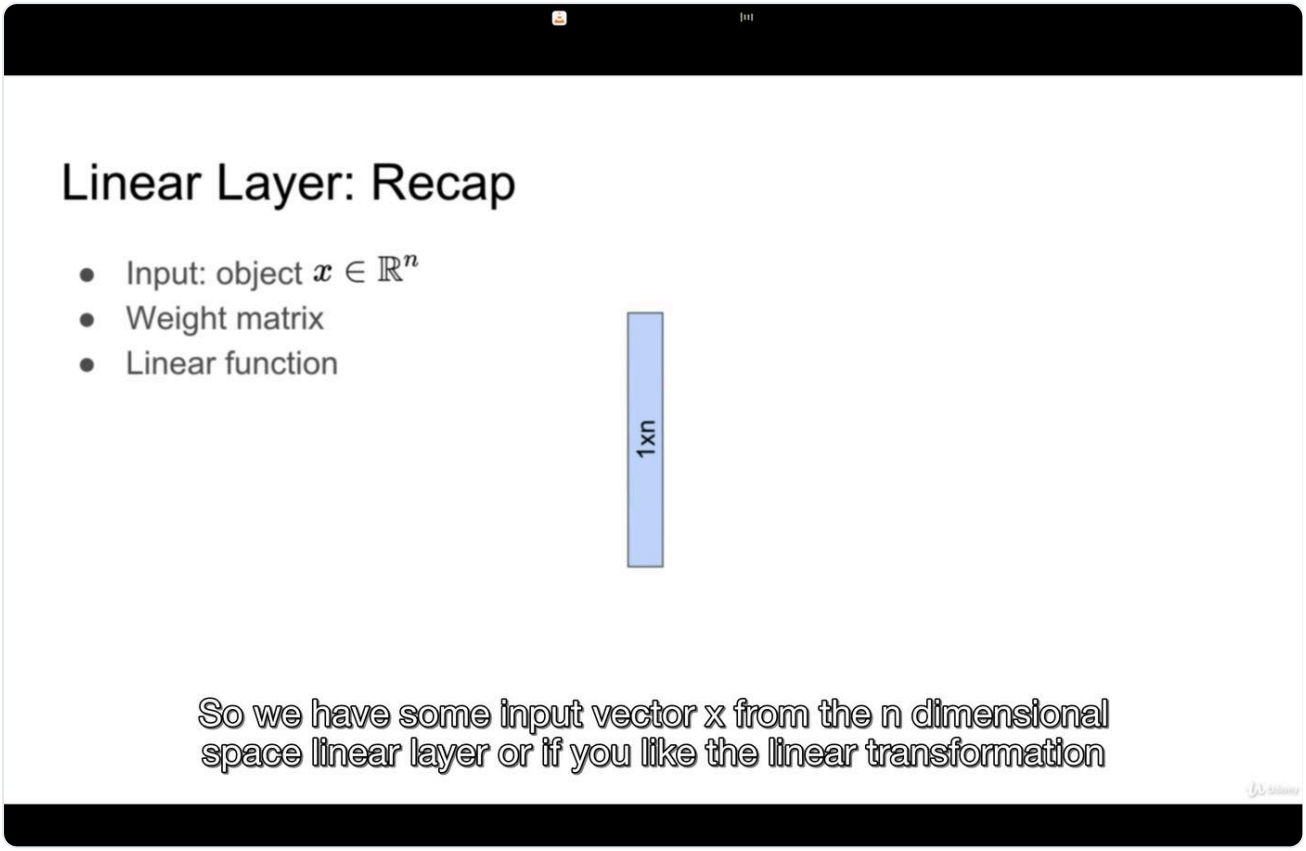


FIGURE 3.2

So we have some input vector x from the n dimensional space linear layer or if you like the linear transformation is defined by the transformation matrix of size and Turkey in terms of neural networks. This is a great matrix which is updated during training in PyTorch.

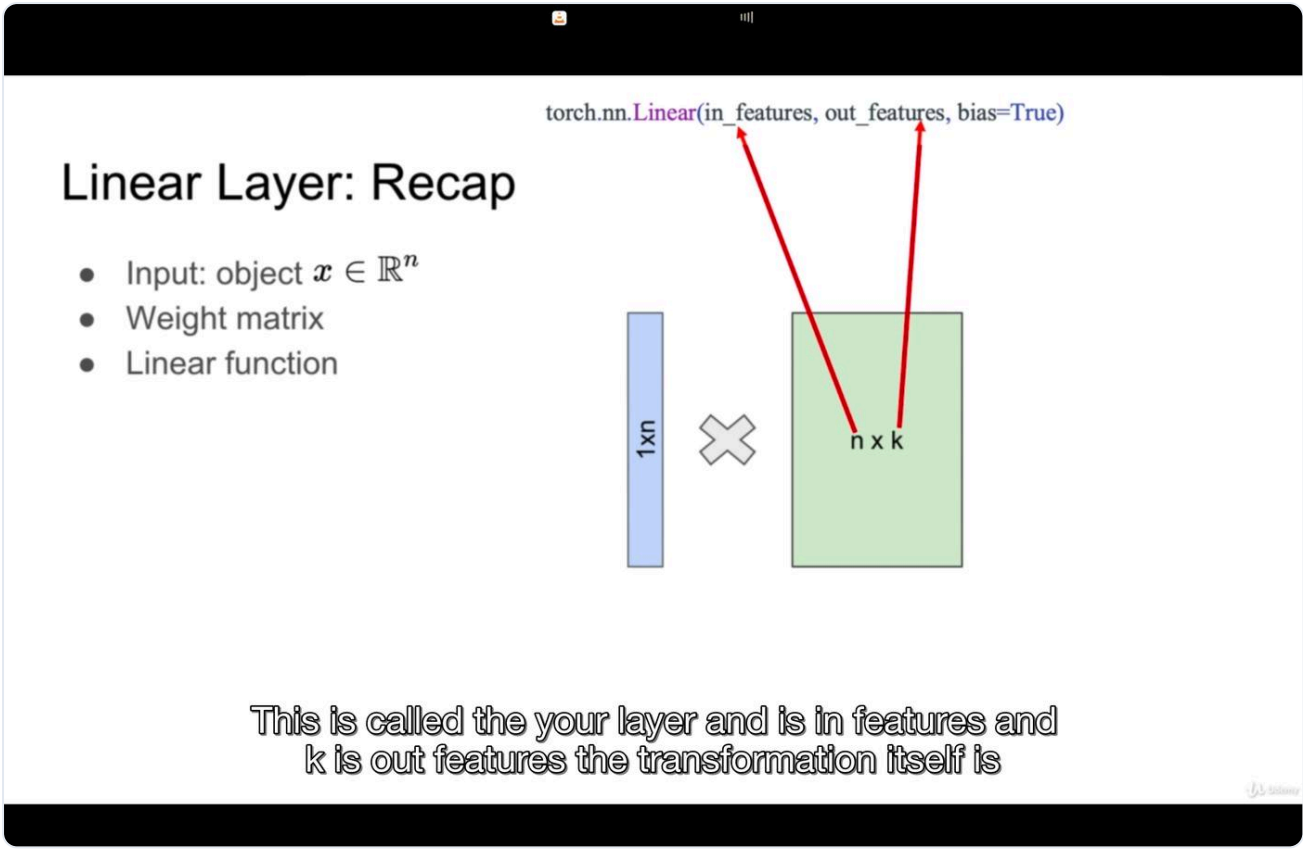


FIGURE 3.3 | Linear Layer: Recap

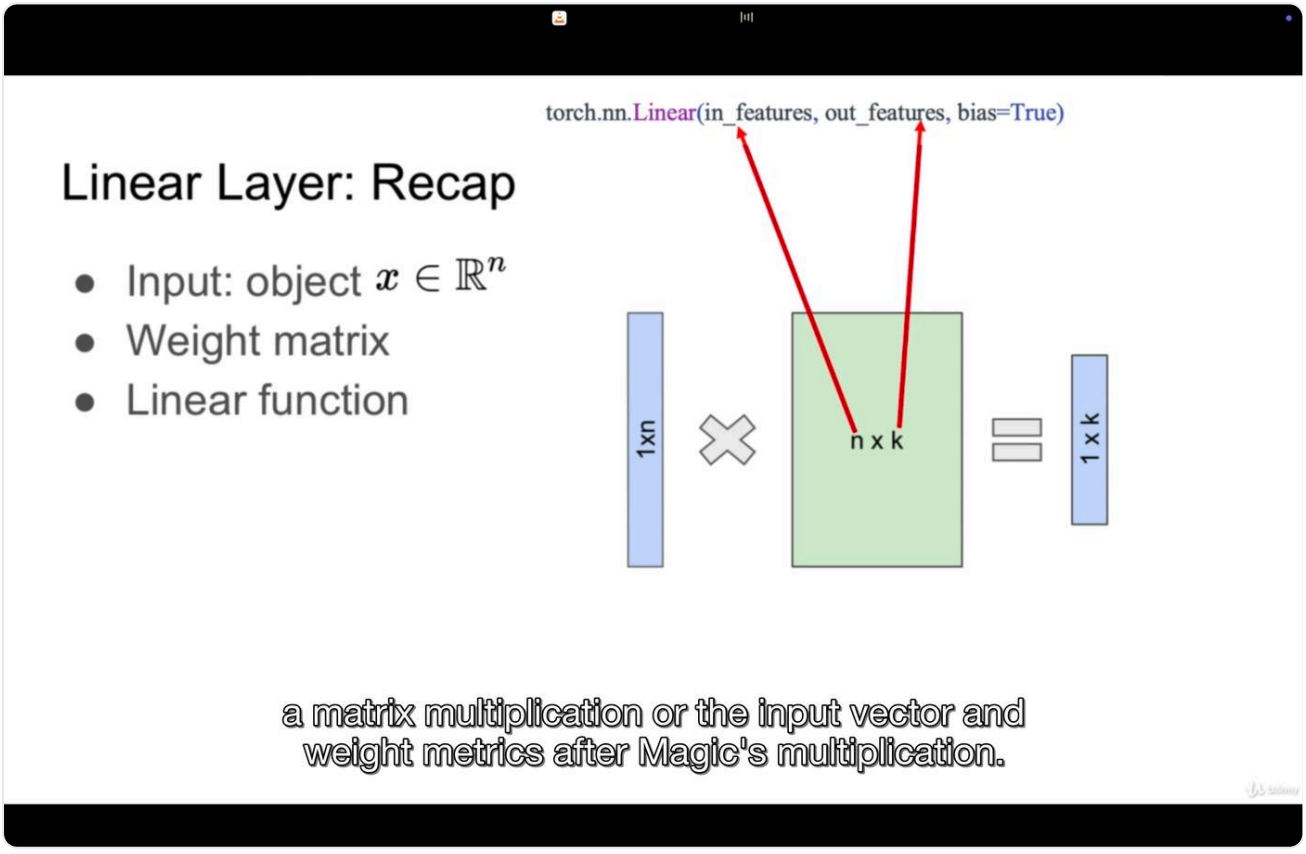


FIGURE 3.4 | Linear Layer: Recap

This is called the your layer and is in features and k is out features the transformation itself is a matrix multiplication or the input vector and weight matrix after Magic's multiplication.

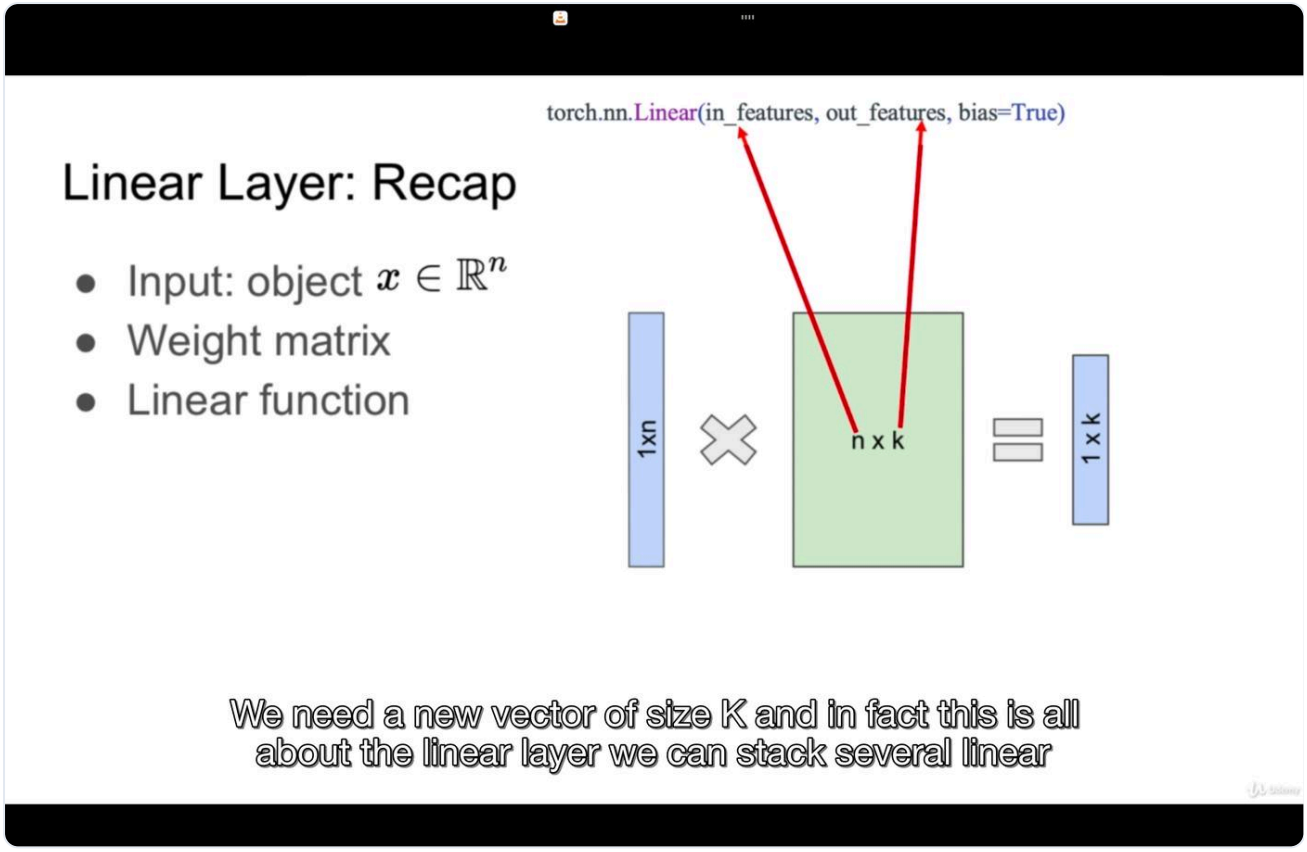


FIGURE 3.5

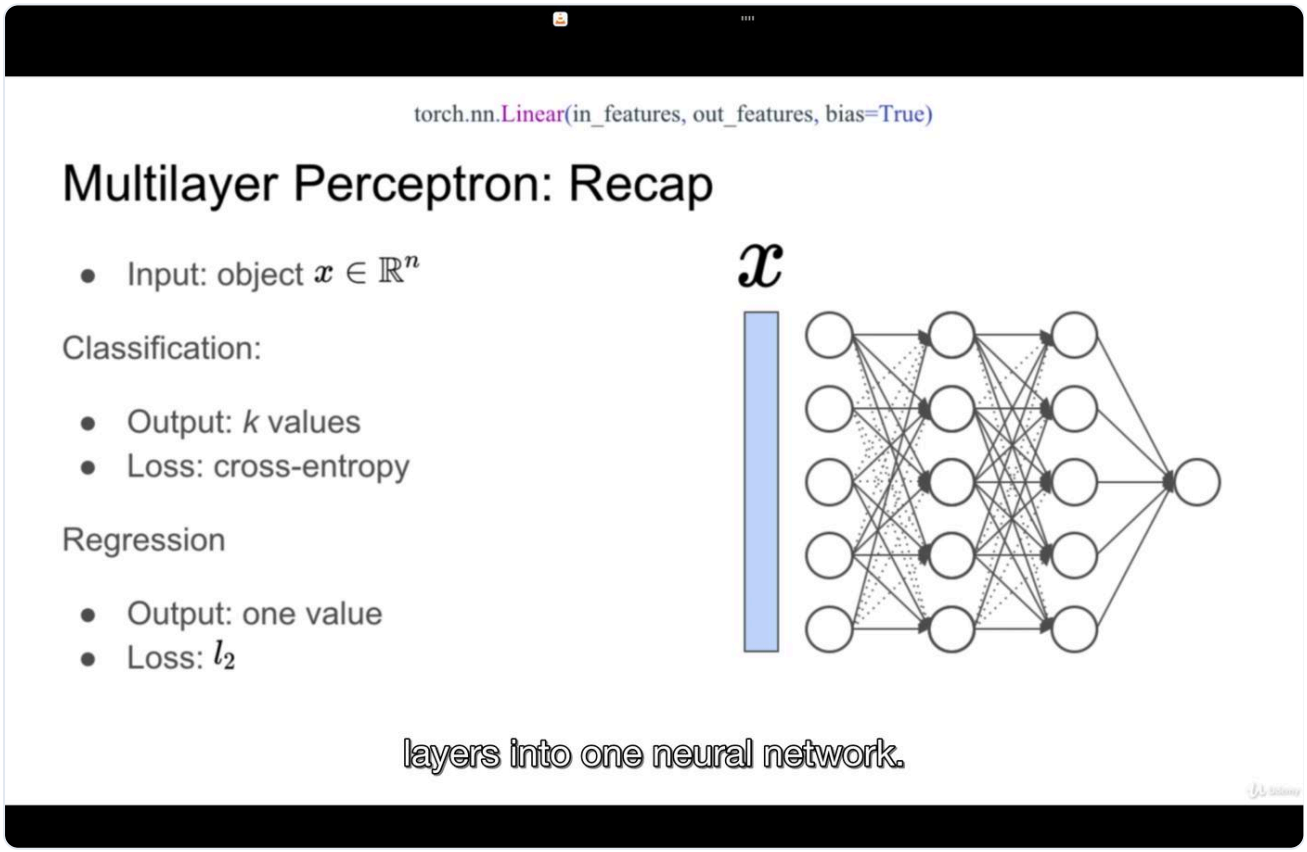


FIGURE 3.6 | Multilayer Perceptron: Recap

We need a new vector of size K and in fact this is all about the linear layer we can stack several linear layers into one neural network. Don't forget that combination of linear transformations is also a linear transformation. That's why you should always insert nonlinearity between them. For example sigmoid function the sigmoid is a bad choice but don't worry we'll study which non linearity to use later in this course depending on the problem. We consider the different number of output neurons in the case of classification to key classes. We should have K output neurons and use cross entropy loss with softmax layer in case of regression problem. We should use one output neuron with any regression loss for example L2 Euclidean distance between predicted and target values.

Now let's talk about classification in this section.

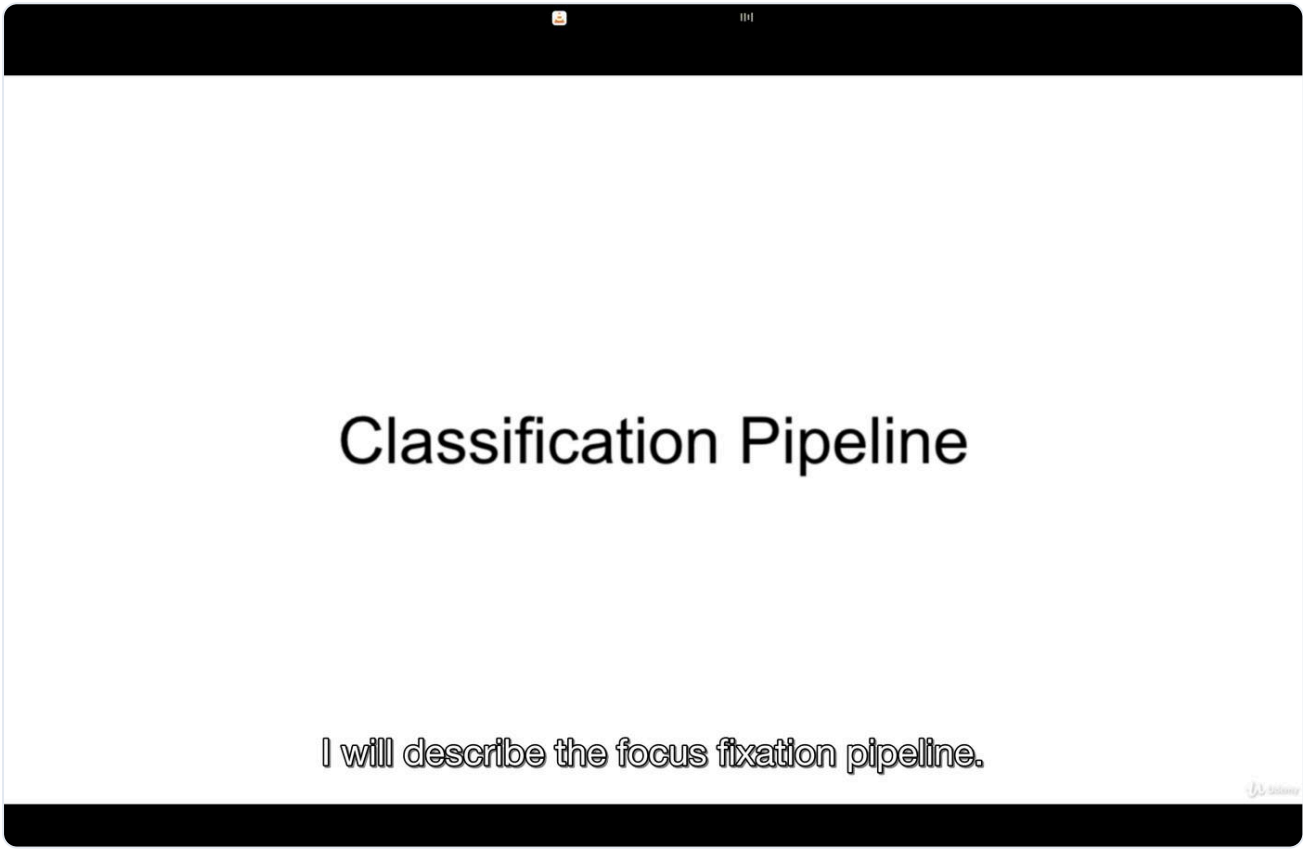


FIGURE 3.7 | Classification Pipeline

I will describe the classification pipeline. We have some input vector each element of the input vector is a feature. This vector is a part of a linear space we can apply a linear transformation to eat. This is called linear layer. As I said before the linear layer is represented as a matrix of learnable parameters then we apply some non linearity such as sigmoid or hyperbolic tangent if we don't we will get the problem from linear algebra. We should remember that combination of linear functions is also a linear function. Thus I did several layers one after another will lead to some bad effects during training. Now we can't repeat dissection several times in order to make our need for deeper between each linear layer.

We're at non linearity such as sigmoid or hyperbolic tangent if we want to perform X vacation to C classes then we need to set C as an output dimension of our network the last layer is now called logits.

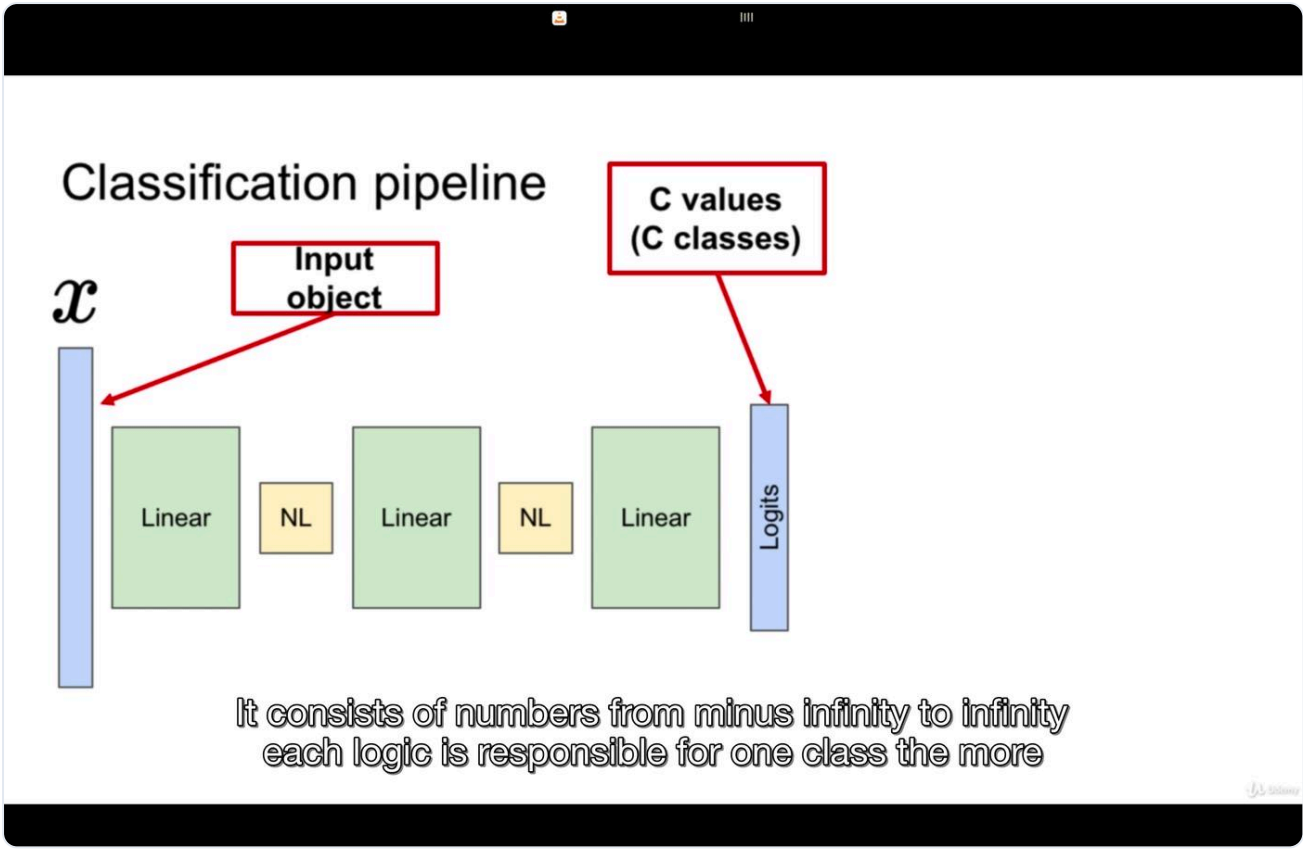


FIGURE 3.8 | Classification Pipeline C Values

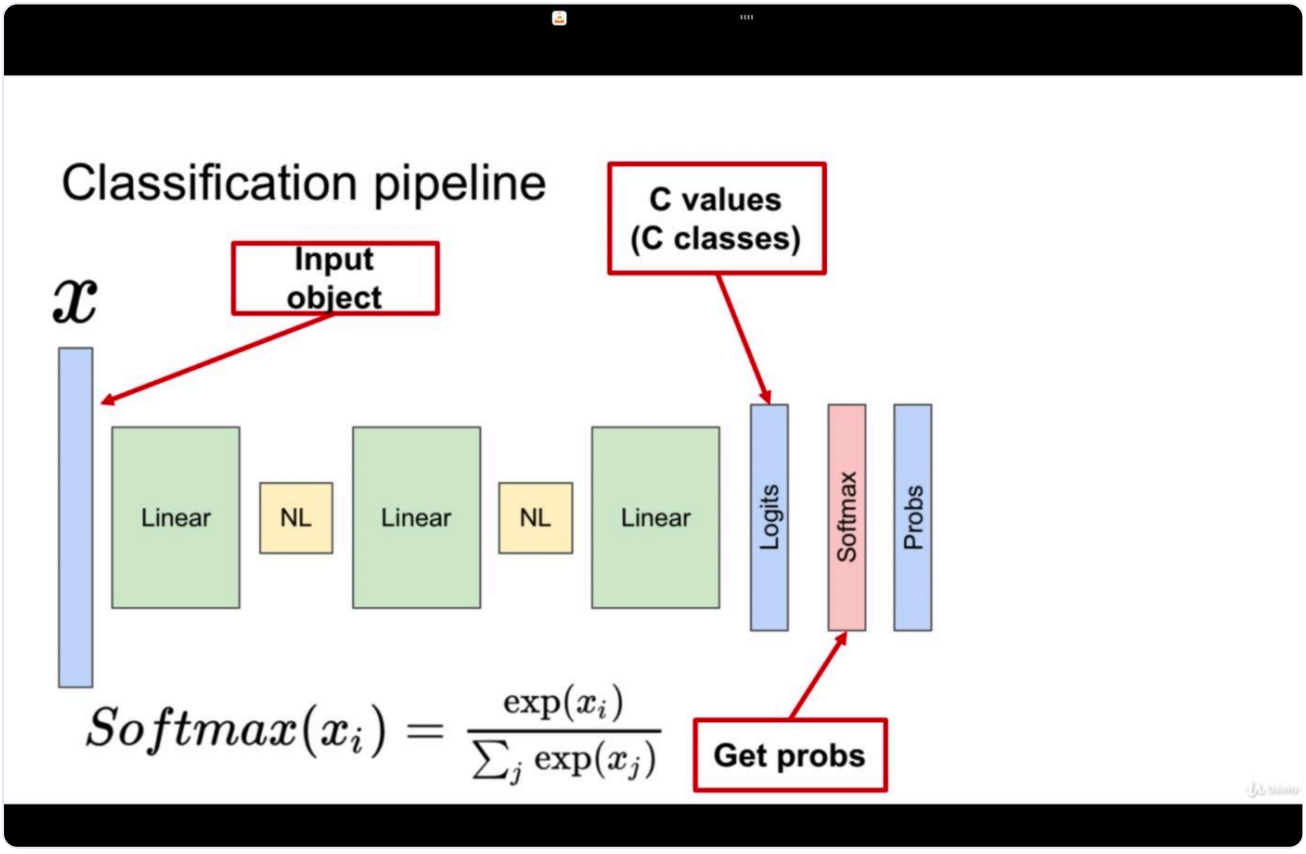


FIGURE 3.9 | Classification Pipeline C Values

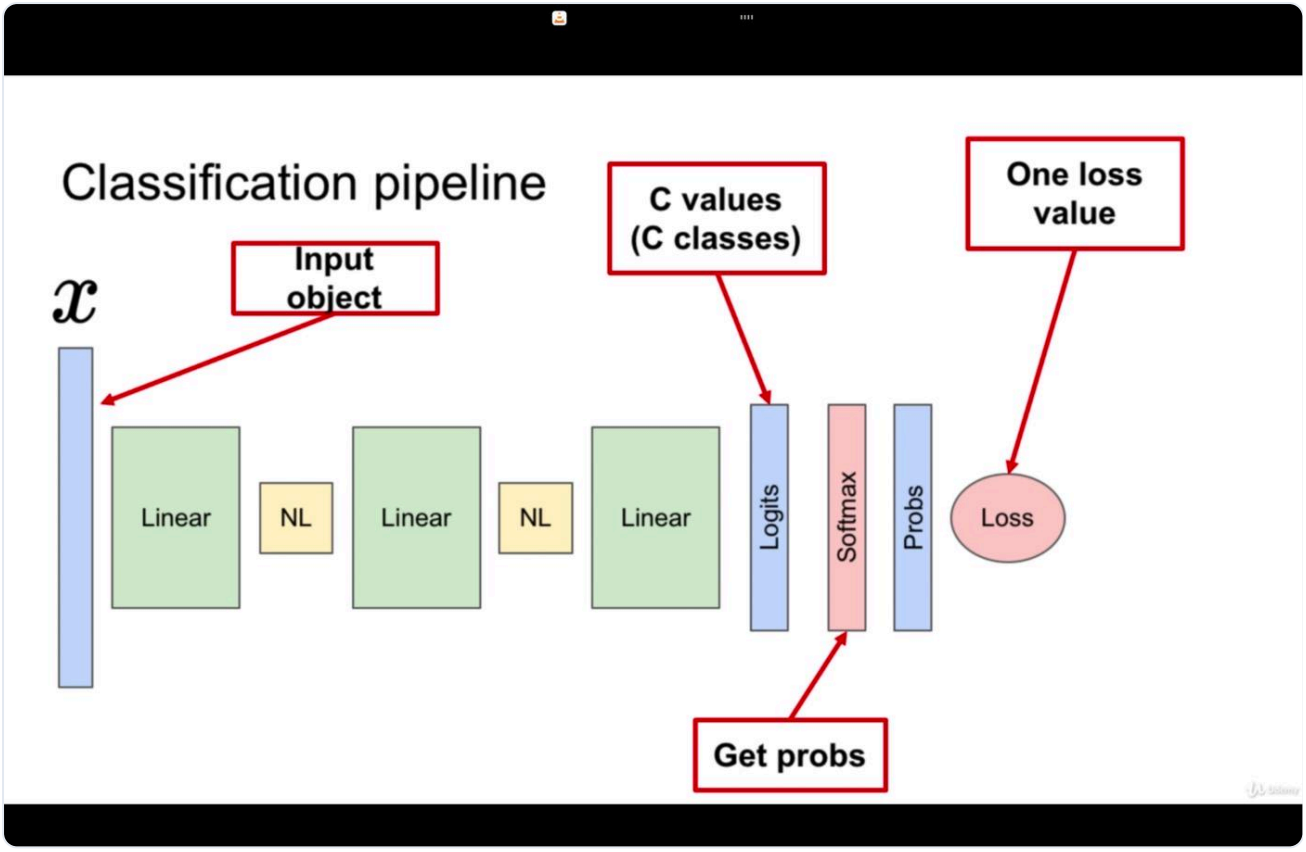


FIGURE 3.10 | Classification Pipeline

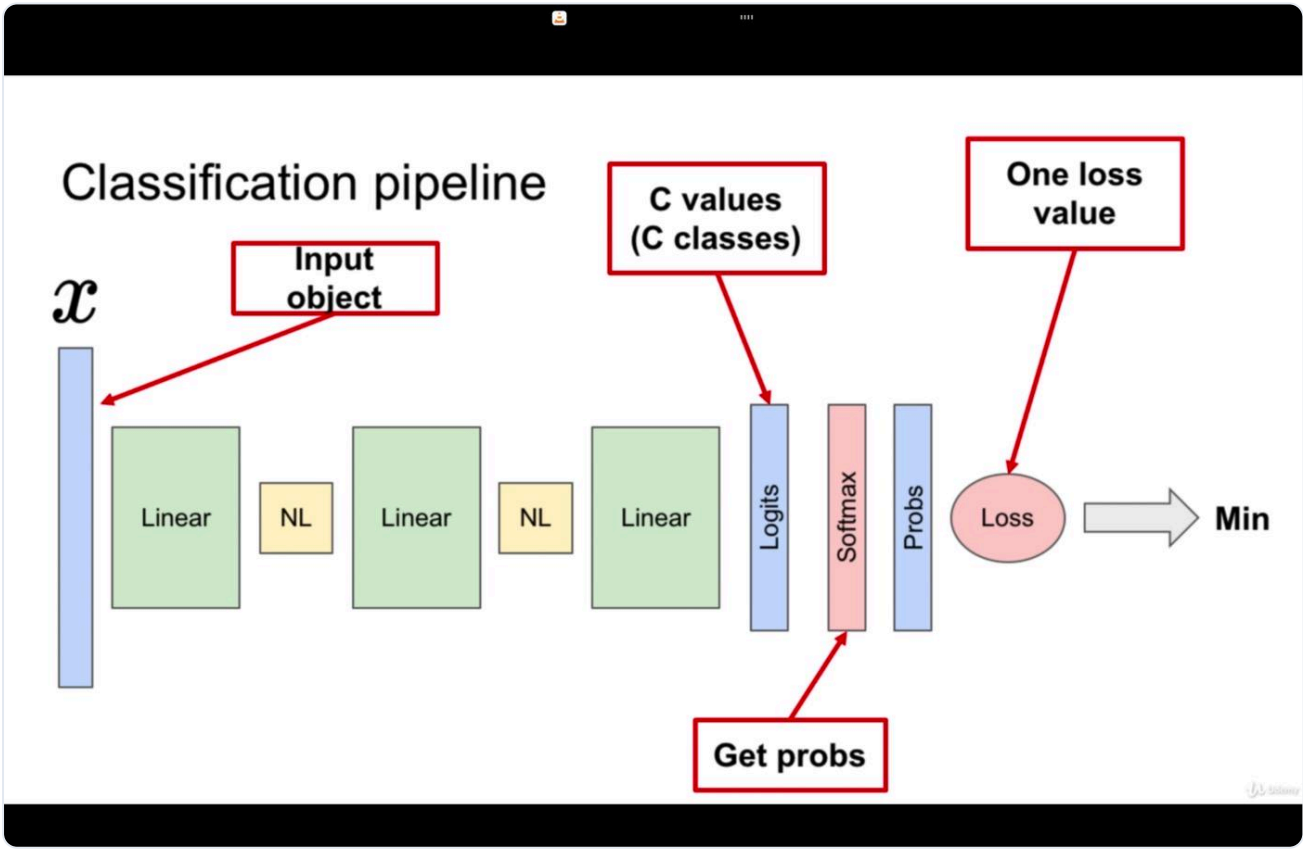


FIGURE 3.11 | Classification Pipeline

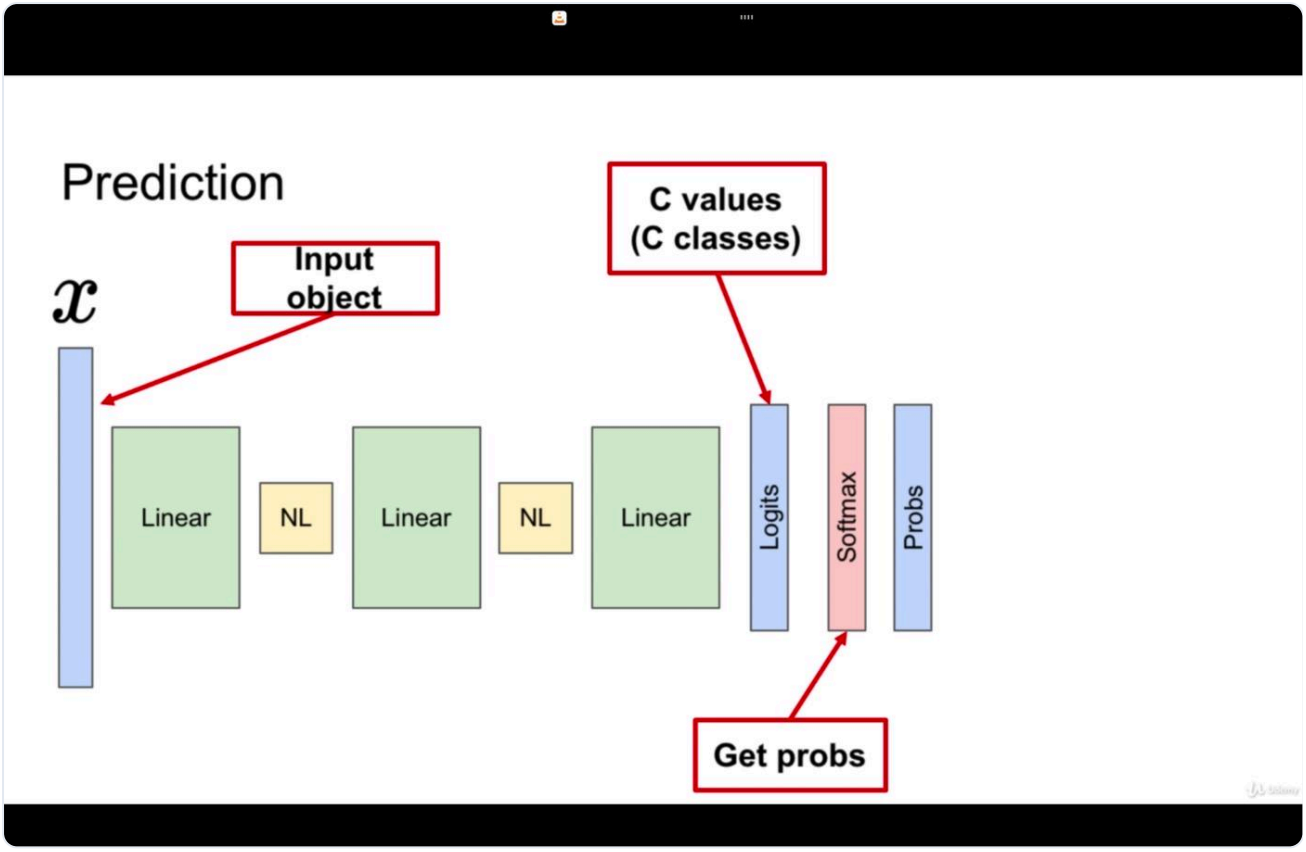


FIGURE 3.12

It consists of numbers from minus infinity to infinity each logic is responsible for one class the more the lawyers the higher the probability of the corresponding class. Usually we should transform our logits to probabilities. Here is softmax layer comes to help us probabilities that were obtained on the last layer can be used in two ways. First prediction. Second training where we use the probabilities for compute the loss value the less our loss the better our predictions are if loss is computed we can minimize it using stochastic gradient descent algorithm minimizing the loss is the process of training neural network to predict the class for the input object.

We use the class with the highest probability and that's it. Thank you for watching and see you in the next video.

LECTURE 4

Softmax Layer

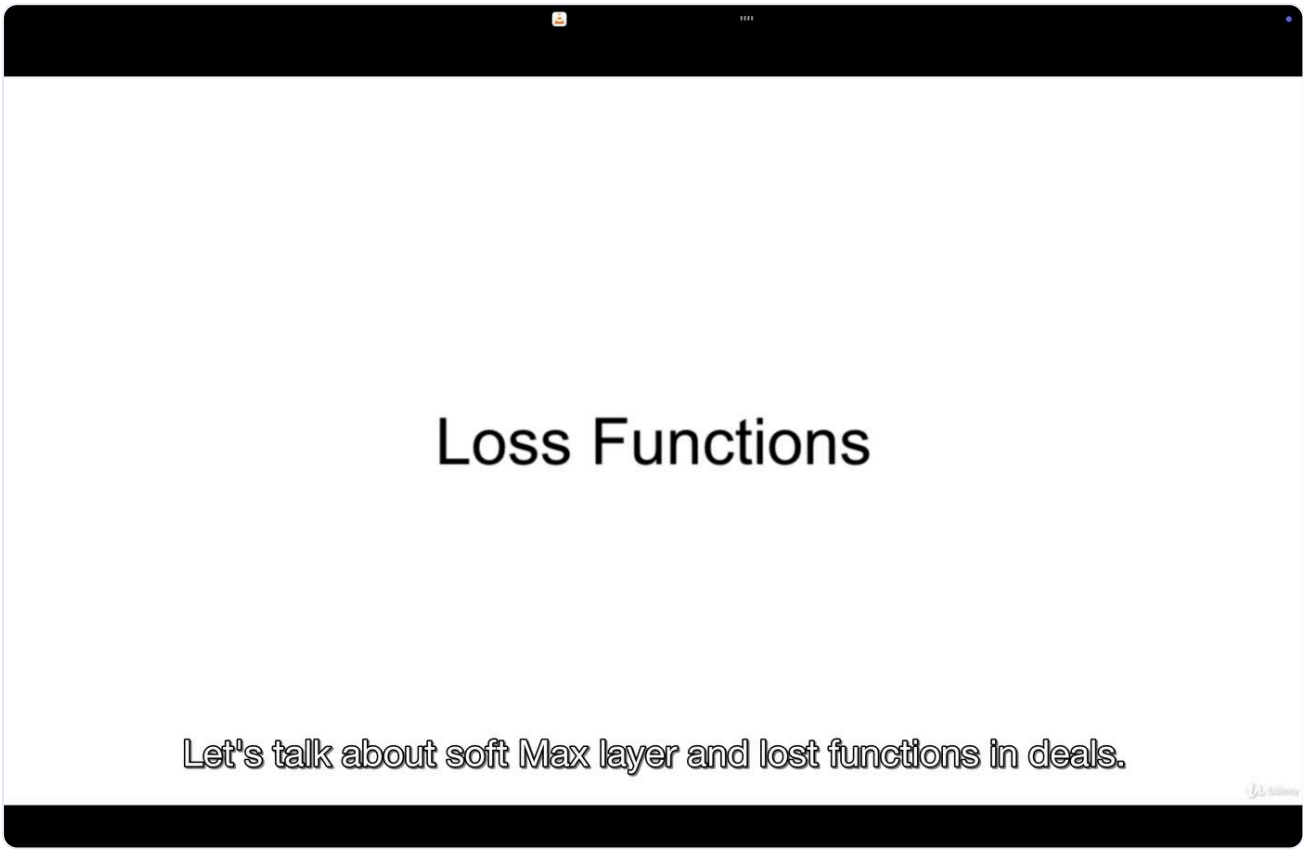


FIGURE 4.1 | Loss Functions

Let's talk about softmax layer and loss functions in deals. The first thing we're going to continue or is a softmax earlier this layer is widely used in classification and it's just transforming logits to probabilities.

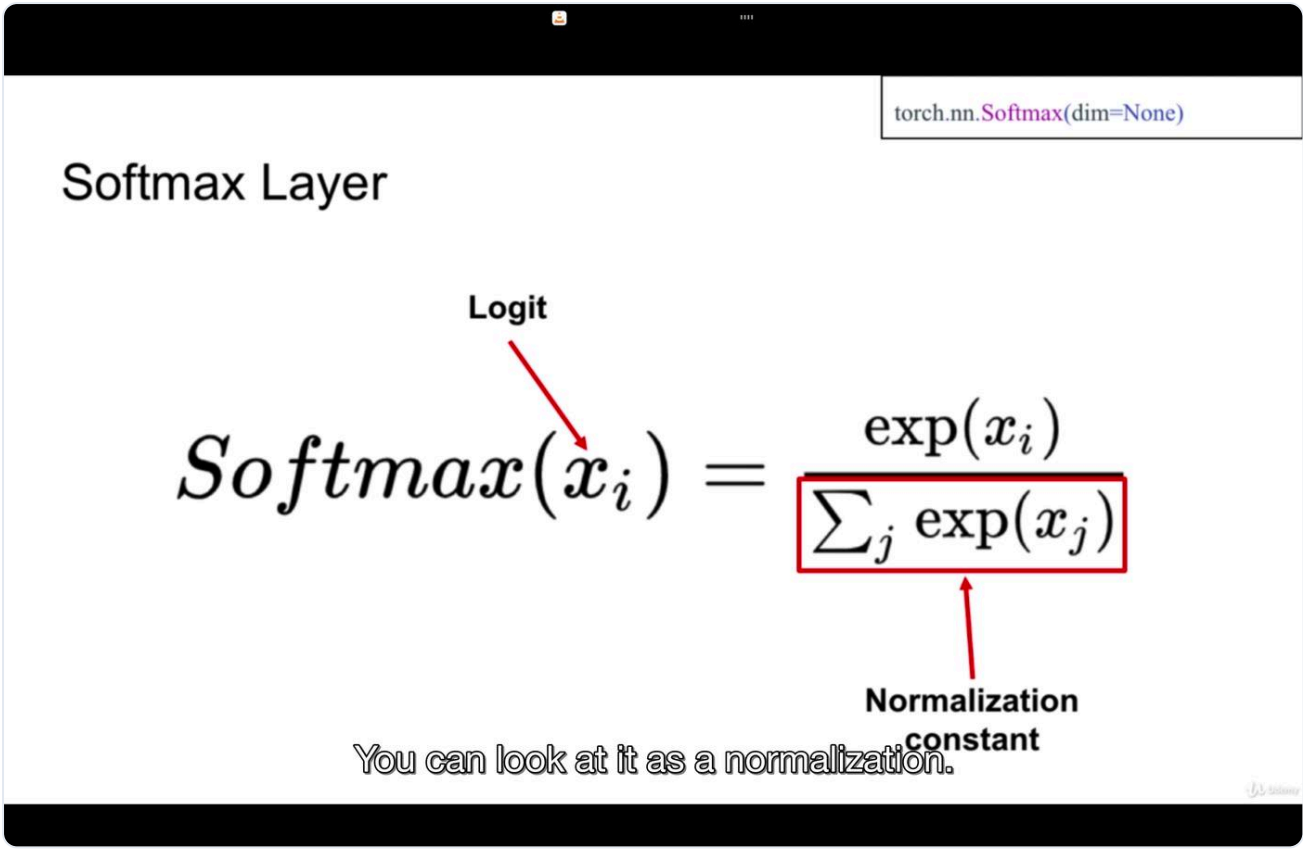


FIGURE 4.2 | Softmax Layer

You can look at it as a normalization. Firstly we transform all logits we their exponential function.

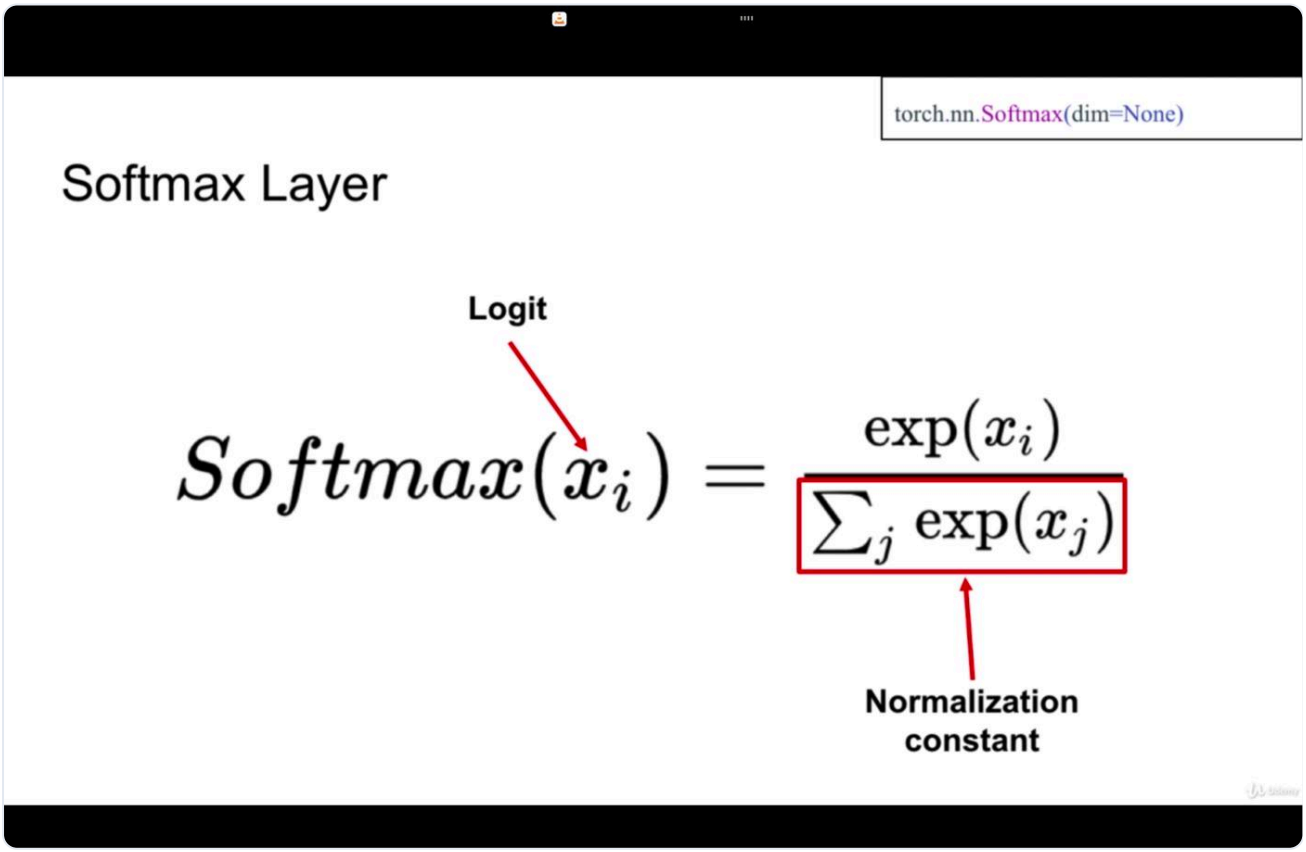


FIGURE 4.3 | Softmax Layer

And secondly we divide by the sum of the exponent for all moderates please stop the radio here and think why do we need both solutions formations exponent for all the logits and normalization and why we call the output of this layer as probabilities let us look at an example of softmax layer we want to classify 2 4 classes cats dogs cow and horse the output of the last layer over the network is logits for 1 2 and minus 8. So if Max layer allows using each of these logits for normalizing them to probabilities which are on the right wild probabilities are computed it is important to speak about the loss function.

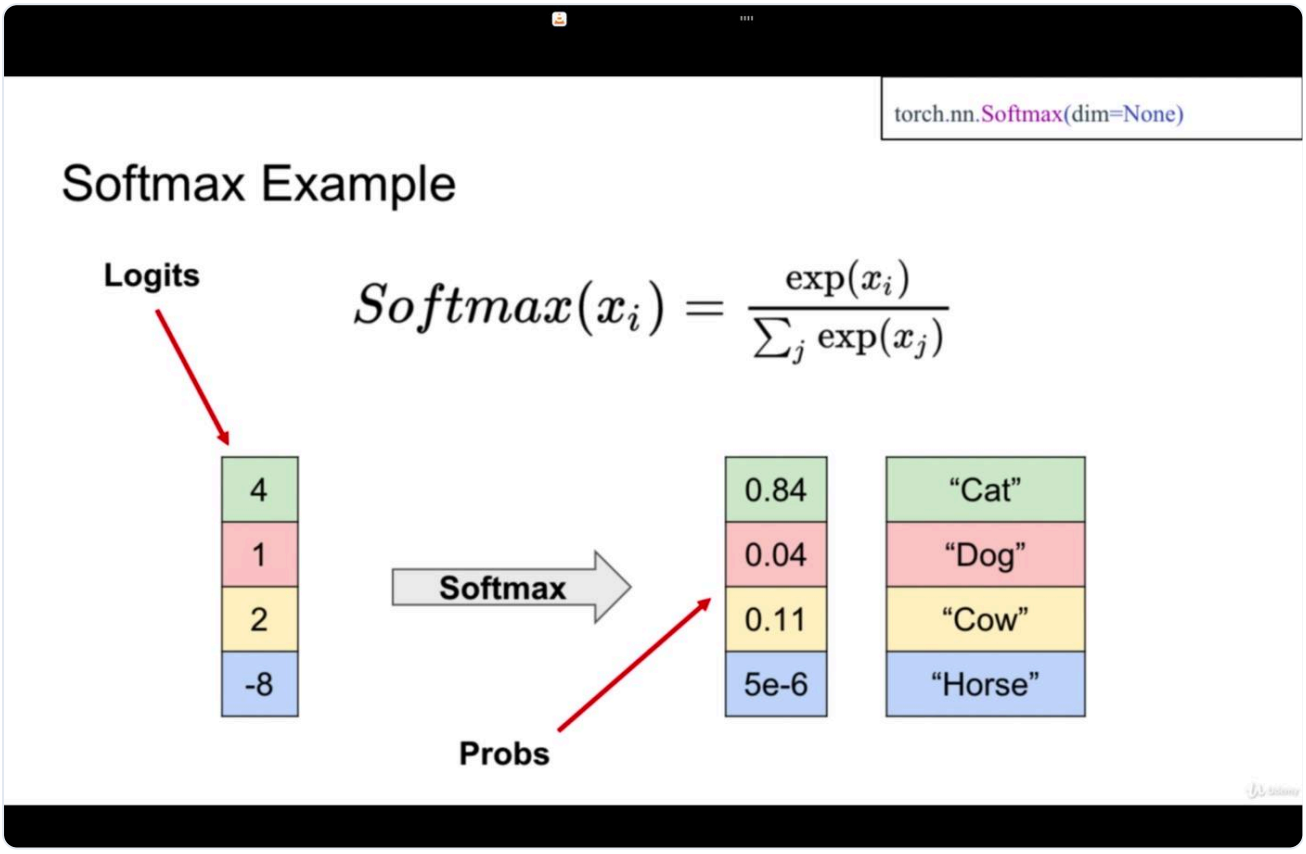


FIGURE 4.4 | Softmax Example

Now we're talking about the binary version of cross-entropy. Let's look at it in details.

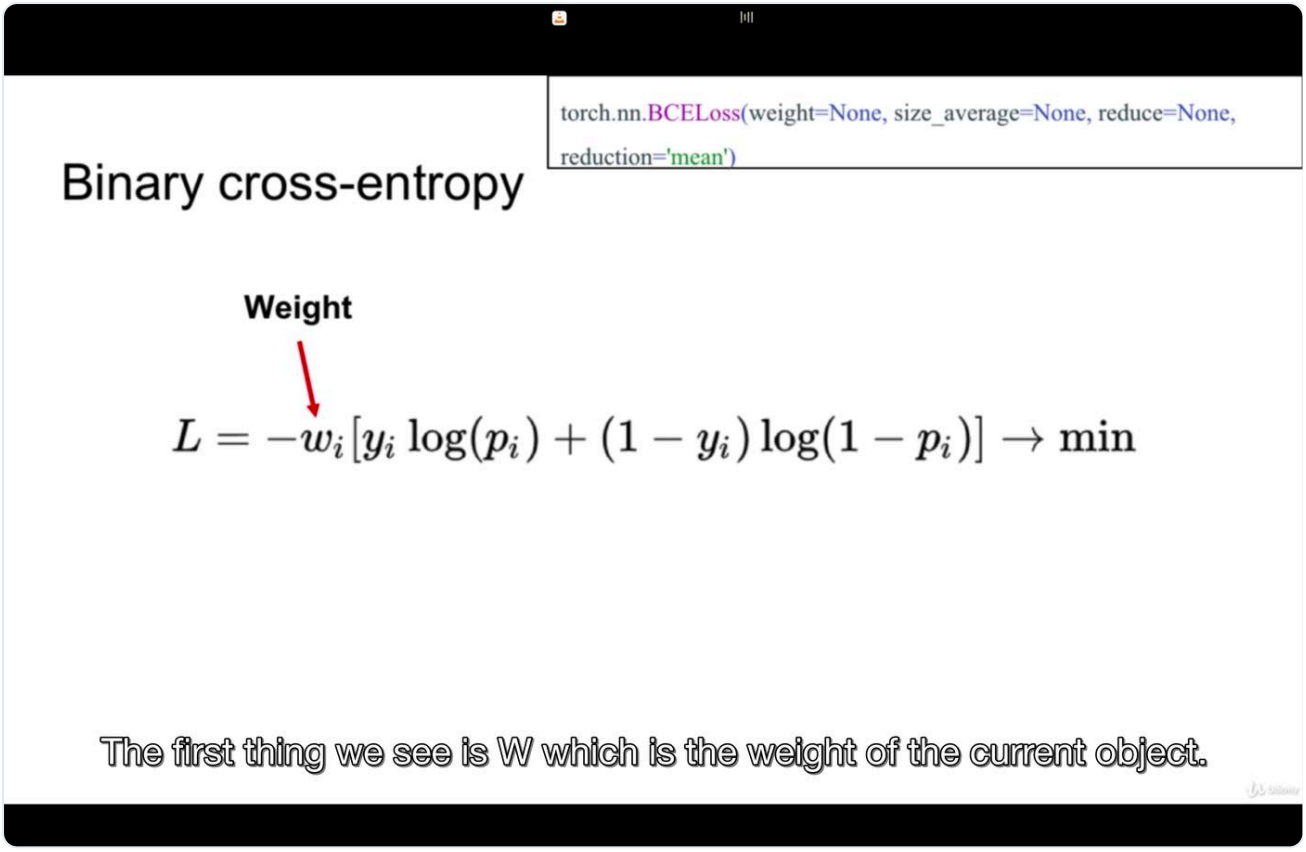


FIGURE 4.5

The first thing we see is W which is the weight of the current object. The loss function is computed for each object and then we averaged that we can compute not usual average but weighted one. However usually All weights are equal to one after setting some weight to the object.

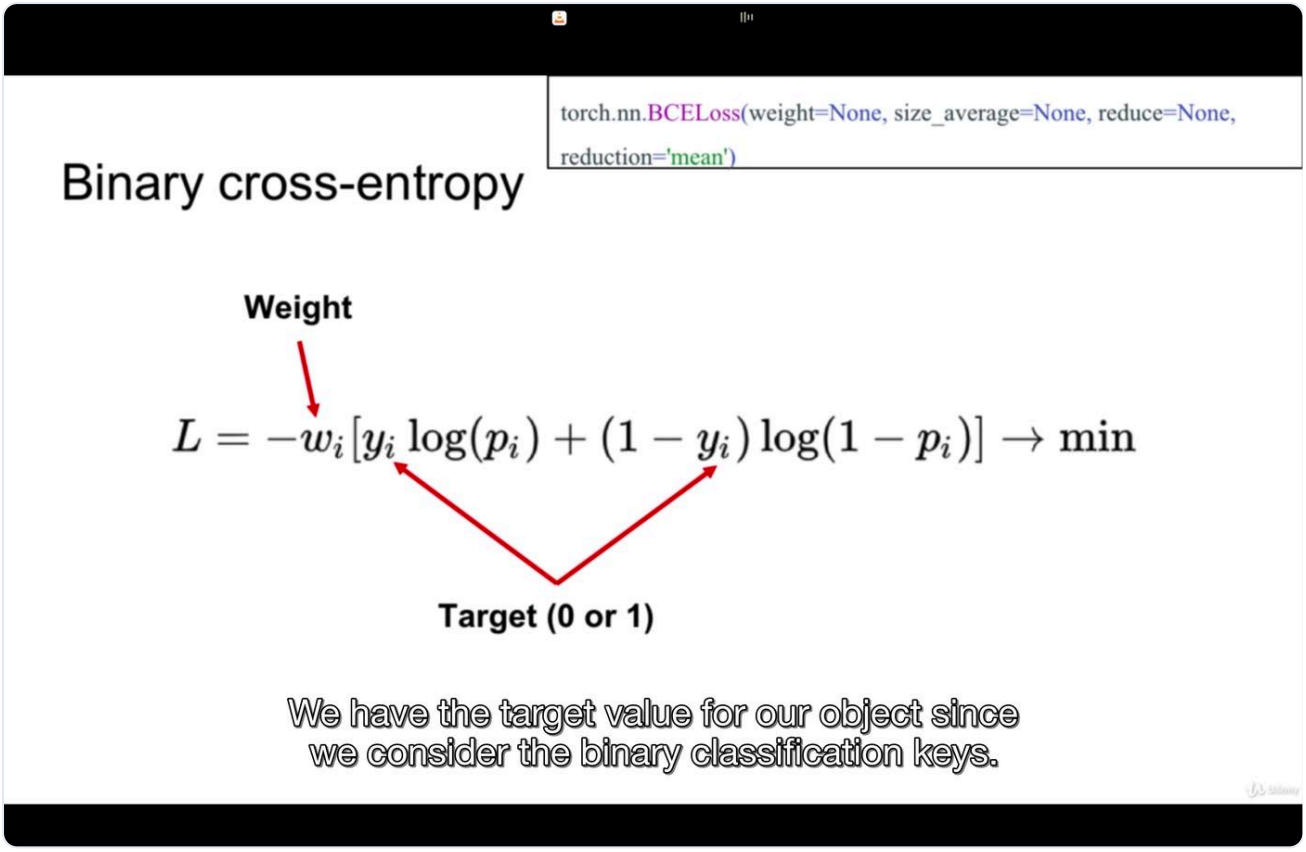


FIGURE 4.6

We have the target value for our object since we consider the binary classification keys. Our labels take values either 0 or 1 depending on need. We will have only one non-zero term left or right we also have the predicted probability of Class 1 which is the output of the last layer of our neural network with sigmoid function.

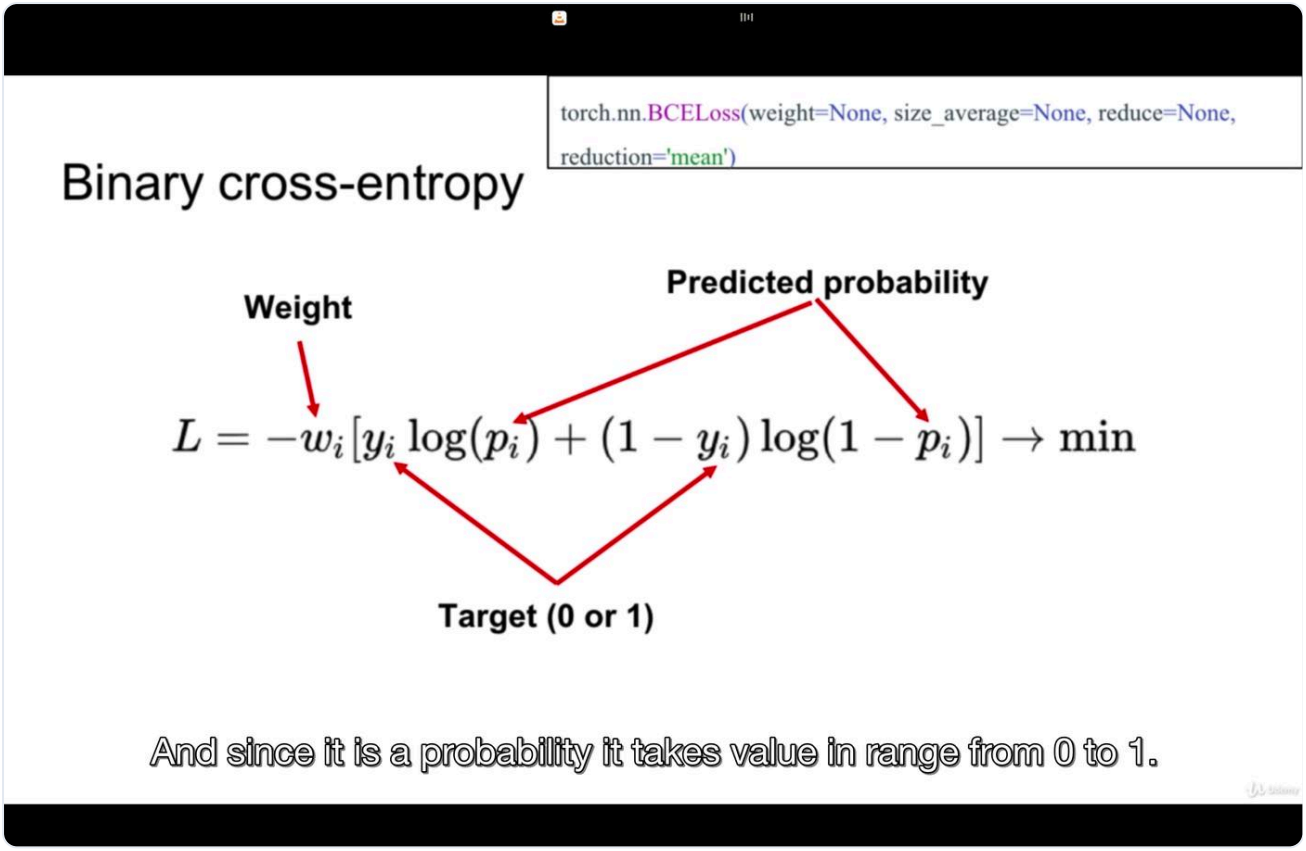


FIGURE 4.7

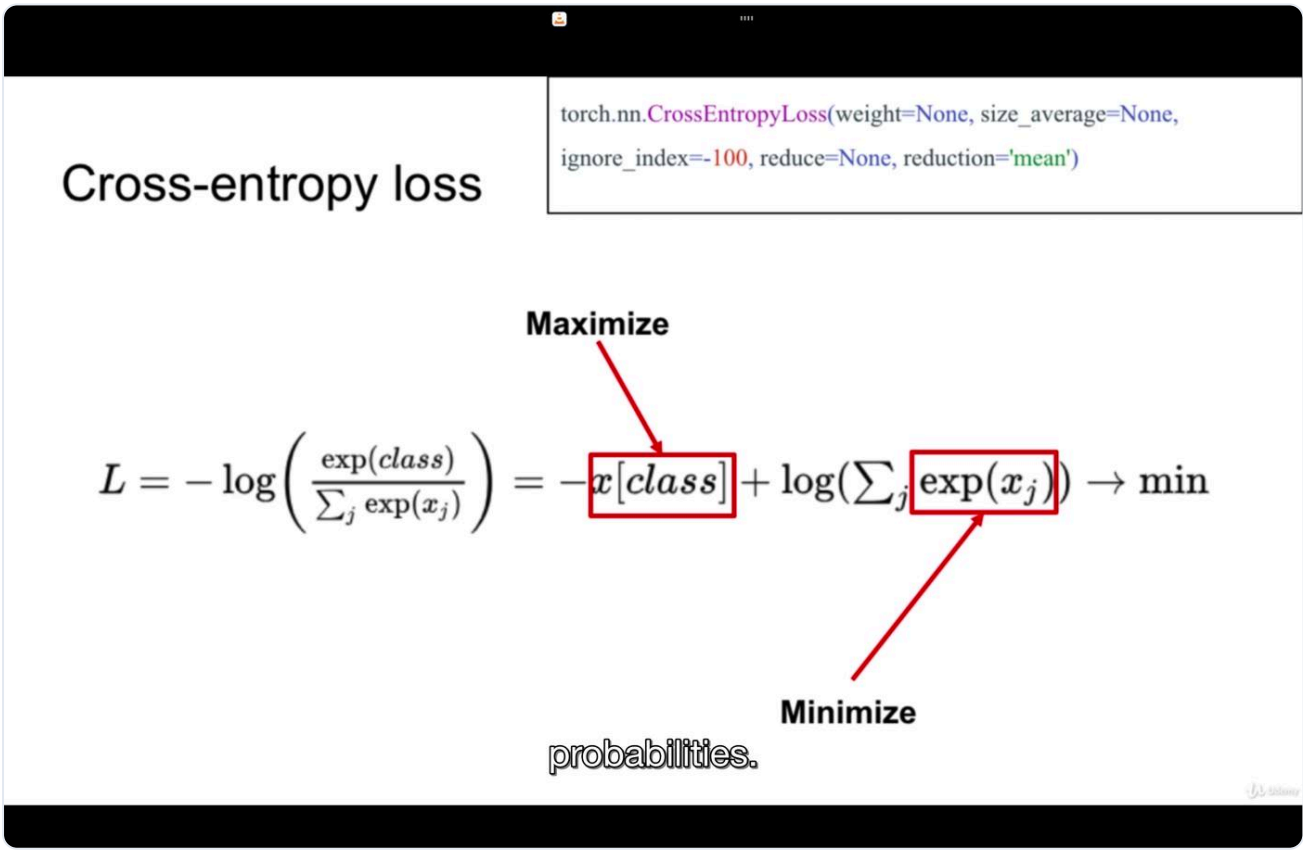


FIGURE 4.8

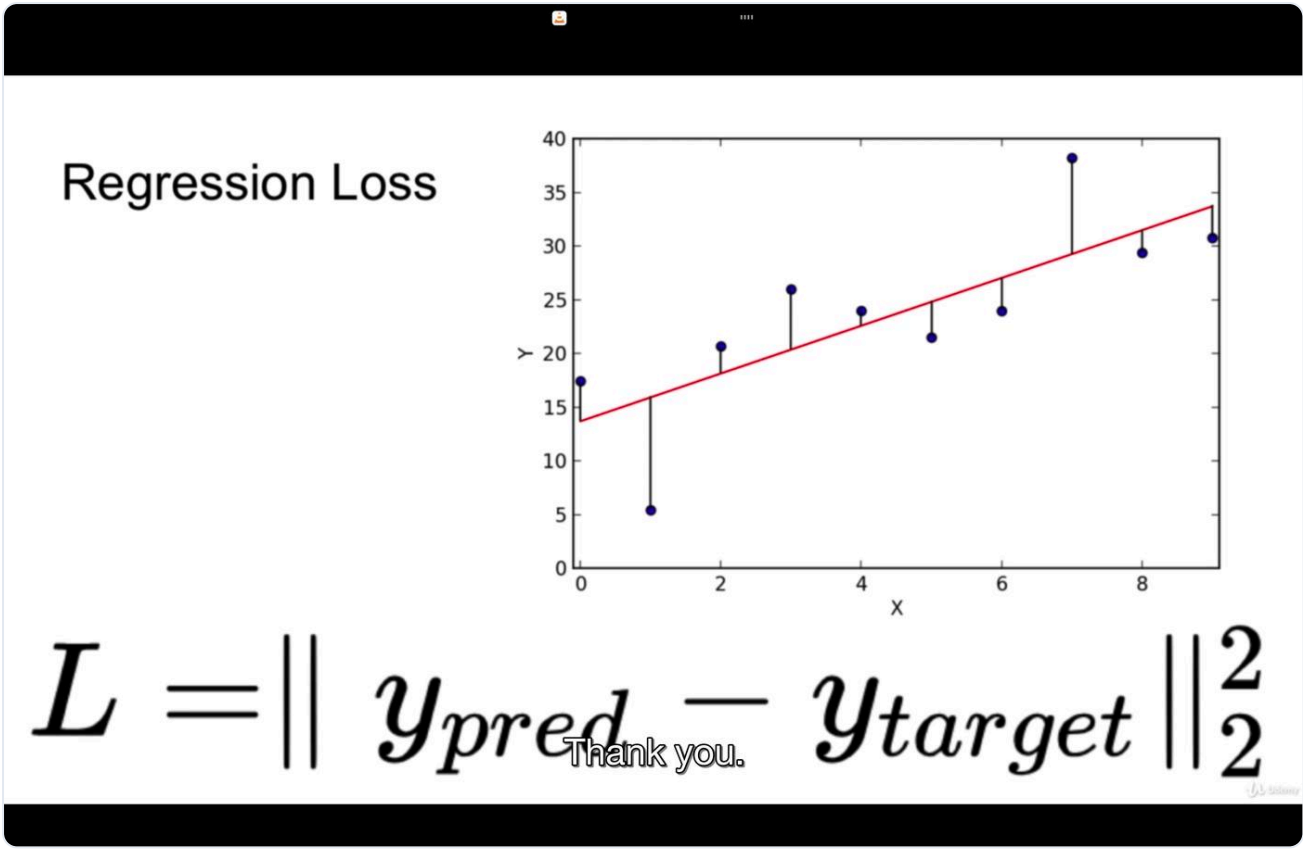


FIGURE 4.9 | Regression Loss 35

And since it is a probability it takes value in range from 0 to 1. If our target label is 1 then we have the only left term and maximize the probability of Class 1 if the target label is 0. Then we minimize the probability of Class 1. Note that maximizing or minimizing is regularized with the miners before the loss. Let's consider the full course entropy loss function in PyTorch it is called Cross entropy loss. What you should know about it is that we maximize the probability of true class and minimize all other probabilities. If you use cross entropy loss in PyTorch. Note that it already includes the log-softmax operation internally. Consider it a regression problem.

You are free to use any regression loss such as L2 Euclidean distance L1 Euclidean distance or any other gestural remind you should compute average distance between your predicted y and target y. The more the distance the worse your predictions. Thank you. See you in the next video.

LECTURE 5

Gradient Descent



FIGURE 5.1 | Gradient Descent

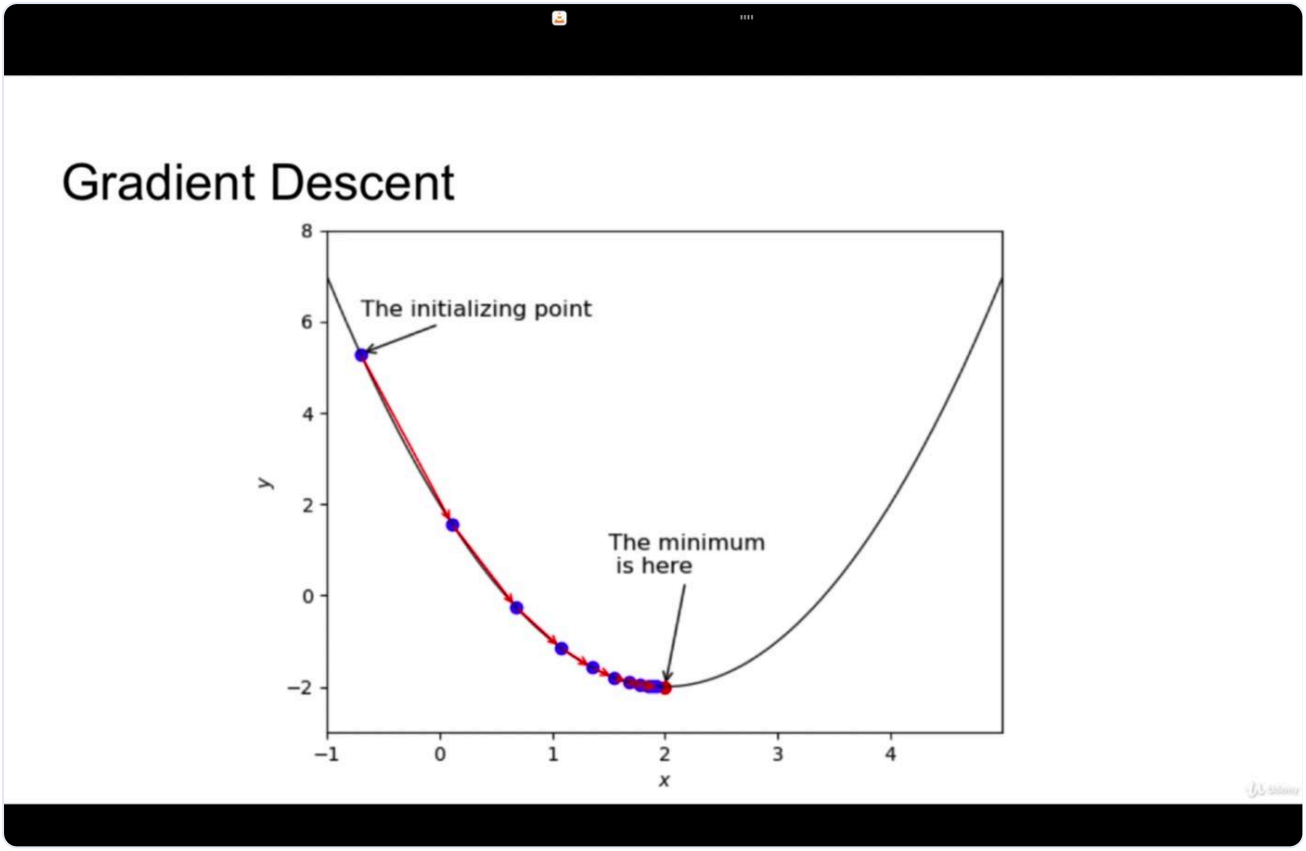


FIGURE 5.2 | Gradient Descent

Now I'm going to briefly remind you this the stochastic gradient descent algorithm the common way to train deep neural networks gradient descent is an algorithm that step by step updates their parameters of the network going to the minimum of the loss function starting from some initial parameters will move in the direction opposite to the gradient vector we have a loss function l of X and the theta that we wish to optimize.

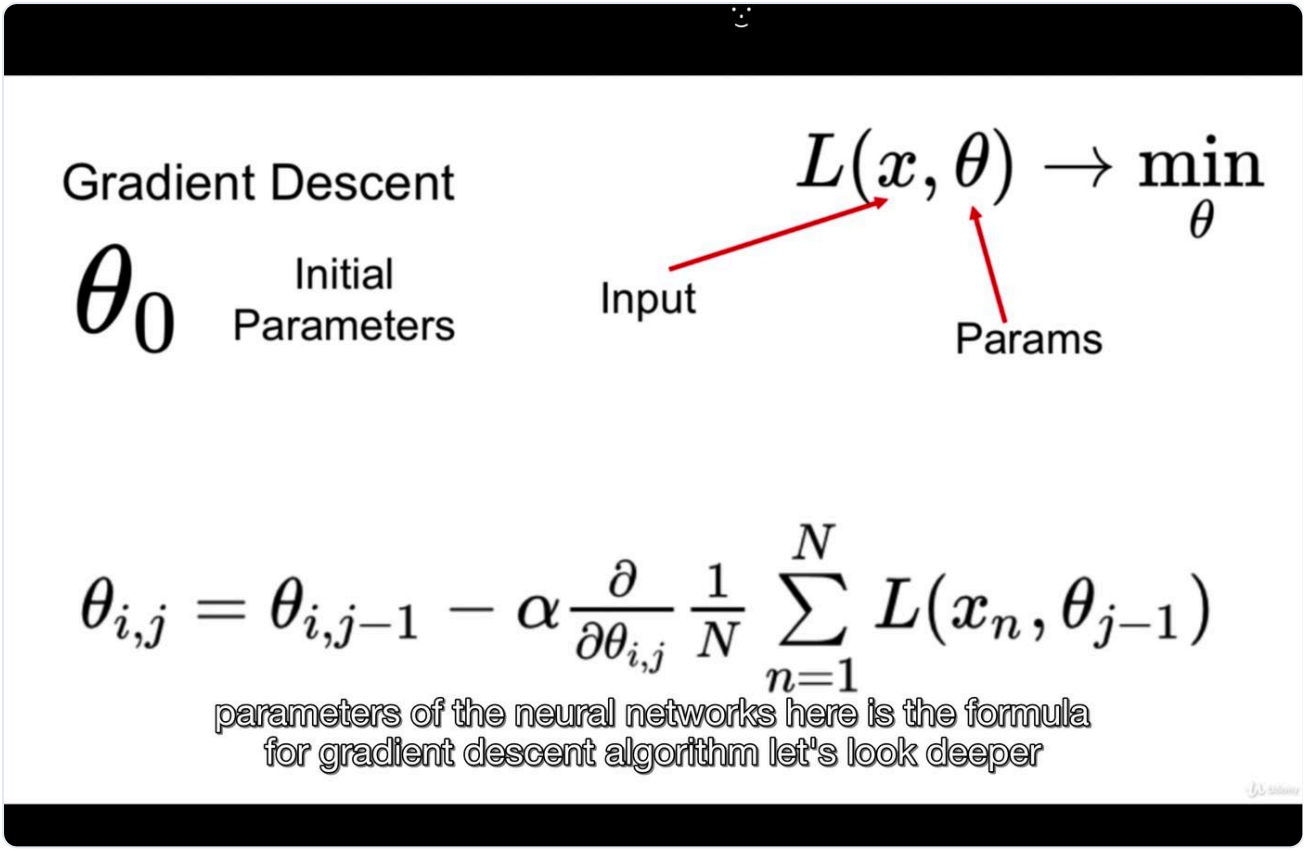


FIGURE 5.3

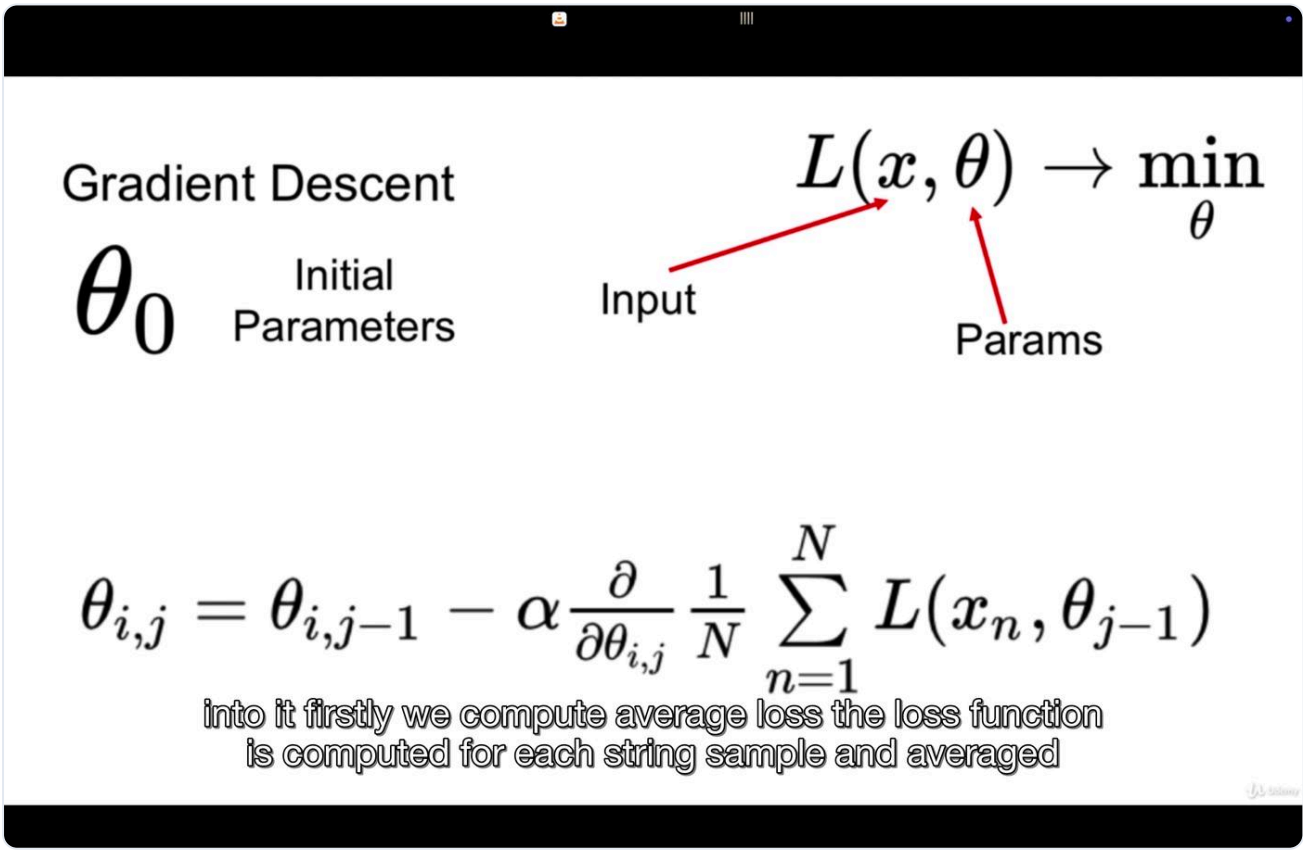


FIGURE 5.4

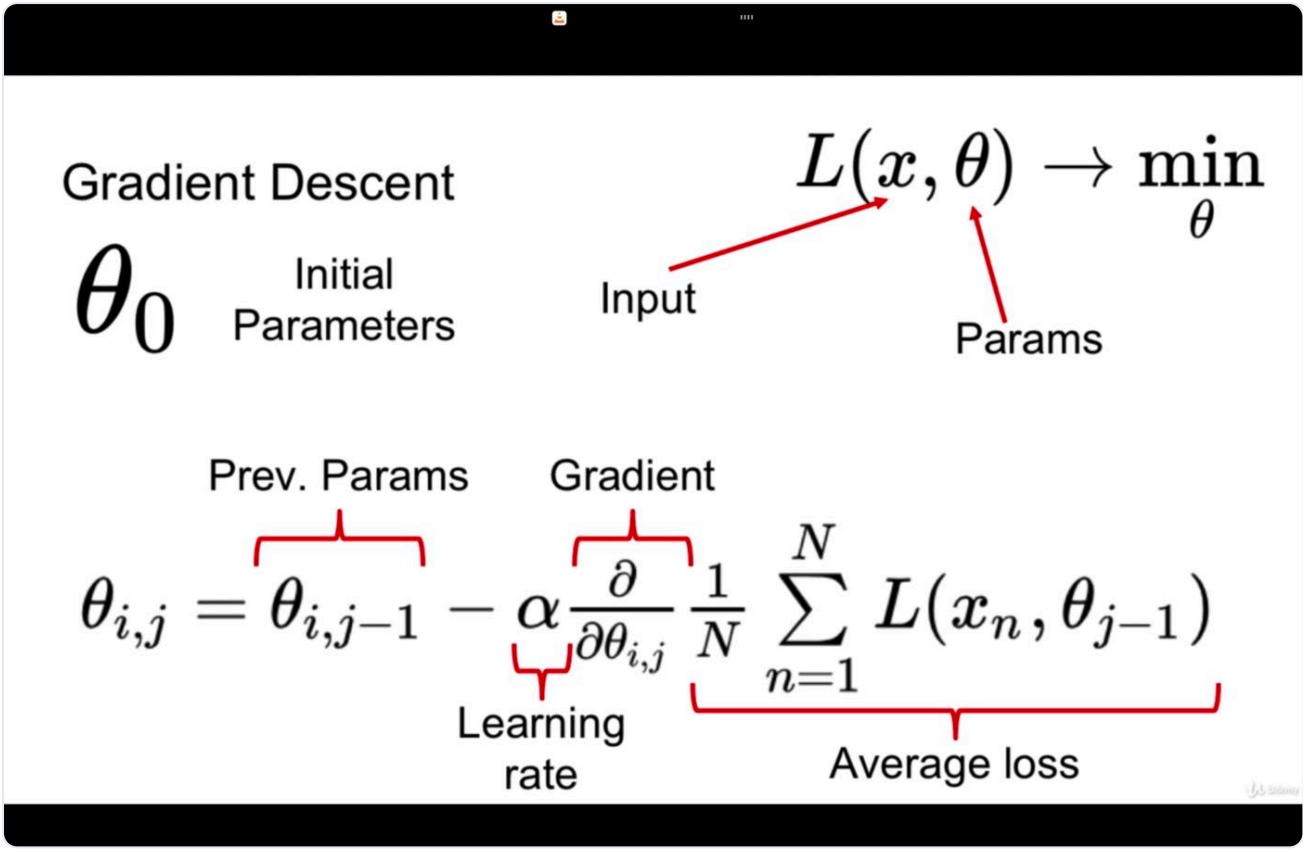


FIGURE 5.5

In other words we want to find such parameters theta that minimize the loss function we set up some initial parameters theta zero in this course we will later discuss how to properly choose the initial parameters of the neural networks here is the formula for gradient descent algorithm let's look deeper into it firstly we compute average loss the loss function is computed for each training sample and averaged secondly would take its gradient according to each parameter of the neural network in this formula we consider the update of that i-th parameter then we should multiply our computed gradient on the predefined learning rate which is the speed of learning.

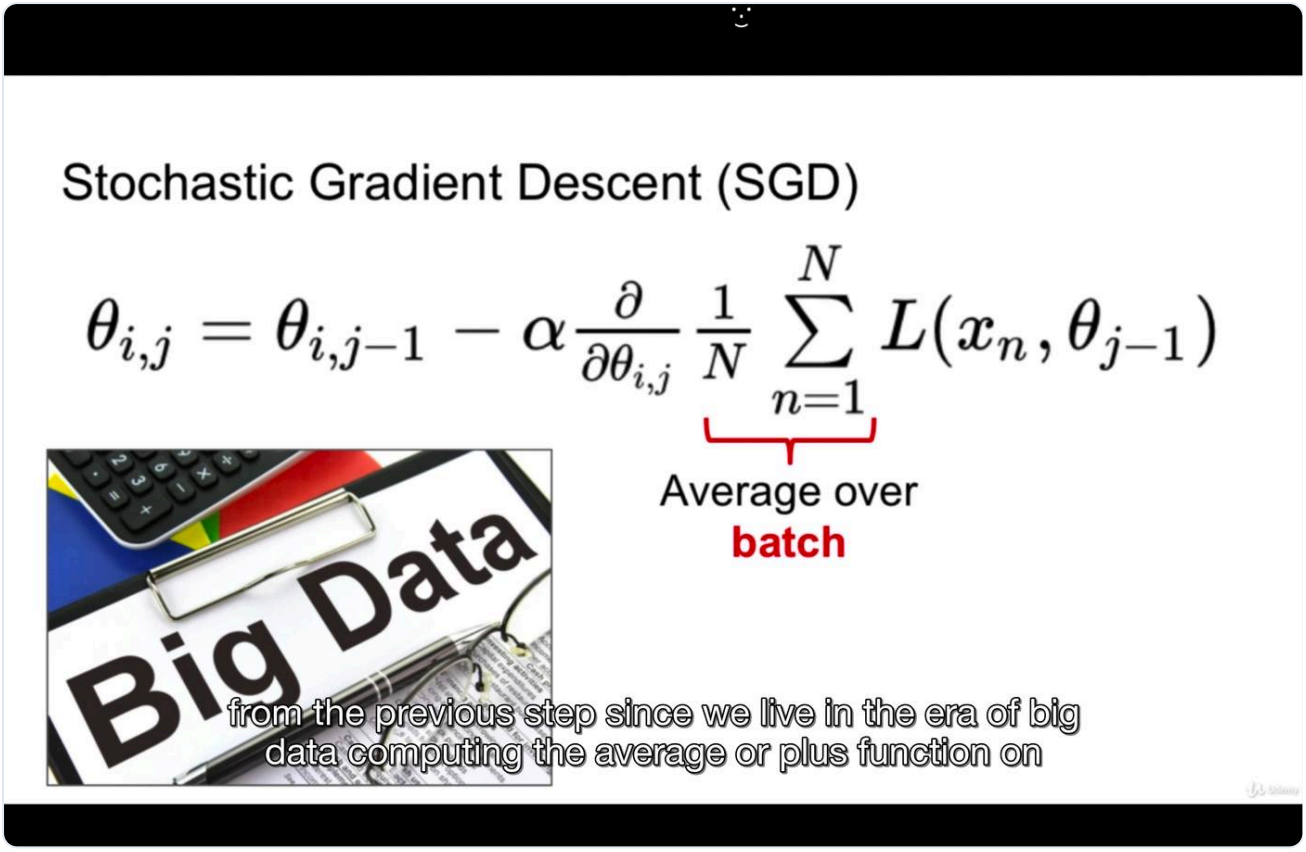


FIGURE 5.6 | Stochastic Gradient Descent (SGD)

Next we update our parameters by subtracting the gradient multiplied by learning rate from parameters from the previous step since we live in the era of big data computing the average loss function on millions or billions of objects seems computationally impossible however we can use not all the data from our training dataset but just the subset of it. This is what people usually call the stochastic gradient descent you should understand some facts about the choice of the batch size the larger your batch size the more GPU memory are required by your neural network therefore you are bounded above in the choice of the batch size on the other hand it is better to take bigger batch size which decreases the variance of your gradient a smaller batch size will lead to higher variance. The third important note prefer to take the batch size which is a power of 2.

The reason for these is the binary system in the lower level computations. It will be just faster.

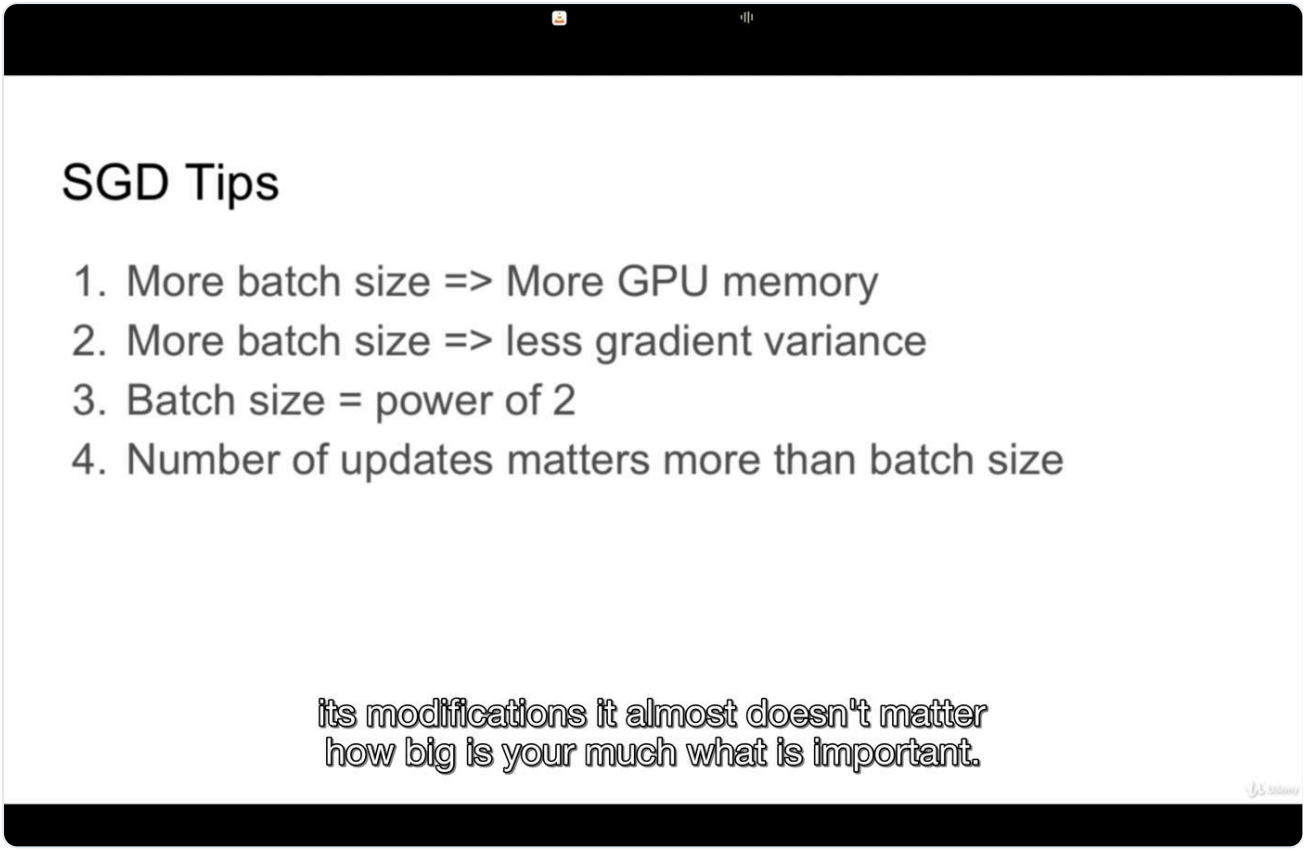


FIGURE 5.7

The usual choice 32 64 128 and so on the last but not the least for us the stochastic gradient descent and its modifications it almost doesn't matter how big is your much what is important. The number of updates it will be almost no difference between batch size 256 and five hundred and twelve. Since you're just slightly improved their estimation of the average gradient let us now consider how we work with data in the mini-batch gradient descent. As usual we have a set of trading data at each iteration of neural network. Look at the batch of data points when the neural network looked at all the data. It is called the epoch. We're now going deeper into details.

The first thing you should do is shuffling your data before each epoch we fix the size of the batch to some value and split all the training dataset into batches of this size.

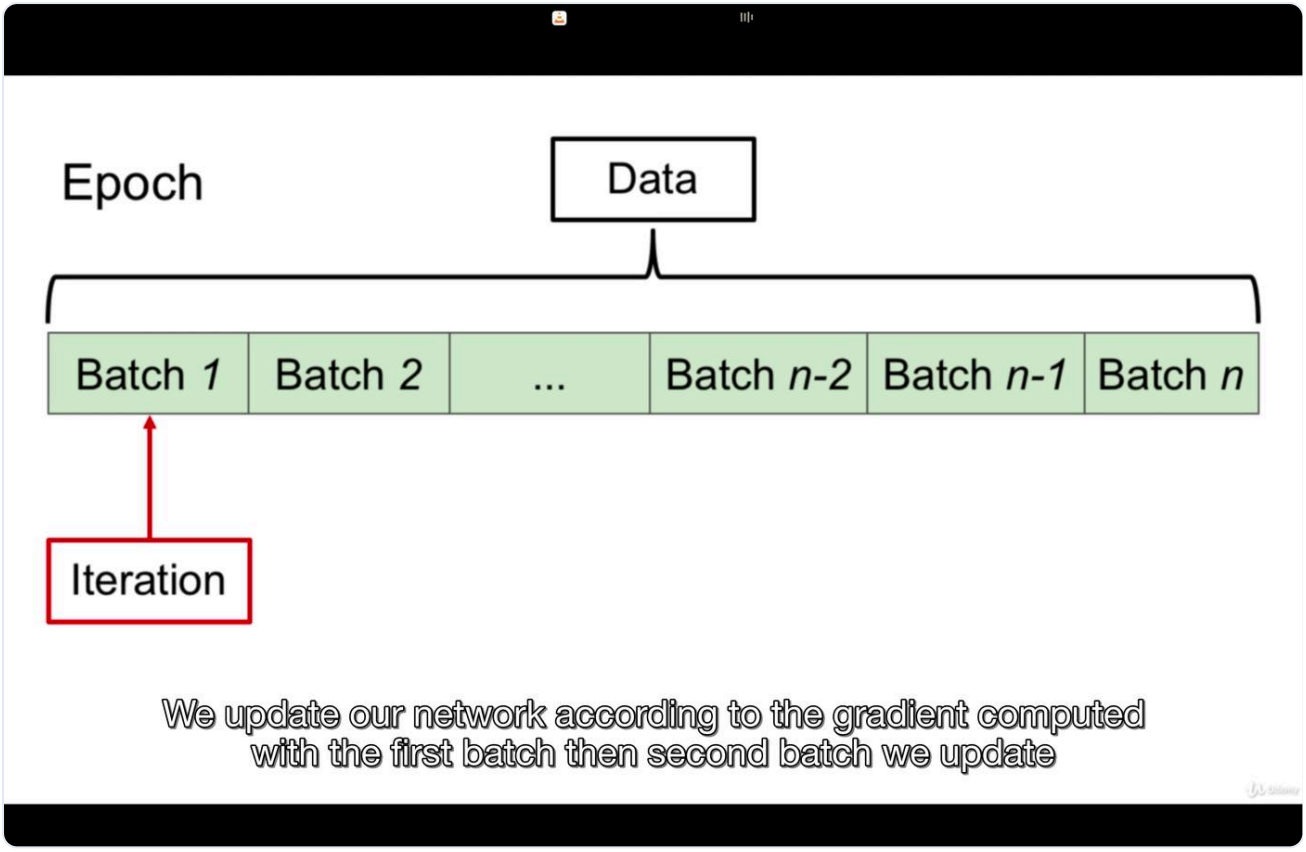


FIGURE 5.8

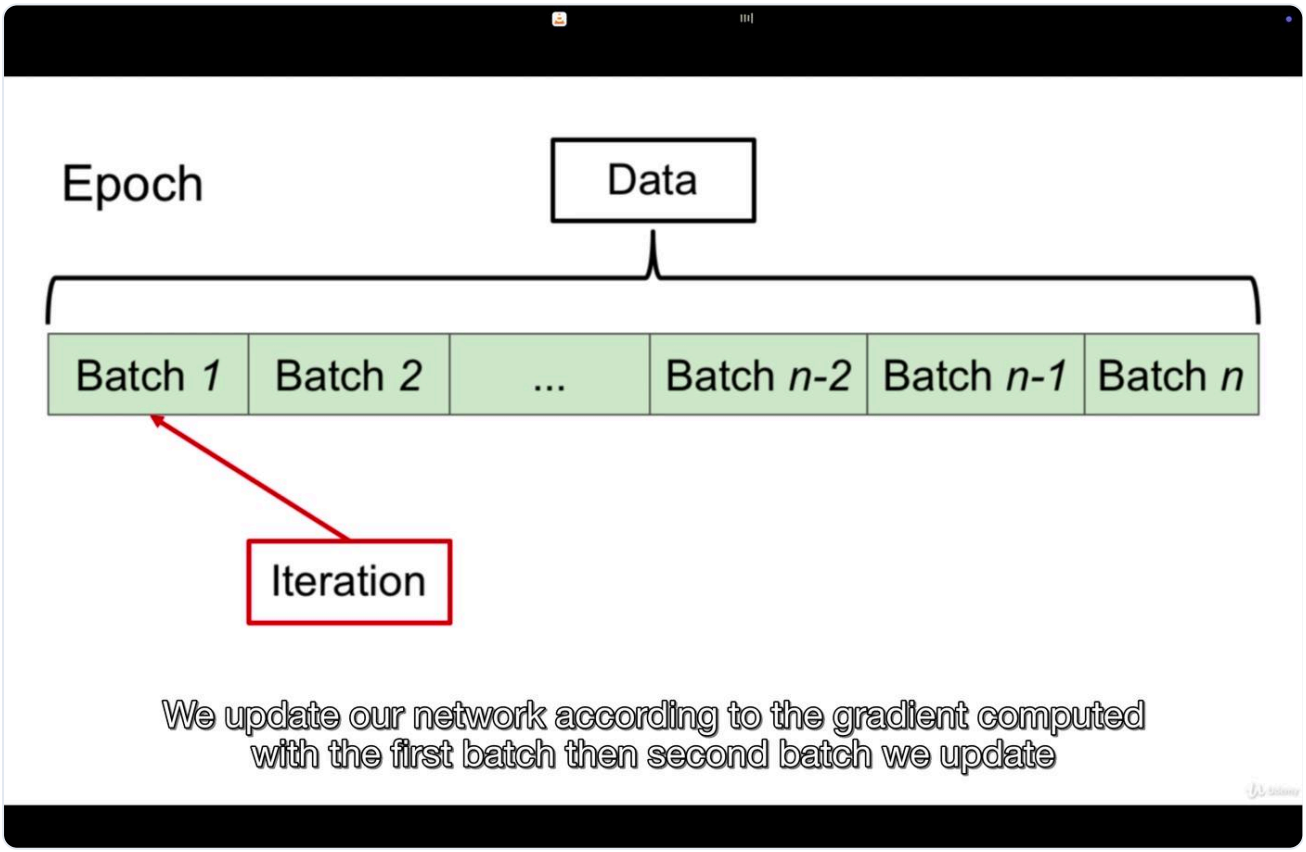


FIGURE 5.9

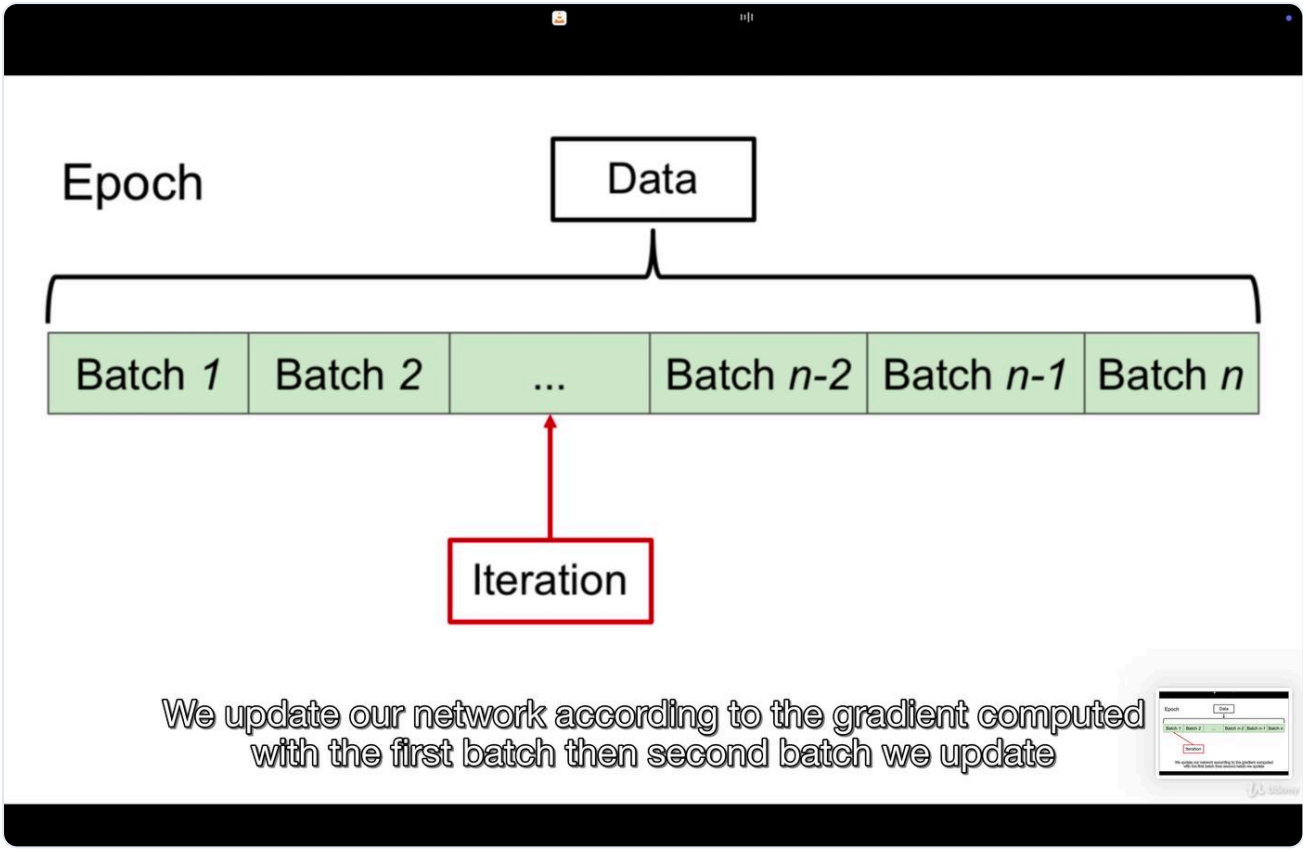


FIGURE 5.10

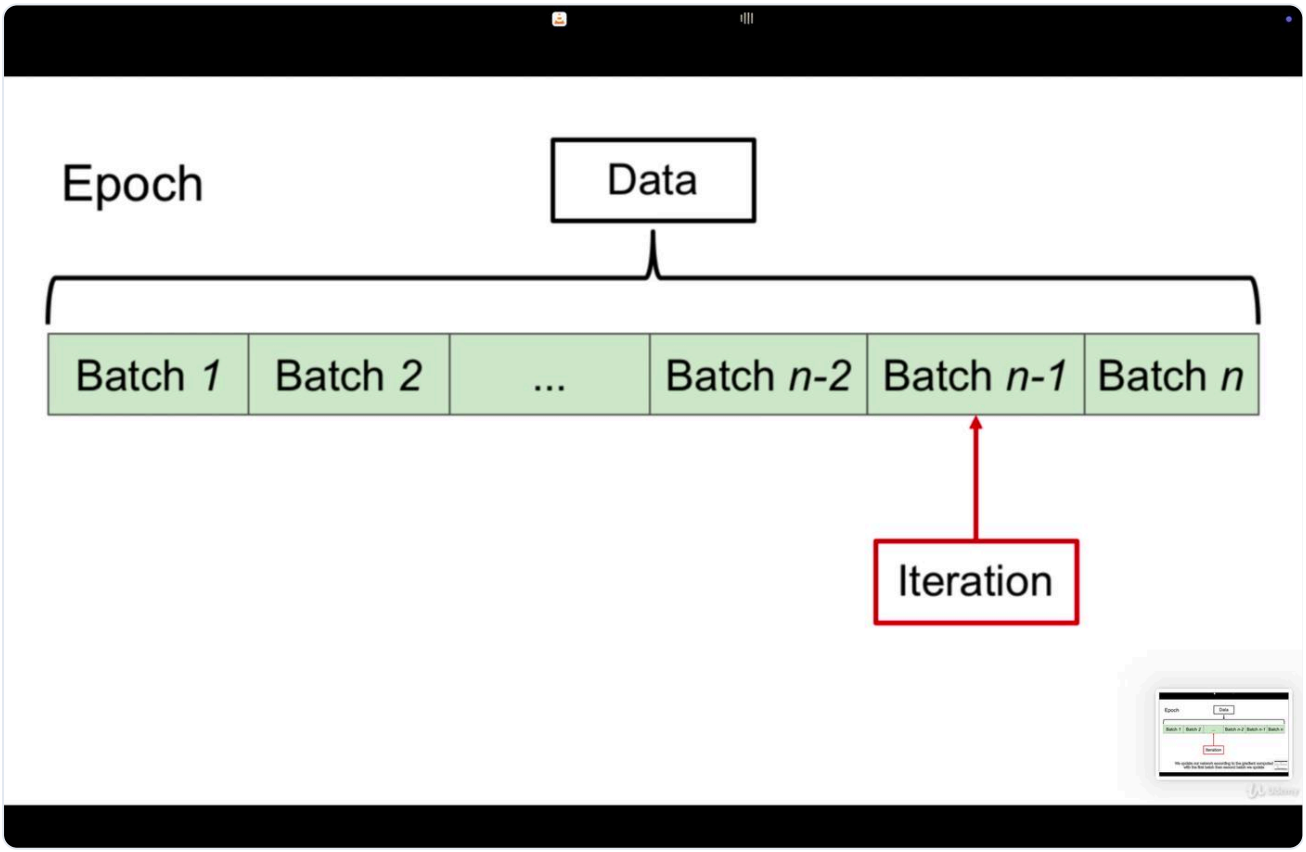


FIGURE 5.11

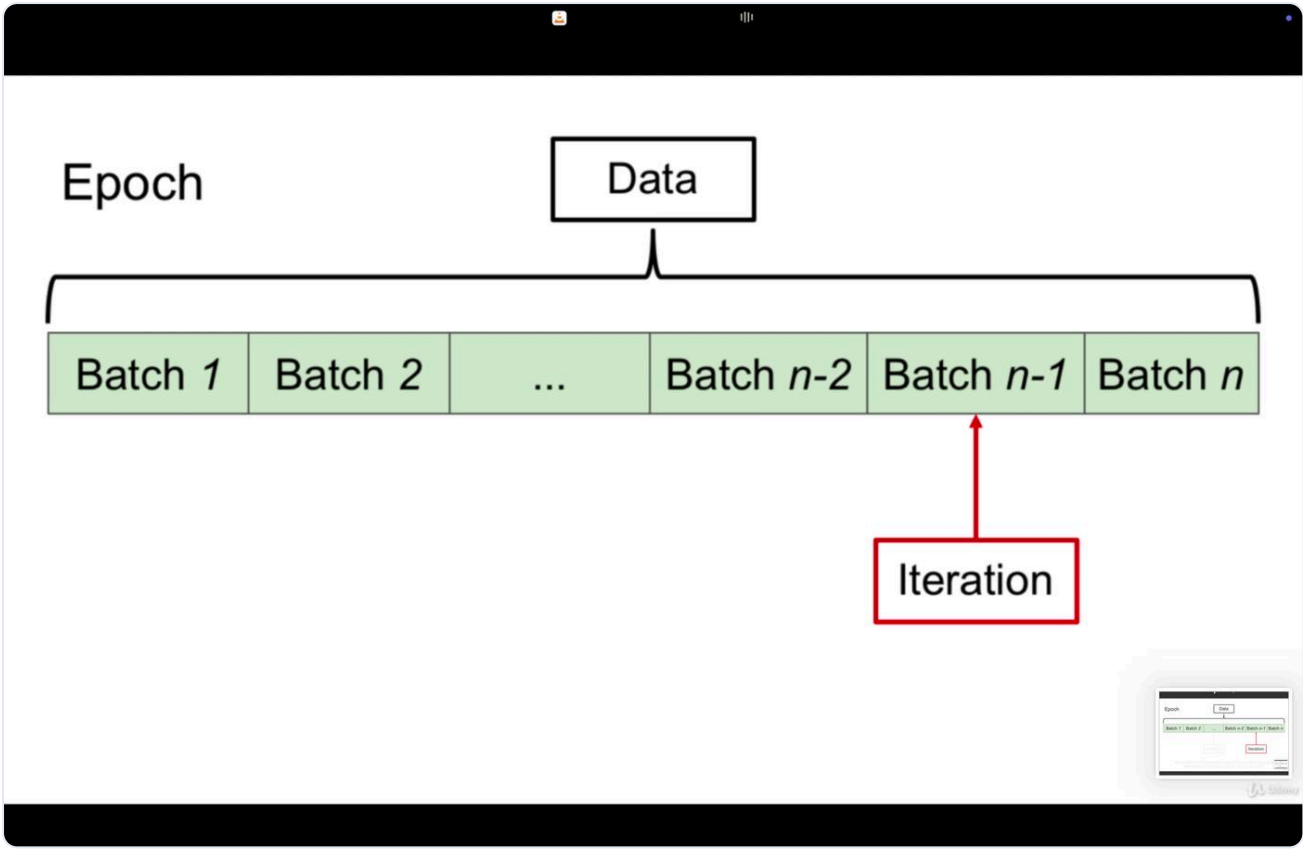


FIGURE 5.12

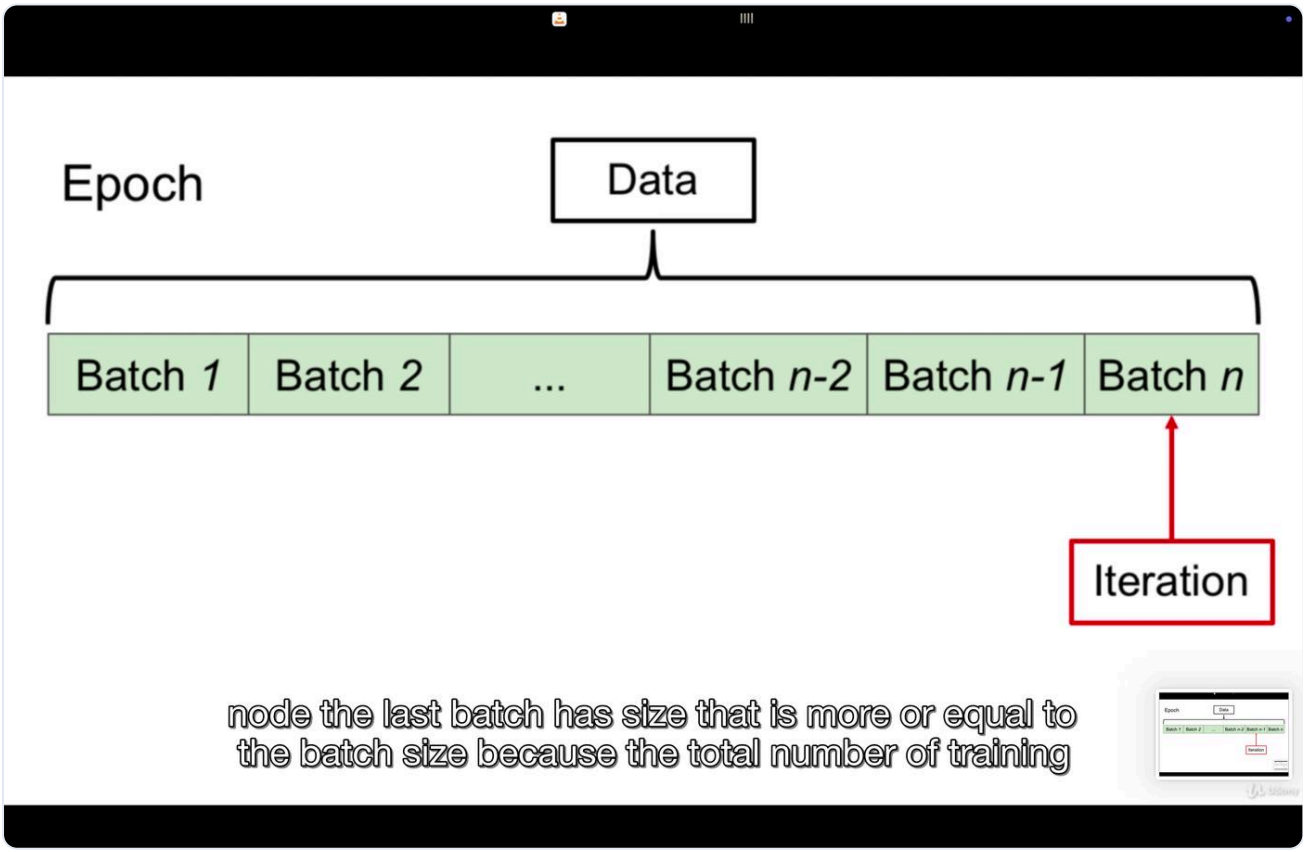


FIGURE 5.13

We update our network according to the gradient computed with the first batch then second batch we update our network batch by batch node the last batch has size that is greater than or equal to the batch size because the total number of training samples is not divisible by the batch size. All I showed you is one full epoch. After it, we start the next epoch of training note that we should shuffle our data again.

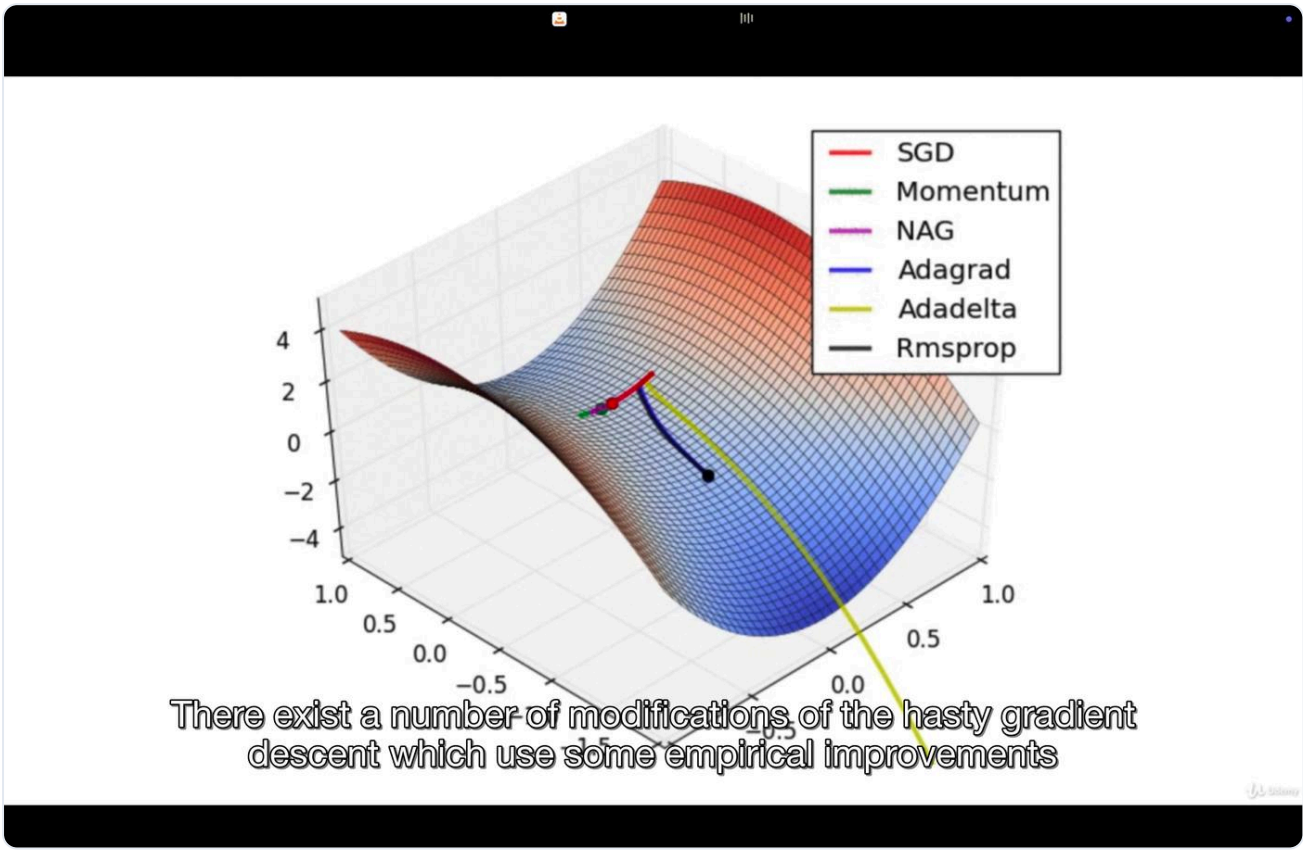


FIGURE 5.14

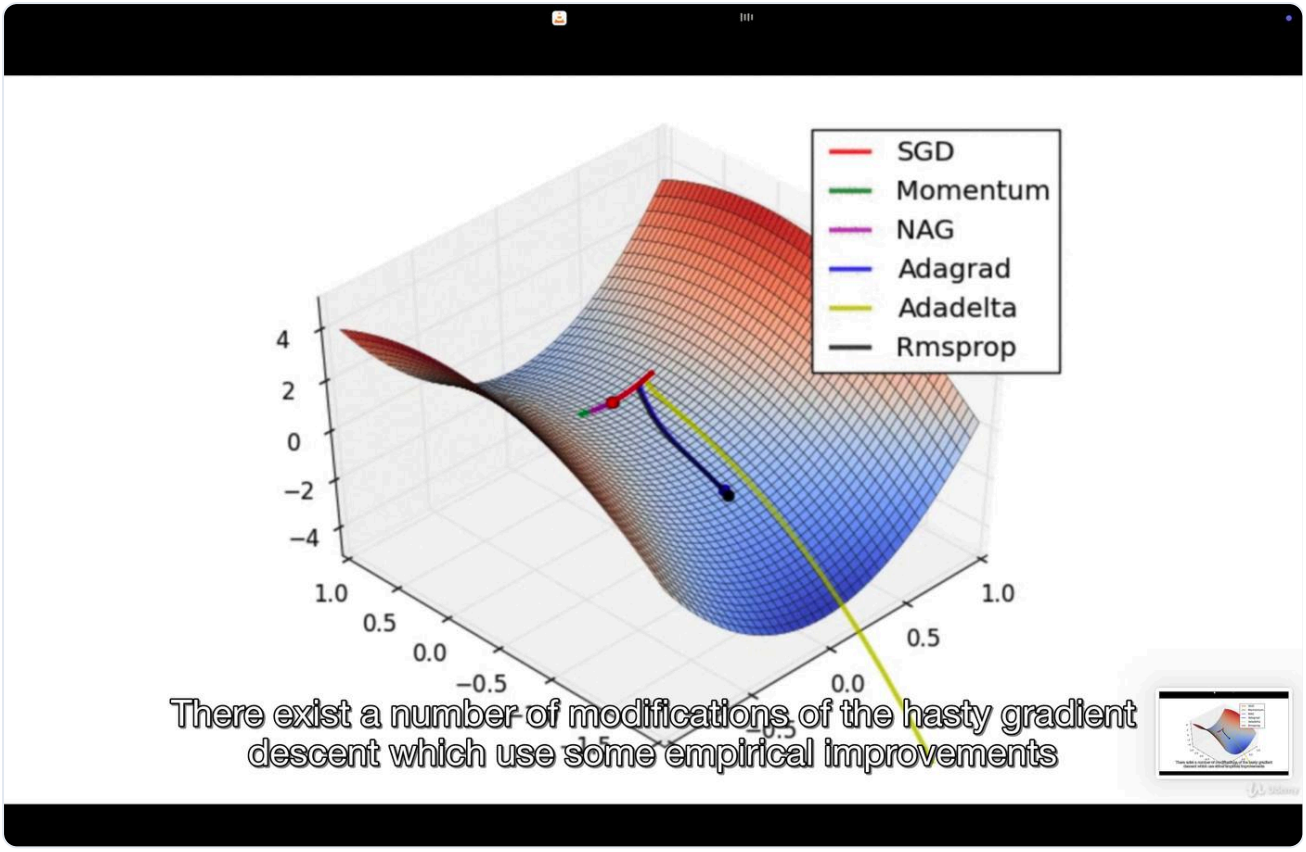


FIGURE 5.15

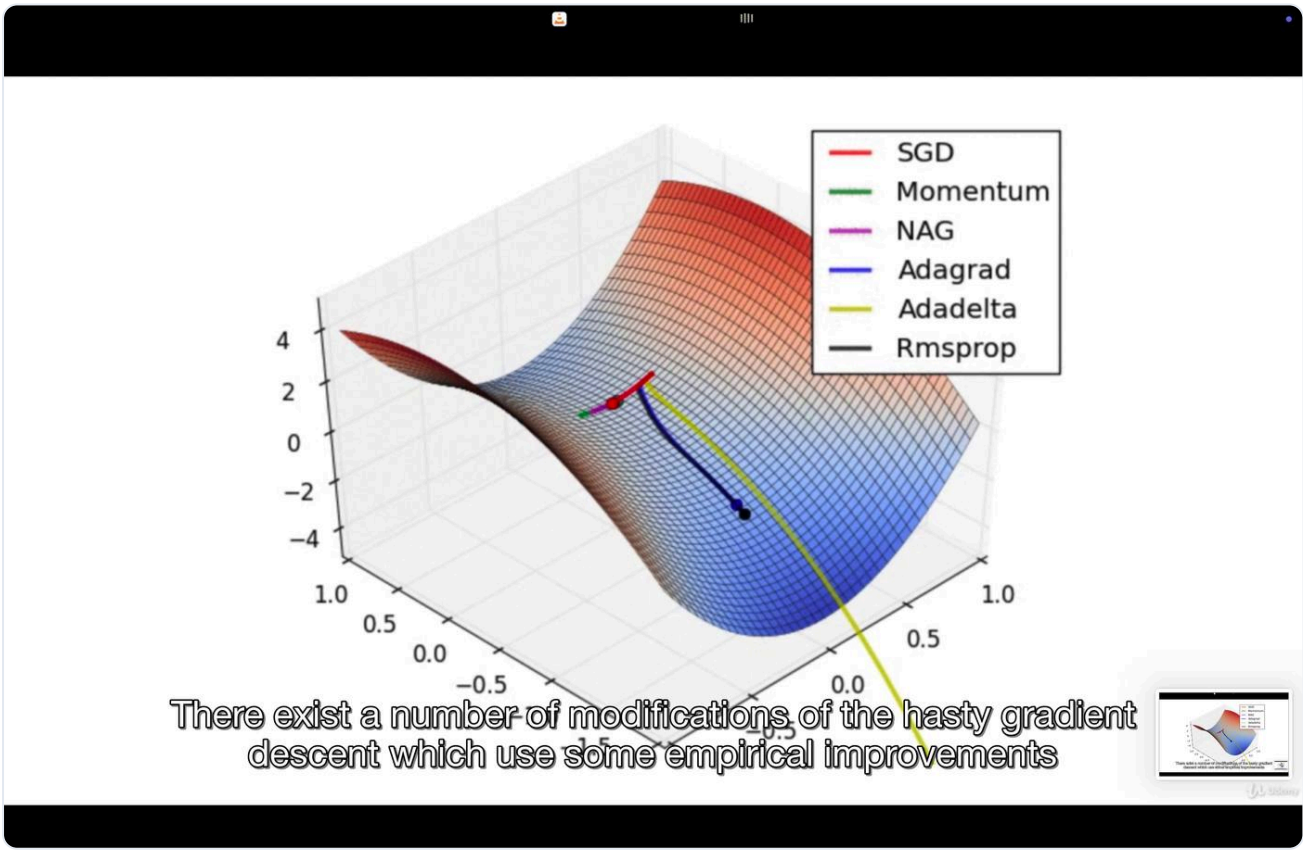


FIGURE 5.16

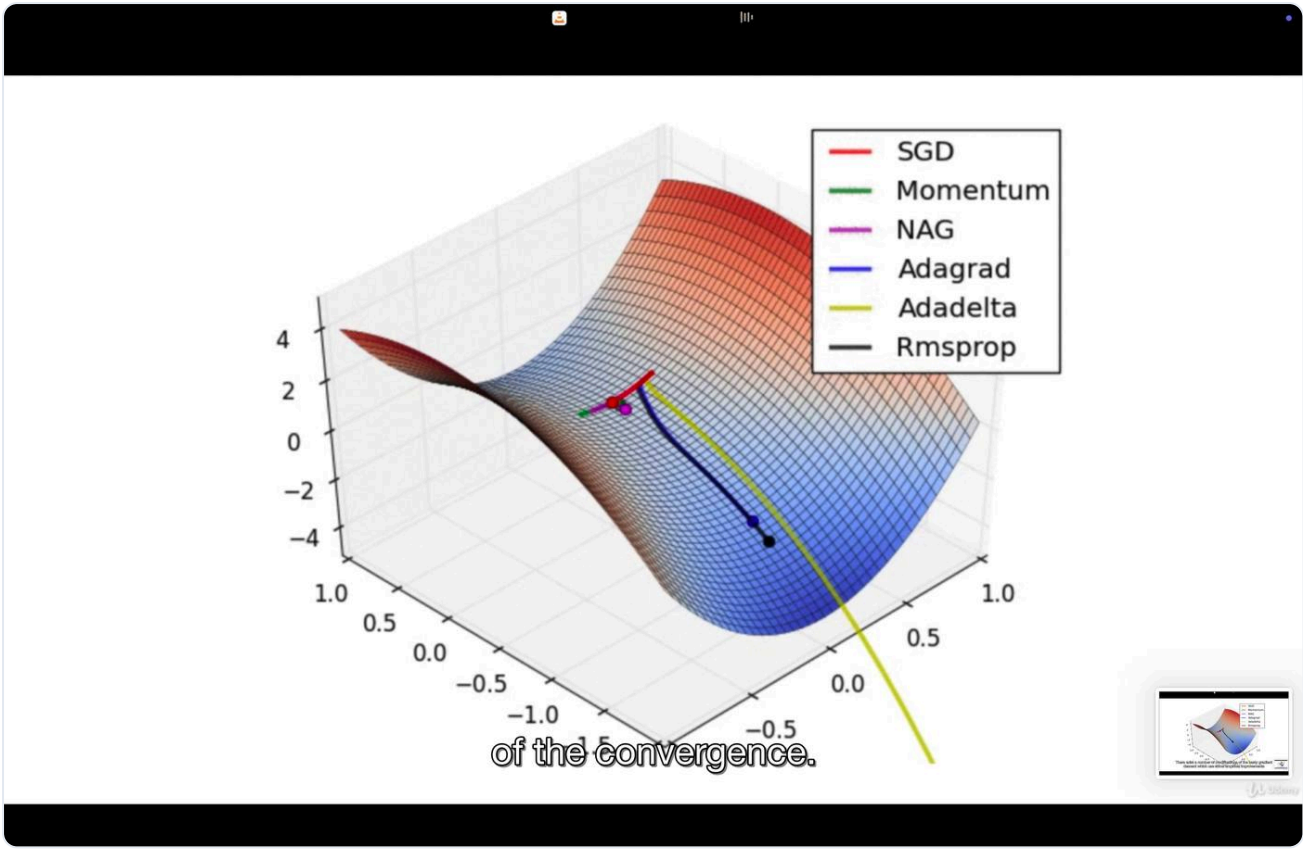


FIGURE 5.17

There exist a number of modifications of the stochastic gradient descent which use some empirical improvements of the convergence.

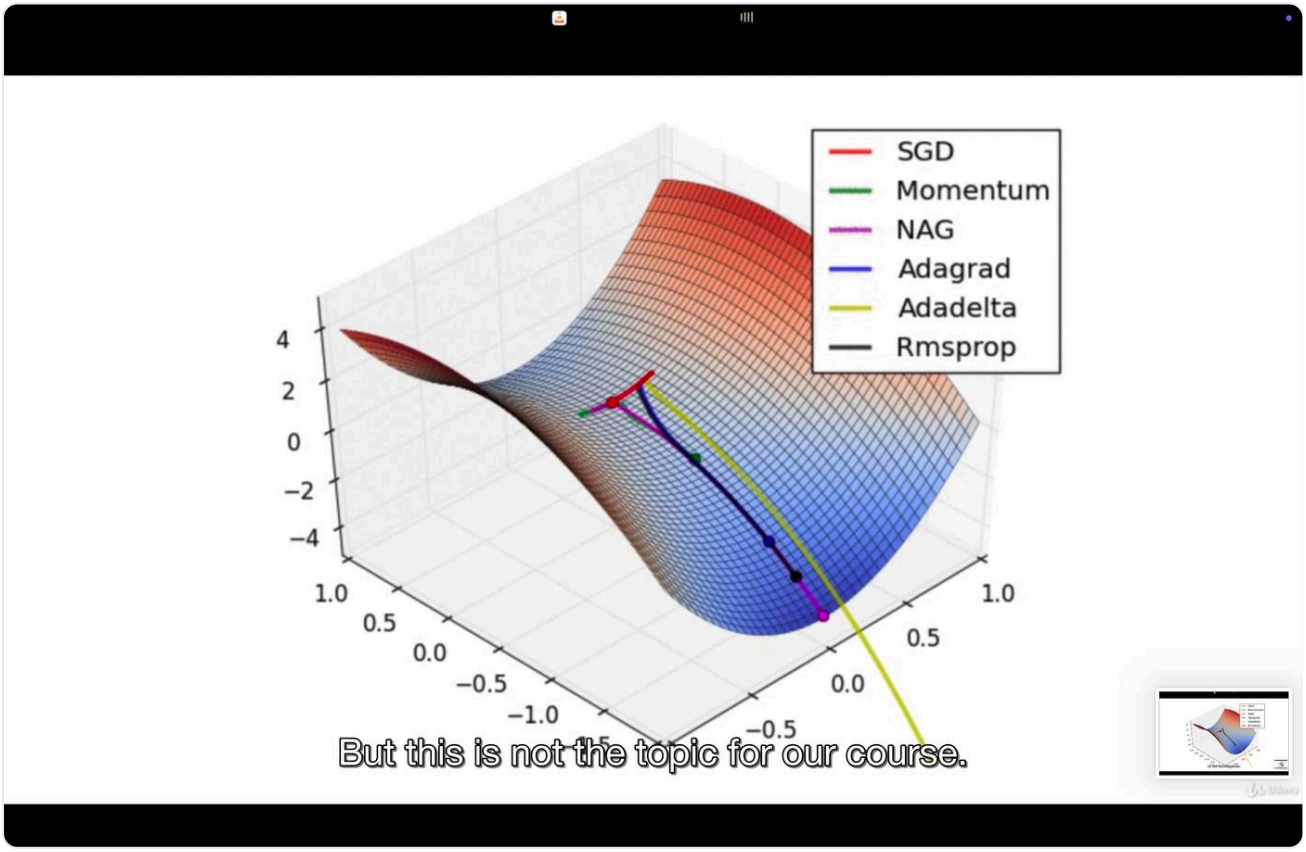


FIGURE 5.18

But this is not the topic for our course.

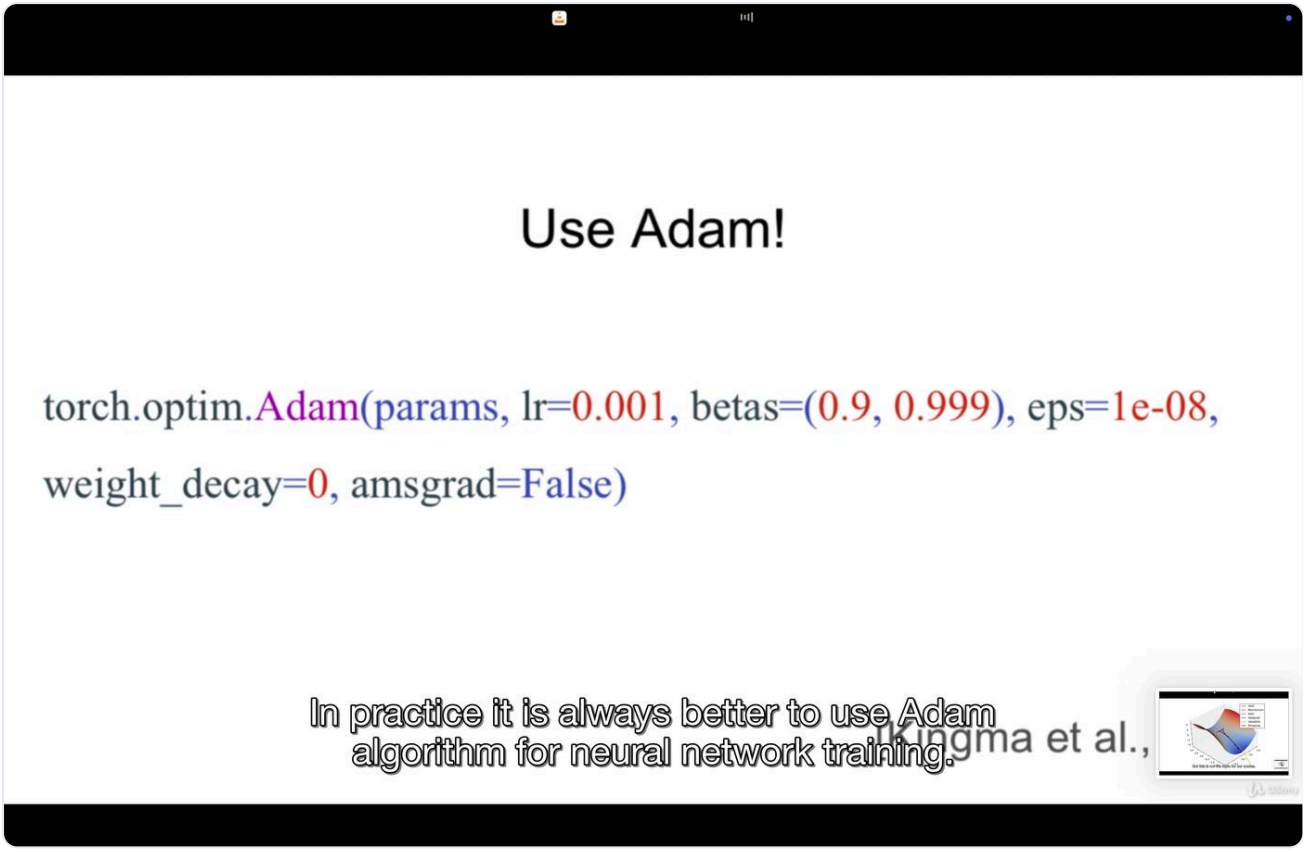


FIGURE 5.19

In practice it is always better to use Adam algorithm for neural network training. Adam is also their modification of this to stochastic gradient descent and is available in PyTorch.

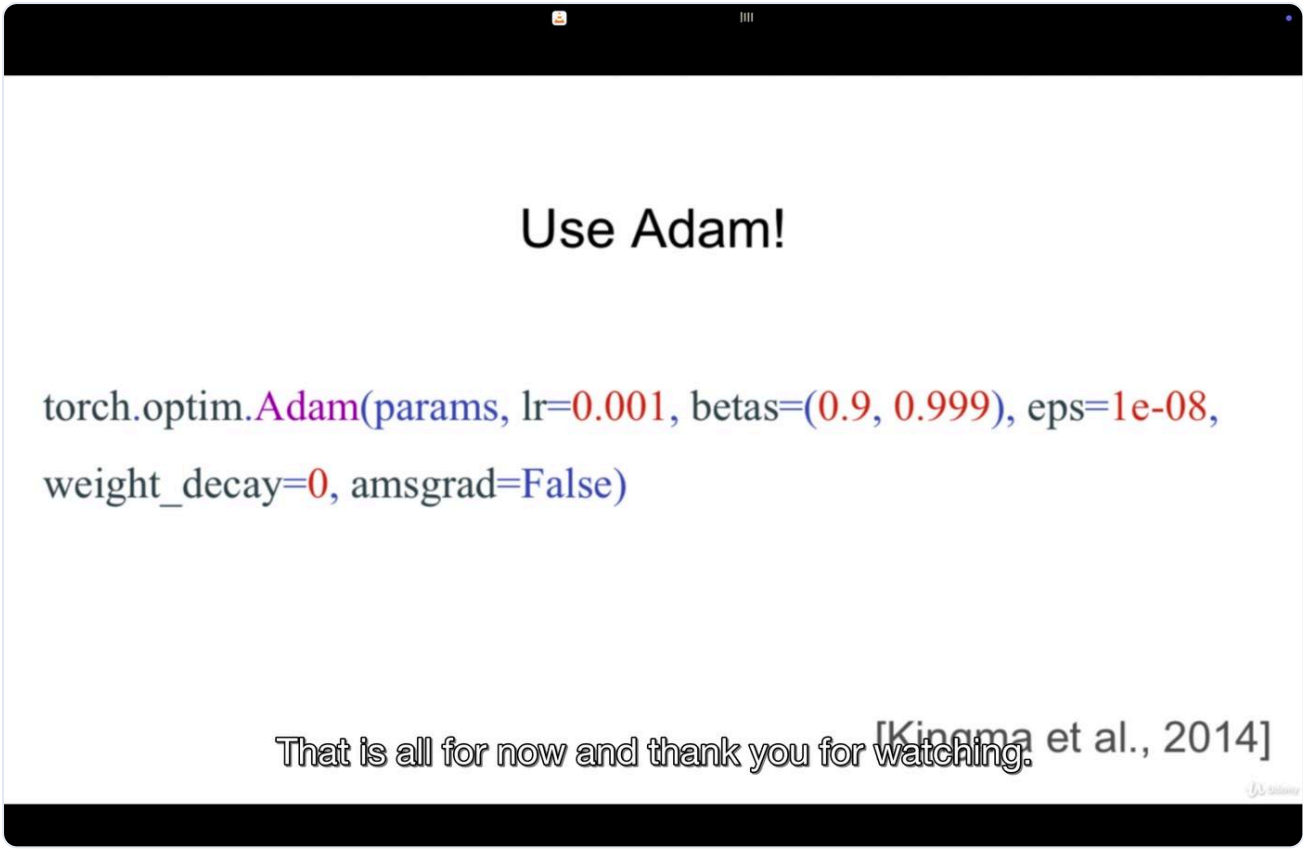


FIGURE 5.20

That is all for now and thank you for watching. See you in the next video.

LECTURE 6

Train Set and Batch Size

See the notebook here

Hello my dear friend. Welcome to the practical part of modern deep convolutional neural networks with PyTorch. In this video we'll build a linear classifier for the dataset which is called CIFAR-10 which consists of 10 classes of images like Plane Car bird cat deer etc.. So first let's look everything we need. This is PyTorch and torchvision and then by libraries in the next cell we have the full in court. First we define that transformation that will apply to all our images what it is. First is transformation from that NumPy array to the tensor this tensor is the basic element of PyTorch library. The same tensor we can see in desert floor or in S.A. or any other library.

Second thing is normalization. If we look at the signature of normalize class from the transforms it says that we will subtract this mean that is zero point five zero point five zero point five and divide by each standard deviation which is again zero point five zero point five zero point five so this is what we will update and scenes our images have will use from 0 to 1. Then after these harmonization our images will be central. By subtracting this mean and we'll have standard deviation equal to 1. So there will be normalized. And this is very good practice before training every neural network after that. After we defined the normalization and the transformation that will apply to all image we should load our dataset for this purpose PyTorch have very good tools which is called dataset and data loader.

So what we do here. First we load the dataset and you see its name its CFR. Then we say that it will be located in the data folder. We say that we want train set not dressed not that set in this section we should download it because we don't have it and we want to apply the transformation that we defined after we defined the train set we should write a law order while there is a class in PyTorch that says what is the what will be the batch size. Do we need to shuffle our data. How many processes we should use for loading these data from the memory and so on. So we will use batch size which is equal to 4 will shuffle all our data and we will have 2 processes to be workers and to load our data from our memory we do the same with train set.

The difference is now we'll have train equal to False so let's apply it. Finally in this cell we define classes that we have plane car birds etc. Totally. There are dance classes. So now you can see that we are waiting on you PyTorch will download all the necessary files and bag them and so on. So as you see files already downloaded anywhere fight good. Now let's normalize these data with it so we get some random train samples which is for because we have batch size 4. As I said before and we use very good tool for making me the images which is called make_grid function in the torchvision library. And here you can see images. They all have size 32 to 32 and have three channels red green and blue.

Okay so you can see plain horse bird and bird. We in fact can repeat dissection and obtain another four images.

LECTURE 7

Linear Classifier and Output Neurons

See the notebook here

Now we have to write some linear classifier in PyTorch for this we need to define a class which is which we'll call linear classifier. Here you can see an end module and we want that it will obtain an input neurones and and return an output neurones a bit later I will describe borders and input neurons and output neurons. So we have to define our linear classifier linear classifier consist of one layer. And this is linear. This is called linear so and then don't linear. And then these torch dot and then and here we see an input neurons and output neurons. How many neurons. I will describe do. I said Okay this is how well meaning earlier we should f we should add self not linear in order to make a goal of these class in forward function.

So now in order to complete this narrow network we should define forward. Plus we don't need to define any backward parsing because because byte arch will automatically do backward pass for us. So in forward bus we do the following. We want to apply self not linear from X and we should return it. Okay Will it work. And I see you know because image X X have have shape batch size 3 hate with. That's why has. That's why we can't apply linear layer because it's an image. And what do we need to do is to shape this X to that of a bunch of vectors and how we can do it. We should define the wearable which size which is called X that shape zero. Then we say that X is equal to X dot view which is equivalent of a shape in num by.

And he will have badge size and minus 1 which will mean that first dimension will be badge size and second dimension will be three. Don't hate dot width and good. This is it. And we defined that when their own network. Now we have to define this class so linear classifier and it takes some input neurons as a parameters and an output neurons and input neurons is equal to some thing and an output neurons is again M equal to something. Okay. What was what should we do here. Here we should say that it will be three dot hate hate hate dot with because here here we multiply we have a product of these shapes here because if minus one this why that dimension of hardware into the input space will be three dot.

Hey Dad. Okay. What is Hayden with. Let's check the size of our image. We have here images. Let's say images that shape and look. It is 4 3 32 32. So this is bench size. This is number of channels and this is hey this week. Okay so we can see that hate. We with 0 2 images that shape from second. Okay. And let's check that it will be 32 to 32. Yes. Yes we are right. Hate and we is equal to 32. And how many neurons we should have in our output. And I say to you that it would be exactly 10 Why. Because we have exactly ten number of classes and when making a four bus we should obtain one score for each class and this score will with soft marks layer is translated to probabilities and again we should have 10 probabilities because of 10 classes.

So this is how a cool of our class not cool. This is the creation of object from our class. And then we apply dot quarter which means that we should transfer our network to GP. Now we have to define their criterion which will be across entropy loss which I mentioned on their lectures and we

use optimizer which is Adam which is now a state of fire. And the best optimizer that was invented and is good for deep neural networks we passed the parameter so of our network and was said to learn and create to some will you which is zero point zero one.

LECTURE 8

Train Loader and Images Images

See the notebook here

Now we have a loop for training. What we have here we have five a box. epoch is that I describe he described you in the lecture. So we want to look at each image from our dataset five times now. At each epoch we use How A Train loader that we defined at the beginning here. Train loader and test loader. And as I said it is very convenient that we can do the following. So we just for save for data in train order and then it is. So each our data is input and labels we translate these inputs. Input images. Sorry. This should be images images images as an input then labels is our data. We translate these images and labels to Jeep you for speeding up training after that in by torture we should make our gradient to zero.

Here we should make for a bus and compute loss. So how to do for us. We just need to apply our network to images and these will be our output. And in order to compute loss function we need to say that loss is equal to criterion from output and labels. And this is it just two lines of code. And after that we make backward. Plus we make optimization step. And this is just debug printing. Let's run this. Okay. Our training now was finished and you can see how the lowest was how the laws decreased from two point three to two point twenty eight. And now let's check their result for these proposals. We will use test loader in the same way that we use train loader in training which is here.

The difference that we don't use any gradients or backward bus and that's why we write torch dot no grid here. What we do is just make for a bus calling our net class. And after that we use Tom's torch dot max function in order to obtain the class with the maximum score and let's compute the accuracy so we have the accuracy is equal to 31 percent. It is not random which is better than random because random classifier will will have 10 percent accuracy because we have 10 classes. This is 31. This is better than random. But again this is just linear classifier. In next video we'll concede there multilayer oppressive tone and then we'll move forward to convolution all neural networks.

Thank you and see you in the next video.

LECTURE 9

Linear Classifier and Linear Layers

See the notebook here

No. Let us modify the linear classifier that we built in the previous several videos into multilayer perception. The difference between that multilayer oppressive Dawn and the linear classifier is that now we have not one in your layer but a set of linear layers and some non-linear narrative between them. So we take the same court for data prep pre processing such as import data PyTorch says that we already downloaded and verified all files. Then we can again with our light our dataset which is remind you is CIFAR-10 which consists of data classes of images of size 32 through 32 which has three channels each. Then now we have a linear classifier that we build on the previous videos.

Now let's bring the meat to their multi-layer perception and let's modify it so as I said we have several linear layers. Okay let's do the following. Let's say that now we'll have what is called Main sequential block sequential is a nice tool in PyTorch that allows us to write several layers with as a sequence. So the first layer will be and then that layer which has an input neurons and lets say two hundred and fifty six then. So continually it will be and then that here and now will have the input neurones 256 and output. Let's say one hundred and twenty eight. The third linear layer will be our last one and we'll have one hundred and twenty eight input neurones and then output.

No not done and output neurones here. Yes we do it d and instead of self doubt linear we just call self good might. And that's all. So no. Please stop the video and think Where is their mistake that I made. I hope that you guessed that as I said in lectures it is very important to insert some nonlinear bridges between linear layers because the combination of linear transformations in the linear space is again a linear transformation. It is the first reason and the second reason we can obtain some bad effects with gradients if we have several linear layers one after another without non-linear narratives. So just let us insert leak here remove here. And as far as I remember we should say that place should be true.

And let's Let's insert the same layer here. Okay. And if you see the parameters here is negative slope which is now a zero point zero one. We can set it to see zero point two zero point two. This is a high parameter and of course we usually should do it you need with some validation set. Okay. So we are done. Then again remind you we obtain height and weight of our image. Then we create not linear classifier but MLP multilayered drawn. Again input neuron sees three dot Hey Dot with an output is done and set it to CUDA would we created our network then a game criterion and optimization algorithm. And again we have 5 a box of training and lets string known network good.

Our train was finished let us obtain test quality using the same function that we used for linear classifier so and now we have 45 percent of accuracy in linear classification we obtained 32 percent of accuracy. Now we have four to five when that random classification will give us 10 per-

cent accuracy. Okay that's not very good result of course because we don't use convolution layers at all and we don't tune the network would just take some random layers but we again improve the results in the next section we'll consider convolution layers and and we'll significantly improve their results so continue watching. Thank you. And seen in the next video.

LECTURE 10

Deep Learning and Descriptors

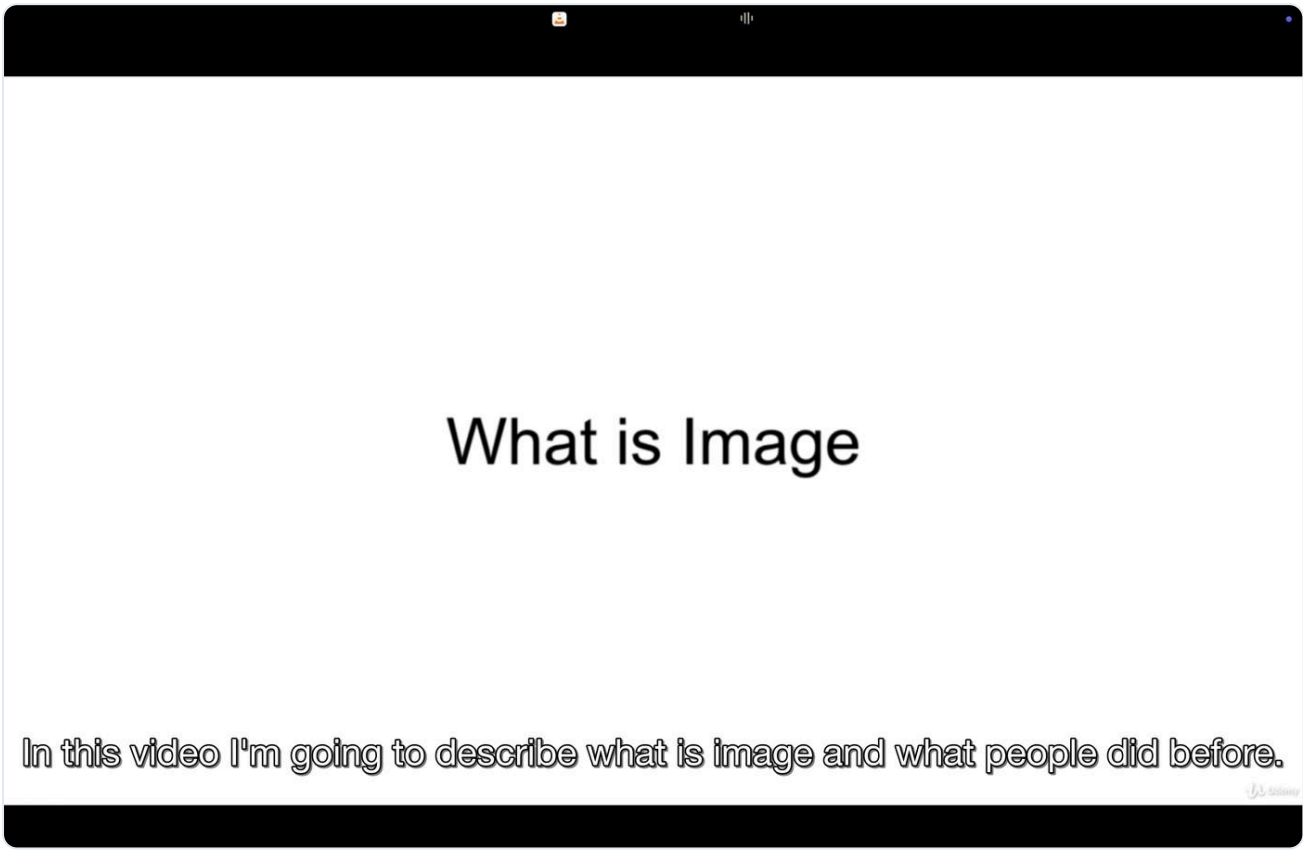


FIGURE 10.1

In this video I'm going to describe what is image and what people did before. Deep Learning usually Gray. Image is represented as a magics of size. Hey.

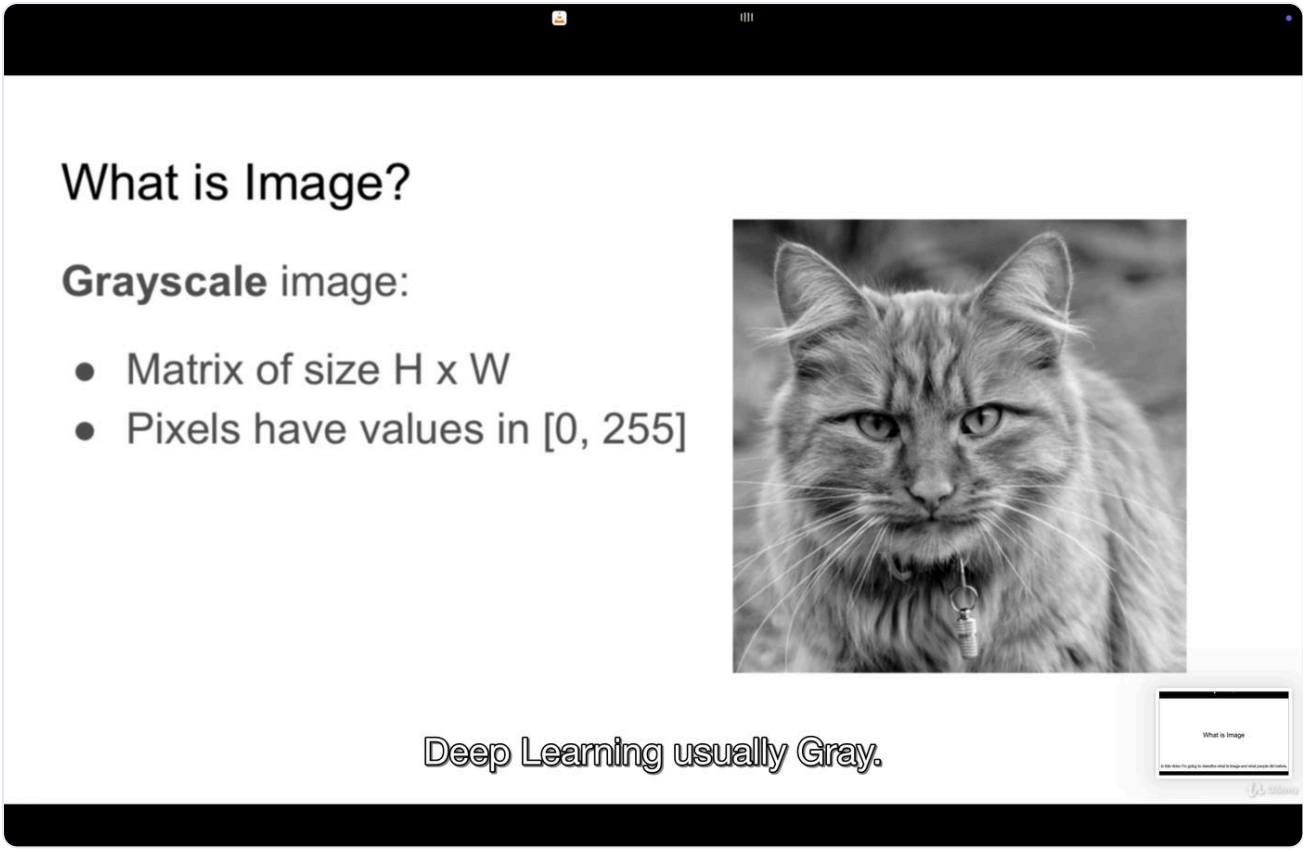


FIGURE 10.2 | What Is Image?

Too weak in this magics there are pixel values which could take well from the range from zero to two hundred and fifty five. Another way to represent an image is matrix of pixels. That takes well use between 0 and 1 if we are talking about color images.

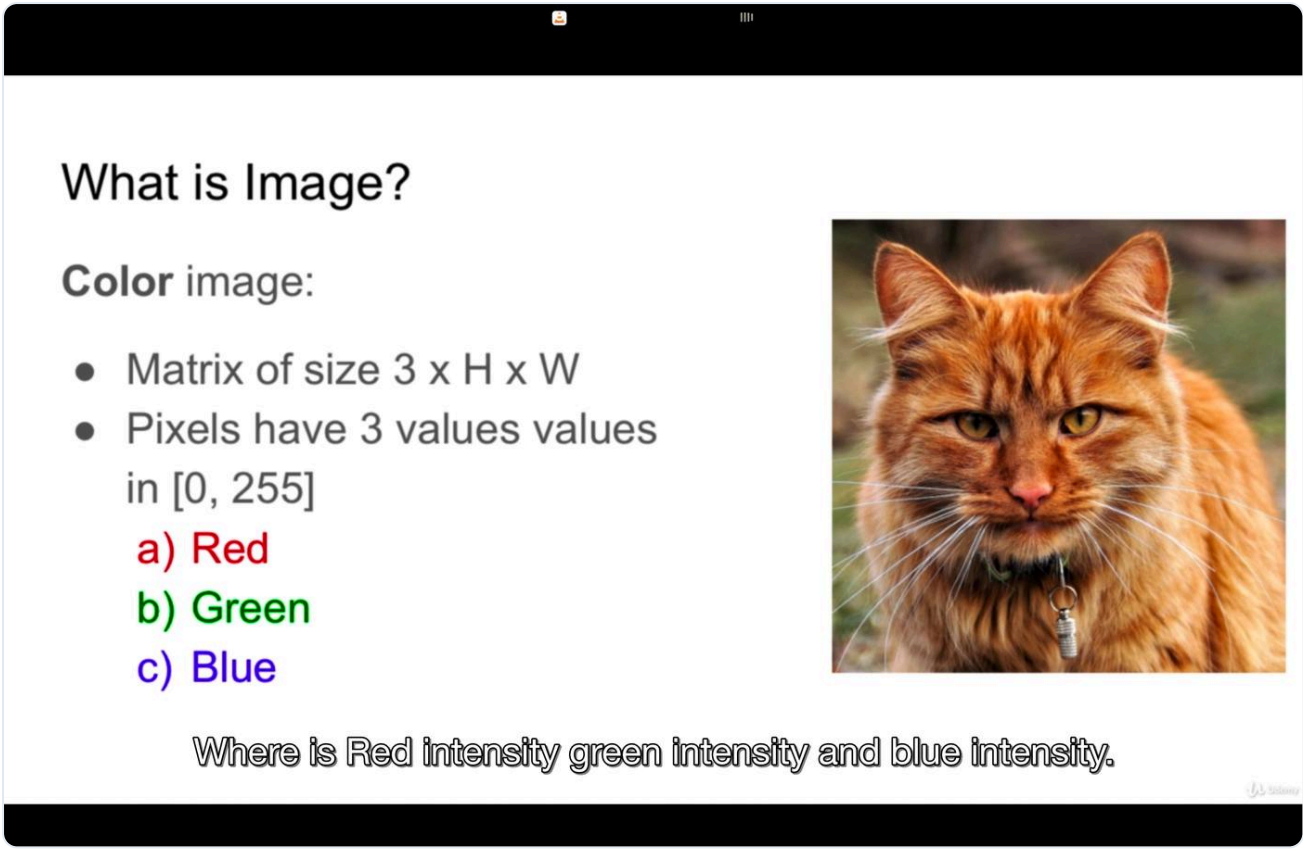


FIGURE 10.3 | What Is Image?

Usually it is represented as a 3D matrix of size 3 to hide 2 with each pixel now is represented as 3 Where is Red intensity green intensity and blue intensity. These these so-called channels on the image. Let us consider old school approach for image processing. We as always start from data. Usually this is a set of images for each image people constructed a set of features by hand using obtained features.

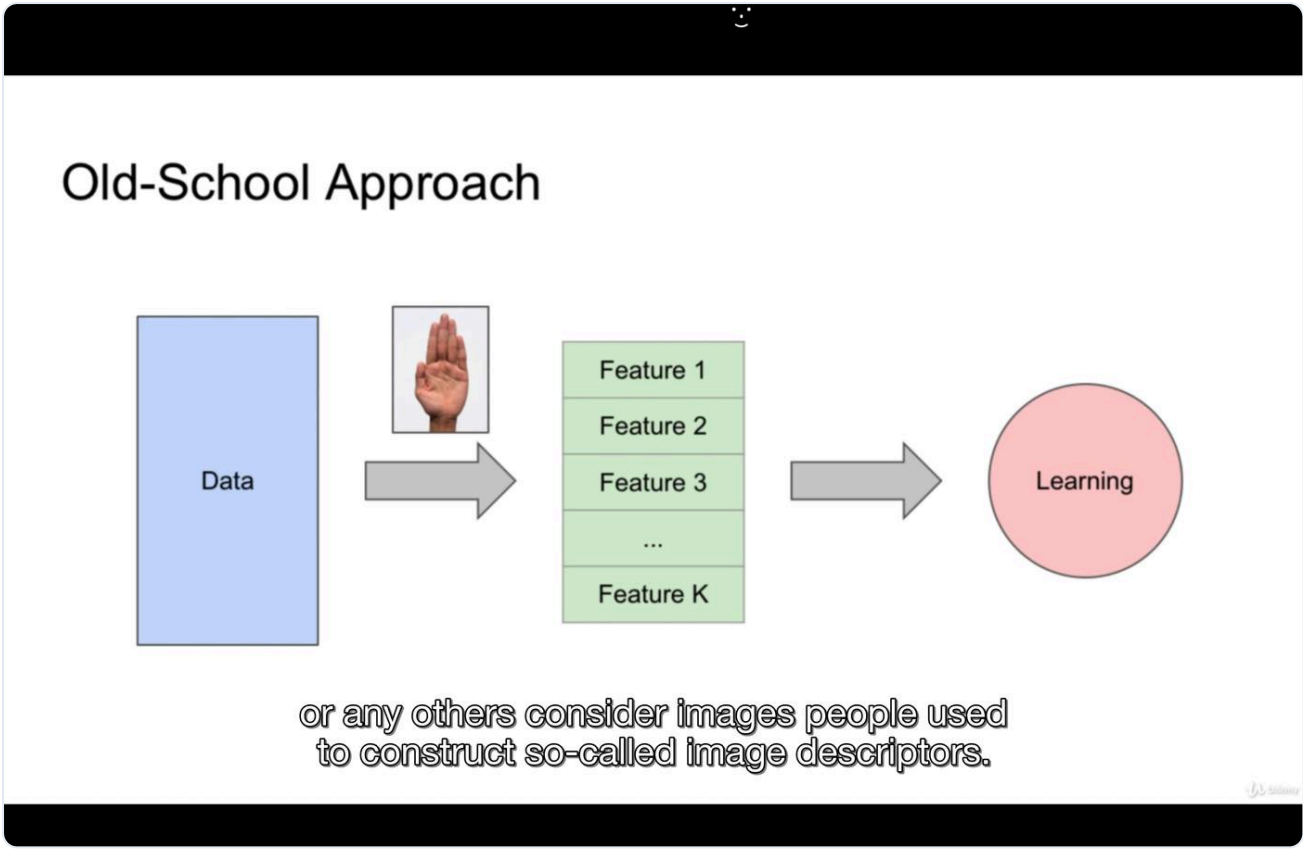


FIGURE 10.4

One can use any standard machine learning algorithm such as linear regression SVM gradient boosting or any others consider images people used to construct so-called image descriptors. These descriptors were not learnable at all. As you can guess the choice of their descriptors influenced training and prediction results.

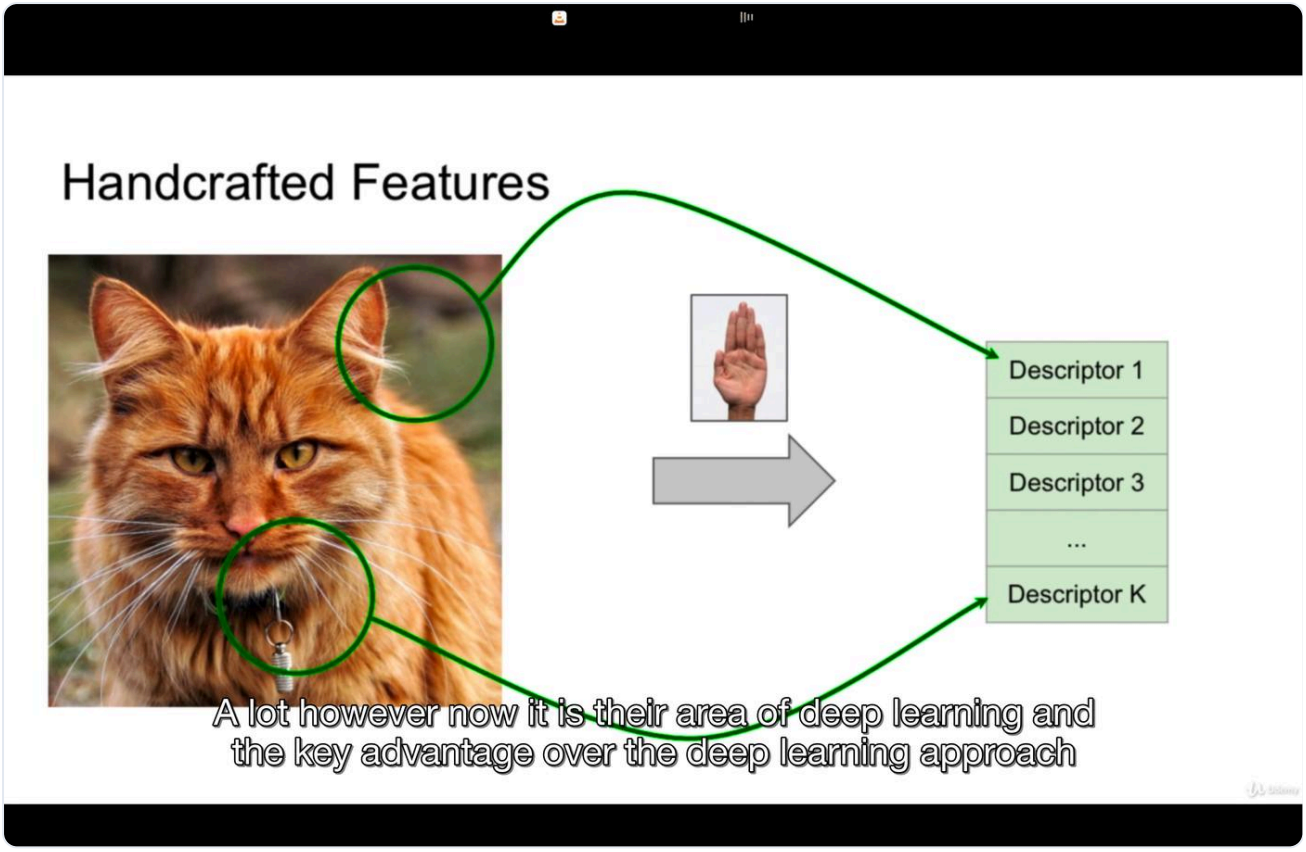


FIGURE 10.5

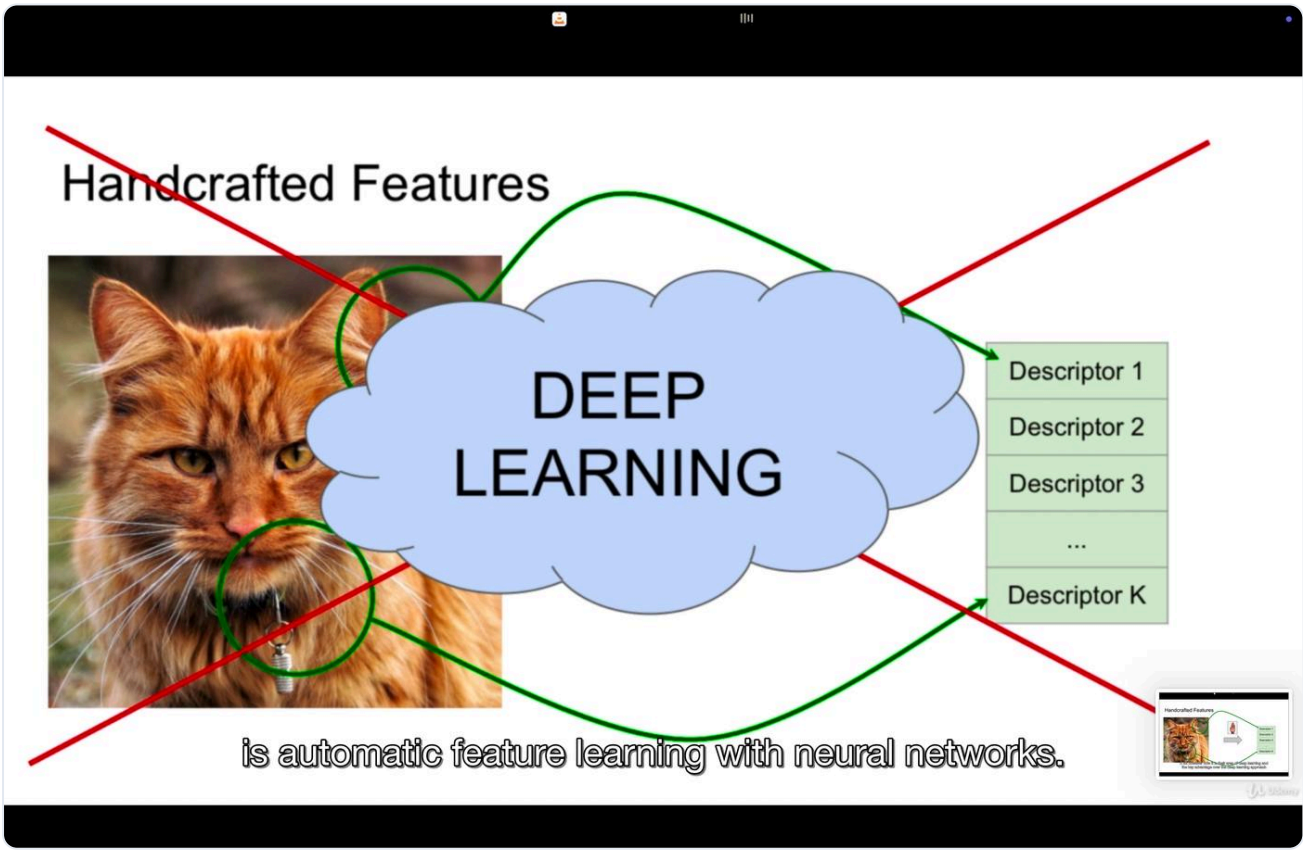


FIGURE 10.6

A lot however now it is their area of deep learning and the key advantage over the deep learning approach is automatic feature learning with neural networks. You don't need to think about features and descriptors you construct a neural network and it learns all features for you thank you for watching you in the next video.

LECTURE 11

Linear Layer: Why Not for Images?

In this course we study conventional neural networks.

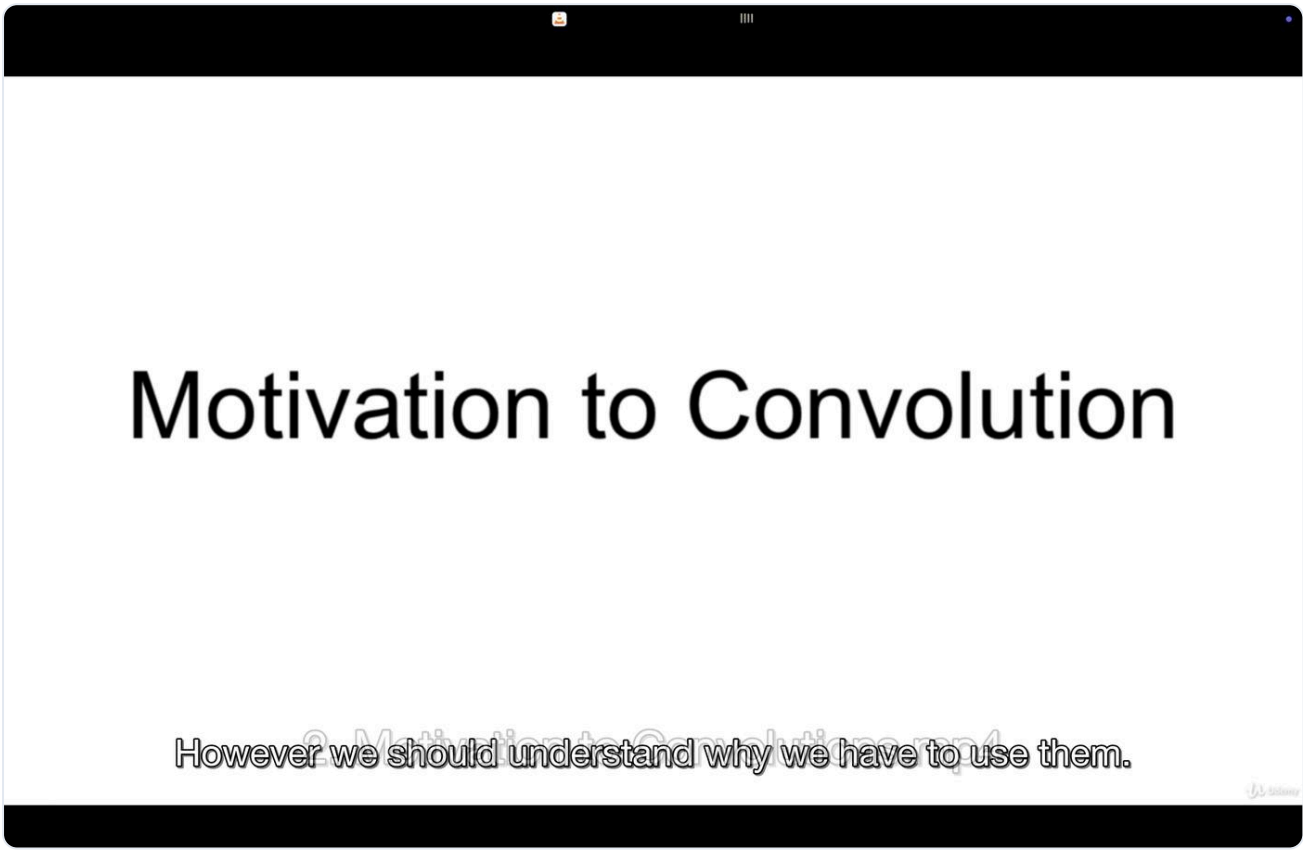


FIGURE 11.1

However we should understand why we have to use them.

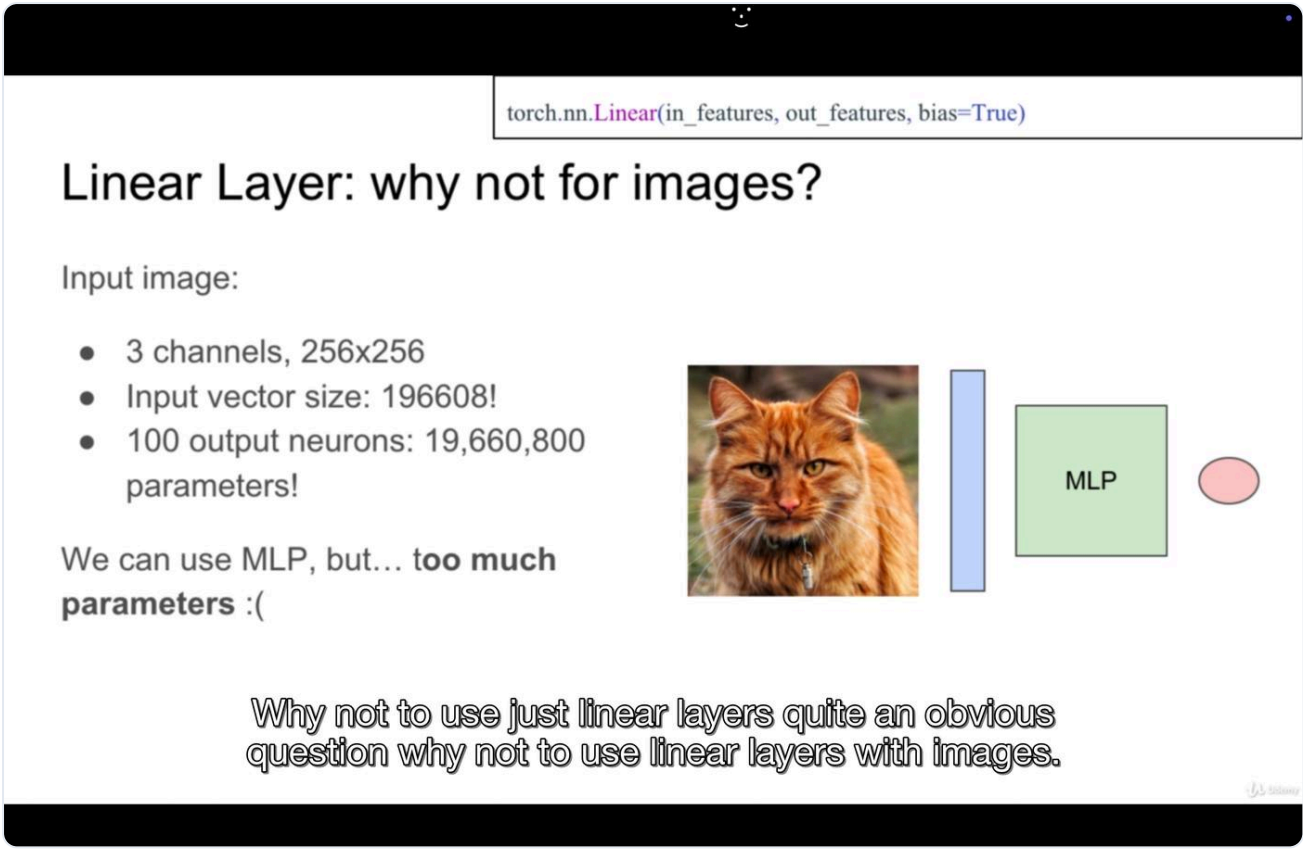


FIGURE 11.2 | Linear Layer: Why Not for Images?

Why not to use just linear layers quite an obvious question why not to use linear layers with images. After all we can just reshape our image to vector and do a forward pass through a set of linear layers with nonlinear narratives. Good idea but if our input image has three channels and size two hundred and fifty six to two hundred and fifty six after reshaping we will have a vector of size around two hundred thousand that's the main situation for introducing convolutional neural networks to match parameters in the linear layer before we start learning convolutions. Let us consider some assumptions.

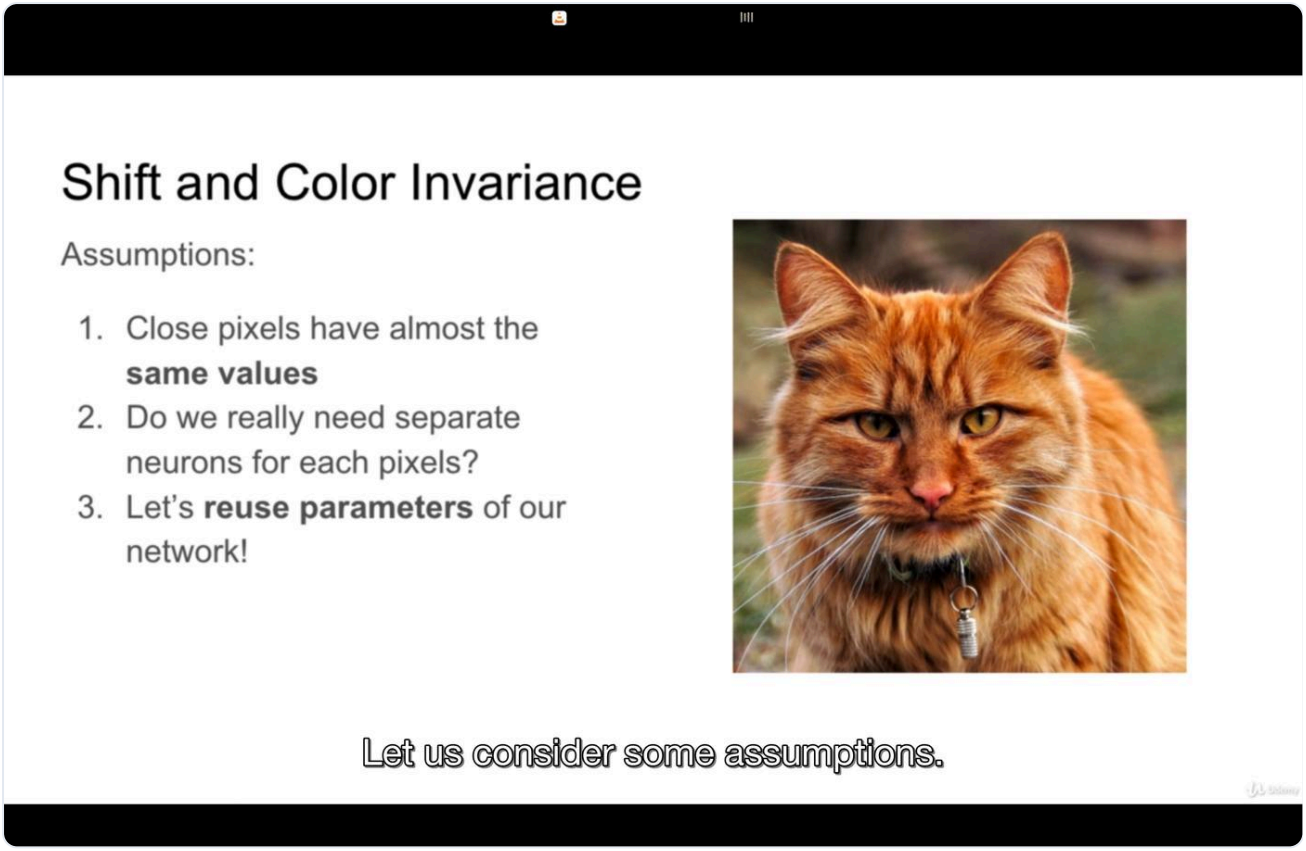


FIGURE 11.3

Could you agree with me that the globe's peak sales have goals values. Actually if we consider the big sale on the CAD which is read with high probability close peak sales will be also red. Okay so we don't need a separate neuron for each pixel.

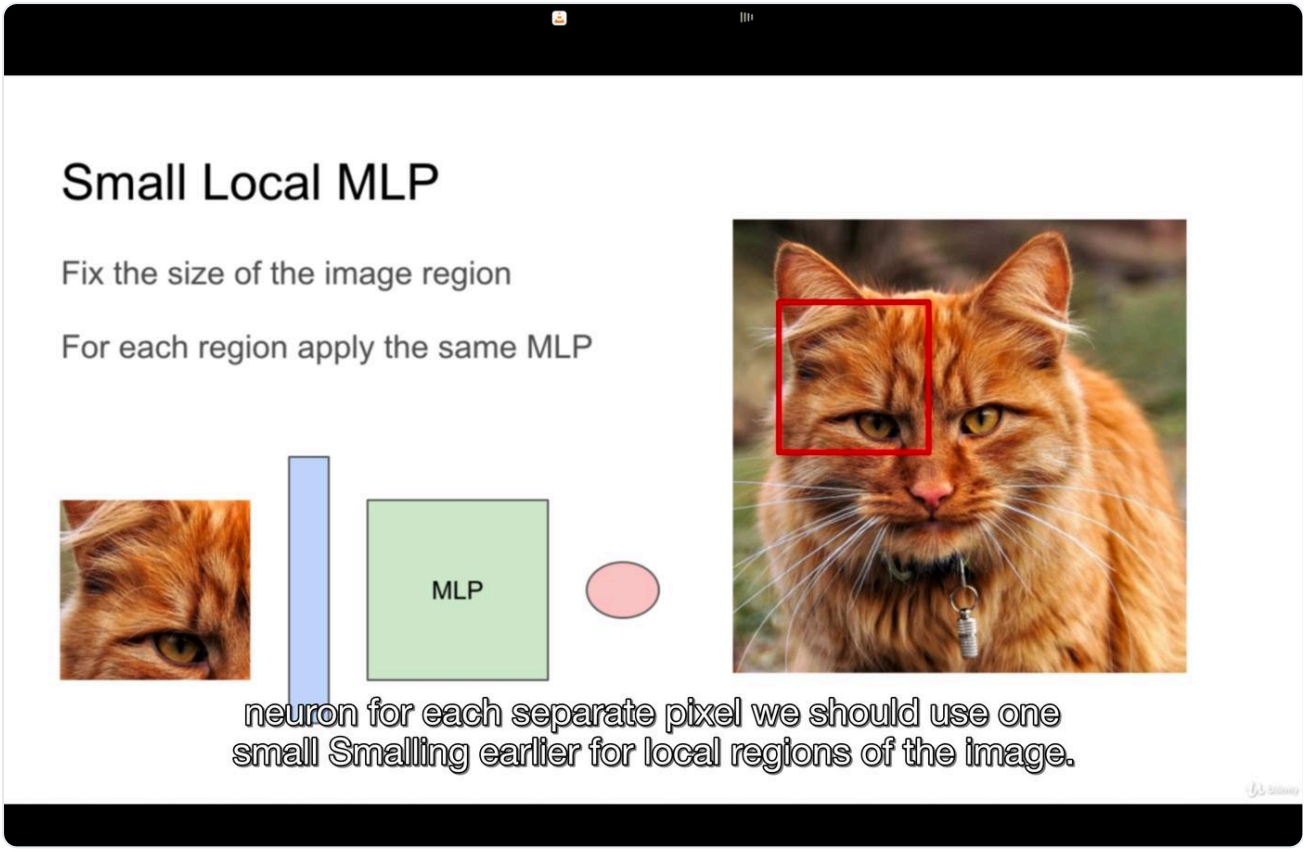


FIGURE 11.4 | Small Local MLP

The open question for now how we can reuse the parameters of our network if we don't need a separate neuron for each separate pixel we should use one small Smalling earlier for local regions of the image. And this is a basic idea behind convolution.

LECTURE 12

One Value

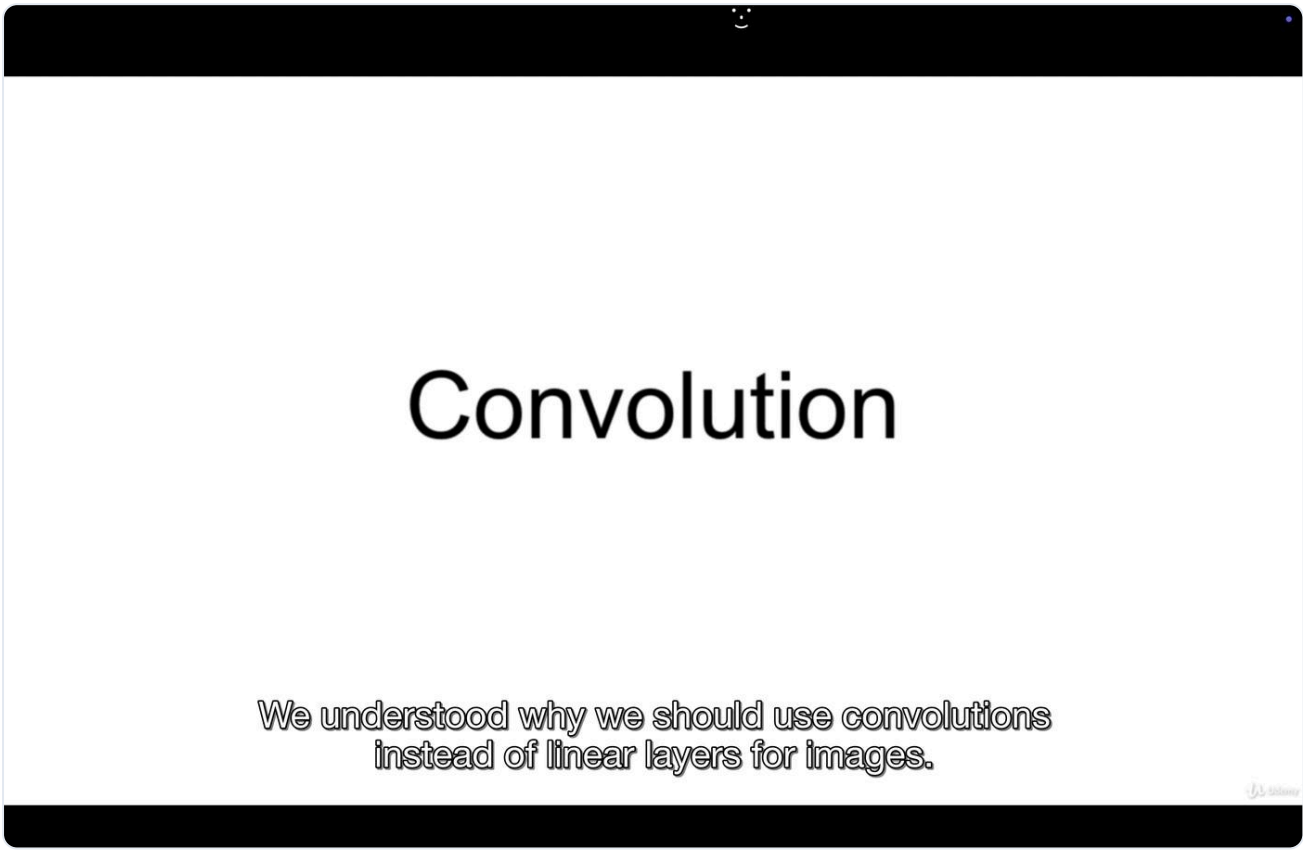


FIGURE 12.1

We understood why we should use convolutions instead of linear layers for images. Let us now consider a convolution in detail let us look at the visual explanation of the convolution.

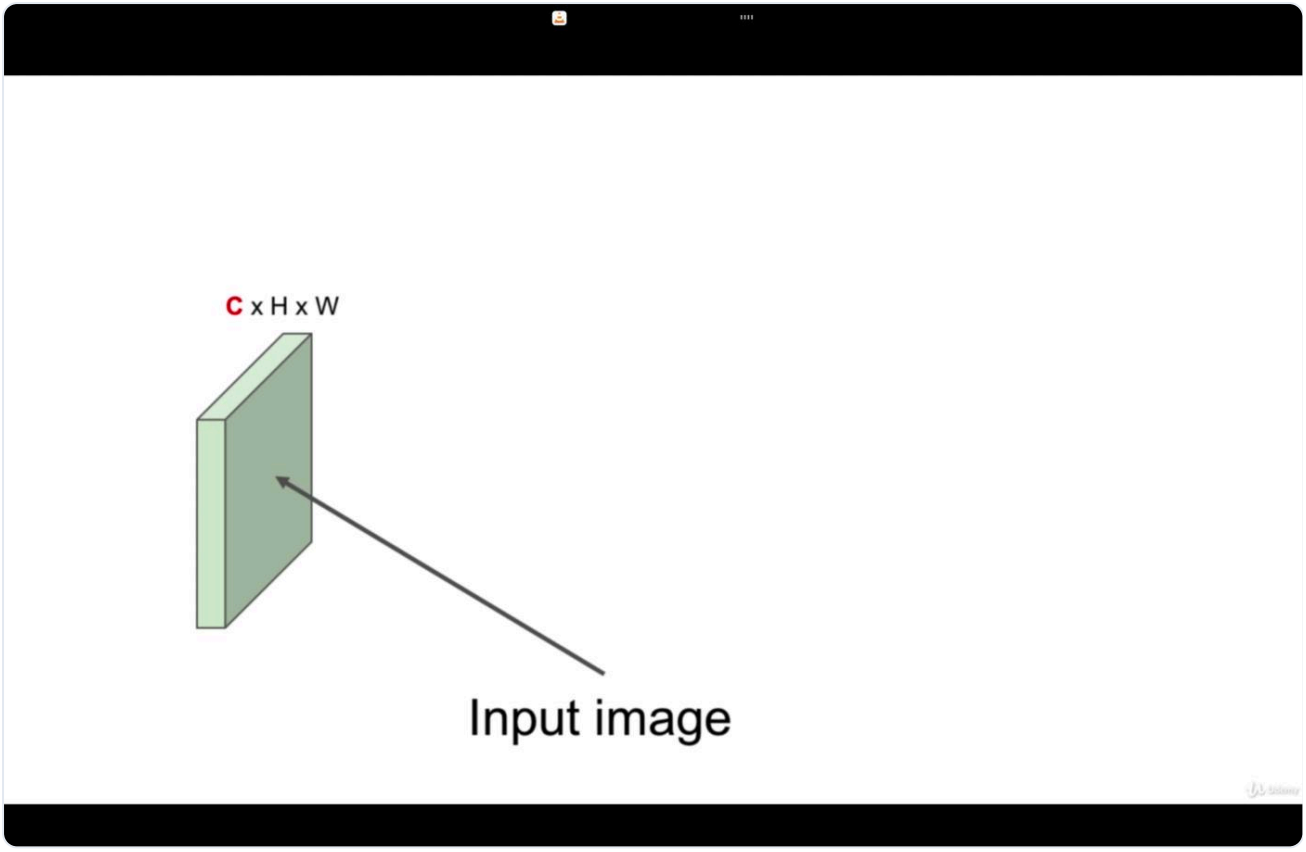


FIGURE 12.2

Again we have an input image of size C x H x W, where C is the number of channels and H and W are height and width, respectively we introduce a convolutional weight tensor of size C x K x K.

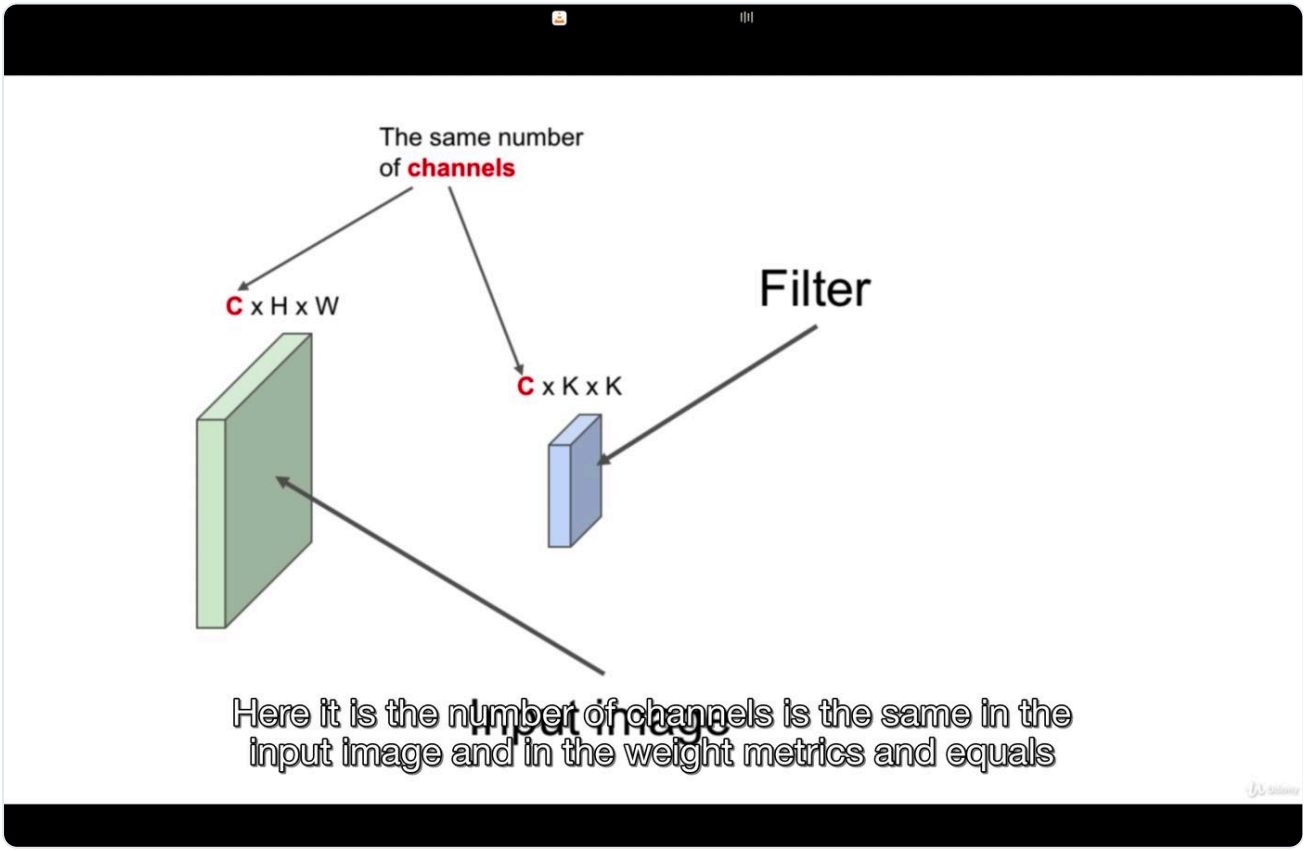


FIGURE 12.3

Here it is the number of channels is the same in the input image and in the weight tensor, and equals C. We slide the convolutional kernel over the image and take the element-wise product of the local region of the image which is green and the weight tensor which is blue.

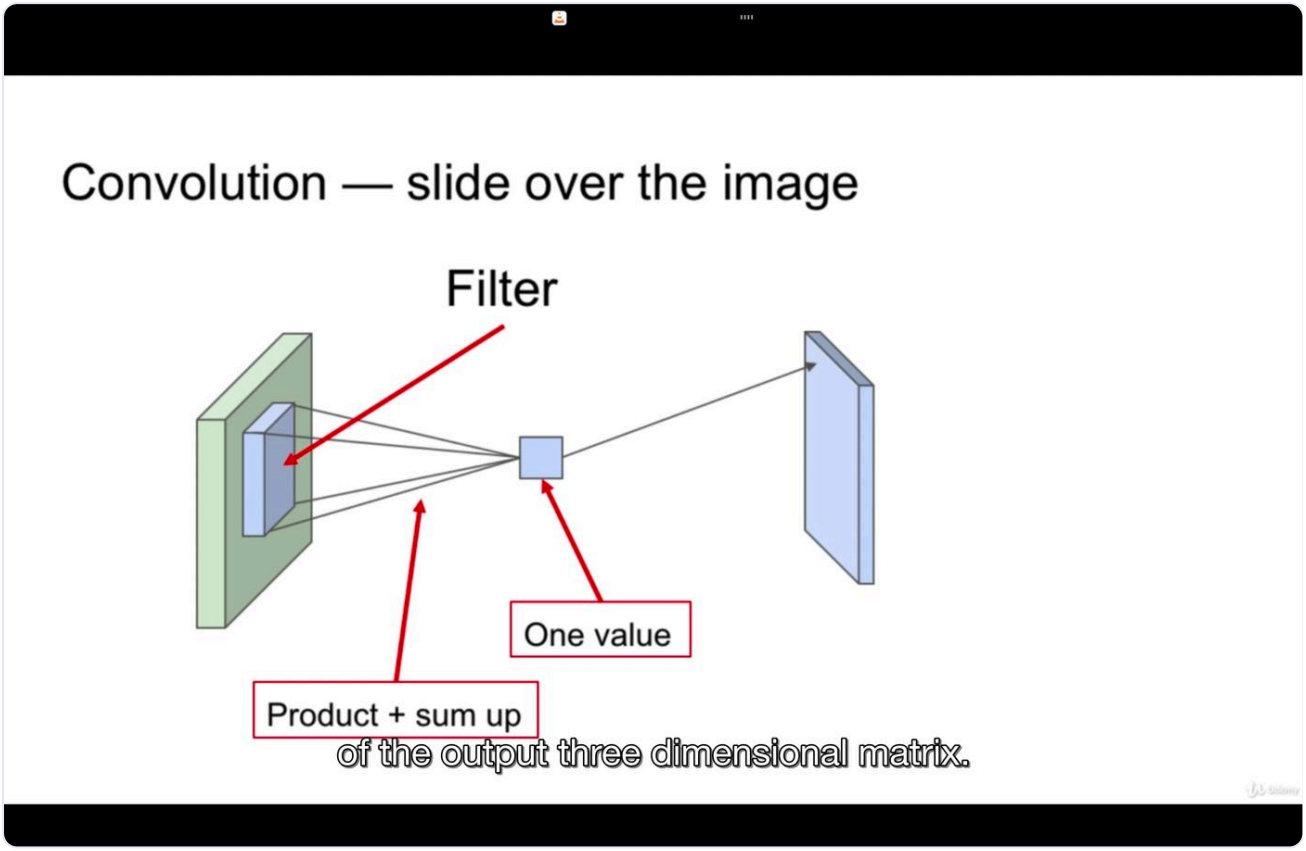


FIGURE 12.4 | One Value

We sum up the result of the product and obtain one value, which we write to one of the locations in the output 3D tensor.

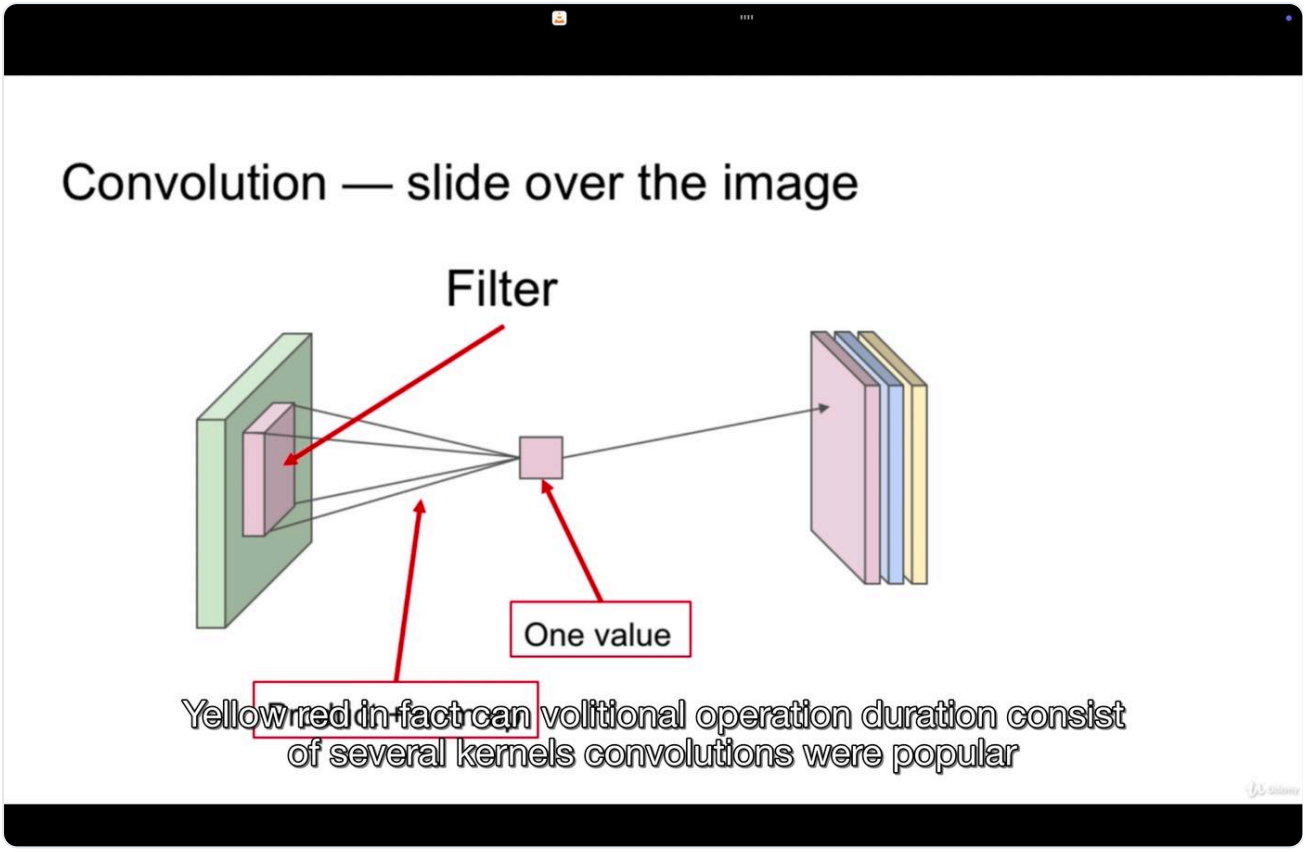


FIGURE 12.5

We repeat these actions several times with different filters. Yellow and red show different filters.

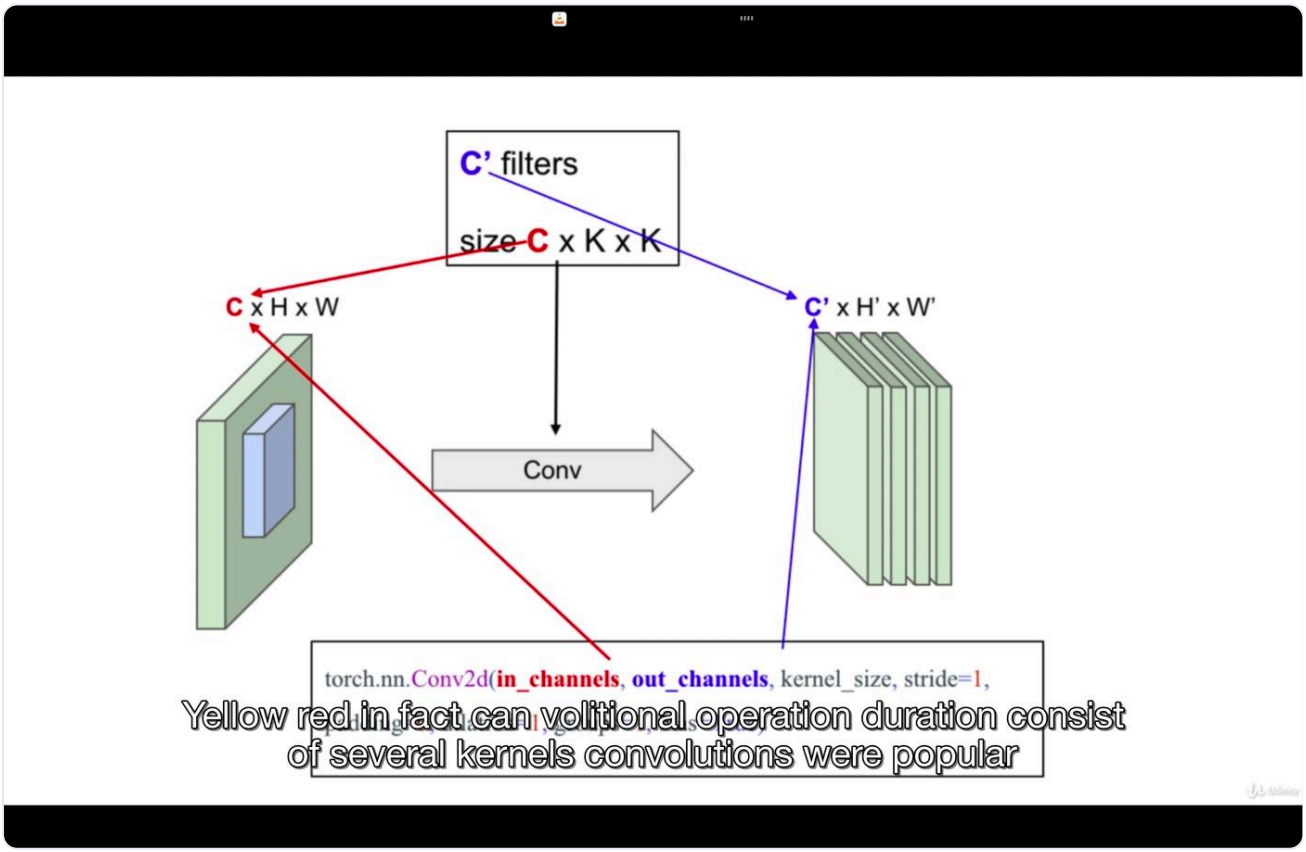


FIGURE 12.6

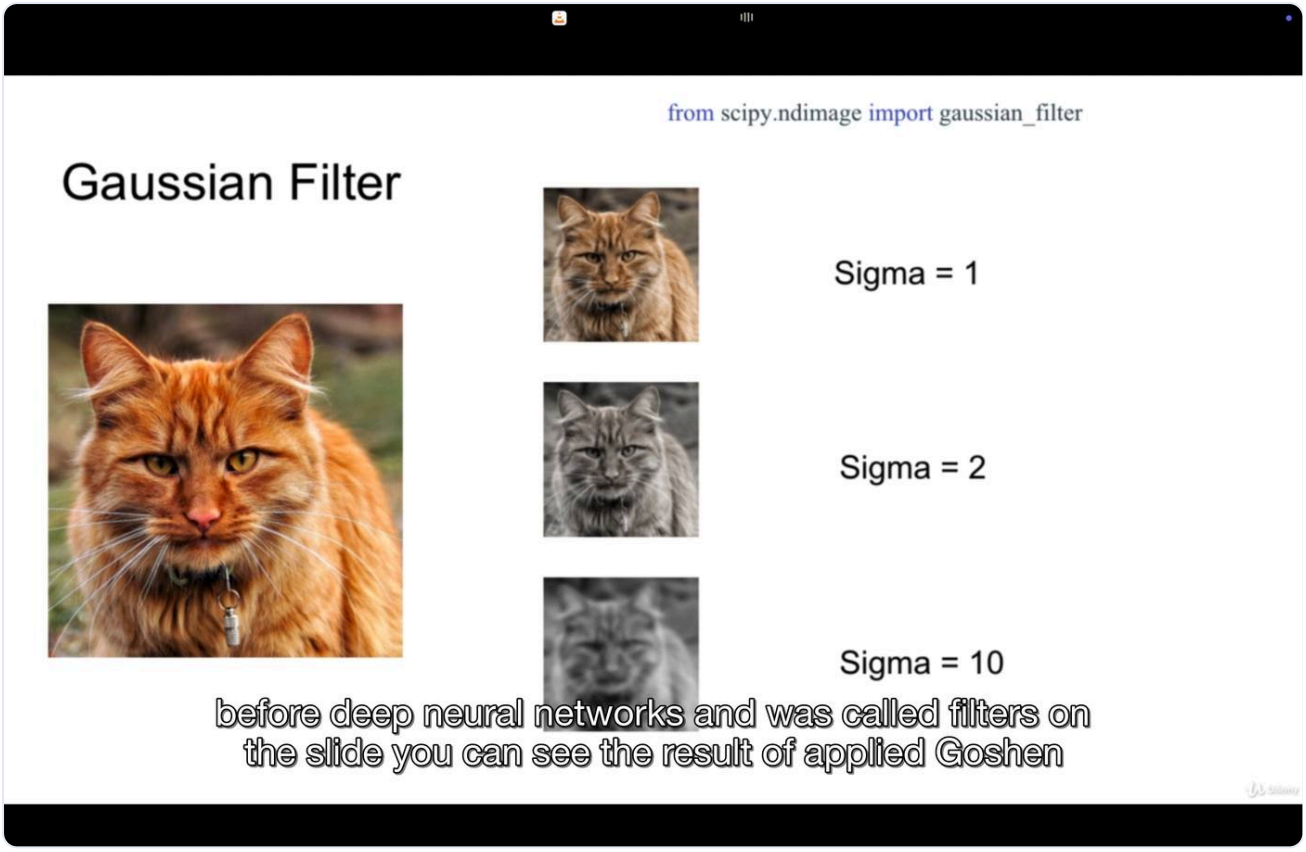


FIGURE 12.7 | Gaussian Filter

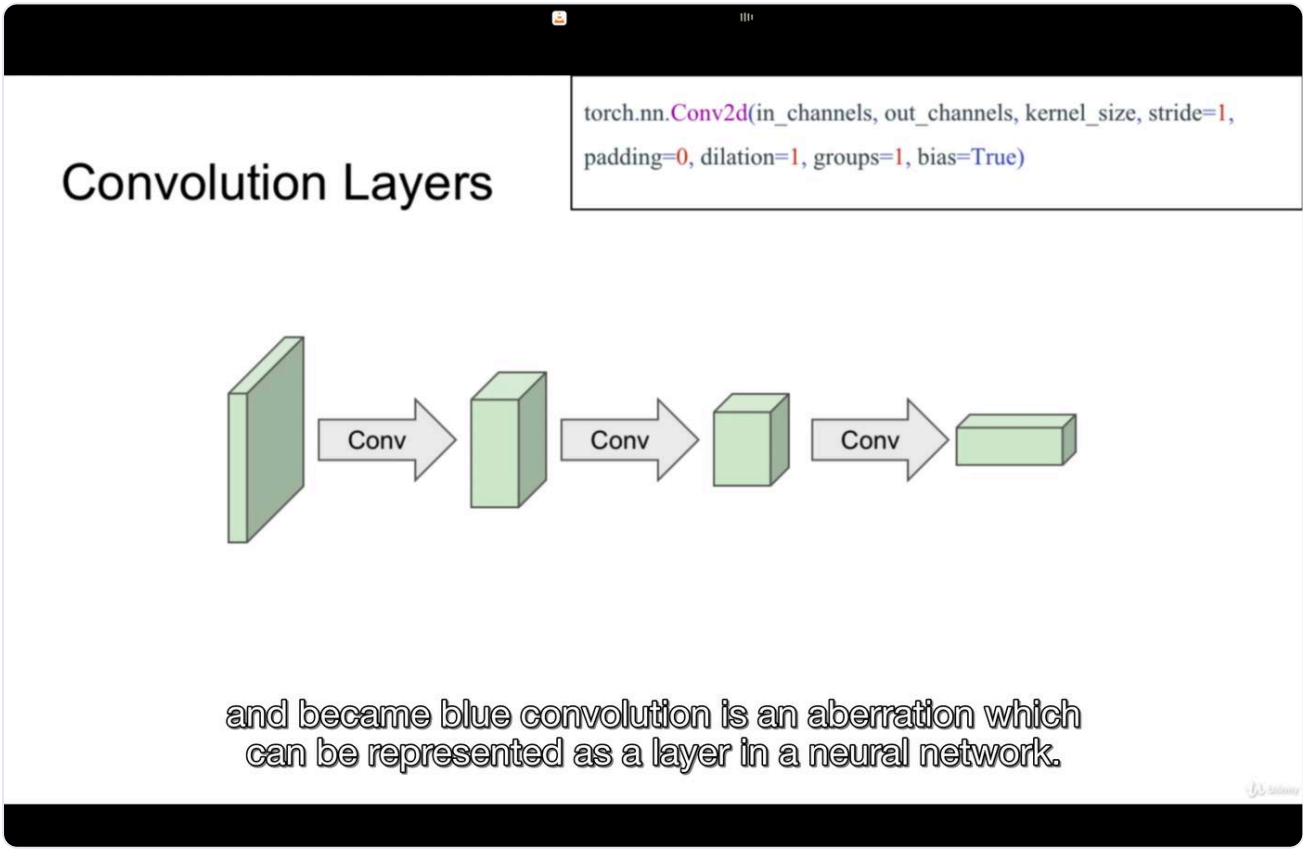


FIGURE 12.8 | Convolution Layers

In fact, a convolution operation can consist of several kernels convolution kernels were used before deep neural networks and were called filters on the slide you can see the result of applied Gaussian filter to an image you can see that depending on the parameter of this filter the image loses its color and becomes blurred convolution is an operation which can be represented as a layer in a neural network.

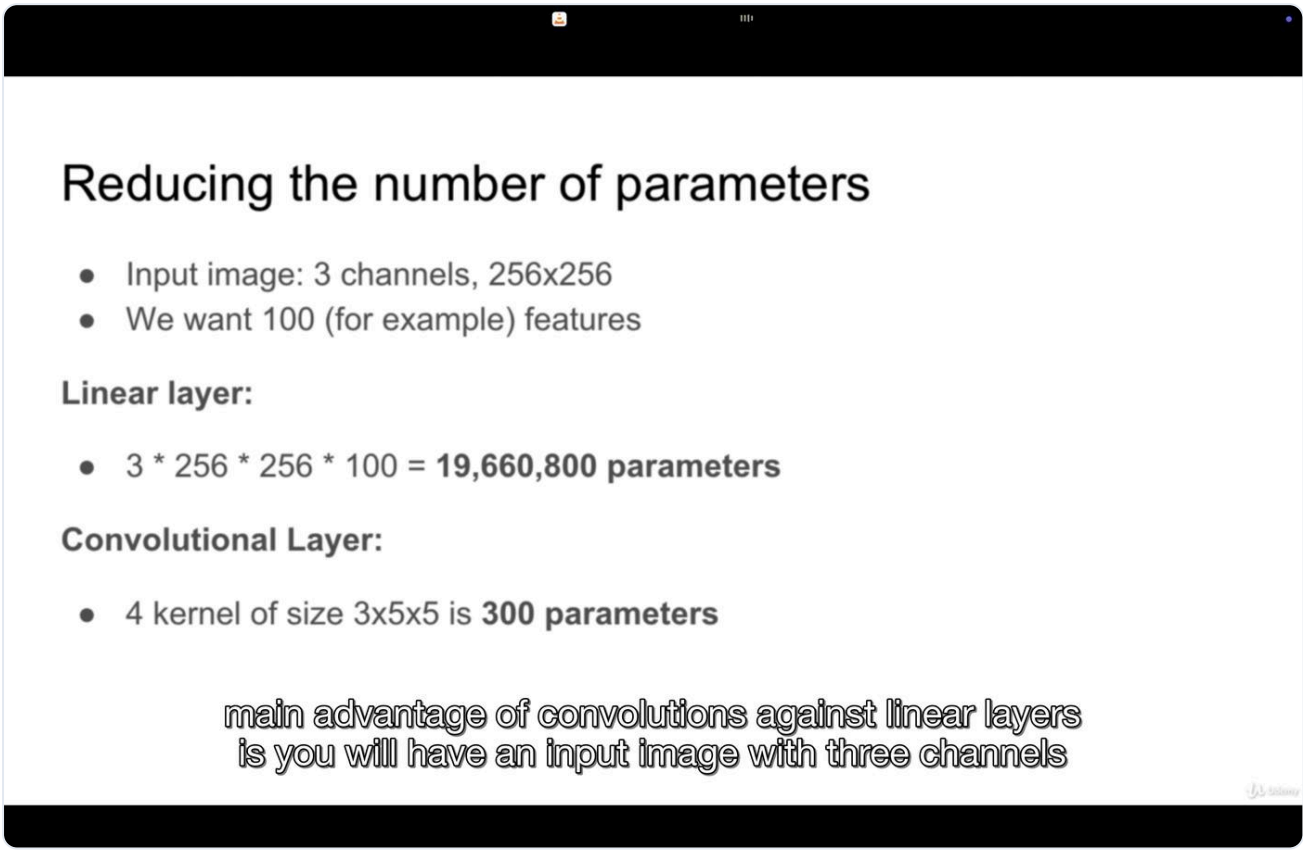


FIGURE 12.9 | Reducing the Number of Parameters

That is why you can combine several convolution layers into one neural network let me demonstrate the main advantage of convolutions over linear layers is you will have an input image with three channels of size 256 to 256 we want to obtain 100 output features after the first layer then using linear layer we will obtain 19 million parameters at the same time the convolutional layer would have only 300 parameters. Now you see the difference in the next video we'll go deeper into convolutions. Thank you for watching.

LECTURE 13

Zero Padding

Convolution operation has several parameters.

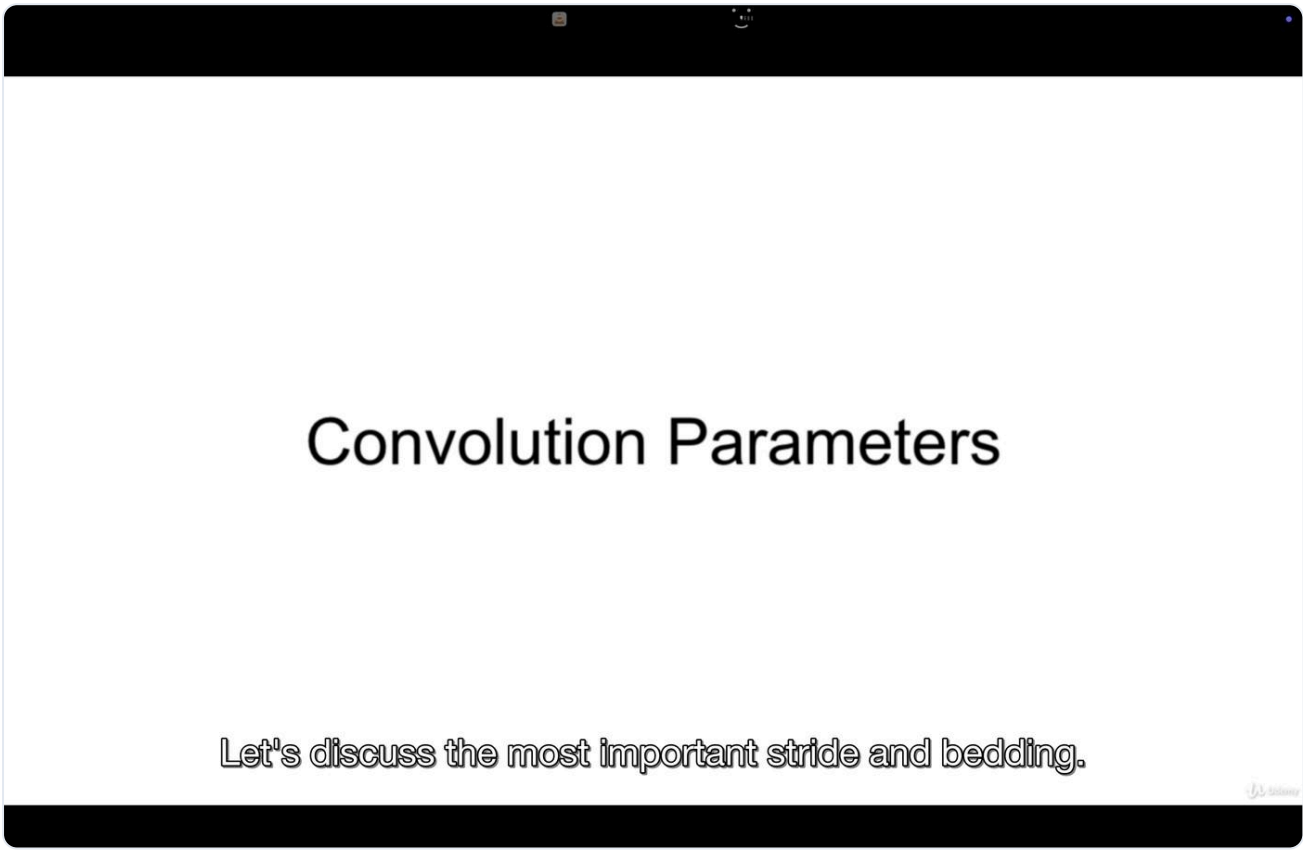


FIGURE 13.1 | Convolution Parameters

Let's discuss the most important stride and padding. The first parameter of the convolution is its stride.

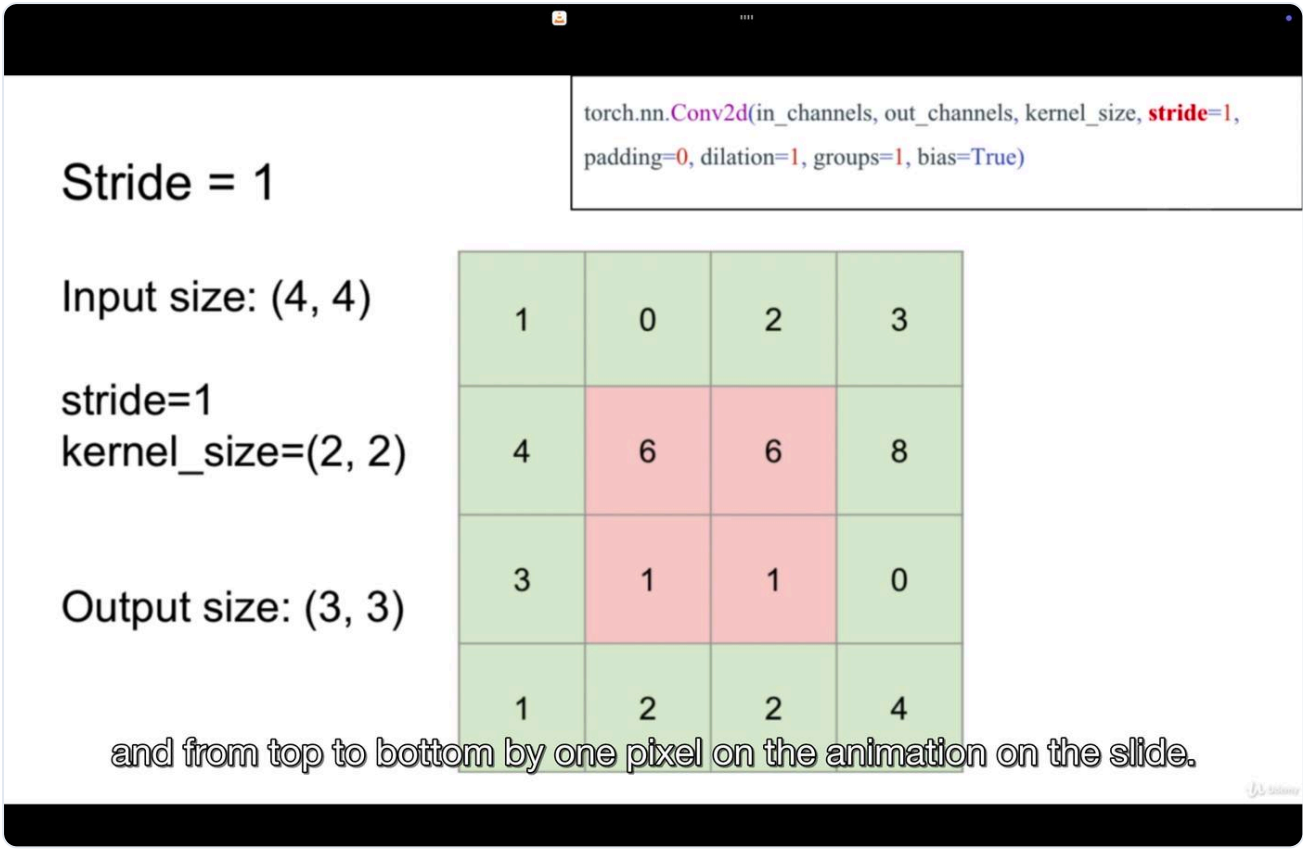


FIGURE 13.2

The default will you of the stride is one which means that our kernel is moving from left to right and from top to bottom by one pixel on the animation on the slide. You can see how their convolution with stride of one works in the animation. The red is a kernel the green is the input image. Know that it will have input size four to four kernel size two to two. Then the output size will be three to three.

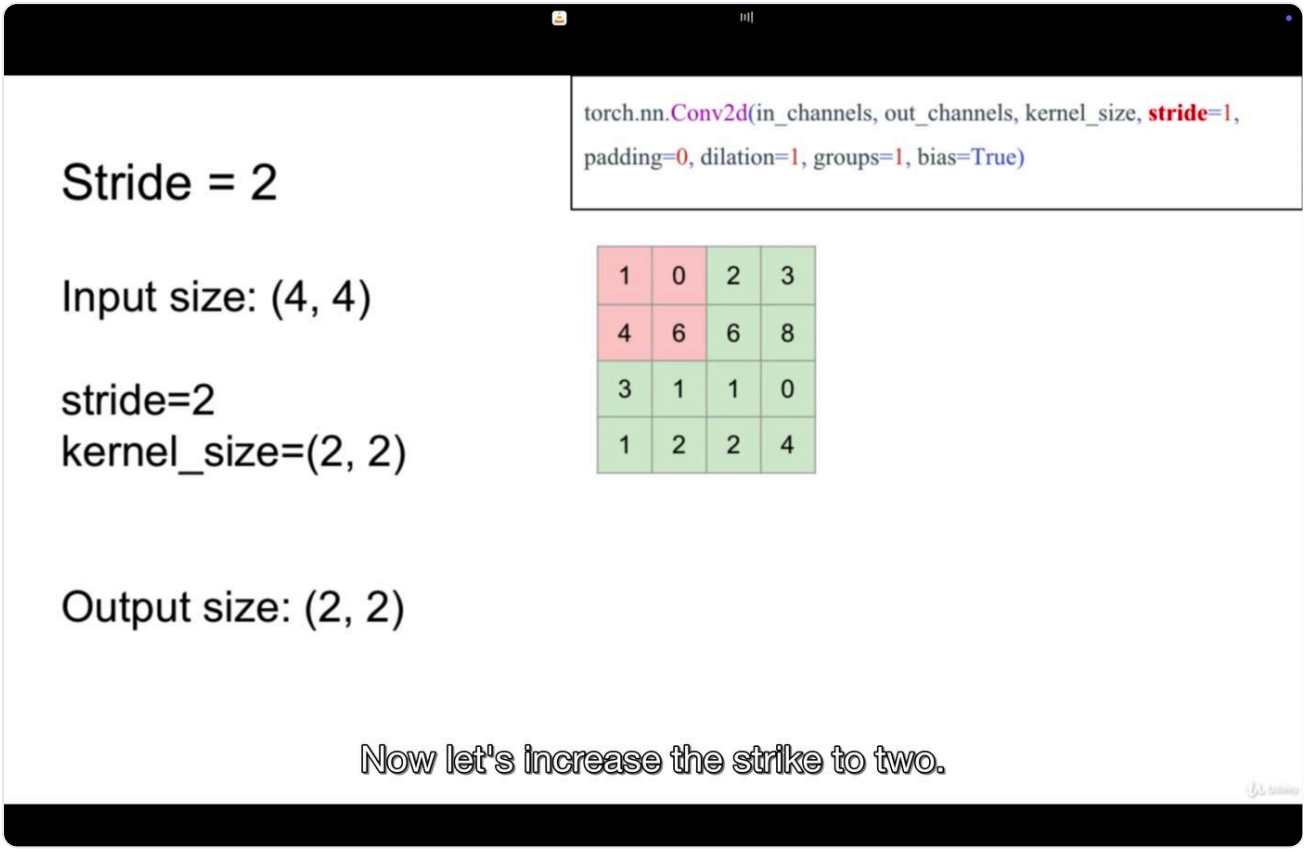


FIGURE 13.3

Now let's increase the strike to two. Look what now happens. We increase this tried and the size of the output of the convolution decreased.

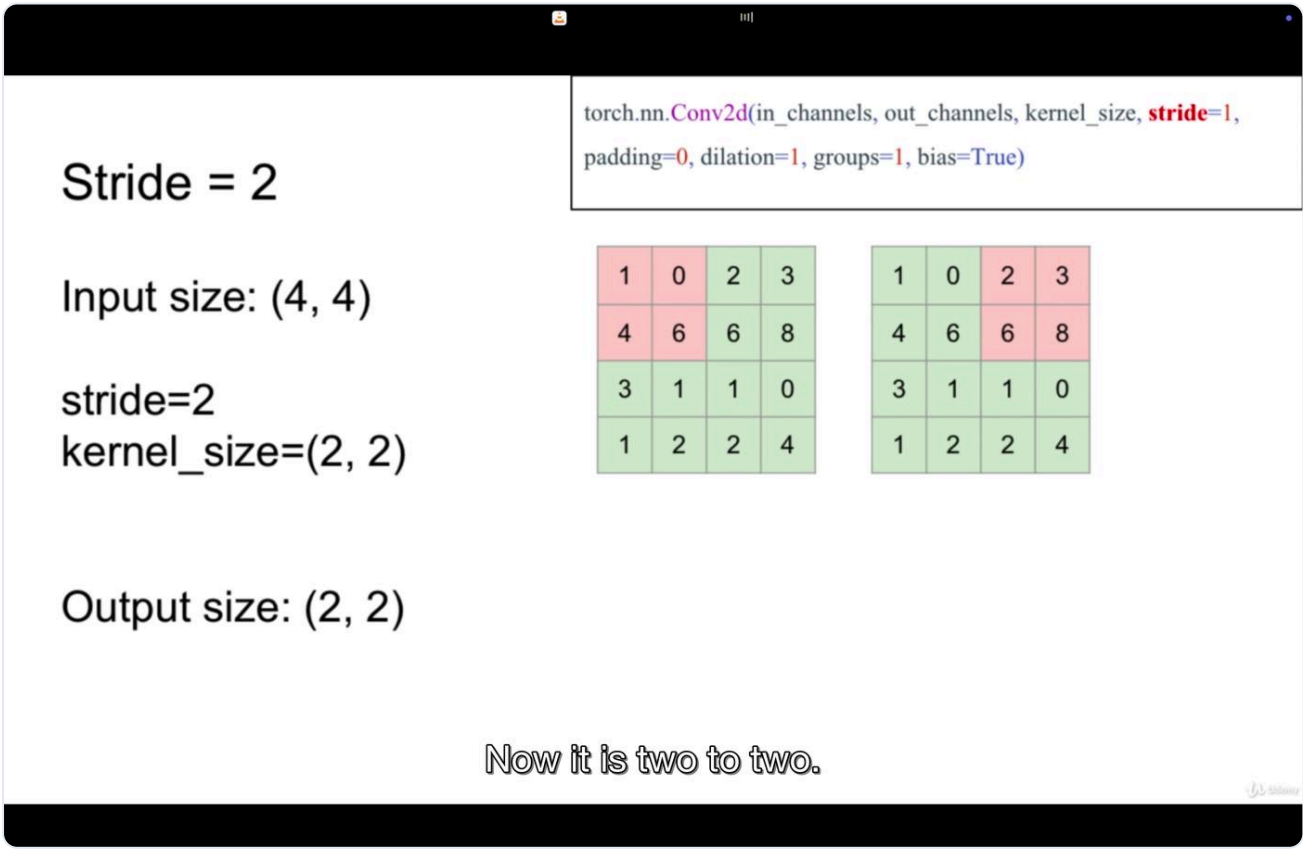


FIGURE 13.4

Now it is two to two. That is why on the first layer of the network it is better to use stride.

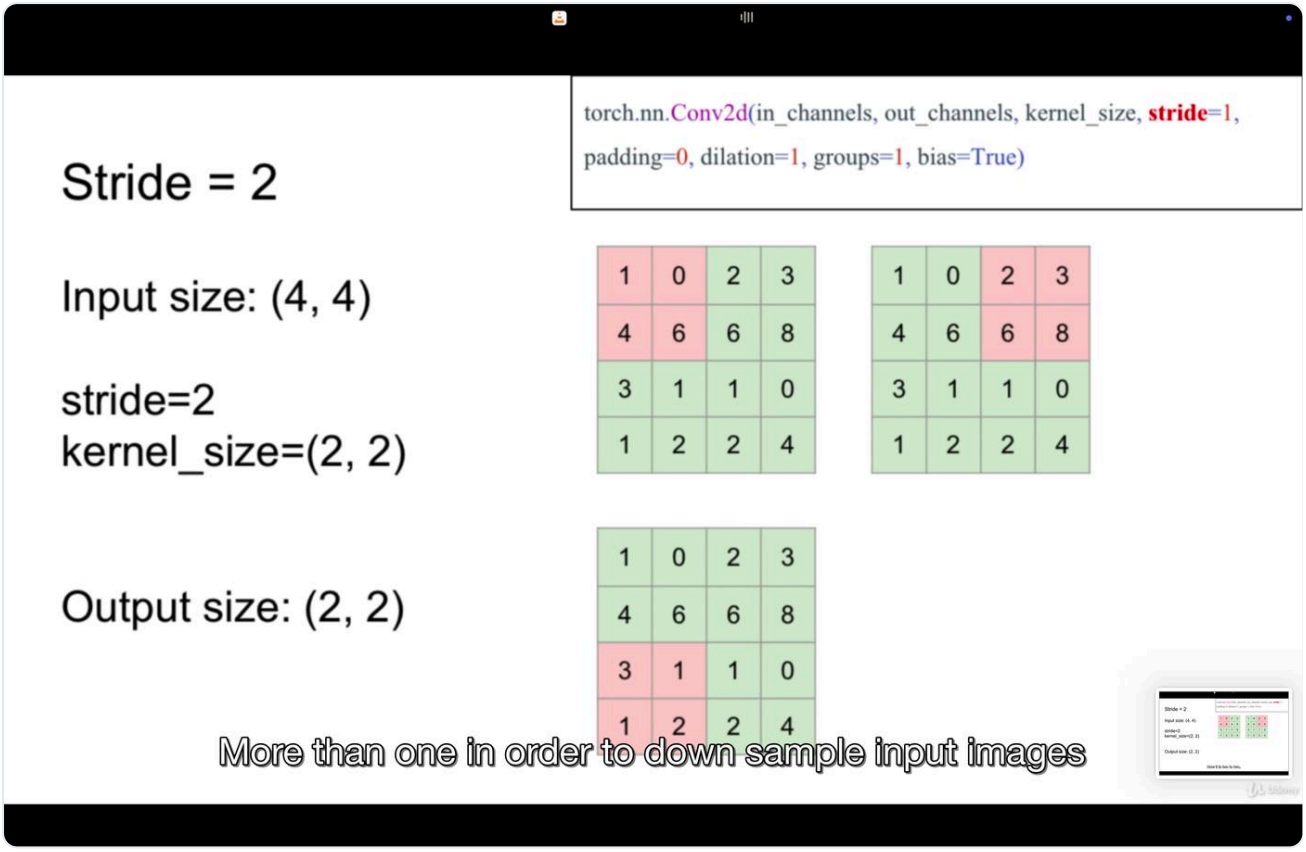


FIGURE 13.5

More than one in order to down sample input images okay.

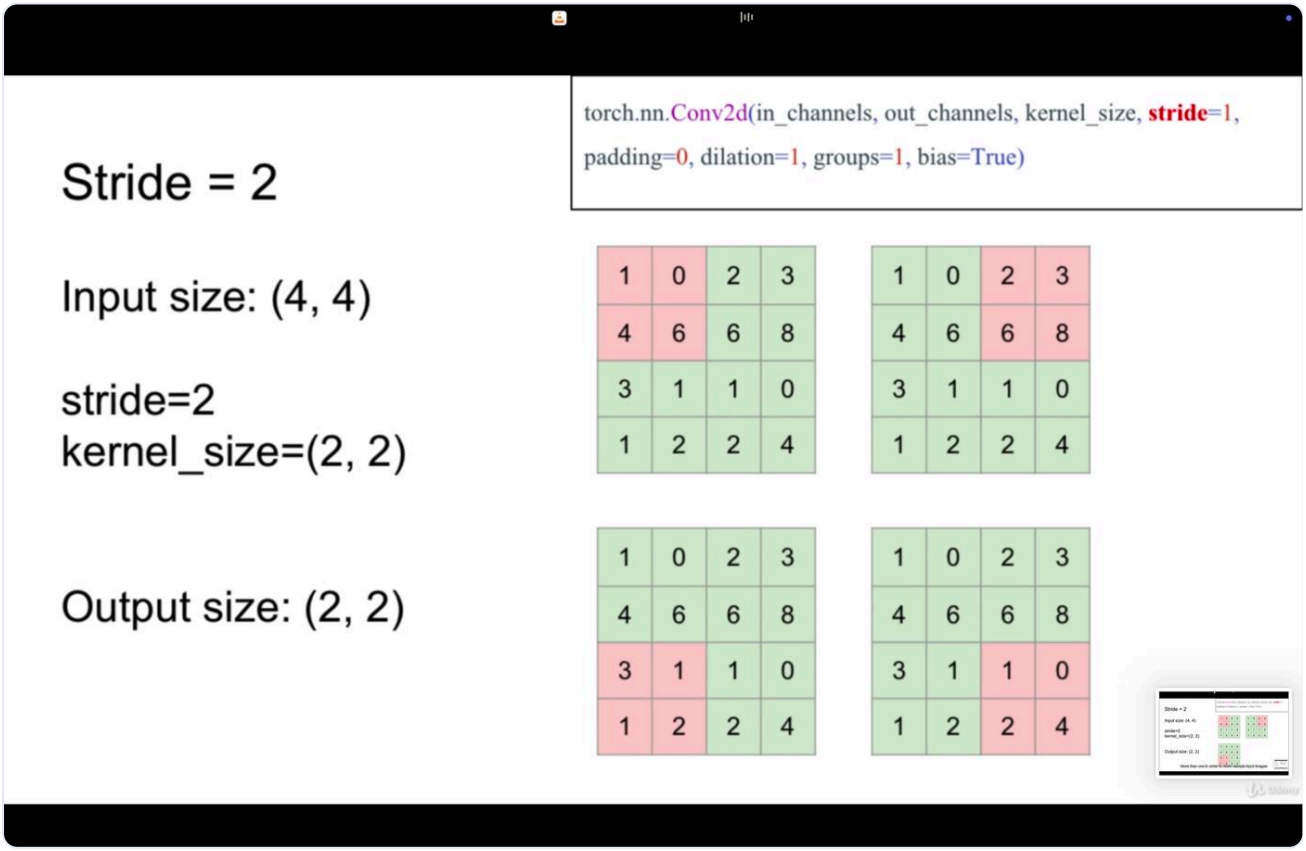


FIGURE 13.6

Now let's send the strike to three.

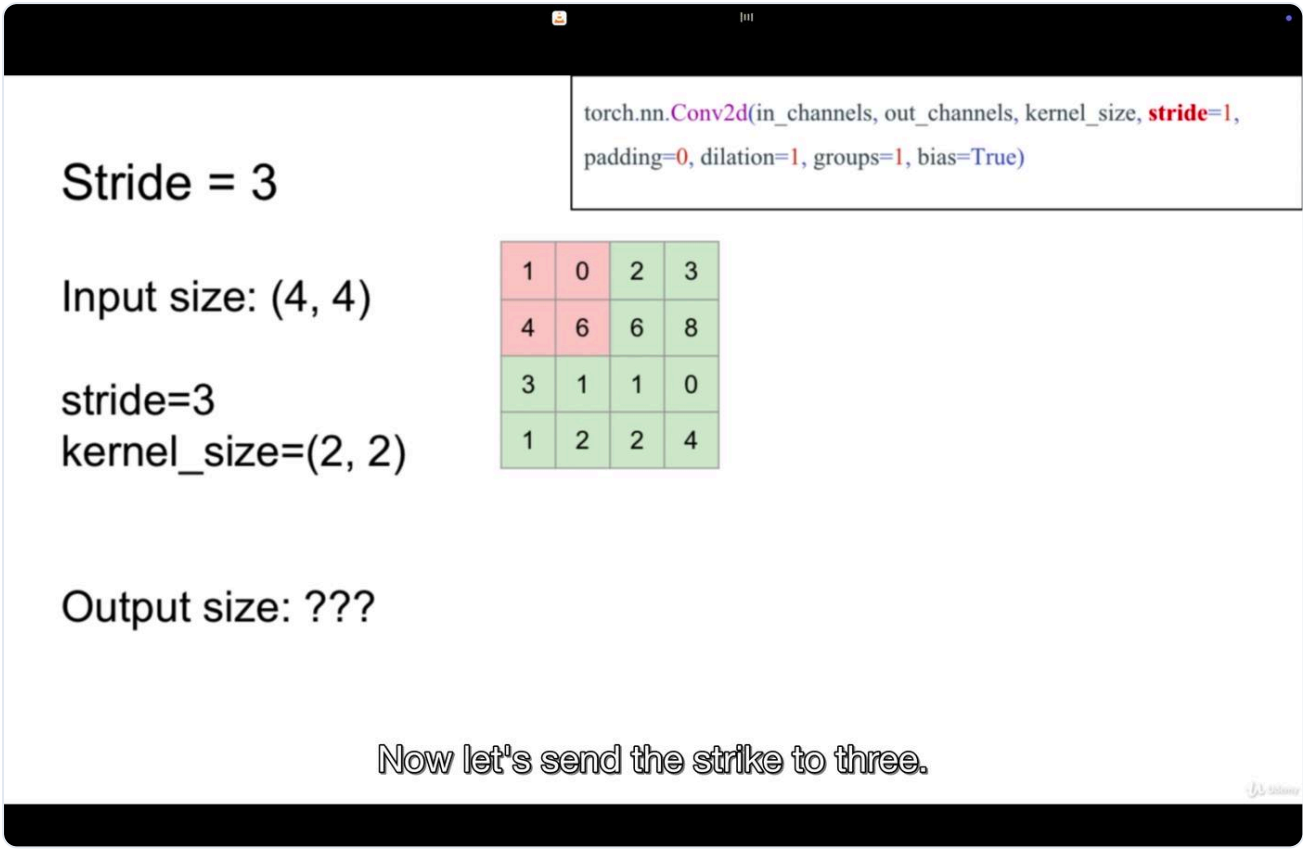


FIGURE 13.7

What will happen probably in this case PyTorch will return an error.

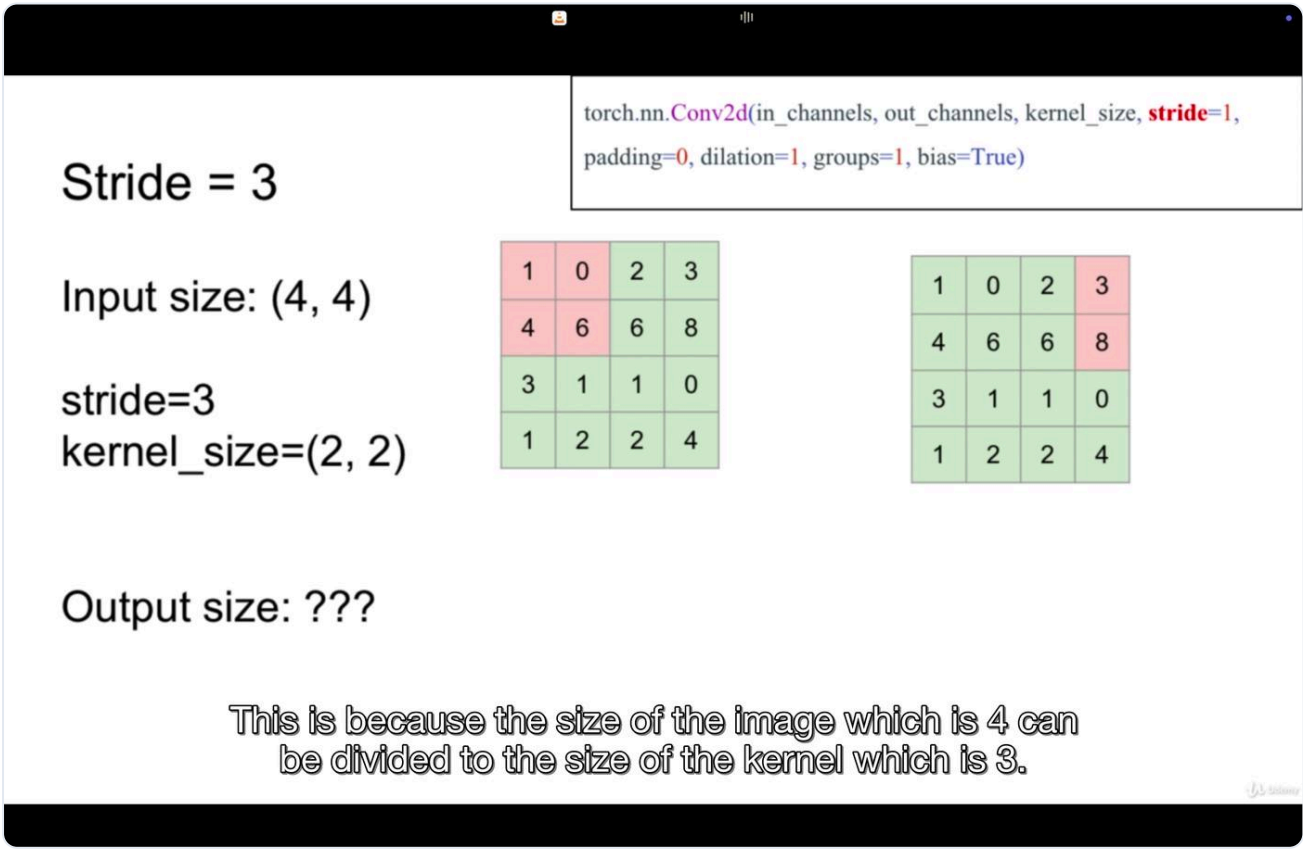


FIGURE 13.8

This is because the size of the image which is 4 can be divided to the size of the kernel which is 3. I hope you understood what is stride. So now let's switch to another parameter padding look at the image on the left.

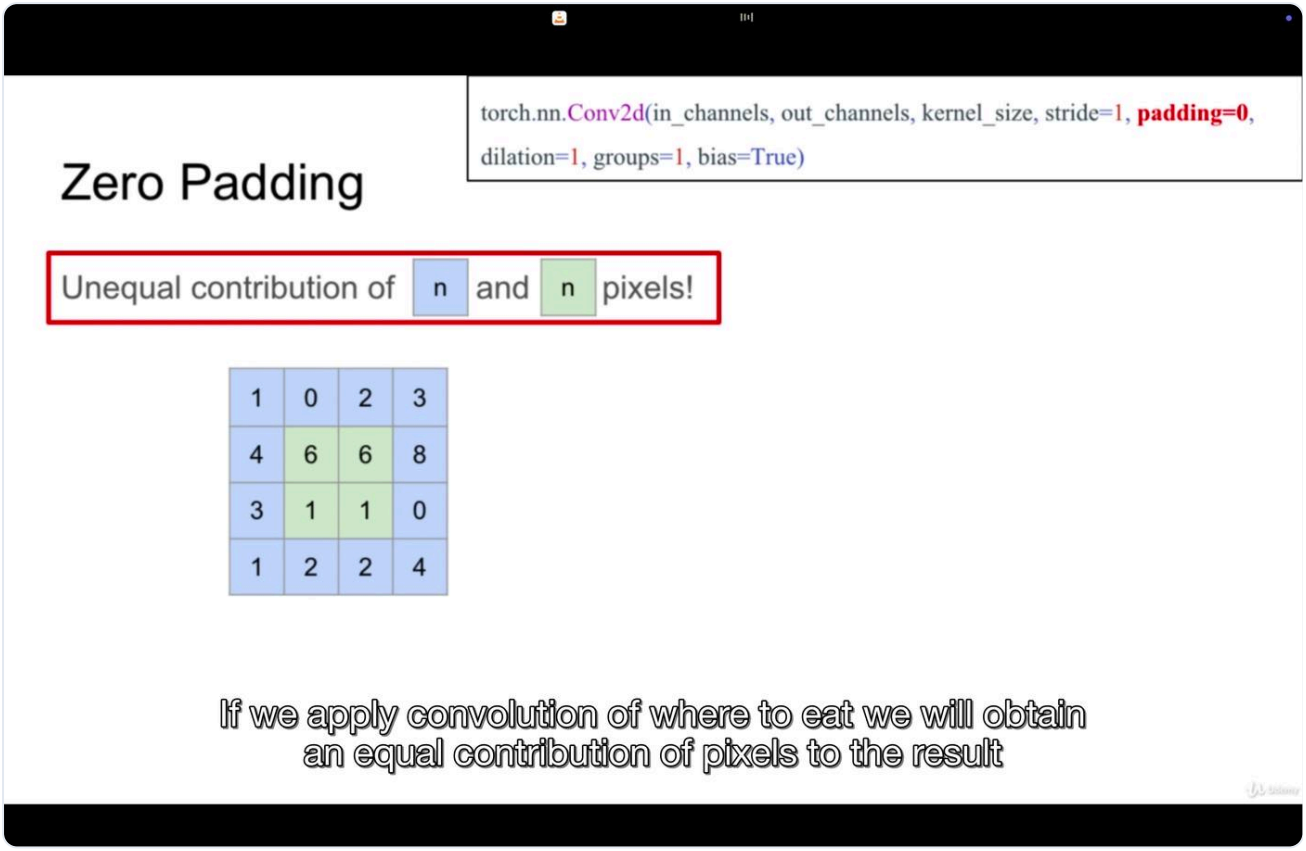


FIGURE 13.9 | Zero Padding

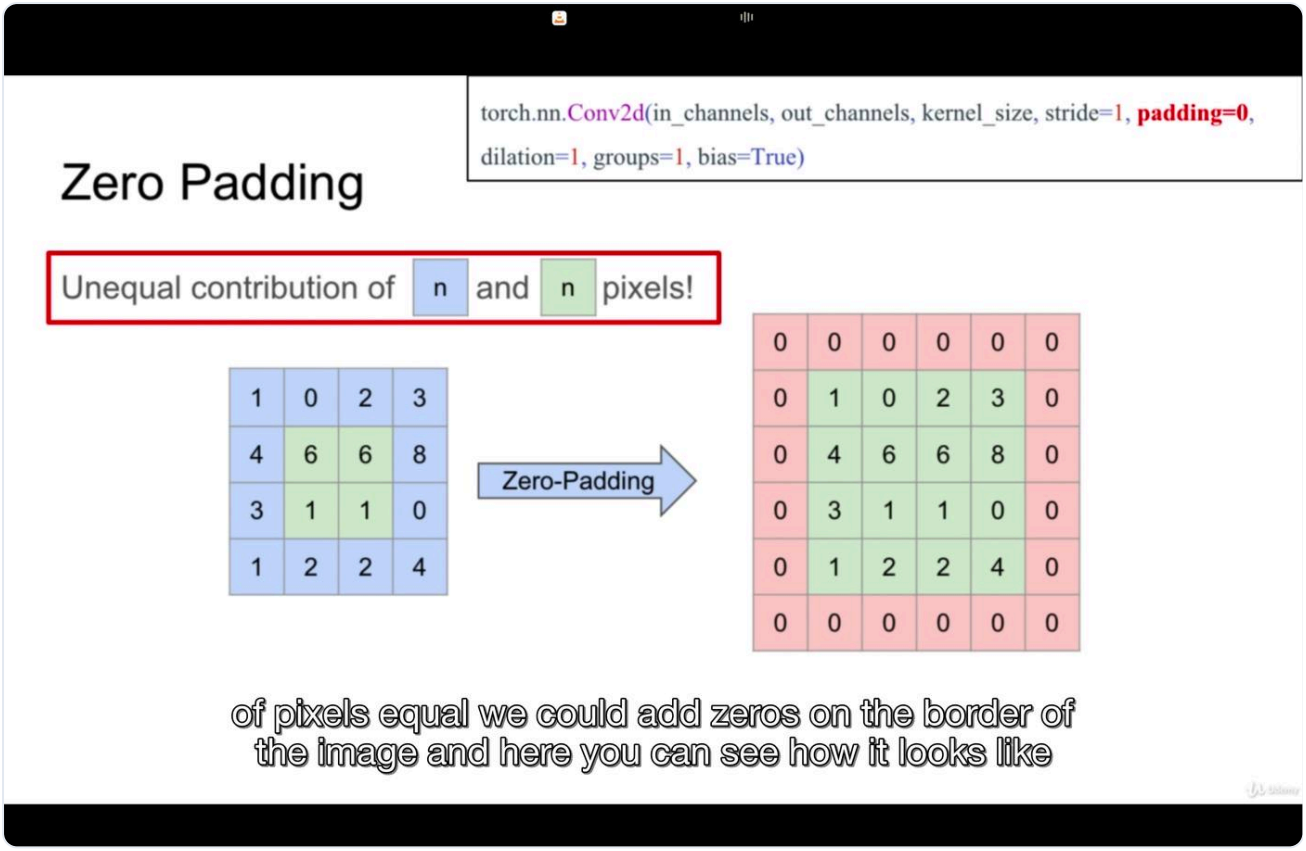


FIGURE 13.10 | Zero Padding

If we apply convolution of where to eat we will obtain an equal contribution of pixels to the result of the convolution green pixels will play there are more than blue pixels if we want to make the contribution of pixels equal we could add zeros on the border of the image and here you can see how it looks like red pixels have now will you zero.

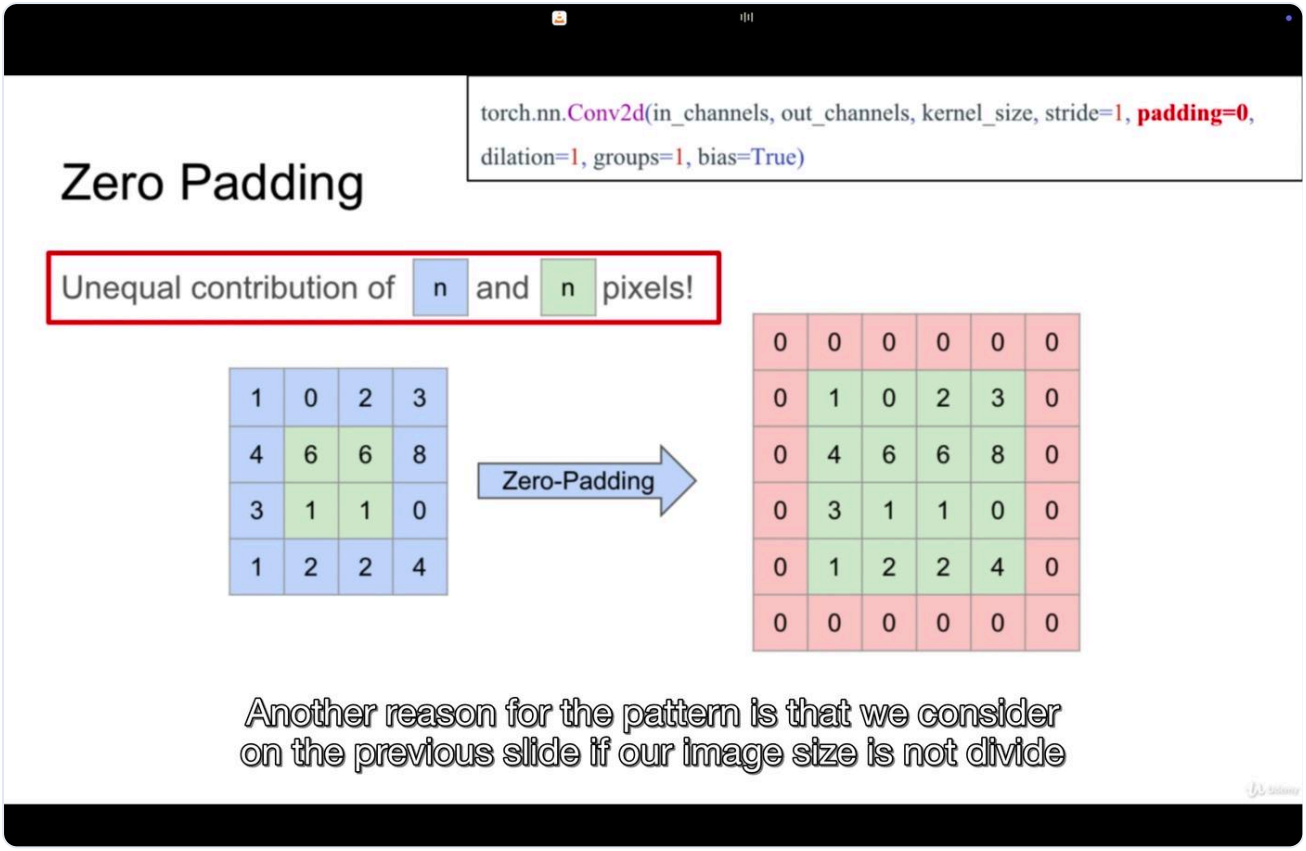


FIGURE 13.11 | Zero Padding

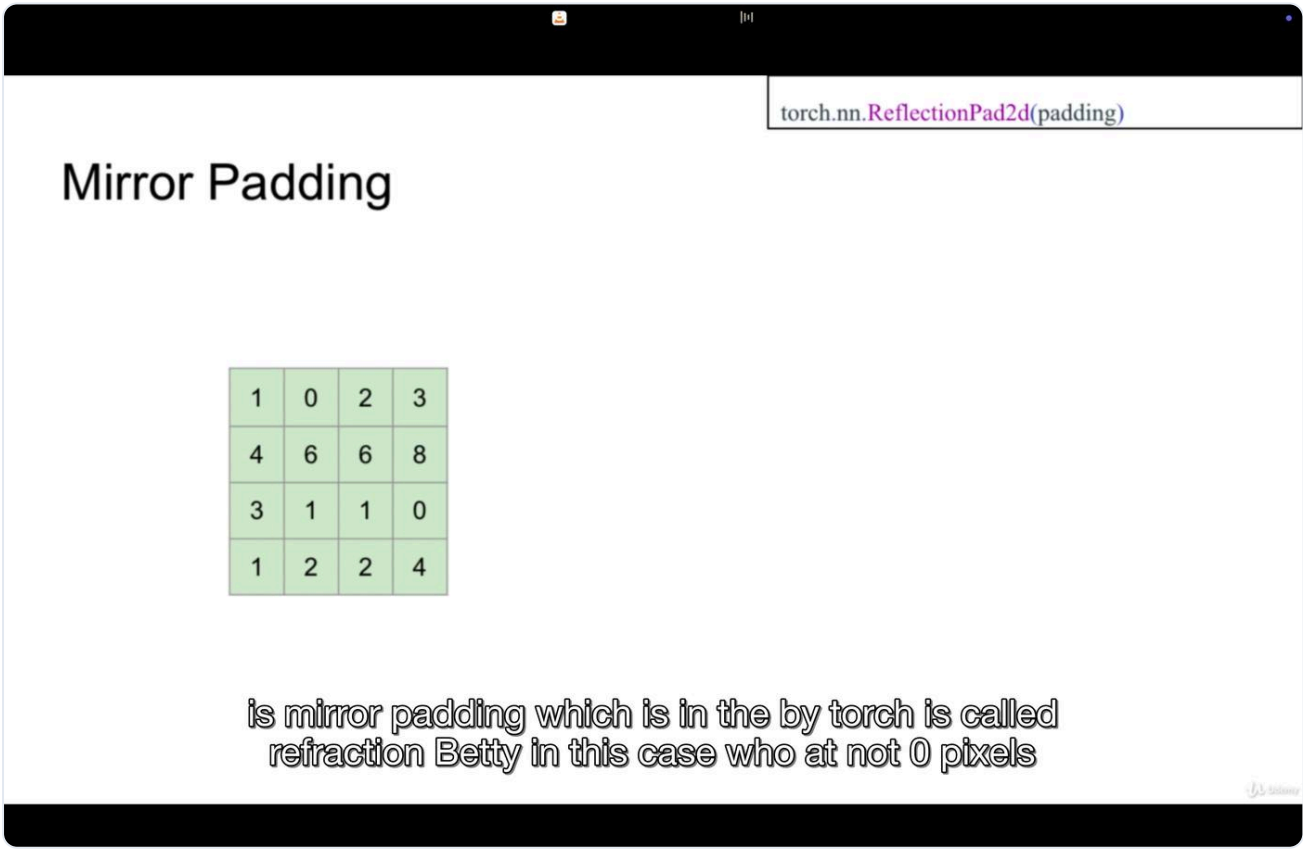


FIGURE 13.12 | Mirror Padding

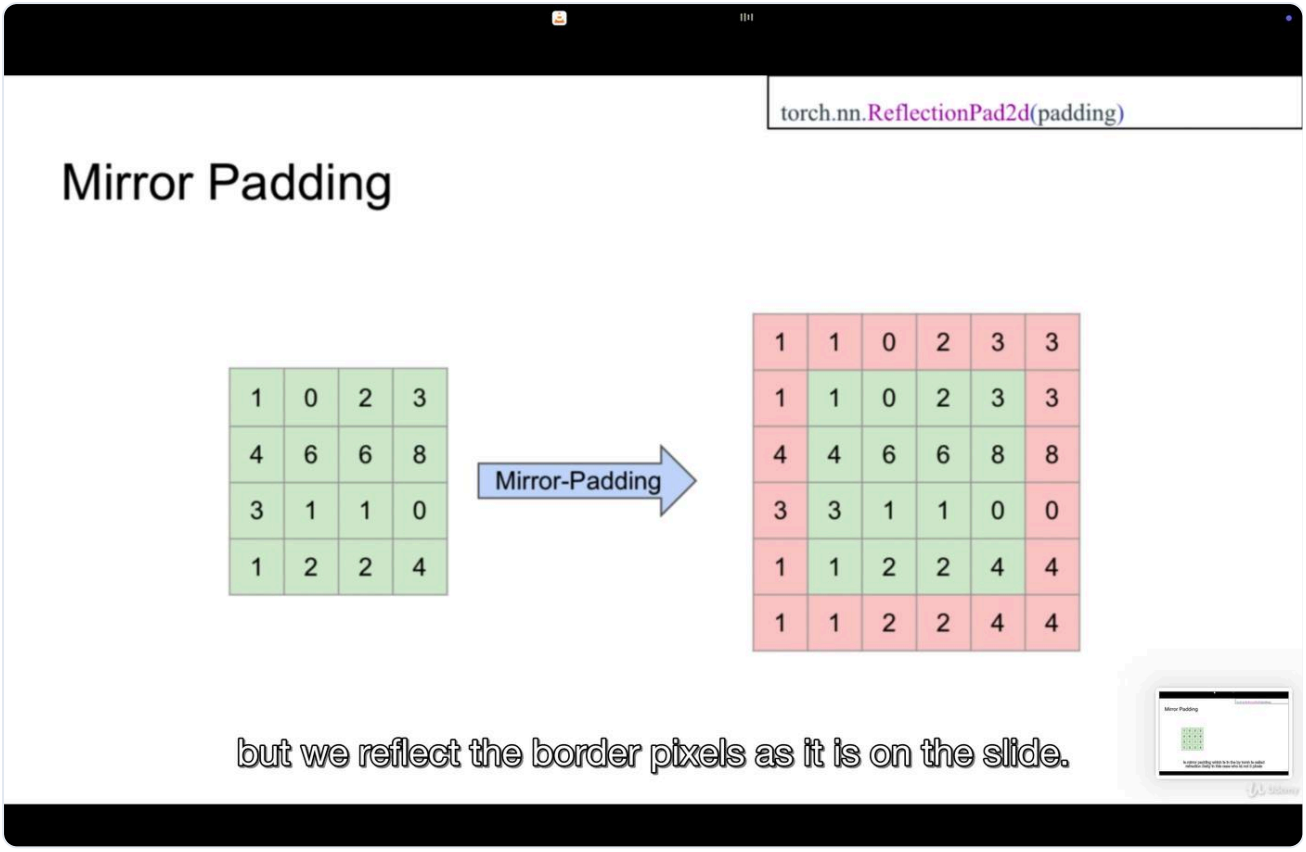


FIGURE 13.13 | Mirror Padding

Another reason for the pattern is that we consider on the previous slide if our image size is not divide by the kernel we can always extend the image with zero padding and now the possible creation of padding is mirror padding which is in the PyTorch is called reflection padding in this case who at not 0 pixels but we reflect the border pixels as it is on the slide. Well this is all for padding. Thank you for watching and see you in the next video.

LECTURE 14

Sigmoid Derivative

There convolution aspiration is a linear function in the linear space.

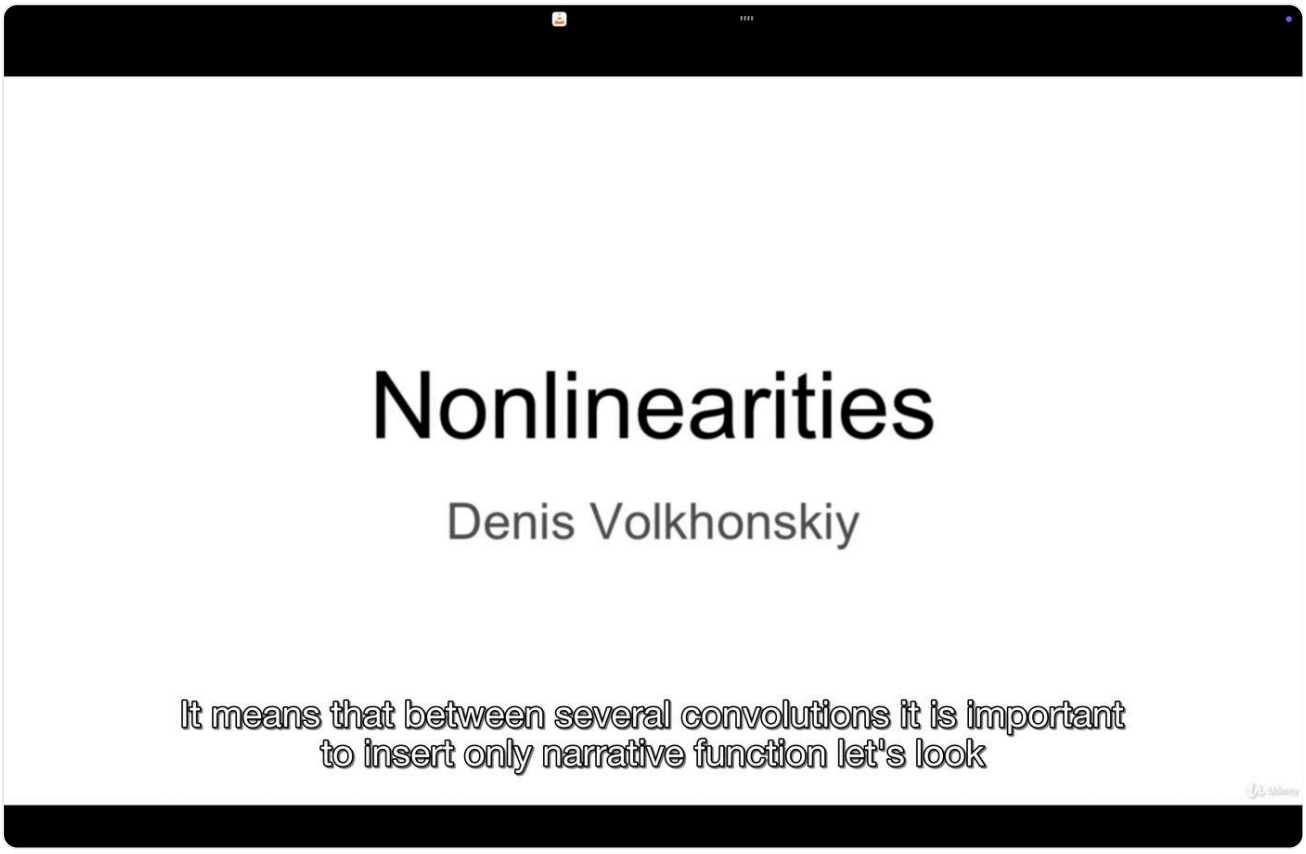


FIGURE 14.1

It means that between several convolutions it is important to insert only narrative function let's look at the most popular choices and discuss their advantages and disadvantages the most popular non linearity are sigmoid and hyperbolic tangent the sigmoid transforms the input to the well you from 0 to 1 while hyperbolic tangent transforms an input to the well you from minus 1 to 1 both hyperbolic tangent and sigmoid are bad choices.

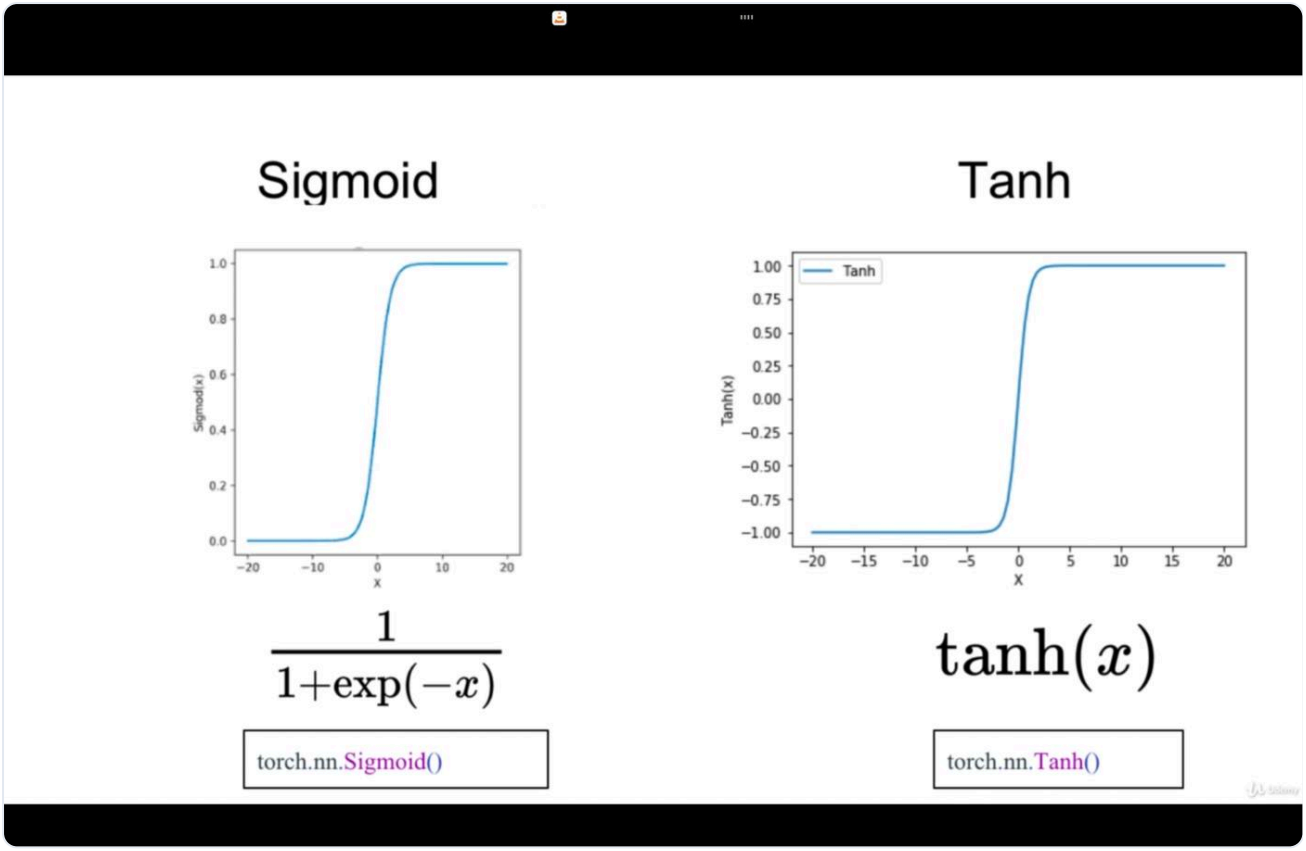


FIGURE 14.2

Let's look at sigmoid gradient. It is equal to zero almost for all inputs it has its largest value in the point zero the value is zero point twenty five.

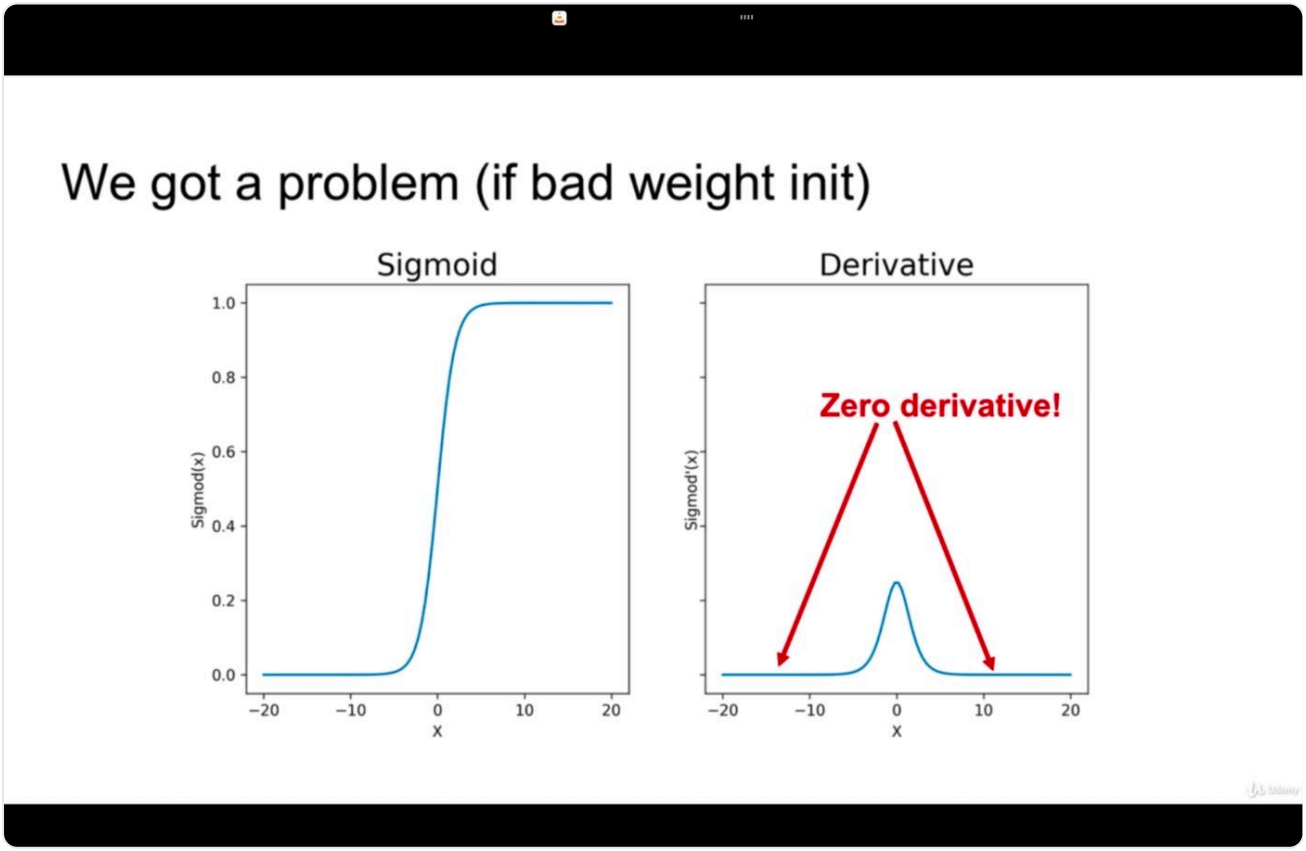


FIGURE 14.3 | Sigmoid Derivative

It means that for most of inputs neural network weight will not be updated at all.

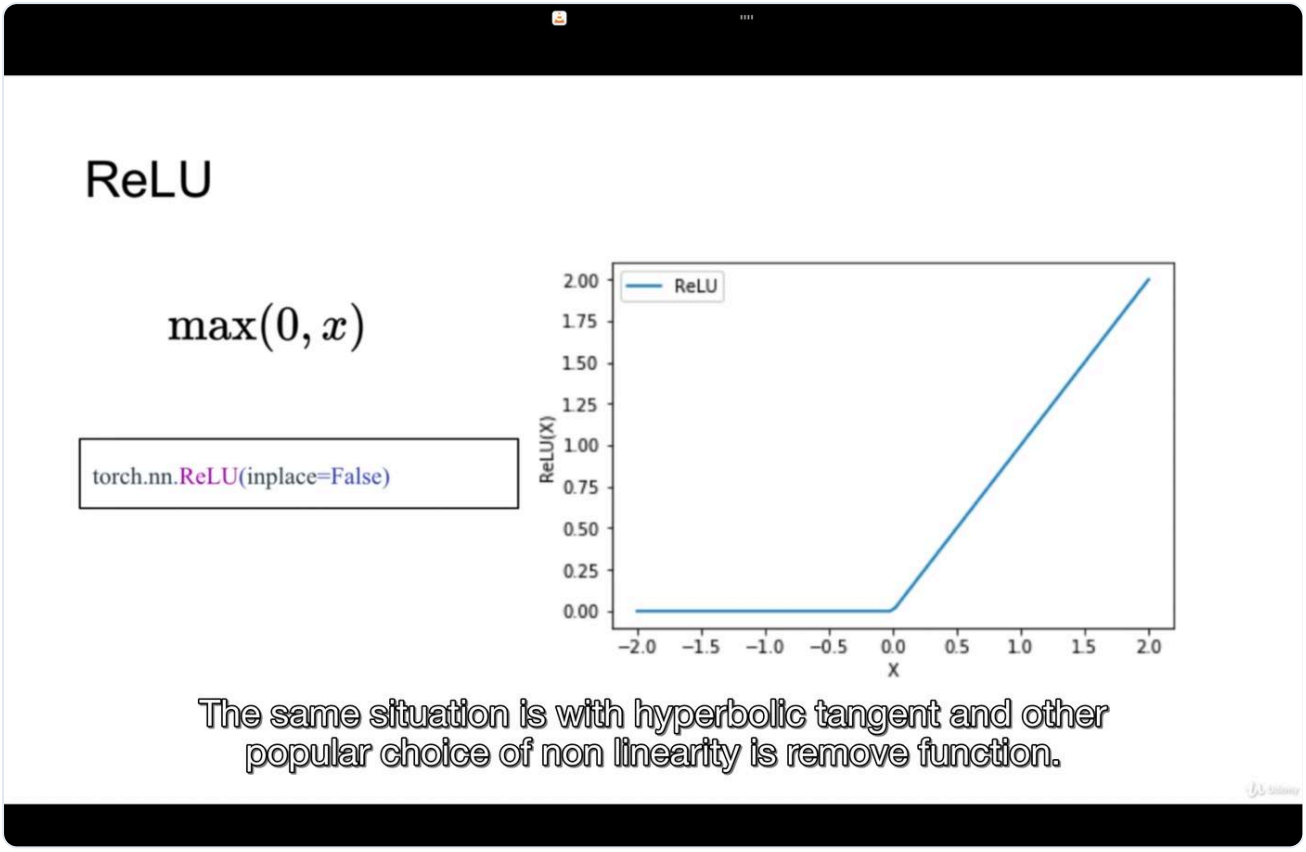


FIGURE 14.4

The same situation is with hyperbolic tangent and other popular choice of non linearity is remove function. It says their input to zero if it is negative it is maximum from zero and X however it sets the derivative to zero in the half of the inputs.

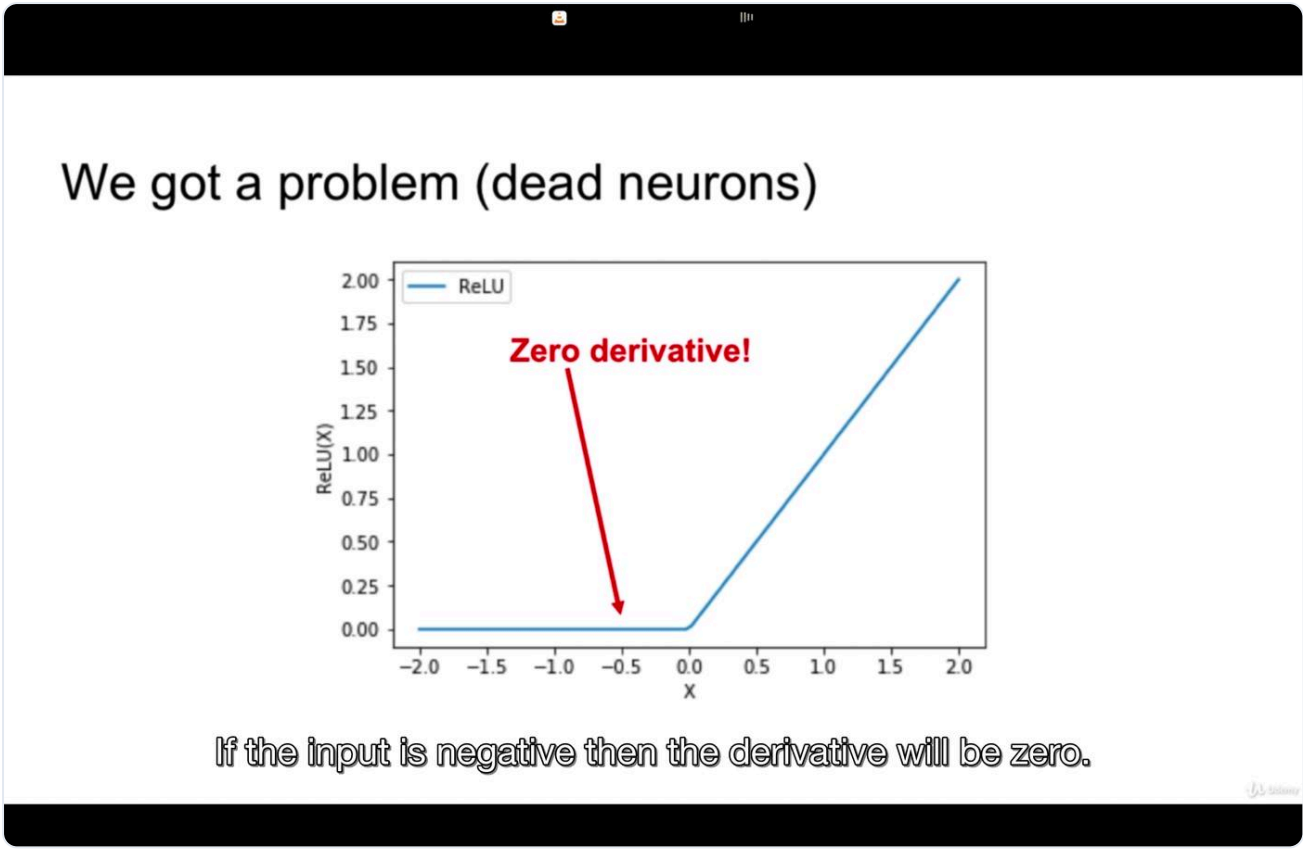


FIGURE 14.5

If the input is negative then the derivative will be zero. Again it means that your weight will not be updated.

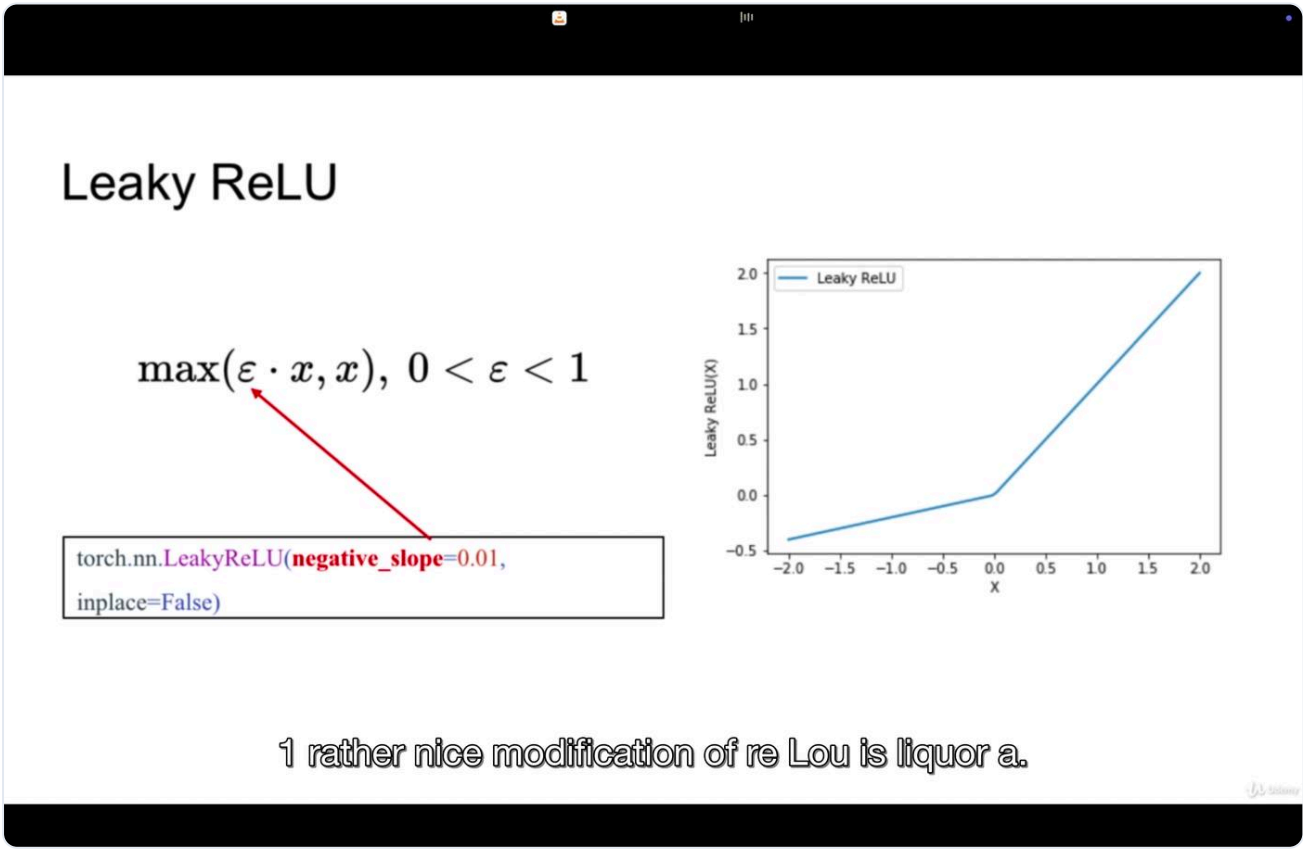


FIGURE 14.6

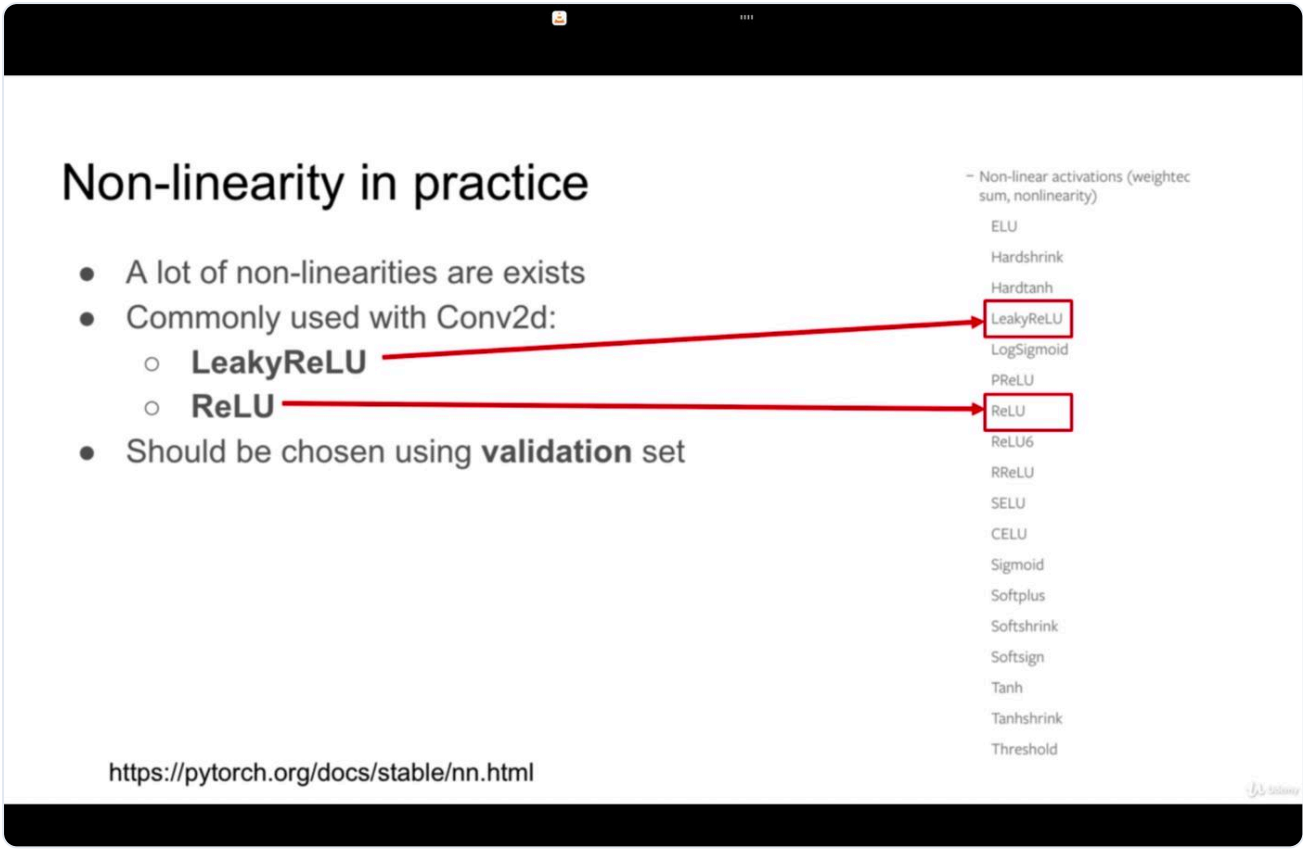


FIGURE 14.7

1 rather nice modification of re Lou is liquor a. It doesn't said the negative input to zero but slightly change the slope of the negative input. As you can see on the slide it is a good choice. Most of cases there are much more non-linear pretty exists in every deep learning library. Usually in practice the best choices will be liquor law and renew. However you may use some validation set of samples in order to choose the best non linearity. Thank you for watching. And tune in the next video.

LECTURE 15

Max Pooling

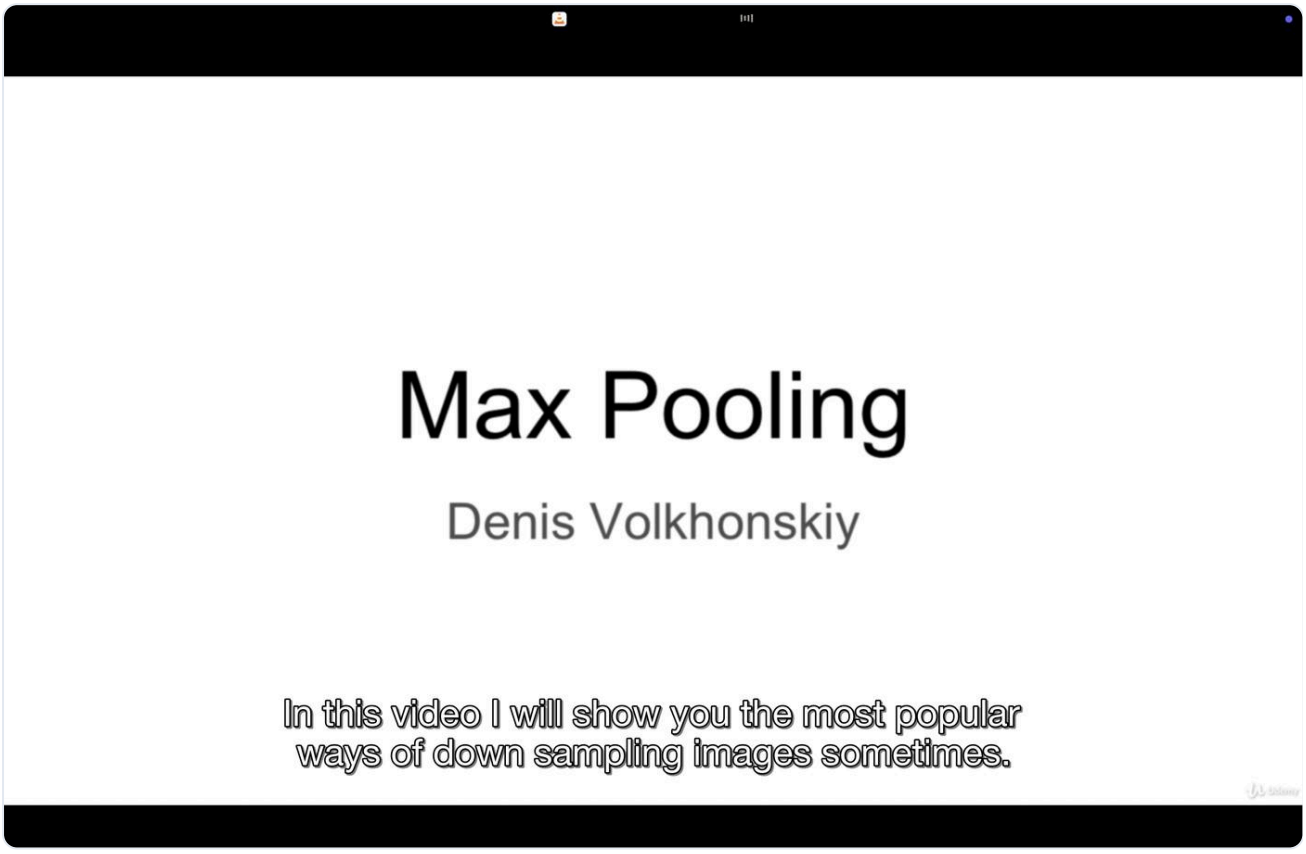


FIGURE 15.1 | Max Pooling

In this video I will show you the most popular ways of down sampling images sometimes. We have a very big images as an input. For example the size can be more than 1000 pixels. If we will process it directly we will need a lot of parameters in the network.

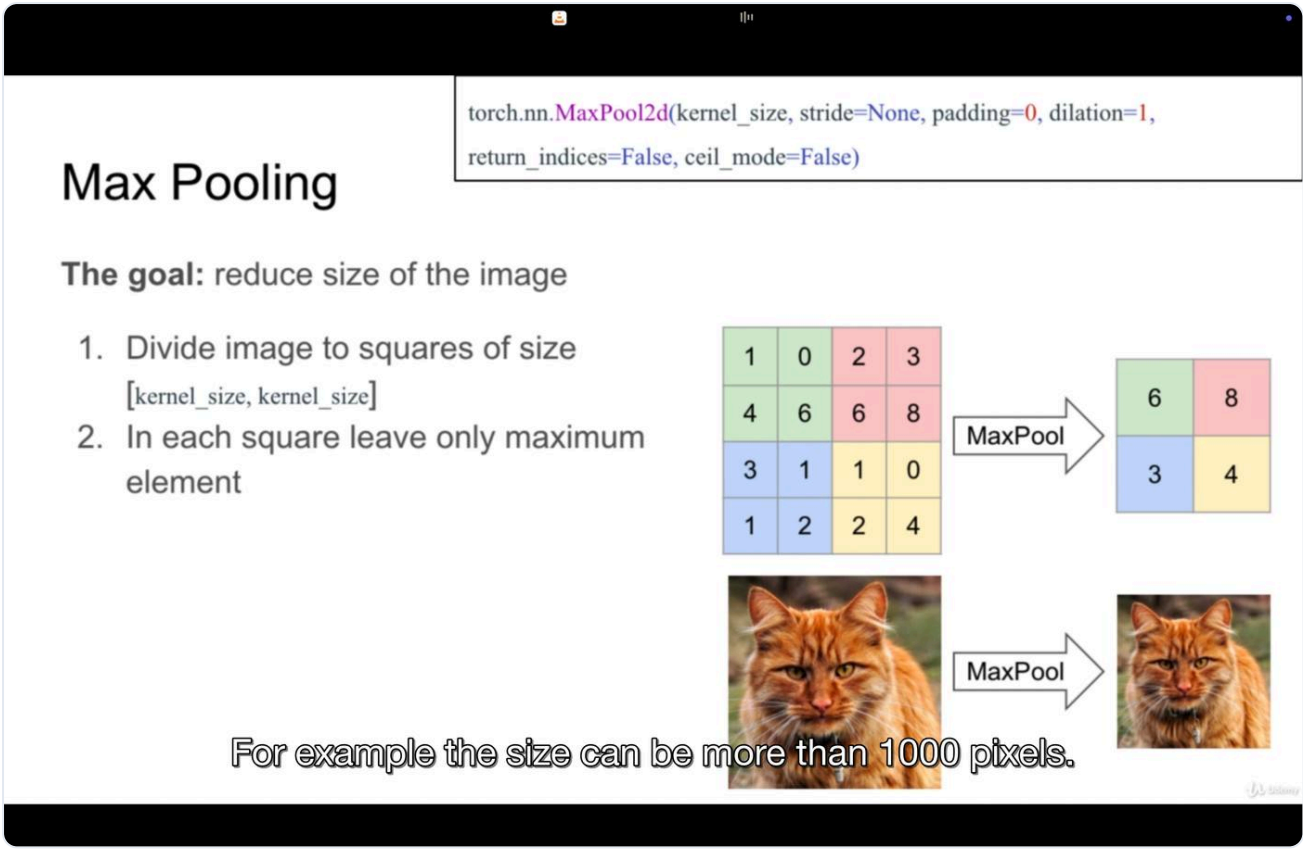


FIGURE 15.2 | Max Pooling

However in order to recognize if it is a cat or a dog on the image we don't need so much information. We don't need so many pixels. That is why our goal is to reduce the input image size for these proposed. People usually use Macs pooling glare.

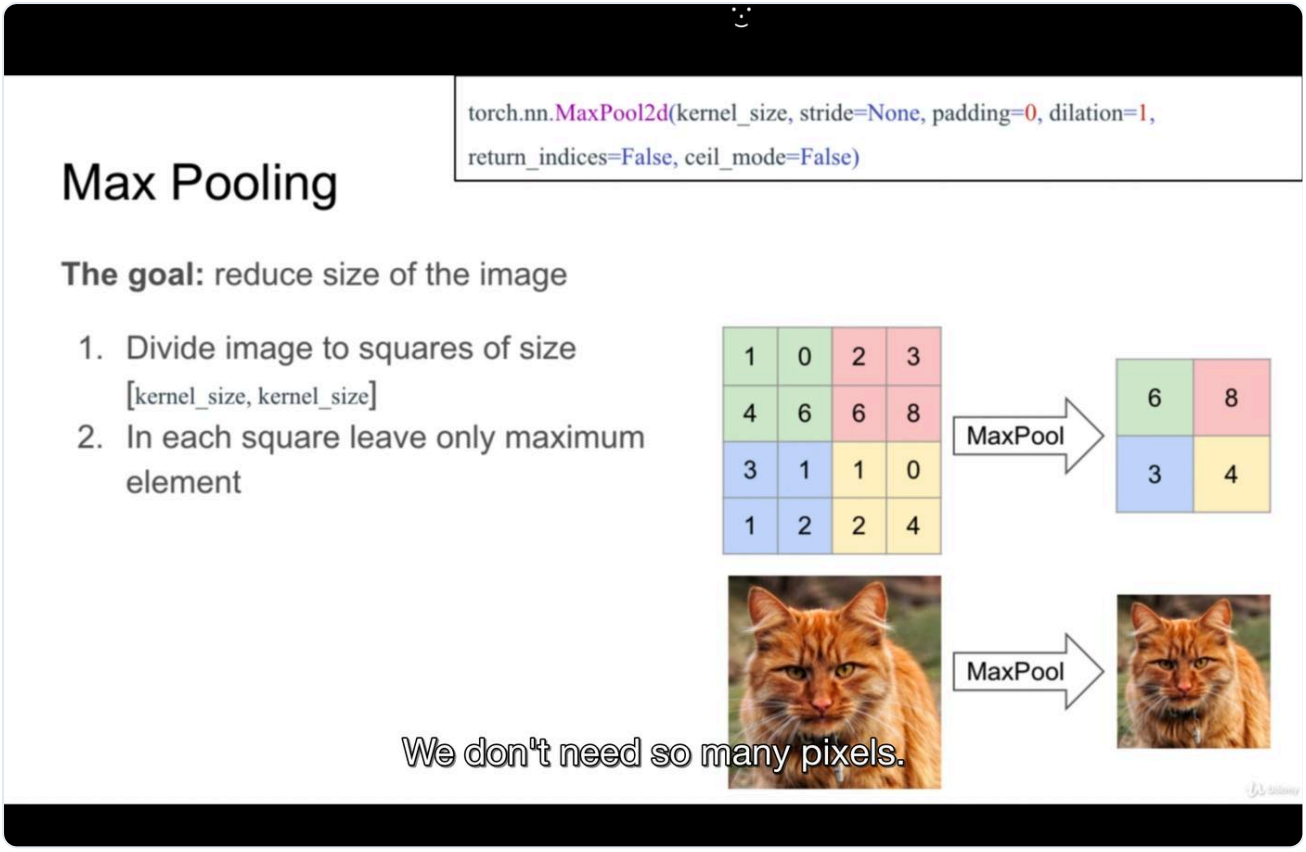


FIGURE 15.3 | Max Pooling

It divides your image into a squares of sound fixed predefined size and in each of the square it selects maximum. Well you as in the example on the right in Green Square. It select Sikhs in Red Square. Eight in Blue three and in yellow four.

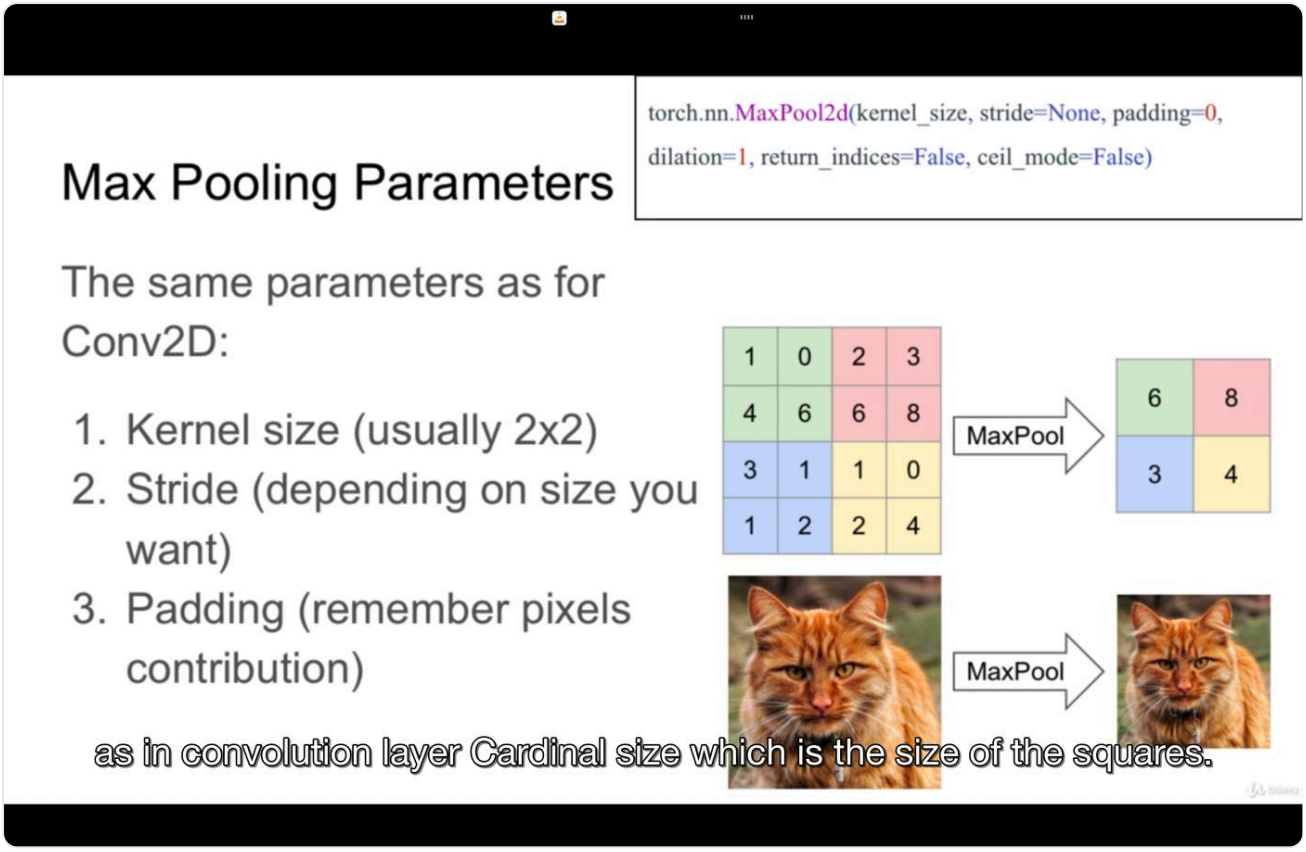


FIGURE 15.4 | Max Pooling Parameters

In the example the size of the square is two by two in the MAX pool where we have the same parameters as in convolution layer kernel size which is the size of the squares.

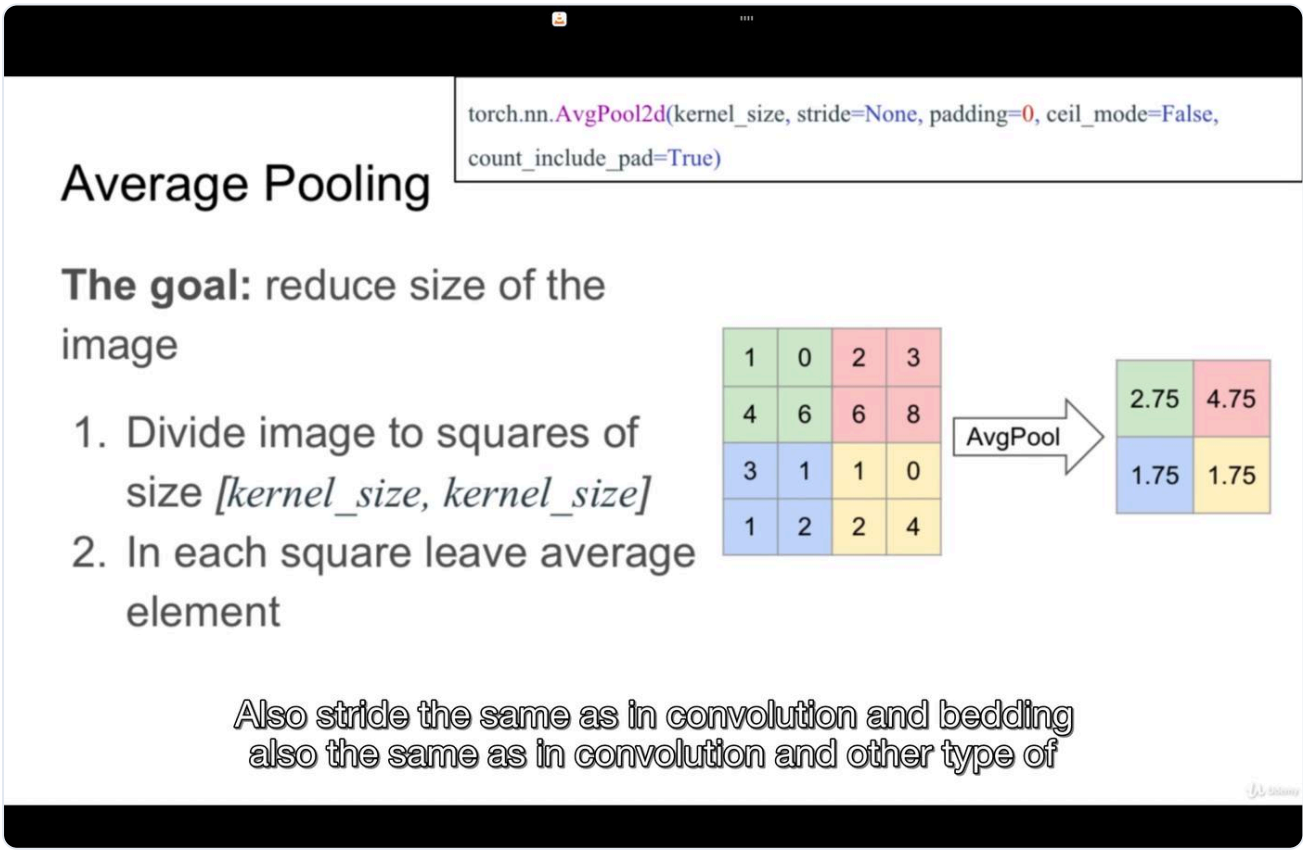


FIGURE 15.5 | Average Pooling

Also stride the same as in convolution and padding also the same as in convolution and other type of boolean for images is average pooling. The only difference from the max pooling is that now we leave not the maximum value but an error each one. You may ask me Dennis. So what should I use max boring or average pooling. Who knows I will answer to you. In practice there is not so much difference between these two methods. Often people prefer to use max pooling instead of Farish boolean. Thank you for watching and see you in the next video.

LECTURE 16

Non Linearity and Build Deep

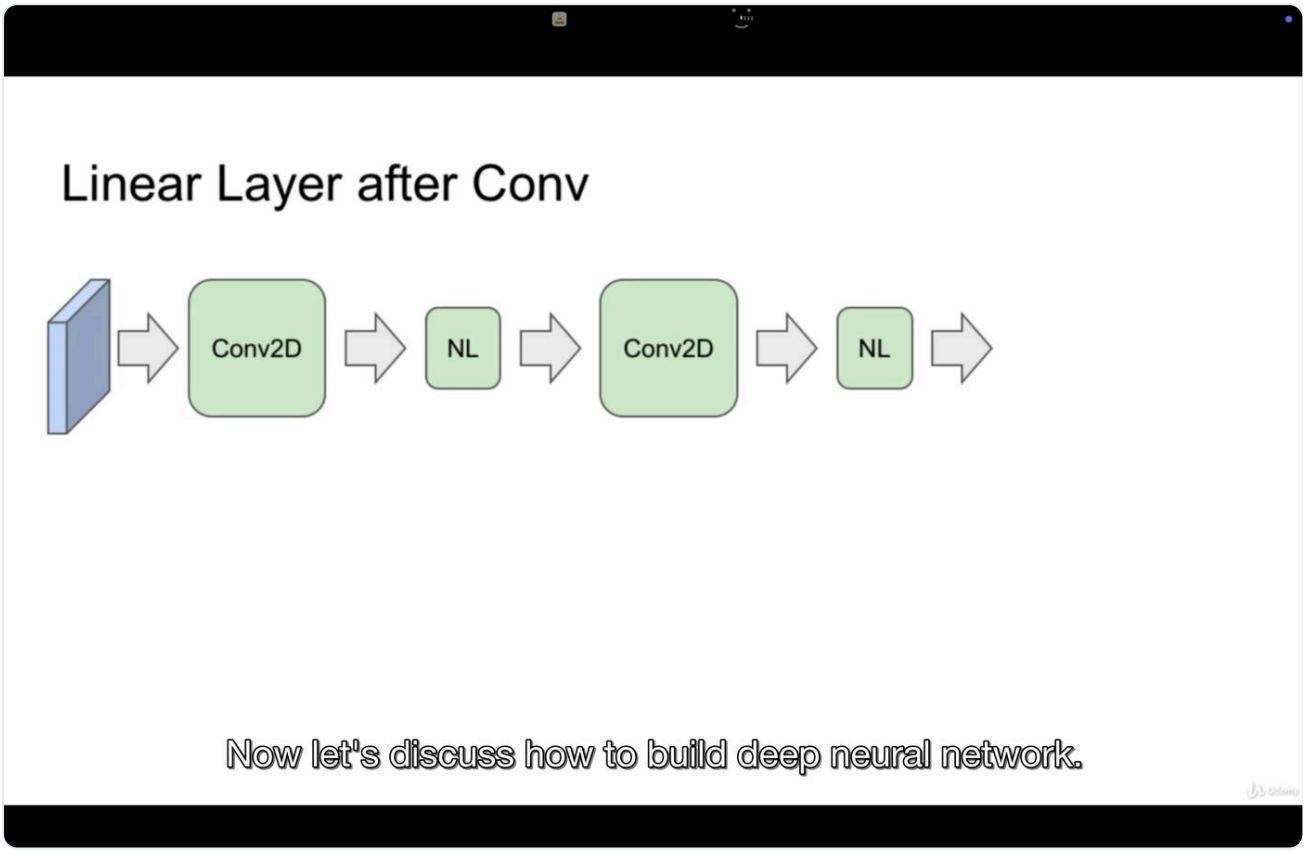


FIGURE 16.1 | Now Let's Cliscuss How to Build Deep Neural Network

Now let's discuss how to build deep neural network. As I already mentioned it is important to add non linearity between each convolution aware because convolution is a linear operation.

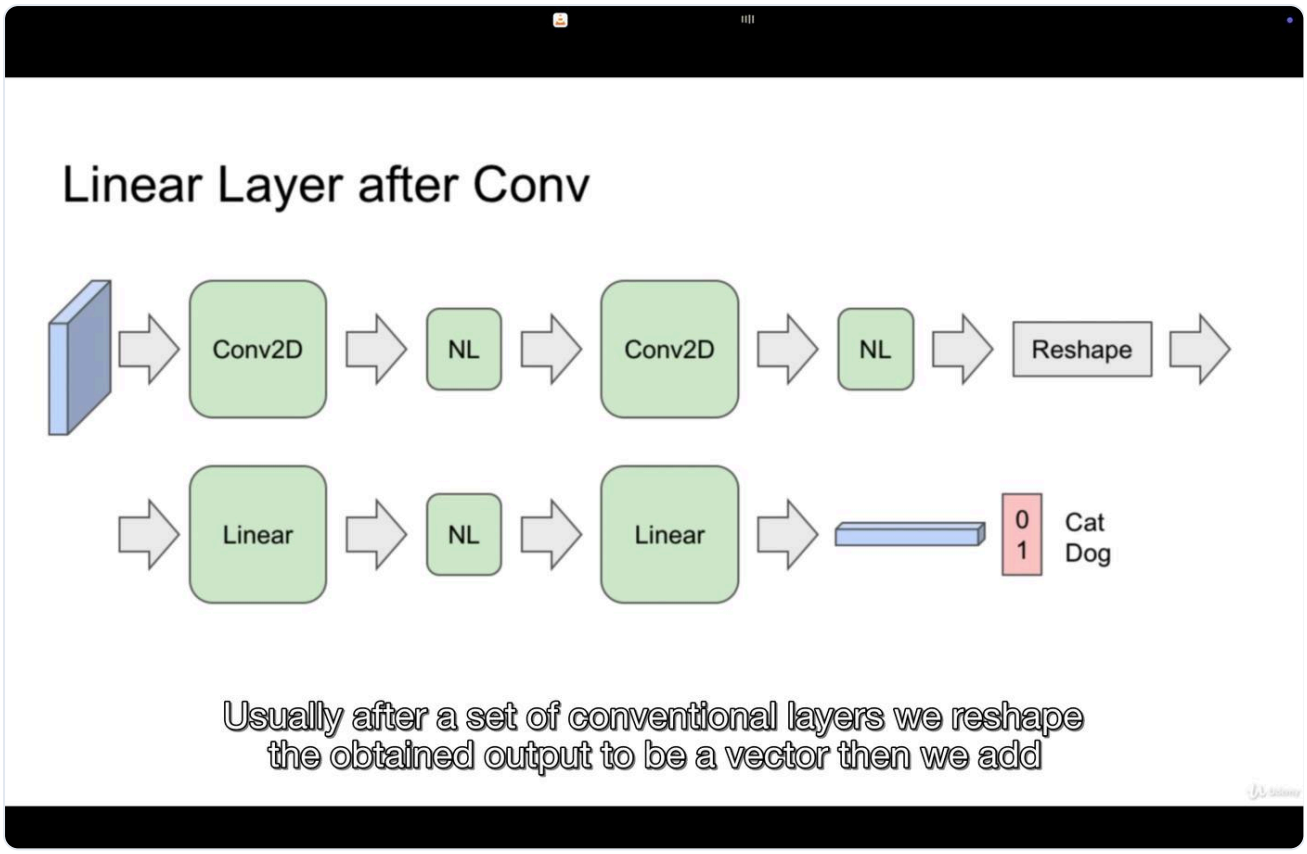


FIGURE 16.2

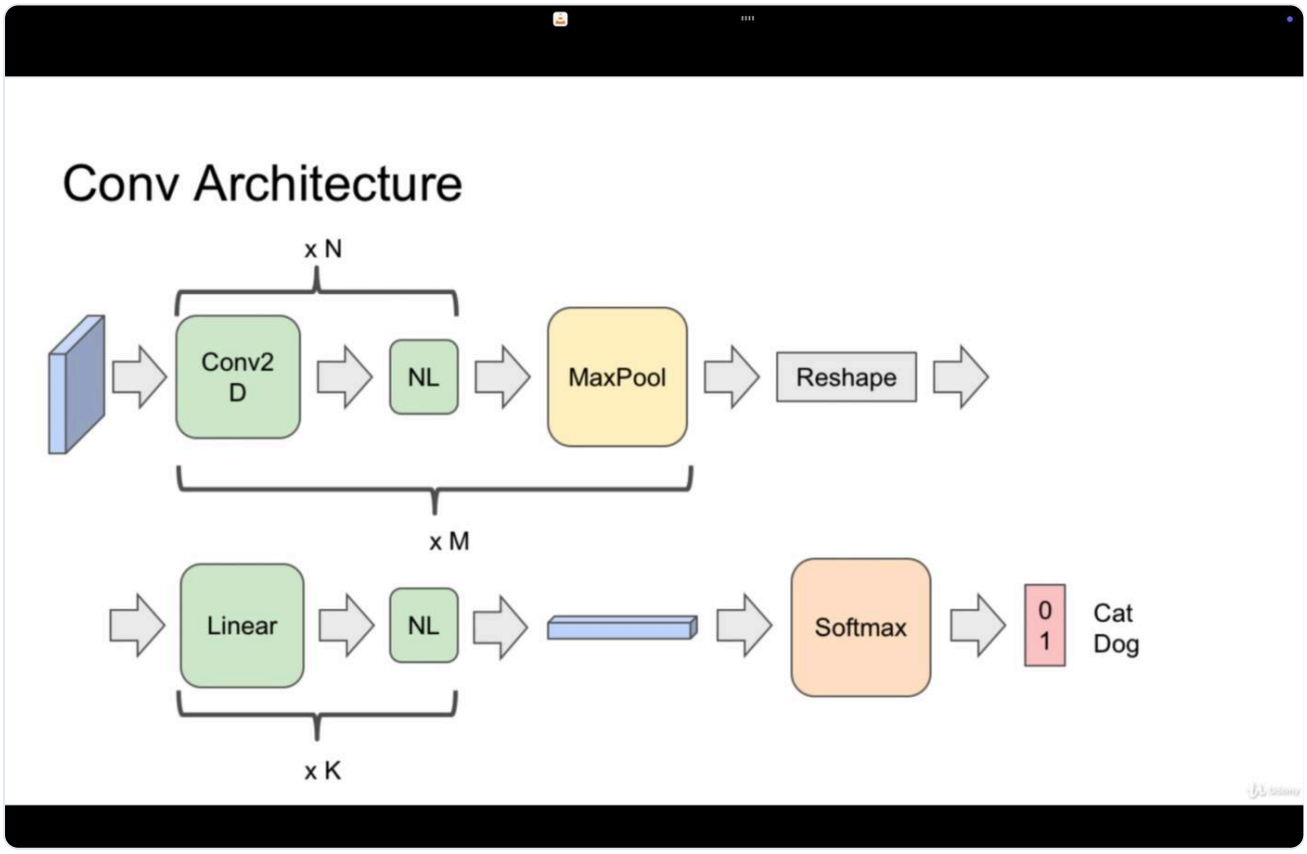


FIGURE 16.3

Usually after a set of conventional layers we reshape the obtained output to be a vector then we add several linear layers with non linearity between them as a result we should obtain a vector of size that is equal to the number of classes in the classification problem if we solve a regression problem. Then the output size should be 1 in order to build deep network. We should stack several blocks that contains convolution with non linearity. After that add max pooling clear. We can't repeat these big blocks several times. After that we reshape the output vector. Use linear layers with soft marks at the end. Now this is our deep convolution all neural network thank you for watching and see you in the next video.

LECTURE 17

Output Size and Channels

See the notebook here

We have already built linear classifier and multilayer passive drawing for classification of CIFAR-10. Now let's switch to conventional neural networks so let me restart the kernel and clear all outputs. Now we again import all necessary libraries. We again load our data which is already downloaded and verified. We again have train set test set train loader and test loader. We can look at our images again. We have 10 classes all of them have size 32 to 32 and and have three channels. Now we have to build add continent from the lecture. You should remember that when we build the Conrad continent Usually we start from the set of convolution of layers. And after that we use several multilayer perceptrons.

Okay so in Python you have two parts. I wrote two parts here. Main part which is called convolutional part and MLB which is multilayer perceptron which stands for set of fleeing your layers. And when we will do forest bus we will apply first convolution apart. Then we will reshape our output to the batch of vectors and then we will apply multilayer purse drawn. Okay. No I will show you the service which is called for more a high which is very helpful when you build a cone of Net. It is good when you want to understand what will be the output size of your and volitional neural network. So what we have here we have their input size 32. It means that our images is 32 by 32.

Then we should define that can volitional where here is kernel size stride dilation and padding. We didn't discuss dilation yet but other parameters I assume that you know Okay let's set the general size to 5 for example. This tried to do and the padding is valid we have two options here valid and same so let's look when we use same padding. It means that all big cells will have the same contribution. This is what I described in Beijing career. If we use valid Beijing then our Beijing will be zero. It means that no Beijing will be applied. Okay so what we have here Layer 1 kernel size 5 means the cardinal is 5 to 5. Let's try this too. Beijing has zero input sizes 32 and the output size will be 14.

It means that it will the output will have a shape 14 to 14 and we will have several channels but there is no new word here about channels because this is not important here. Okay we can duplicate this layer and apply it after the first one. So the input size will be 14 now and the output size will be 5. And for example if you want to delegated then we will obtain the input size 5 and output size is 1. Okay so we can use 3 layers which has the same parameters cabinet size 5 stride 2 and Beijing 0. Let's write them and then that comes to D. Then we have parameters in channels which is and channels and next for the next parameter is all channels. This is our channel sees the number of filters which we will have in our conventional air.

Let's say it will be 16. This is a hyper parameter of course it should be tuned with the nation set. Now we have Cardinal sites kernel size. We decided to set it to five then stride to then Beijing 0 OK. And so now we should repeat this where three times but here we have now not and channels

which is three and in the input. But now it will be 16. And let's say the second player will have 32 shooters. It means that the third player will have 32 channels as input and assume it will have sixty four channels. Now this is our conversational part. As I said to you in lectures convolution is a linear operation. It means that between each volitional layer we should add some non linearity because the combination of linear layers of linear transformations is the same linear transformation and we can obtain again bad effects with the gradients of the network.

That's why we can write here like really do and in place to hear and hear good. So now we have the main module which consists of three can relational and do non-linear bridges after it. We can also apply some really later here after me. After playing me we will have the shape X shape by size 64 because we have sixty four output channels sixty four cardinals in the last convolution layer then 1 to 1 y one because after playing 3 can reduce the layers out size sizes 1 so 14 5 and 1. Now we have to reshape these like image consist of 1 pixels and one pixel and sixty four channels we should reshape it to the batch of factors. Again we use EEGs do it for you but size minus one and after that we apply multilayer basic John.

Okay let's write it and then the linear will have 64 input neurons input elements and the output dimension it will be said to then. Normally the charity lets say it will be like Marilu in place. True. Then again. And then that linear 32 to an output. Neurons and output neurons for our task will be done because we have exactly 10 crisis. So let's compute hate and with and as it was in the previous lectures and now and channels and channels is three. Because if we look at that image is the shape it is for too much size and three a number of channels. So a number of jealousies three and output neurons will be ten. It means that we need to classify to 10 classes.

So again let's create our continent says send it to CUDA and check that if we apply our network to some images it will be it will have the output size four to ten. For stands to batch size and then stands to number of crises and we have some criterion we just cross entropy loss. We have an optimizer and let's run our network for five bucks Ricky. Our train was finished and now let's check the quality. So now we have fifty nine percent if you remember when we trained multilayer person Don we had forty five percent. So we improve. Tell us corn in the next practical videos will improve our score more and more applying on practice what we learned in lectures.

Thank you for watching and see you in the next video.

LECTURE 18

Overfitting and Regularization

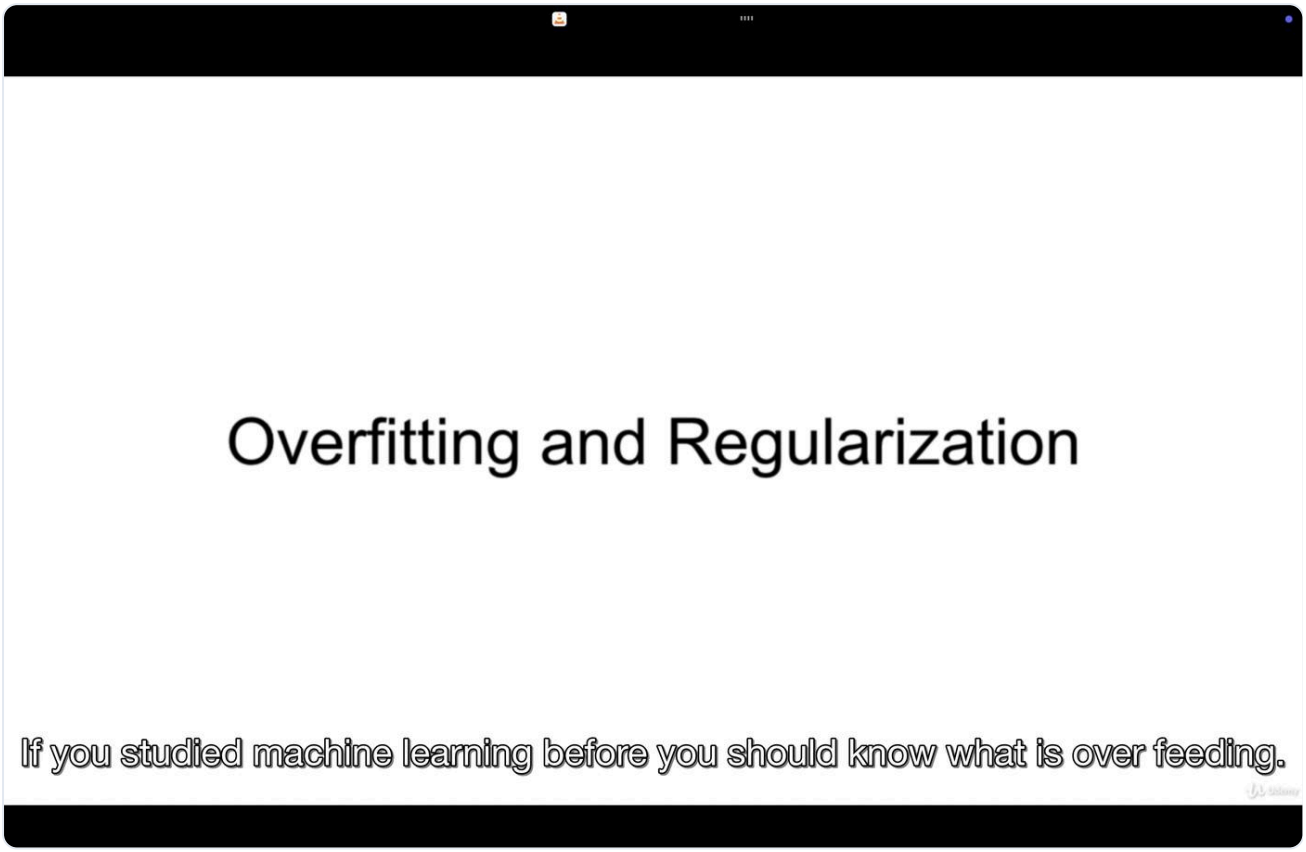


FIGURE 18.1 | Overfitting and Regularization

If you studied machine learning before you should know what is overfitting. Also you should know how to avoid overfitting and what is regularization. However there are a number of regularization techniques that are good for deep learning and image processing.

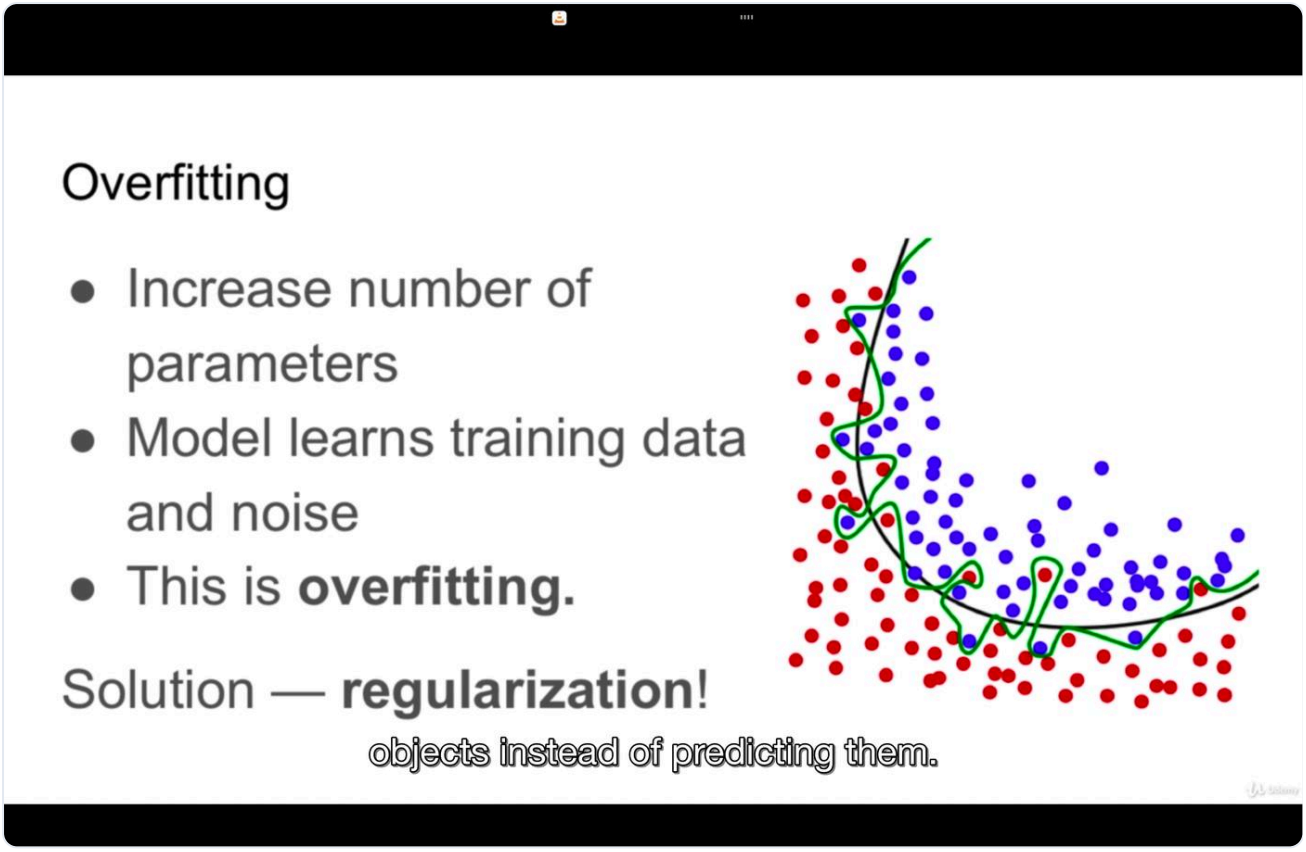


FIGURE 18.2

From the course of machine learning you should know that when you increase the number of parameters in your model it starts to overfit overfitting means that your model learns the answers for the training objects instead of predicting them. This leads to bad behavior of your model on the test set. The prediction will be very bad their solution to the problem of overfitting is a regularization technique. You may ask me why should we regularize the network.

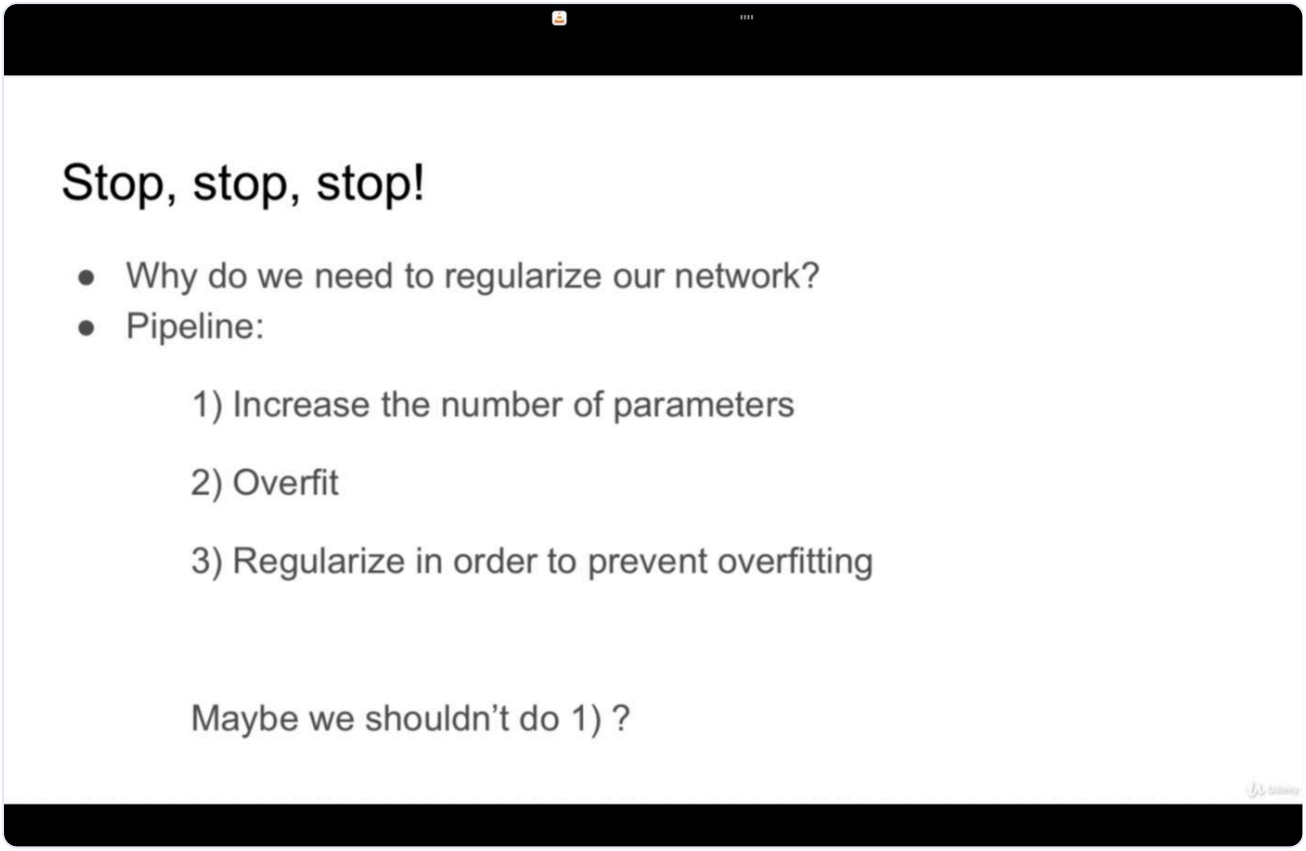


FIGURE 18.3

Maybe it is better not to increase the model size. So we have two possible approaches deeper network work with regularization or smaller network without regularization. Let's think our loss function is non-convex. It means that there are multiple local minimums of the loss function but the local minimum is usually good for us. Yes it is better to reach the global optimum but now optimization methods for deep learning can do that.

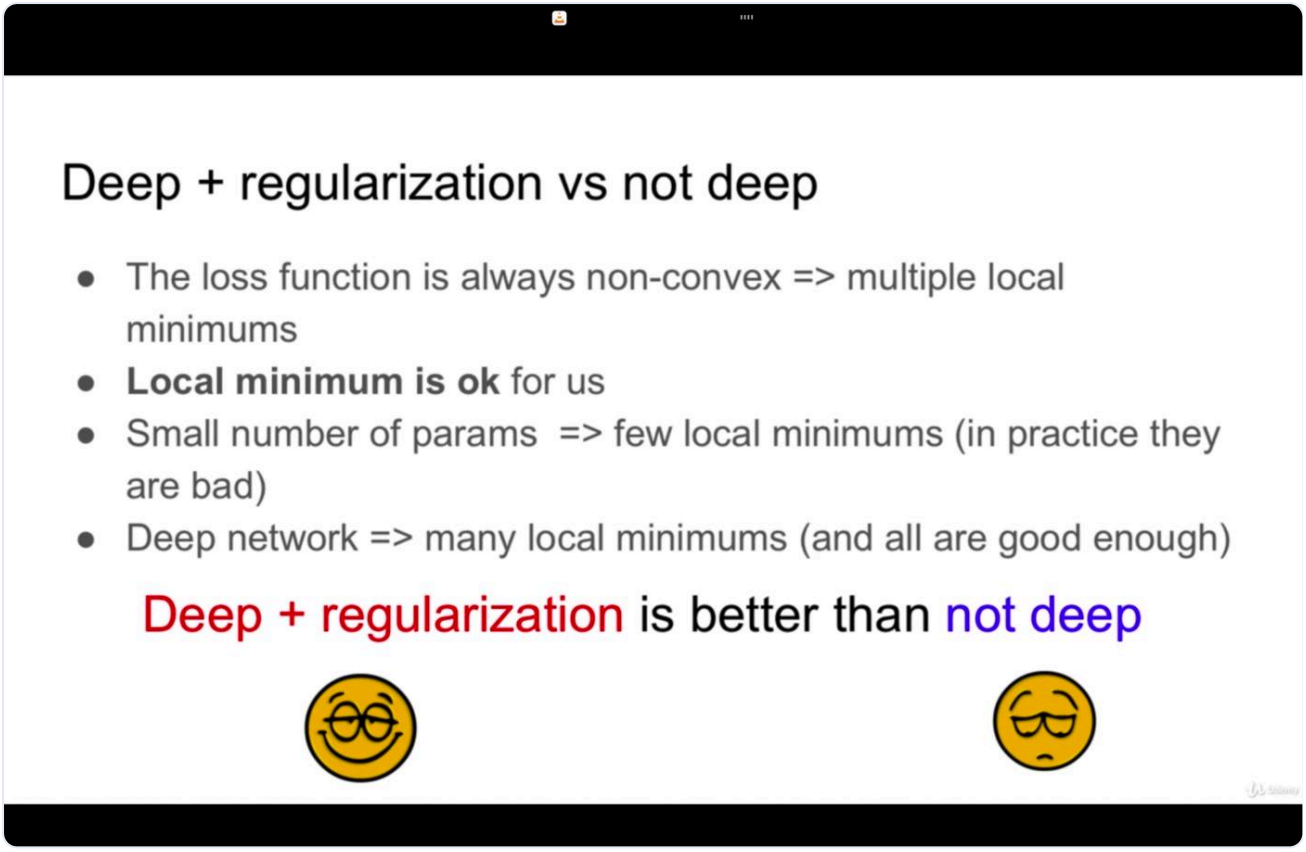


FIGURE 18.4 | Local Minimum Is Ok for Us

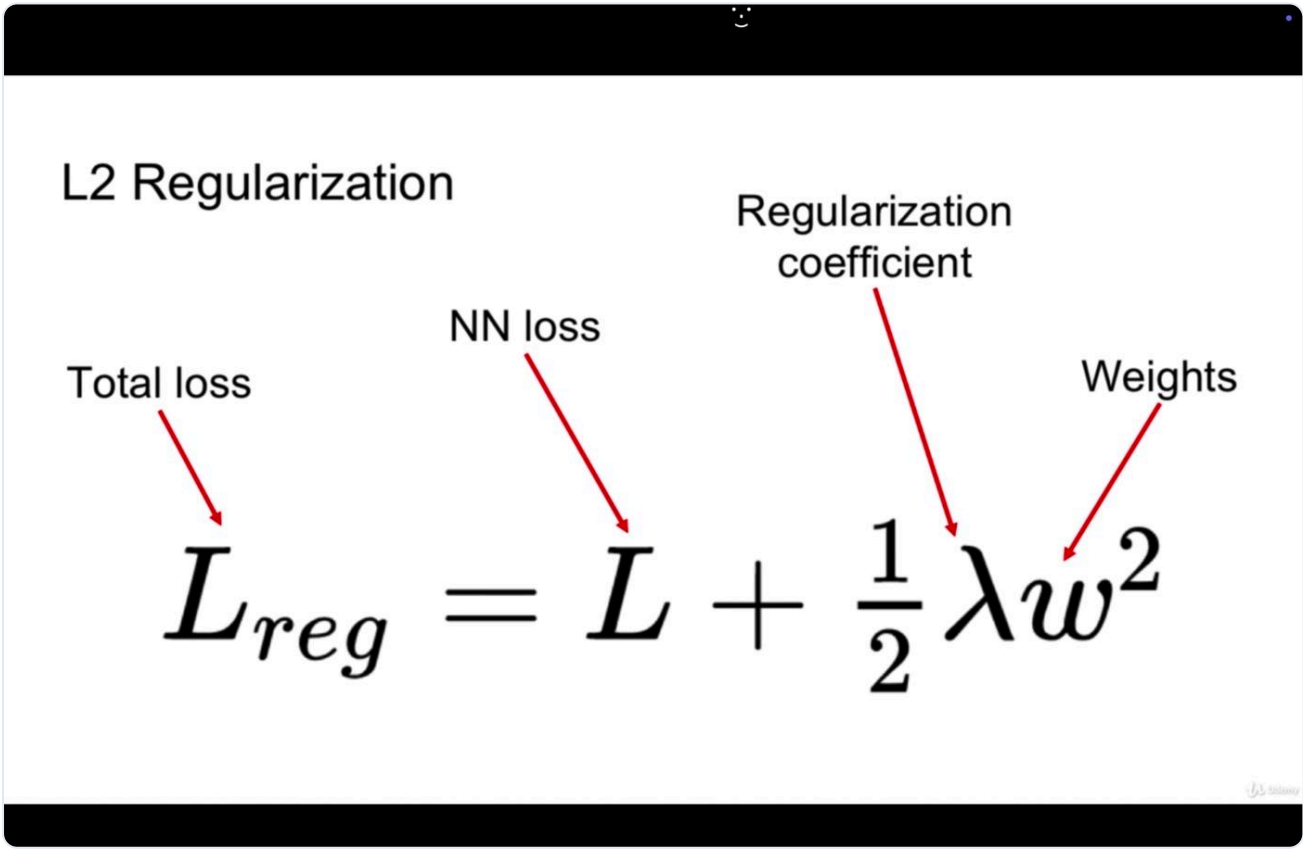


FIGURE 18.5

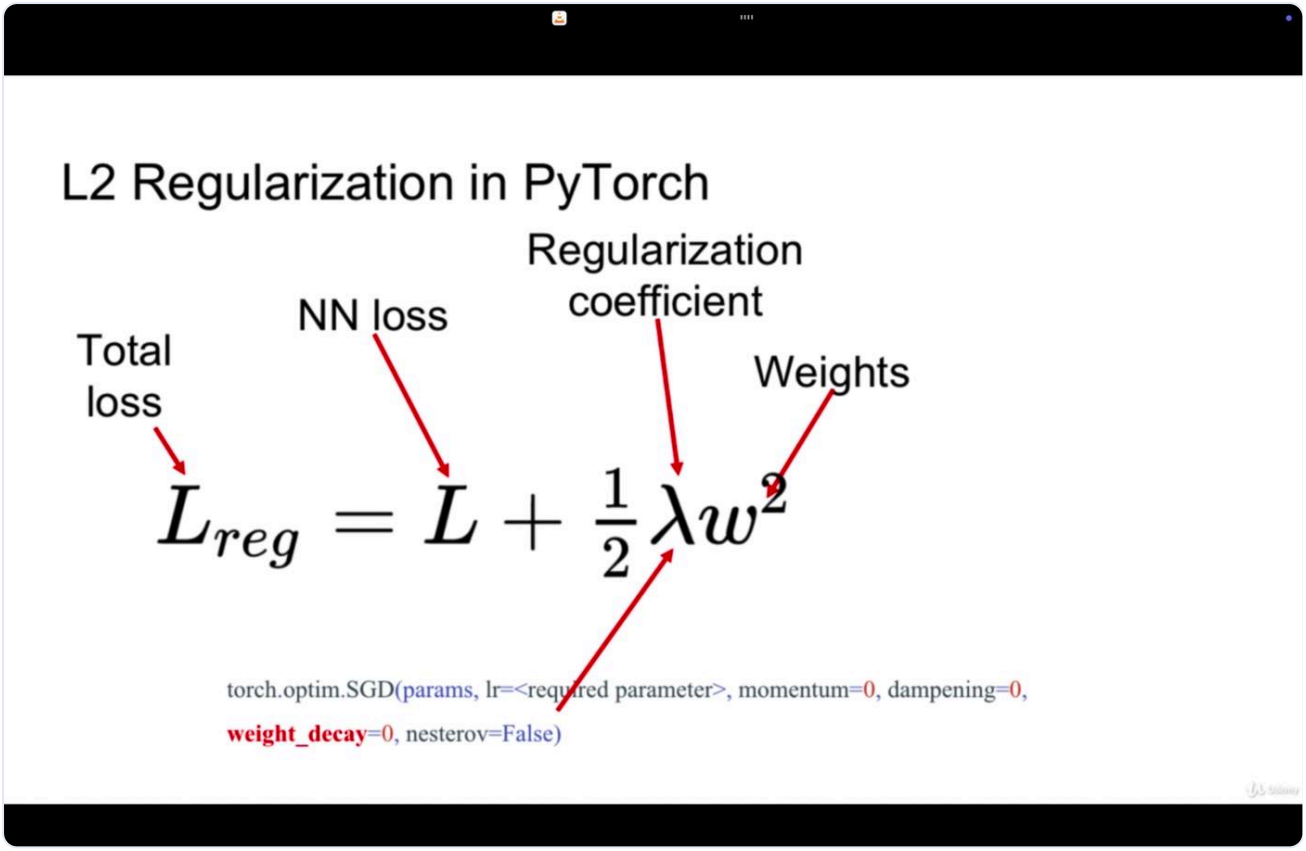


FIGURE 18.6 | L2 Regularization in PyTorch

If we have a small number of parameters then we have a few local minimums and in batches they are bad. If we increase the number of parameters making our network bigger then our loss function will have more local minimums and usually in practice it is better their conclusion is simple. It is better to have a deeper network with regularization than small network without it of course you should understand that this is practical results and in your concrete case the situation may be different. The first method of regularization comes from that classical machine learning L2 regularization. The idea of it is the same as in ridge regression which is a name of linear regression with L2 regularization.

We have some loss function for our neural network and we add that regularization term to the optimization problem. This regularization term consist of the multiplication of lambda to wait lambda E the regularization coefficient which should be tuned on validation set. The more lambda parameter the closer our weight will be to 0 in order to implement such realization in PyTorch you can use the parameter weight_decay in-depth simulation algorithm definition. This is absolutely the same parameter as the lambda in our formula.

LECTURE 19

Dropout Regularization

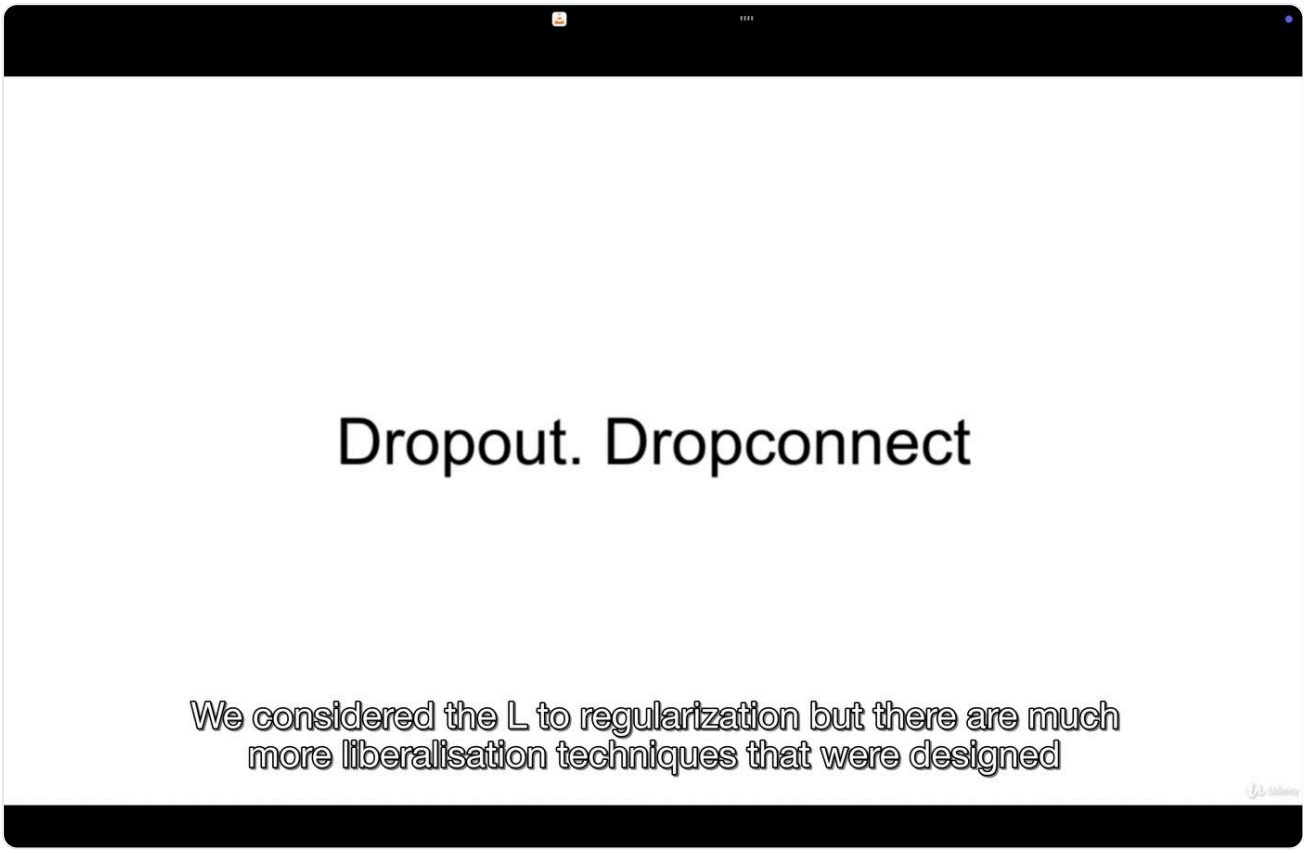


FIGURE 19.1 | Dropout. Dropconnect

We considered the L2 regularization but there are much more regularization techniques that were designed specifically for deep learning the first diploma in organization technique is dropout. It is easy to understand it on their multilayer perceptron on the left will have their original multilayer oppressive drawn with several layers on the right will have the same network but with a bolide dropout.

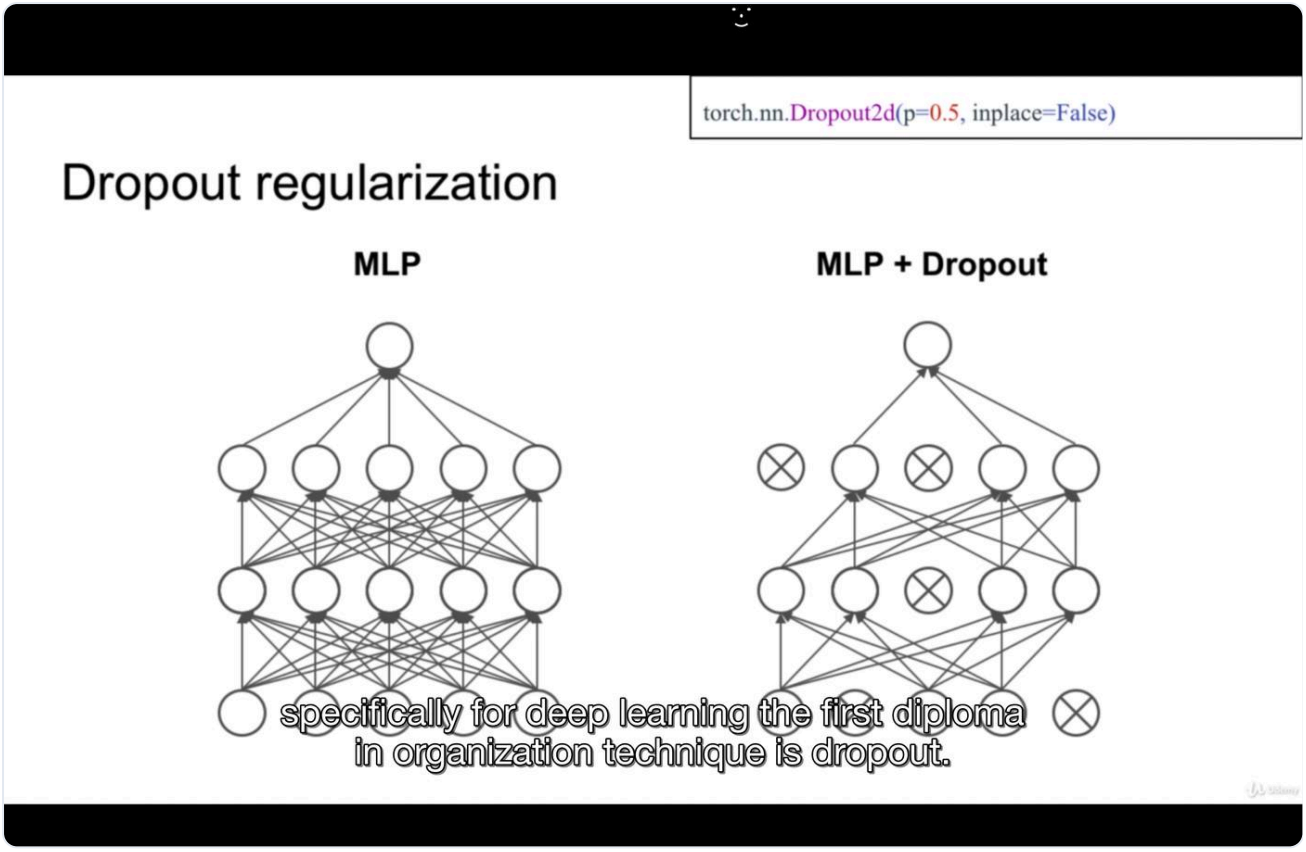


FIGURE 19.2 | Dropout Regularization

Look we turned off some of the neurons.

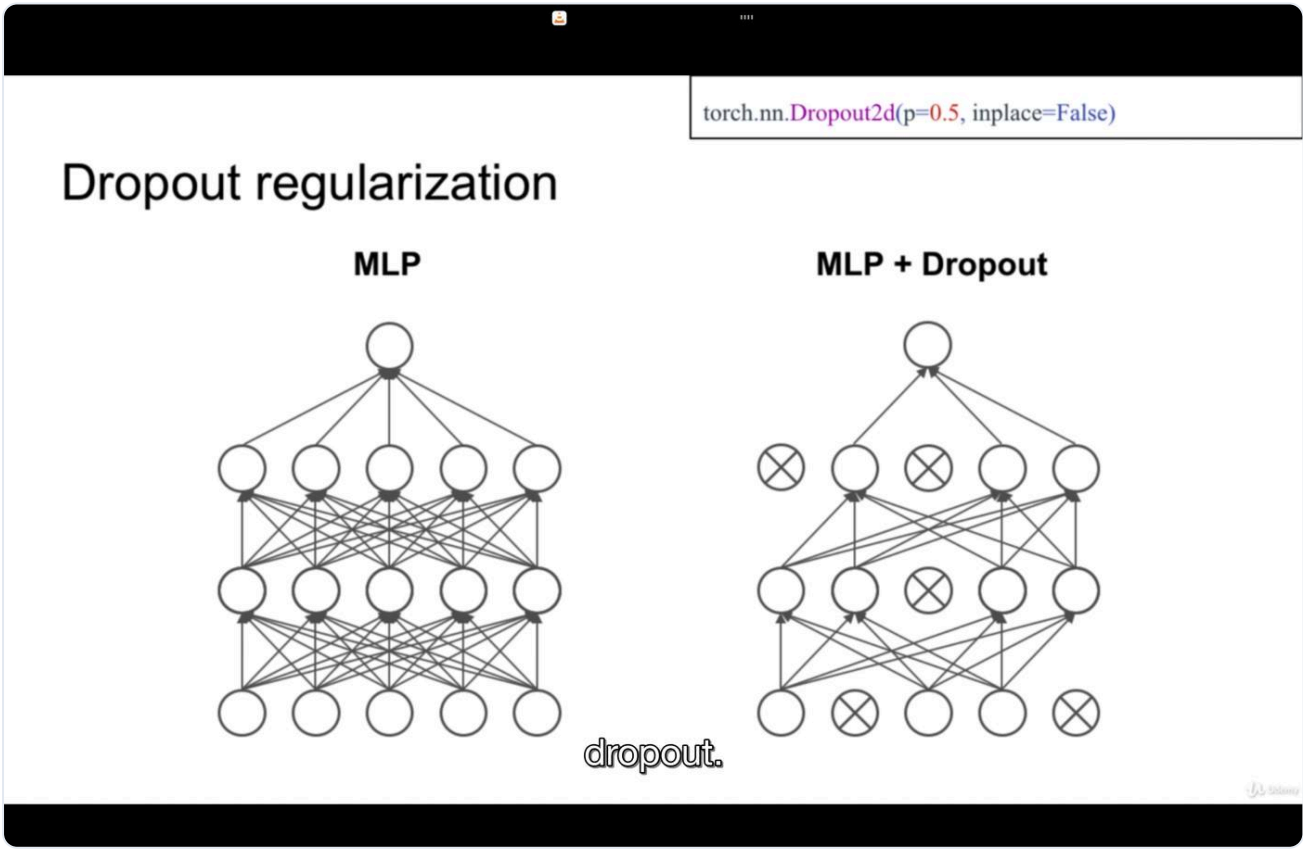


FIGURE 19.3 | Dropout Regularization

In other words we set some input features to zero. And the important thing we did it randomly.

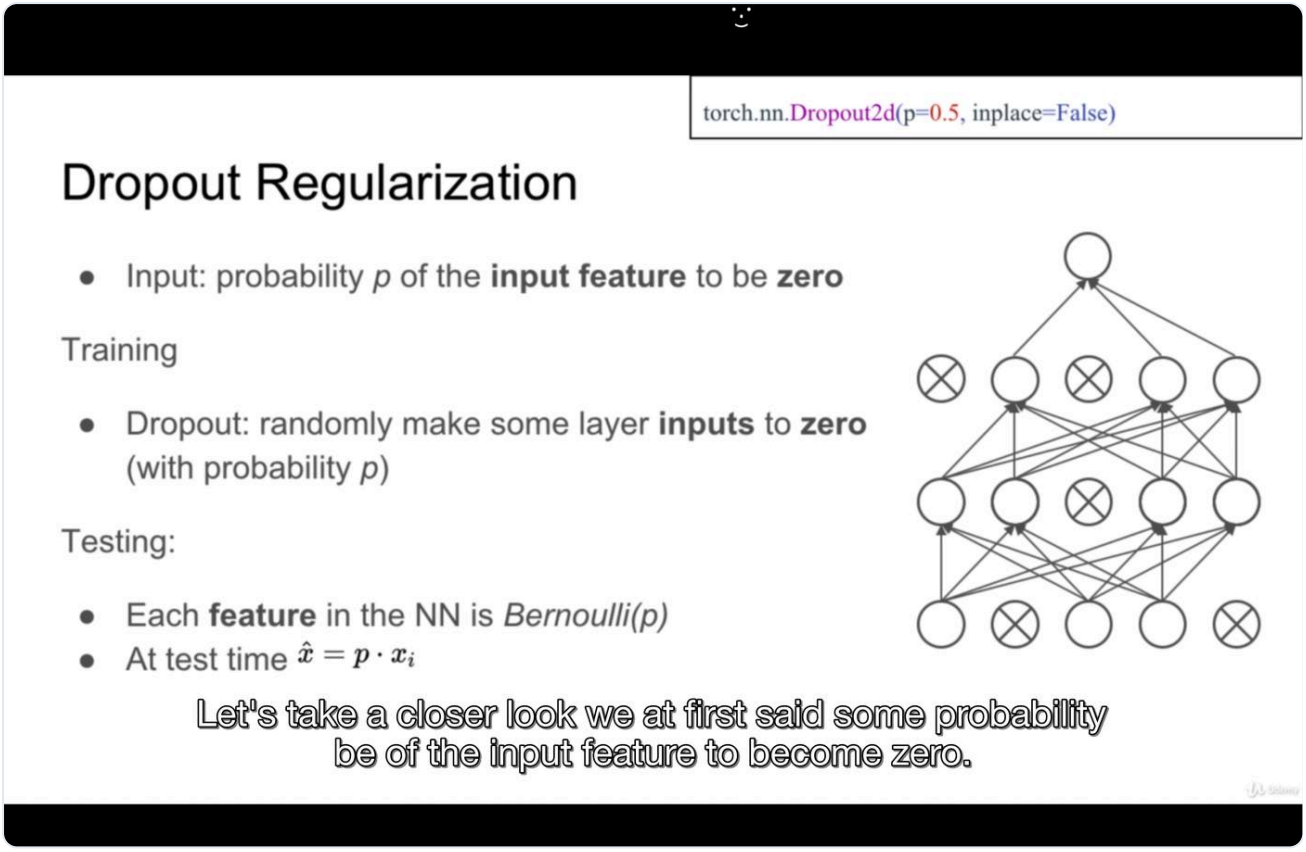


FIGURE 19.4 | Dropout Regularization

Let's take a closer look we at first said some probability p for an input feature to become zero. Now we have two cases. Training and testing. During training we set some features to zero with probability p on each forward pass that is on each iteration. We choose different features we can look at it another way we generate their new mask and multiply it by the features during testing. As we sample the mask from the Bernoulli distribution we should take the average so we multiply our input features to the probability B dropout is usually implemented as a separate layer which randomly sets features to zero.

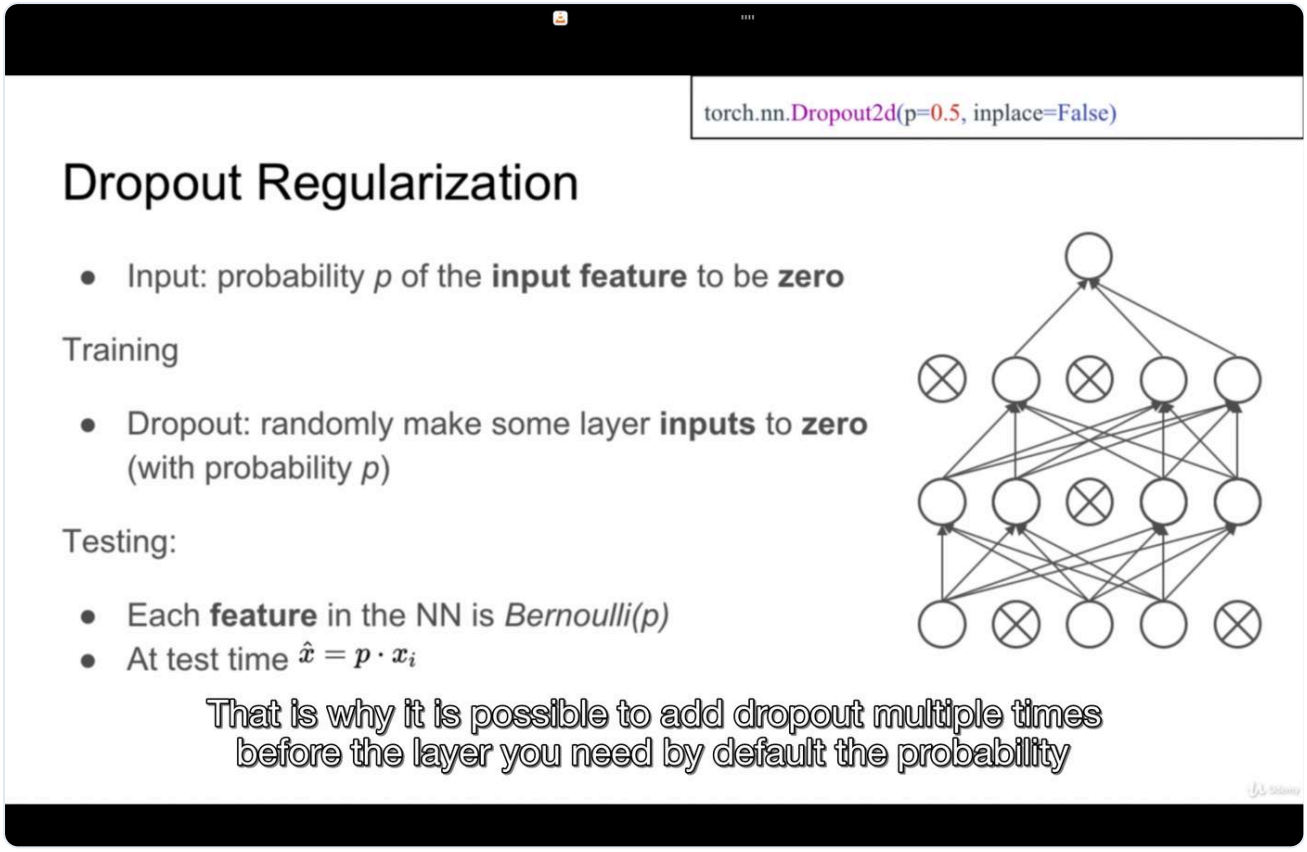


FIGURE 19.5 | Dropout Regularization

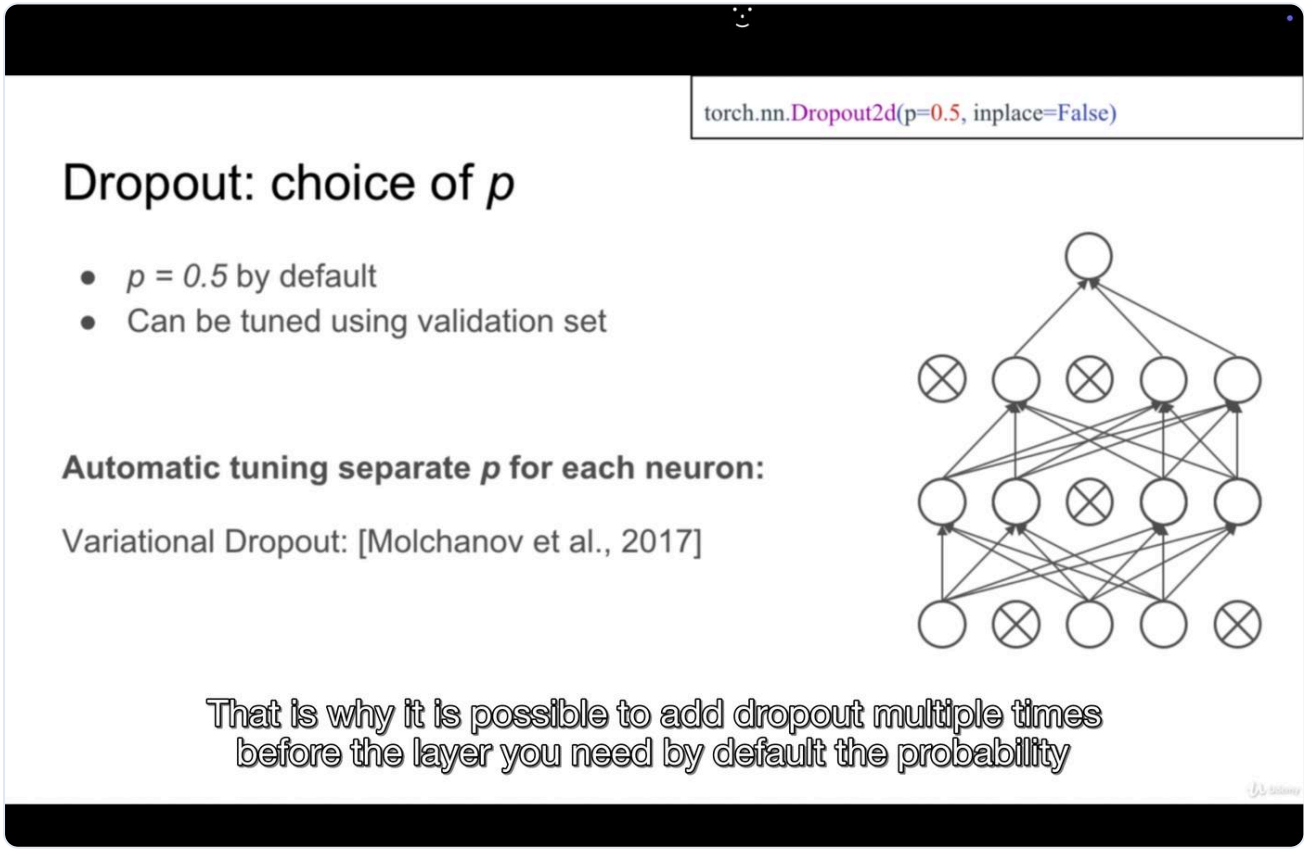


FIGURE 19.6 | Dropout: Choice of P

That is why it is possible to add dropout multiple times before the layer you need by default the dropout probability is set to zero point five however it can and in fact should be tuned on the validation set. Also there exist some research papers that describes the way of automatically choosing that separate probability P for each feature. If you are interested in it read about the variational dropout dropout can be used as a good regularization technique but what if you want to train and ensemble of neural networks as you may know from their machine learning course and ensemble of predictions. For example bagging and boosting or random forest always works better than the single algorithm.

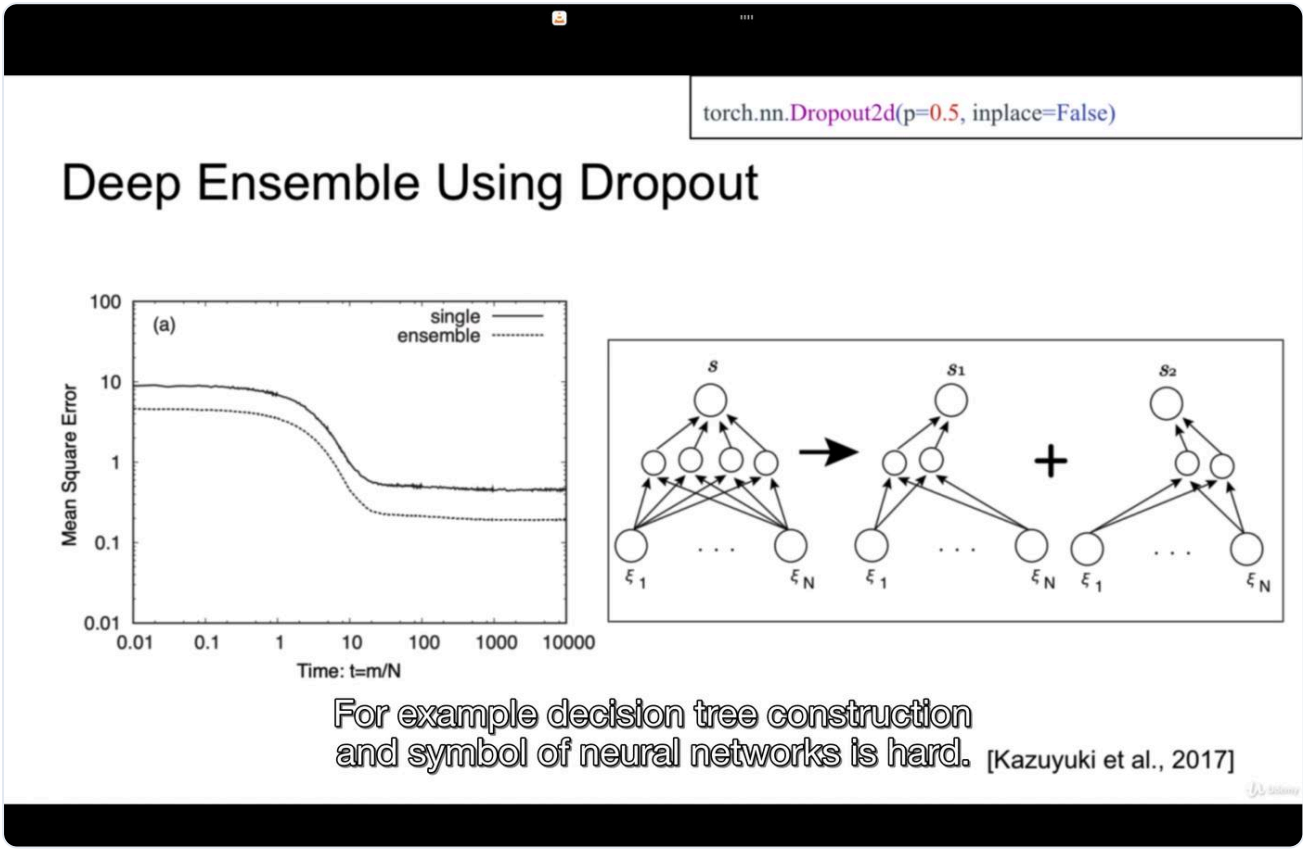


FIGURE 19.7 | Deep Ensemble Using Dropout

For example decision tree construction and symbol of neural networks is hard. It will require a lot of computational time and memory however dropout comes to help here scenes for each forward pass during training. ensemble a random Bernoulli mask. We have a new neural network on each duration that is different from the previous one and we can use these different networks as an ensemble. For example every June their predictions and good news is that we don't need to store them separately. We just put the same data to the network input each time randomly making some inputs for some layers to zero authors of the sides. The article showed that such and dropout and ensemble usually works better in terms of prediction quality.

The other regularization technique is DropConnect which is also can be used in deep learning.

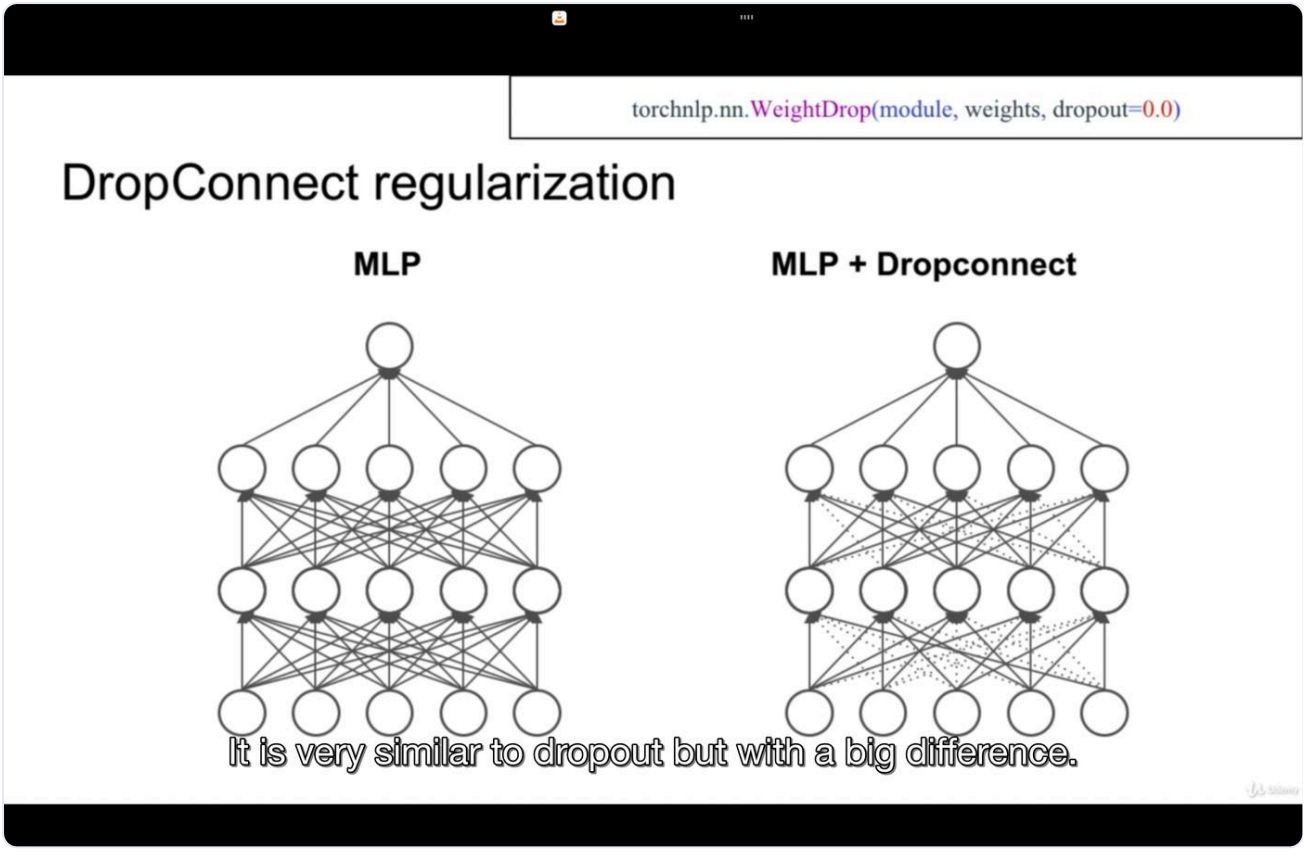


FIGURE 19.8 | DropConnect Regularization

It is very similar to dropout but with a big difference. While dropout said some features do zero DropConnect said some weights to zero. The pipeline is the same as in dropout.

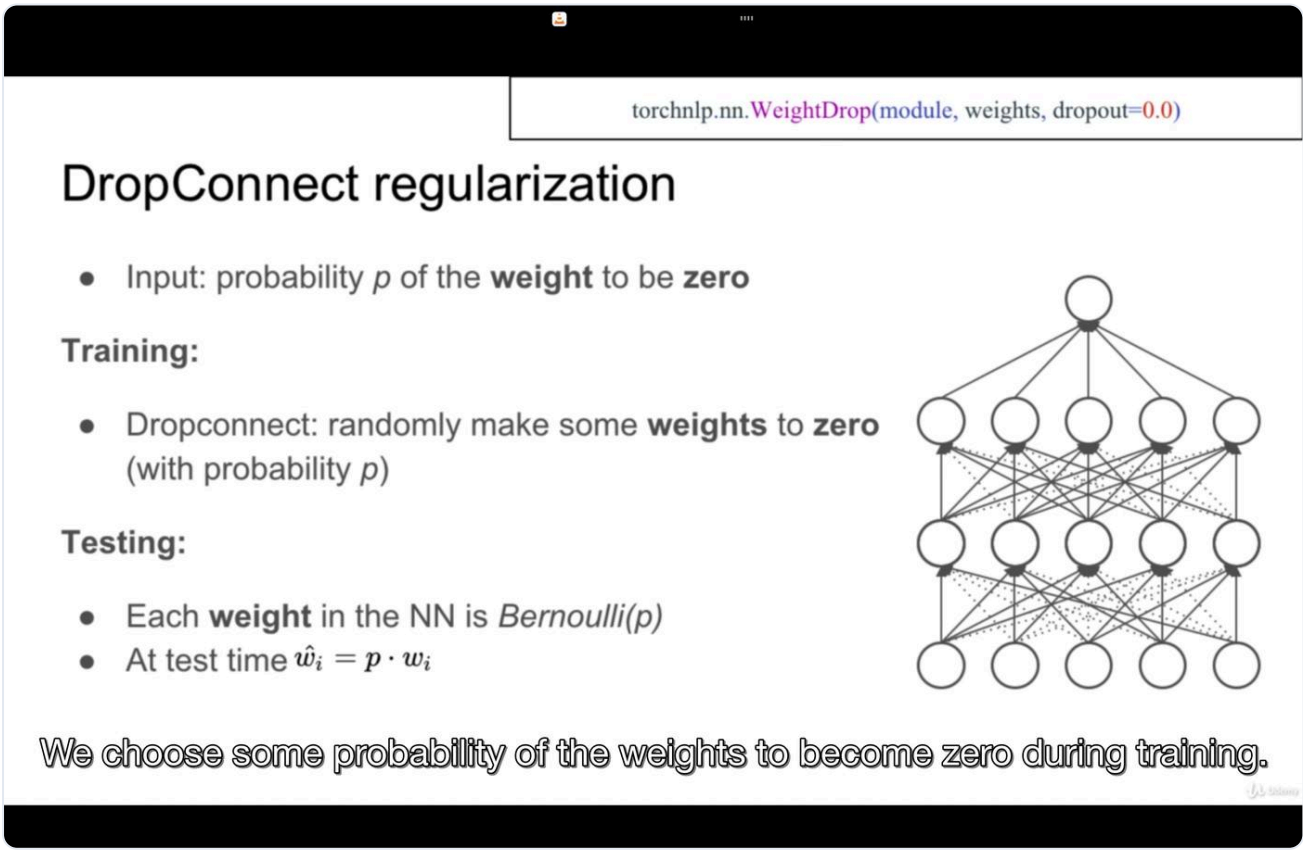


FIGURE 19.9 | DropConnect Regularization

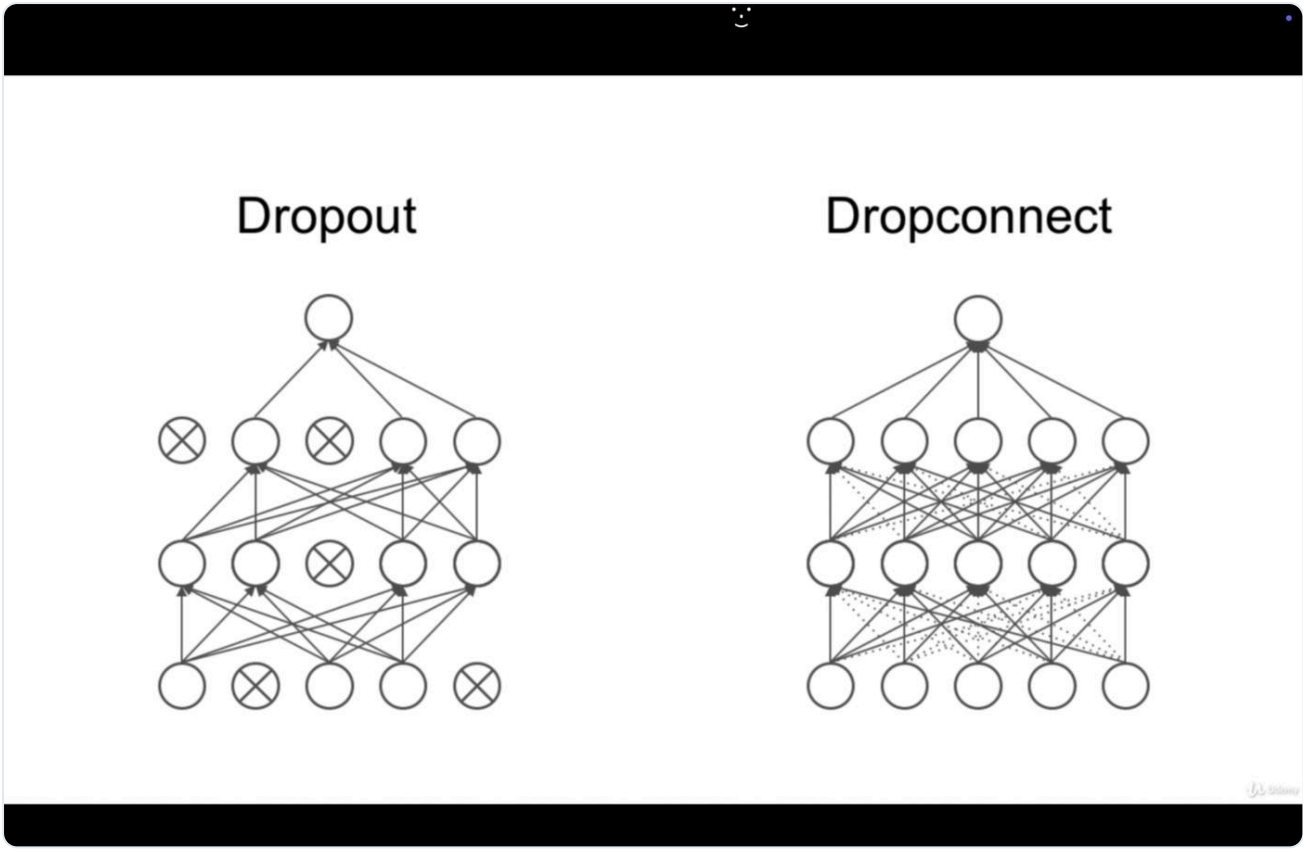


FIGURE 19.10 | Dropout Dropconnect

We choose some probability of the weights to become zero during training. We set the weights to zero according to the selected probability during test time. We multiply our weight on the probability P. Here is the comparison of dropout and drop connect methods. Although the difference between them not that much people prefer to use dropout.

LECTURE 20

Dropout DropBlock

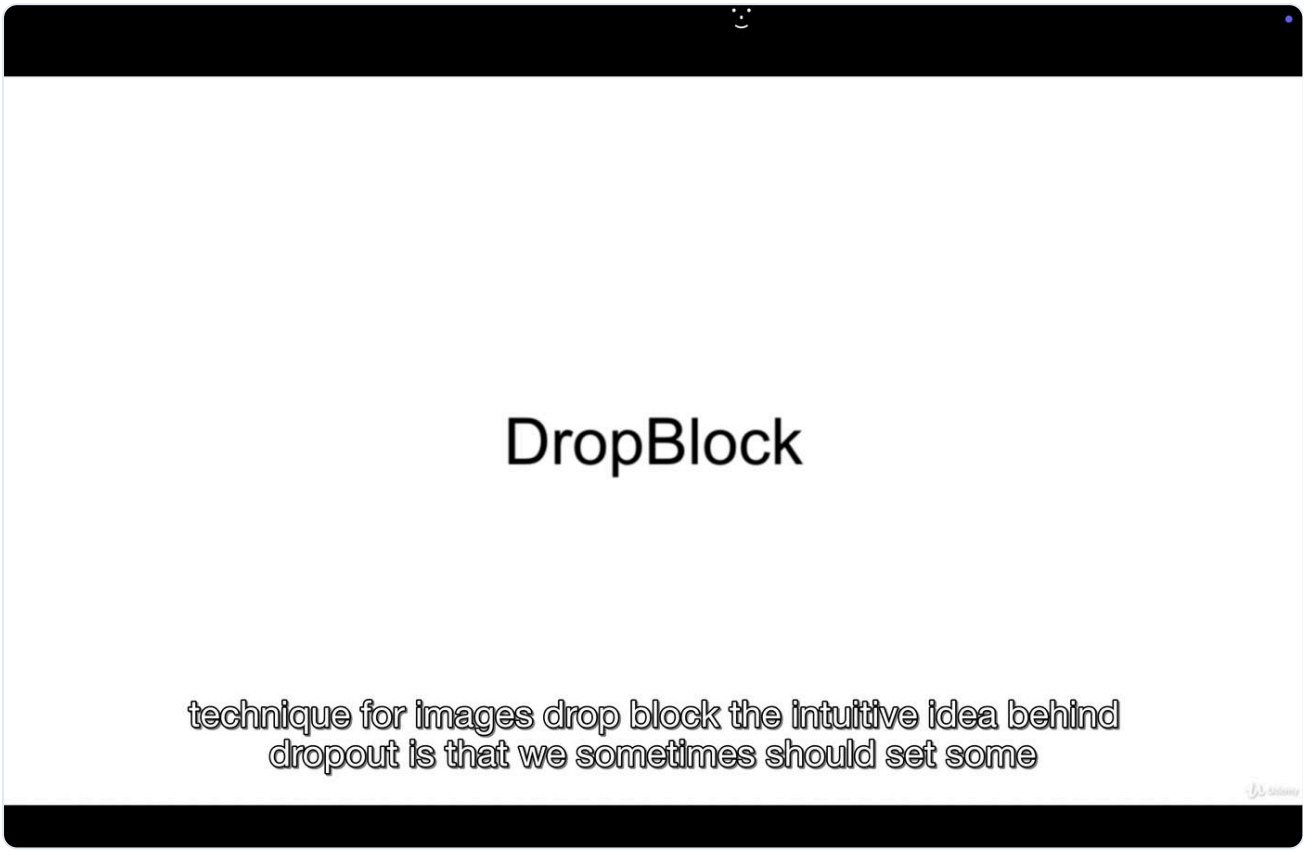


FIGURE 20.1 | Gropout Is That We Sometiines Should Set Some

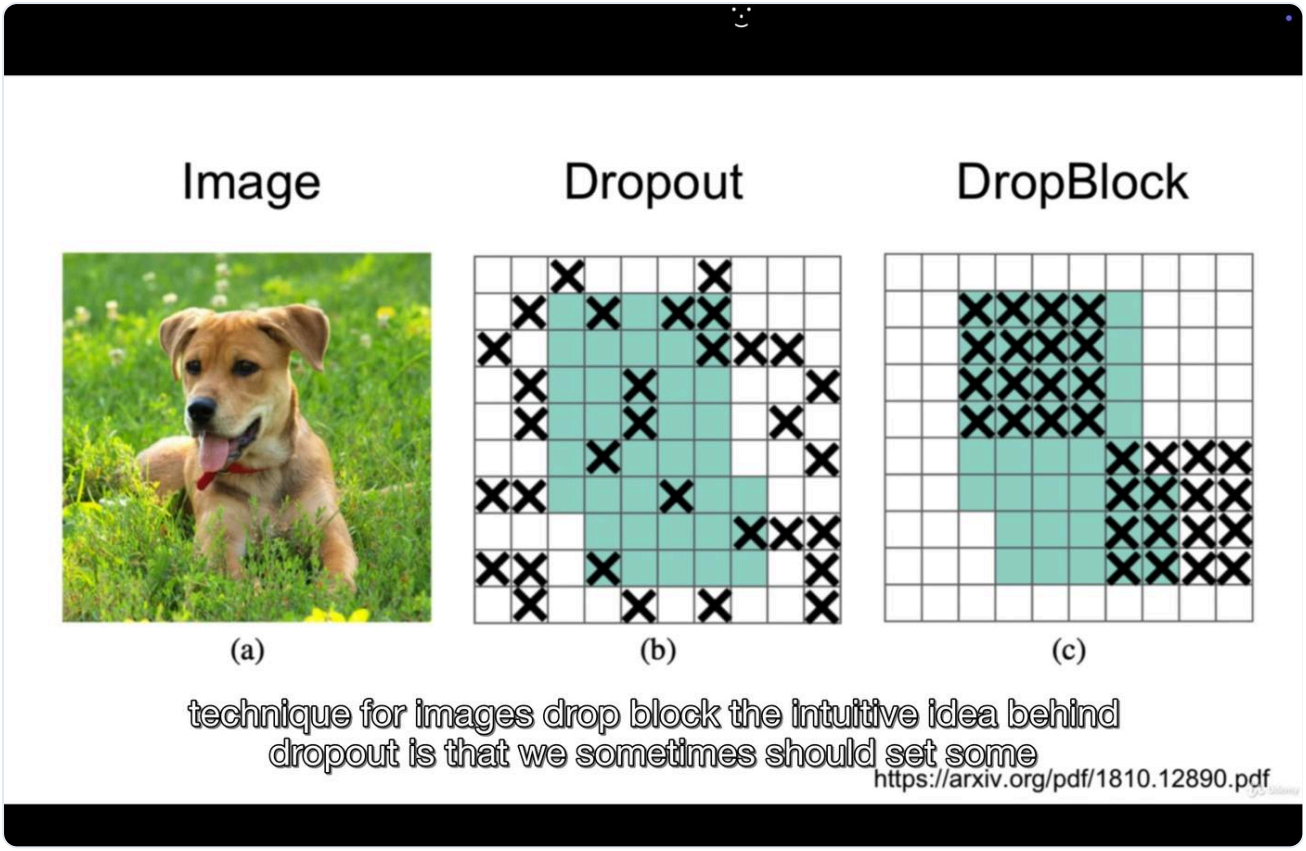


FIGURE 20.2 | Dropout DropBlock

Since we study more than deep image processing I have to share with you recently proposed regularization technique for images drop block the intuitive idea behind dropout is that we sometimes should set some pixels to zero but we know that usually on the image there are only a limited set of pixels that contains useful semantic information. For example for the dog photo useful region is green on the middle. Image if we learn a classifier with two classes gets a dogs then we don't need the information that surrounds the dog which is in these photo or the grass. The disadvantage of the dropout as our organization technique is the following. If we said pixels of the grass to zero then it will not affect the prediction results at all.

Since the grass is useless information in other words sometimes our organization will not work because we want our regularization method to debrief our network for solve semantically important information during training only. It will help it in preventing learning.

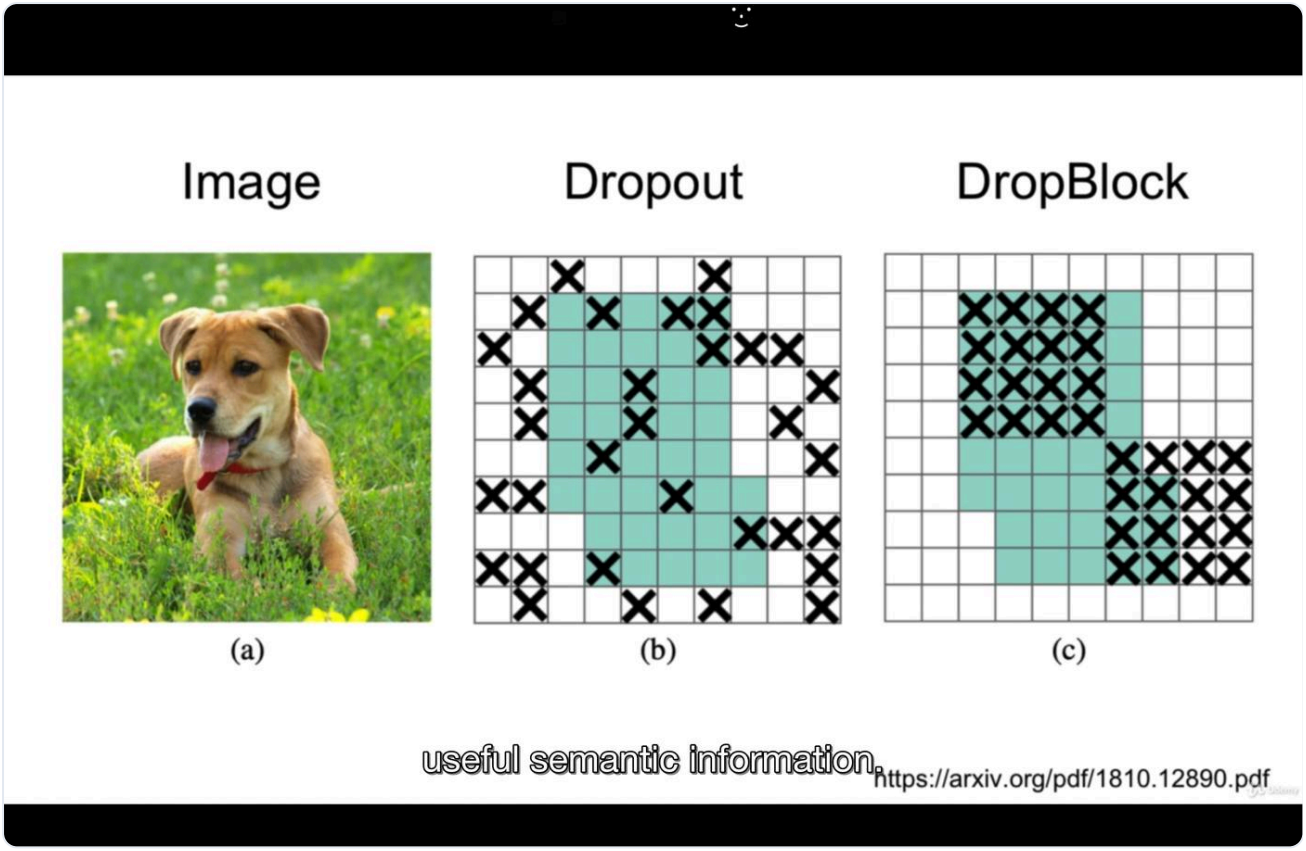


FIGURE 20.3 | Dropout DropBlock

The training samples and here come the DropBlock algorithm on each training iteration. It said that random square of pixels to zero as it's shown on the right image you can think of it in the following way. If our network could recognize the dock on the footer It is also should be able to recognize a dock without a near or without a month and this is the idea behind the book. How it should be implemented in fact rather simple at first we set the land on the side of a square which is called block size which also should said the probability over the block with the central at each pixel to become zero then no in the block size we can select a region which potentially can contain the center of the block.

Of course we can store in the center of the block to the corner of the image. This region is green on the left image. Here we have the block size equal to 5 after Z. We generate absolutely massive size that is equal to the green area.

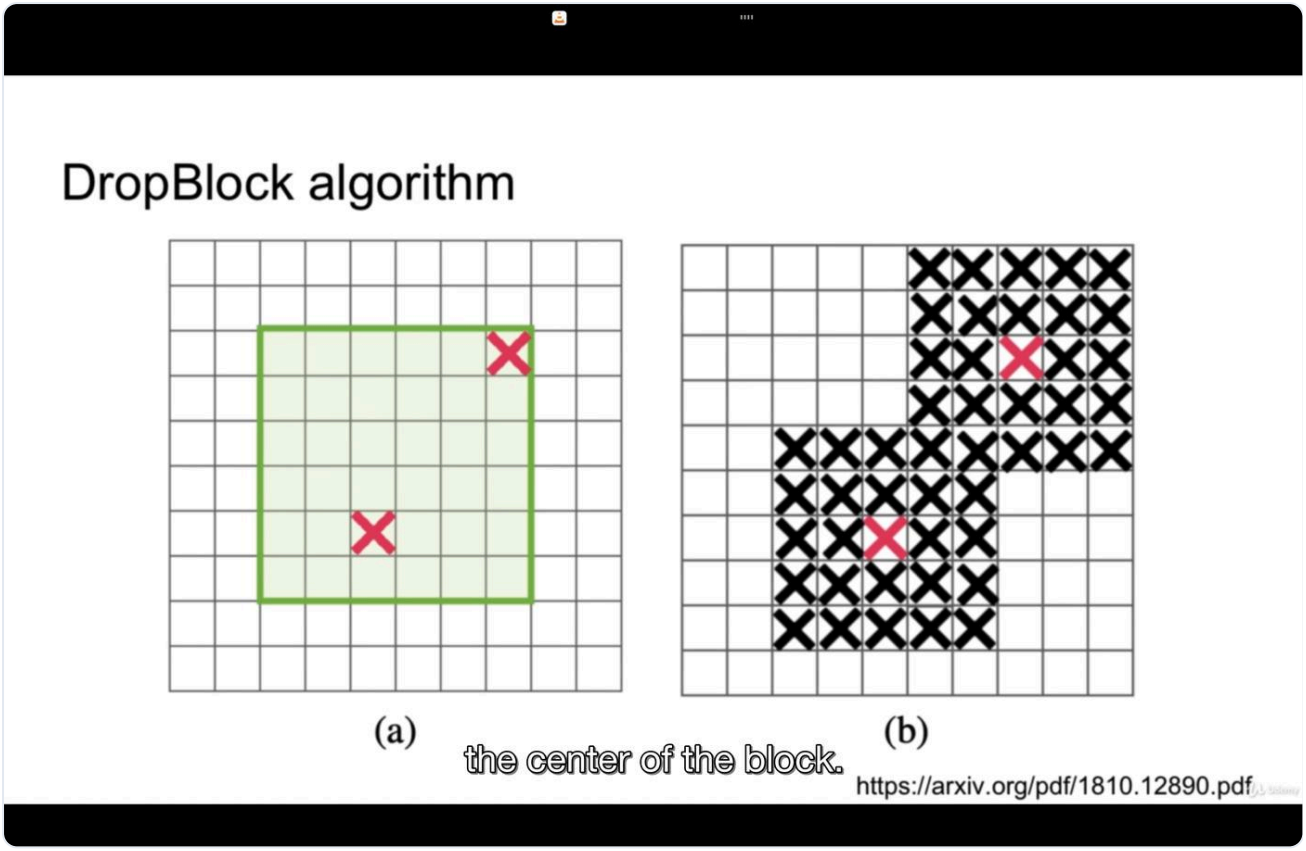


FIGURE 20.4 | DropBlock Algorithm

For that we renew our mask. We use the probability that we set up before as you see on the left image there be renewed newly mask. Give us two centers of block to make zero. And on the right image you can see the result of the DropBlock. Some pictures became zero as in their dropout. The DropBlock is random on each of the region and don't play his role during testing. If you want you can read the original paper by the link in the right corner on the slide.

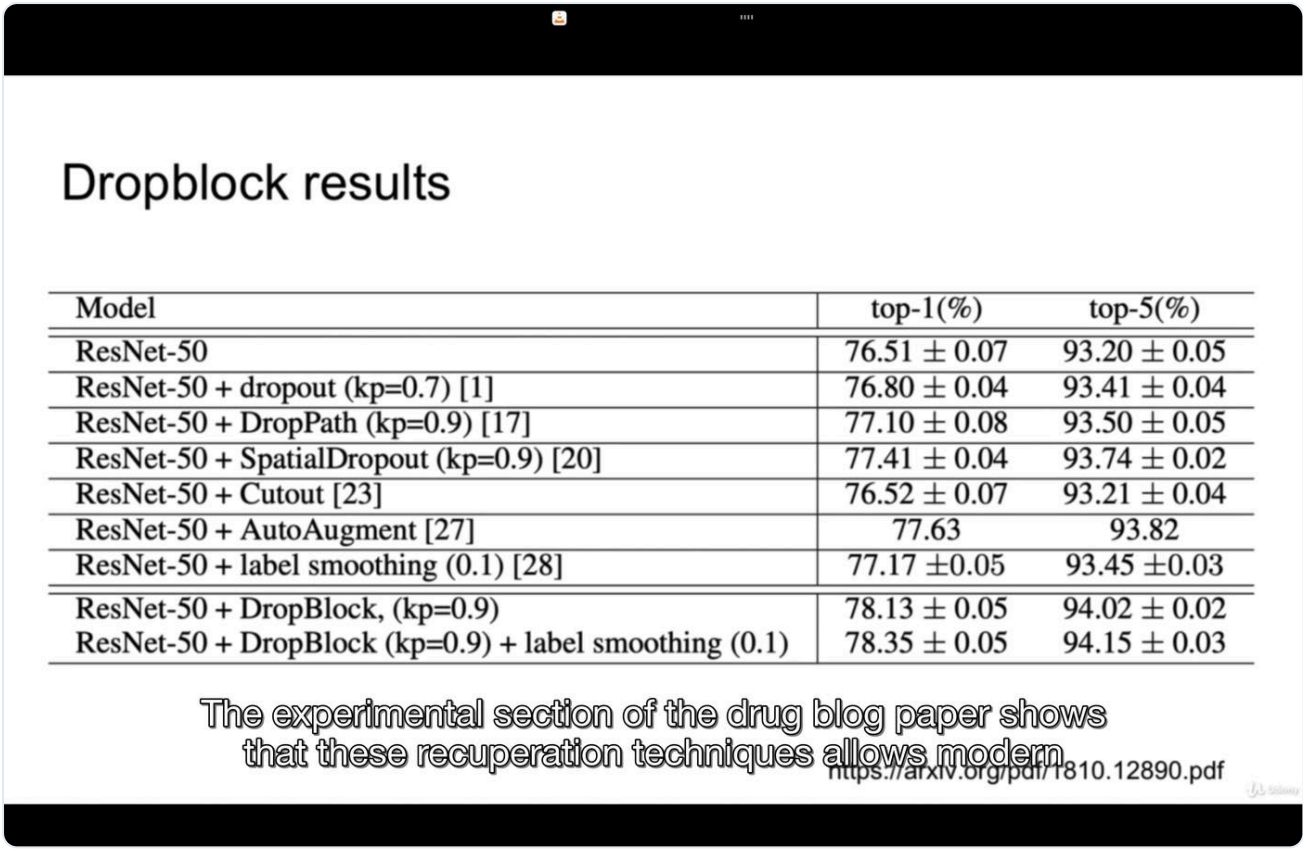


FIGURE 20.5 | Dropblock Results

The experimental section of the DropBlock paper shows that these recuperation techniques allows modern architectures such as ResNet-50 which will study later in this course to overcome the same architecture with dropout in classification accuracy.



FIGURE 20.6

Here is another interesting visualization from the authors of this paper. Class activation mapping at the first row. There are input images and other three rows a regional model model with a draw block with block size 1 and with block size 7 Model 1 trained with DropBlock learns spatial distributed representations that induce high class durations on multiple regions. On the other hand the model with DropBlock tends to focus on one or few regions so here it is all four DropBlock. If you are interested please read the original paper.

LECTURE 21

Early Stopping

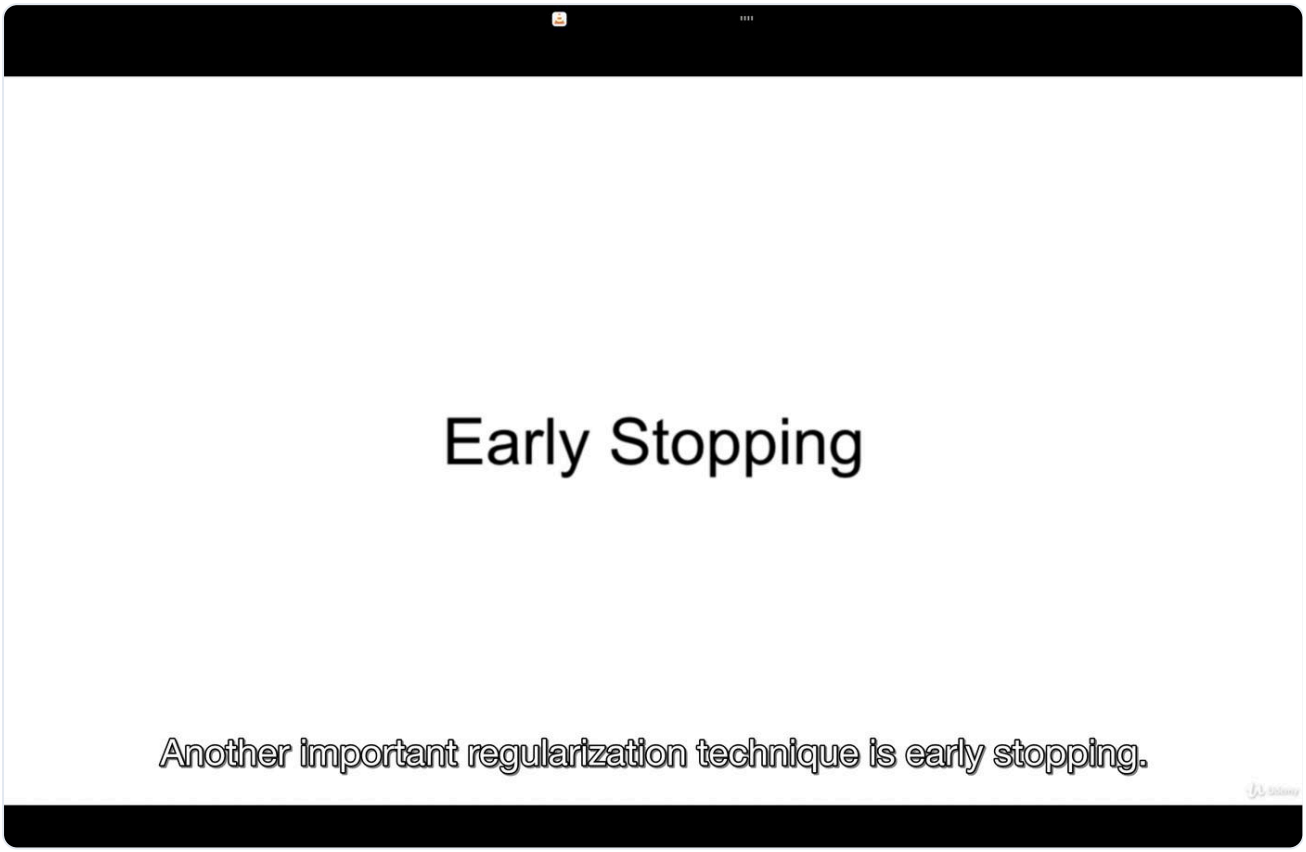


FIGURE 21.1

Another important regularization technique is early stopping.

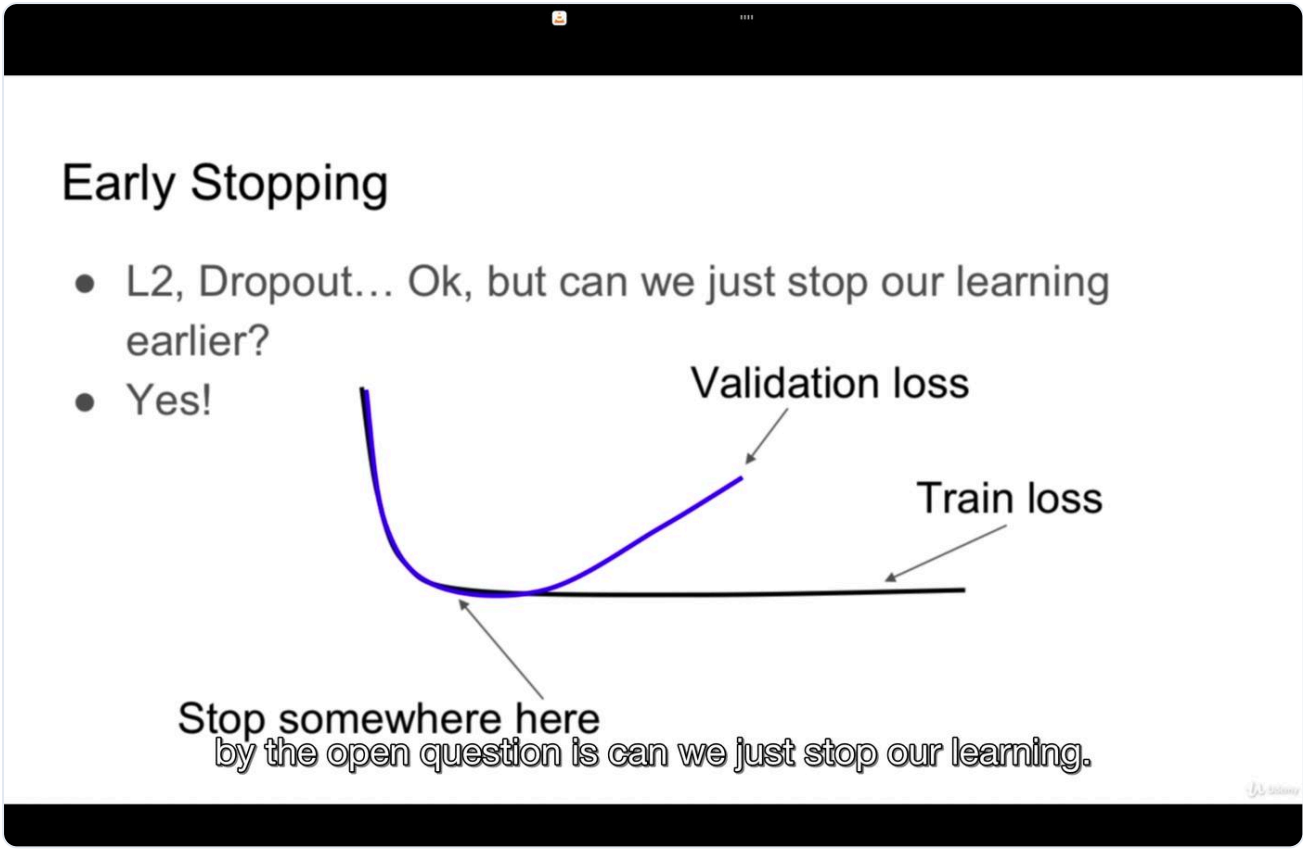


FIGURE 21.2 | Early Stopping

We already considered several other organization techniques such as how to do about DropBlock and others by the open question is can we just stop our learning. Earlier and the answer is yes we can and in fact we have to. Usually when you train your neural network you plot two metrics some loss function on your train set and tested. It may be the loss that you minimize or the accuracy of the classification or any other and while training you train error should always degrees and your tester degrees. Only some limited time and after that starts increasing. This is what is called the learning curve and I hope you studied it in machine learning course in order to stop earlier.

You should define where to stop your model.

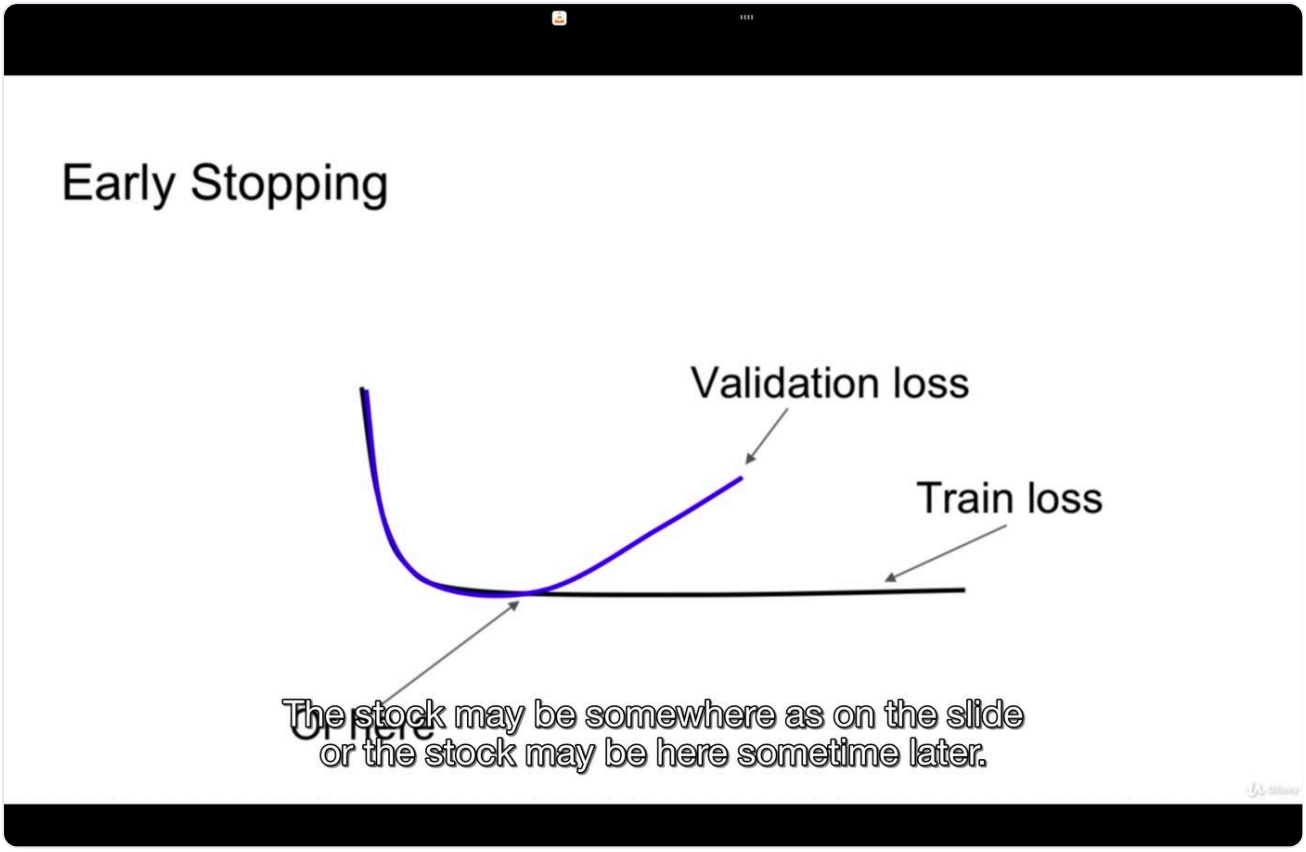


FIGURE 21.3 | Early Stopping

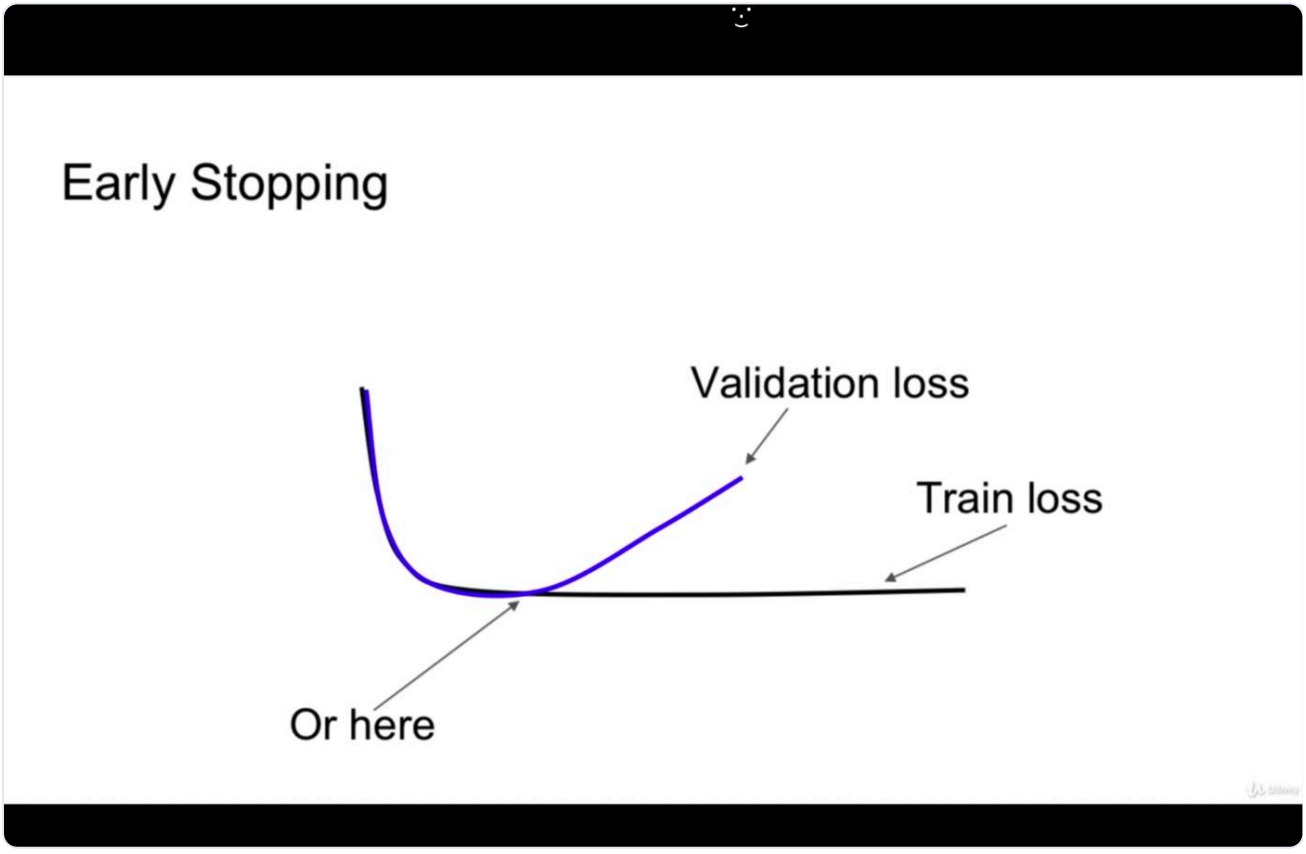


FIGURE 21.4 | Early Stopping

The stock may be somewhere as on the slide or the stock may be here sometime later.

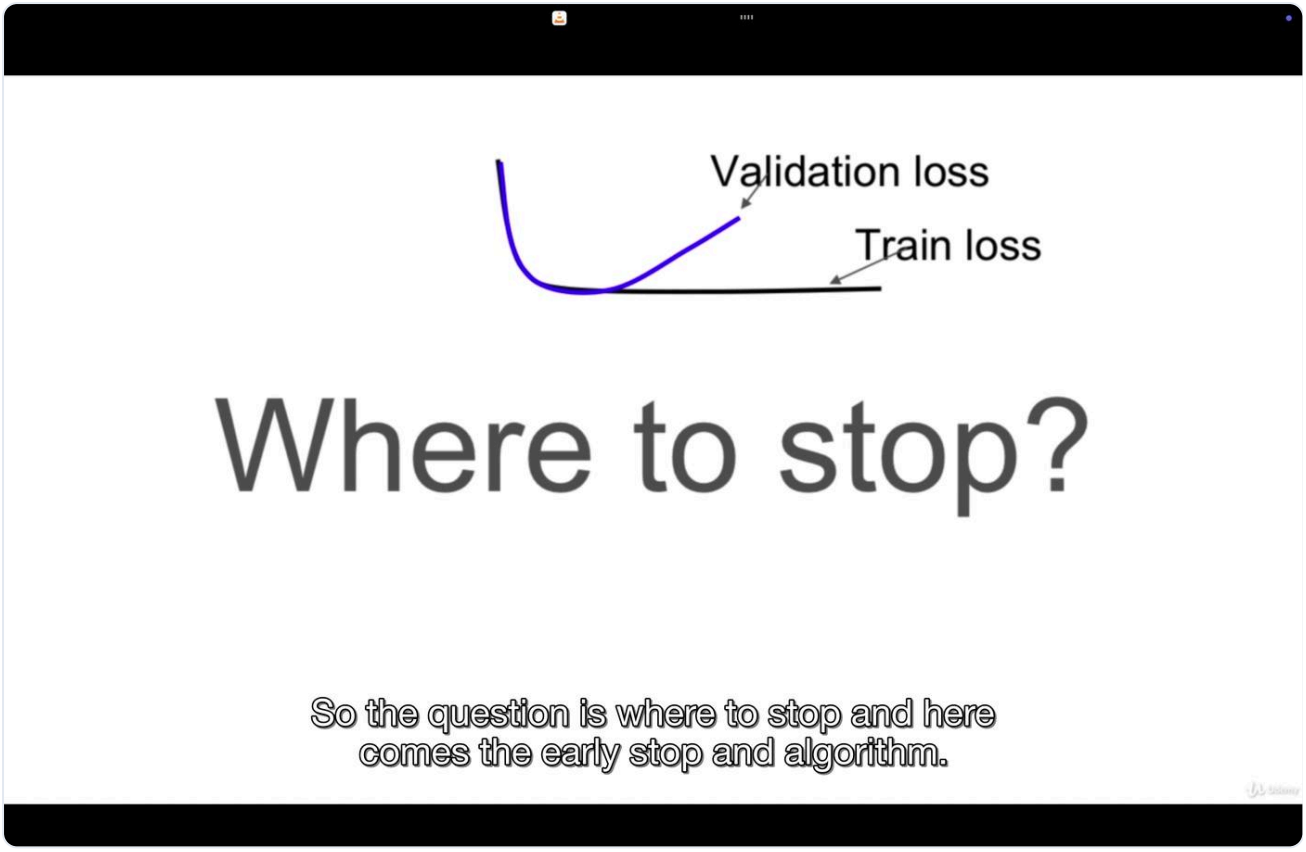


FIGURE 21.5 | Validation Loss

So the question is where to stop and here comes the early stop and algorithm. It is important to say that scene we use bias gradient descent at each iteration our laws can decrease or increase. This is normal theme that you will always see while training. That is why we need to introduce parameter and steps.

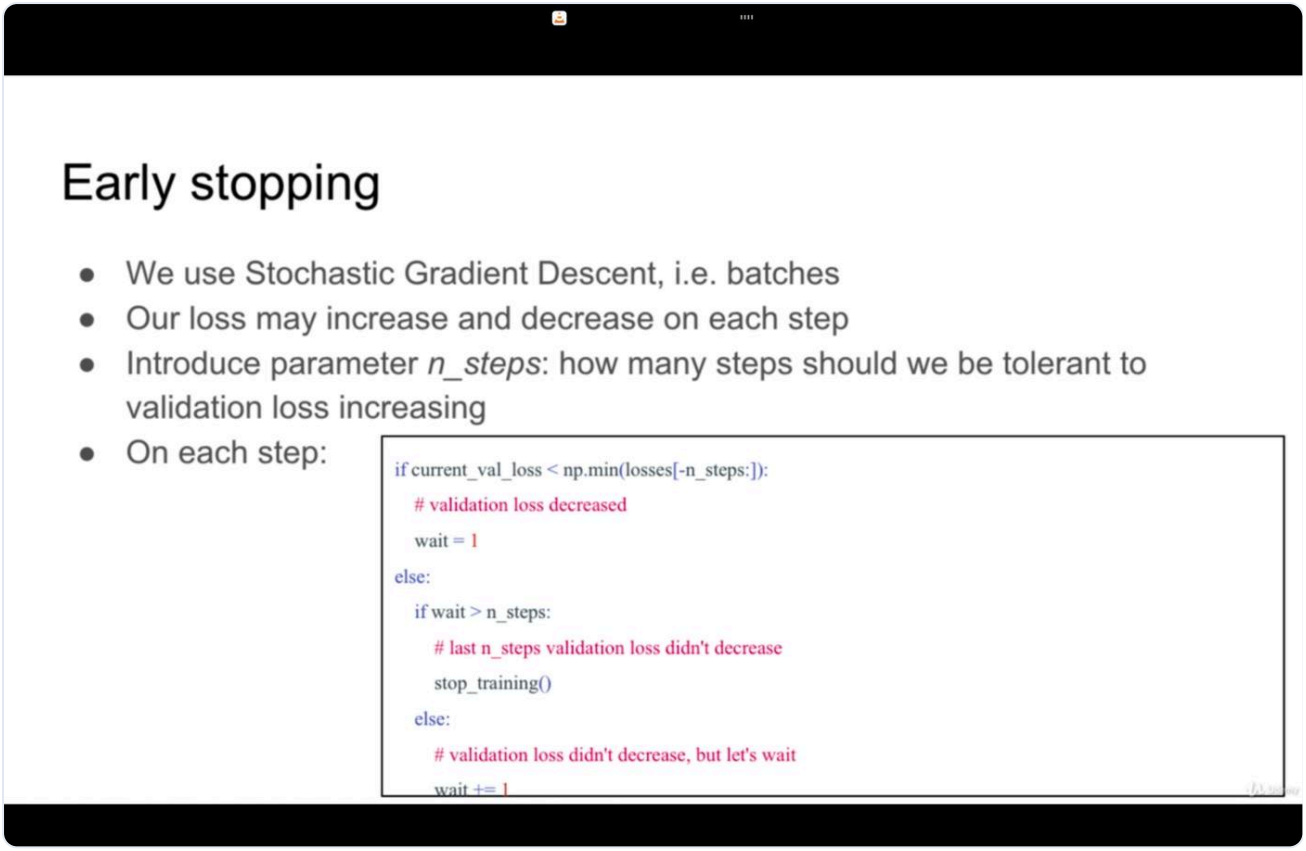


FIGURE 21.6 | Early Stopping

It means how many steps should we be tolerant to validation laws increasing in as award. How many iterations should we wait while the loss is not decreasing. Let us look at the algorithm implementation in Python at each iteration. We check if our will additional laws decreases comparing to last and steps. Of course if you can see their accuracy it should increased. So if the law is decreased then it is OK.

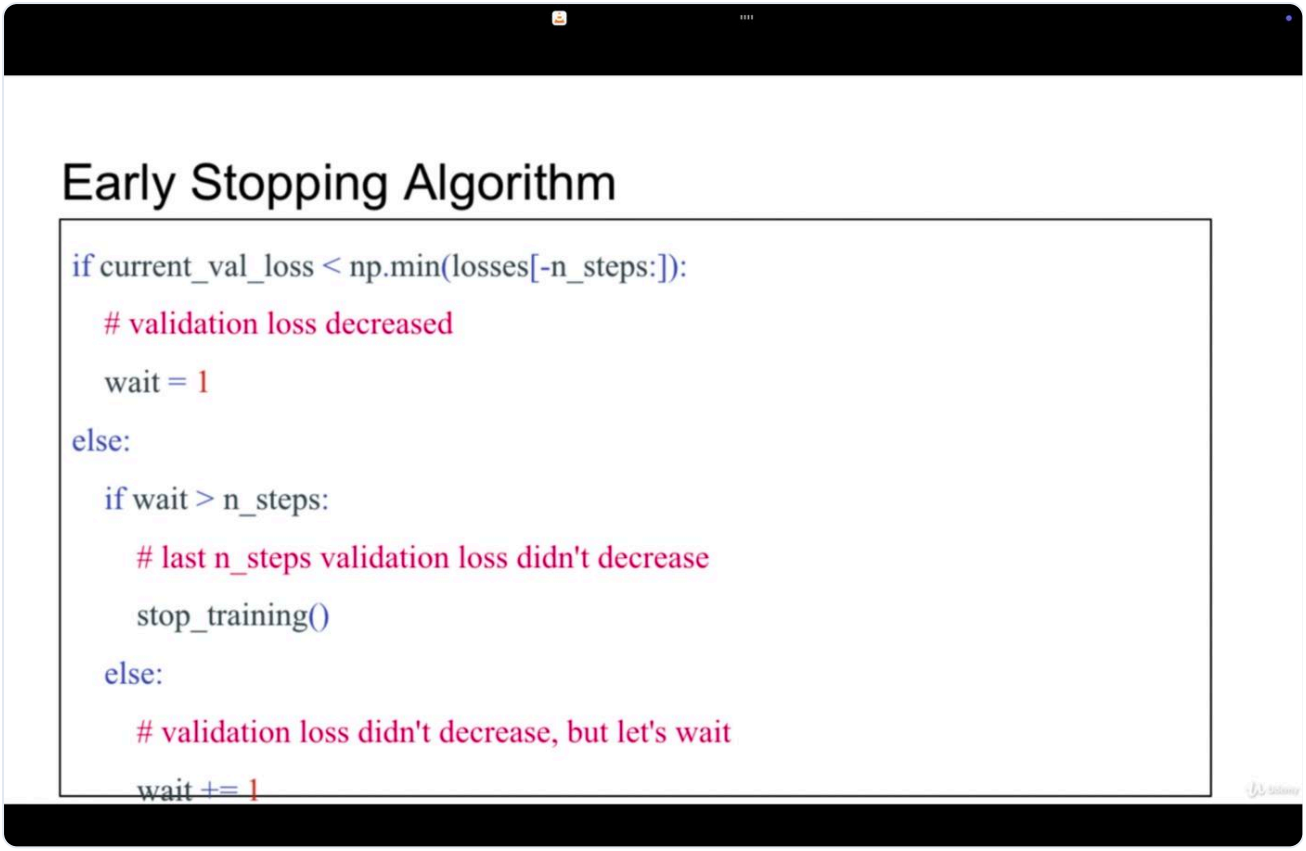


FIGURE 21.7 | Early Stopping Algorithm

We said wait parameter to 1 if it didn't decrease we should check 2 options first. If we waited for increasing more than an steps in this case we should stop training. Otherwise we increase the parameter weight which is responsible for how many iterations we are already waiting. And that's it. Again the only idea behind the early stopping if our will edition laws doesn't increase for any steps then we should stop our training.

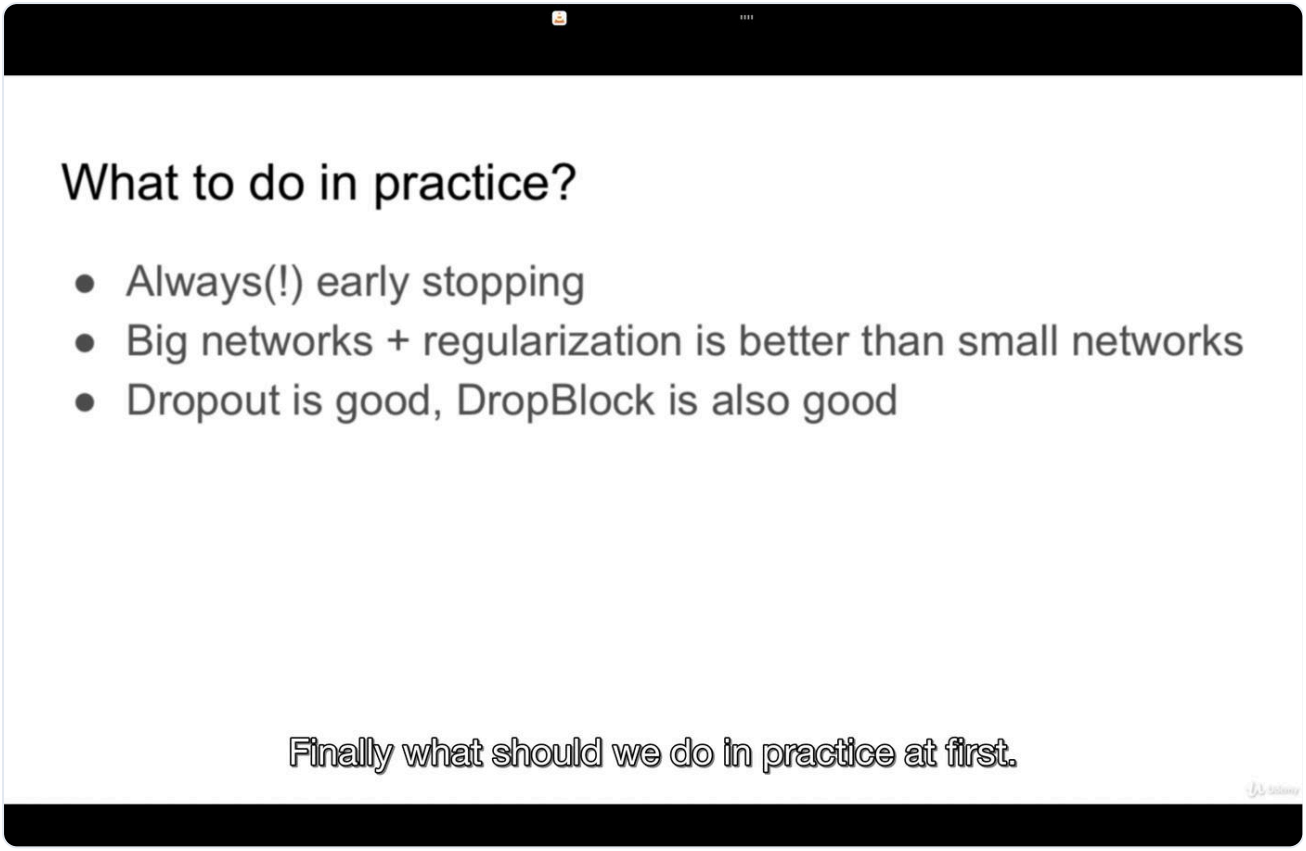


FIGURE 21.8

Finally what should we do in practice at first. We should always use early stopping always Secondly big networks plus organization is better than small networks as our regularization technique. People usually use dropout DropBlock is also good but it is recently proposed method which is not implemented in deep learning libraries yet that is all for this video. Thank you for watching.

LECTURE 22

Batch Normalization

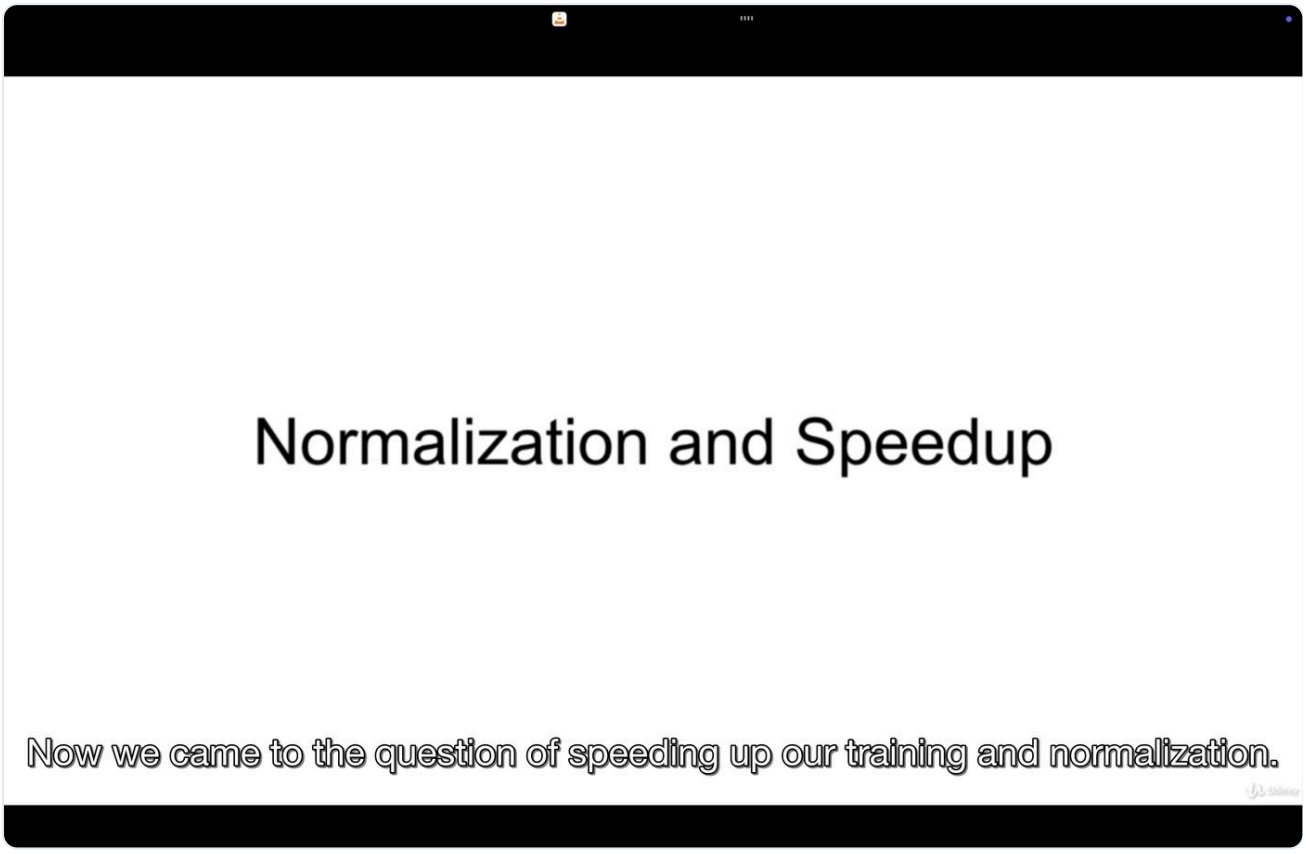


FIGURE 22.1

Now we came to the question of speeding up our training and normalization. Train of modern convolutional neural networks may take up to several weeks and the question is how can we reduce this time. Of course we can use several GPUs but it is not always available.

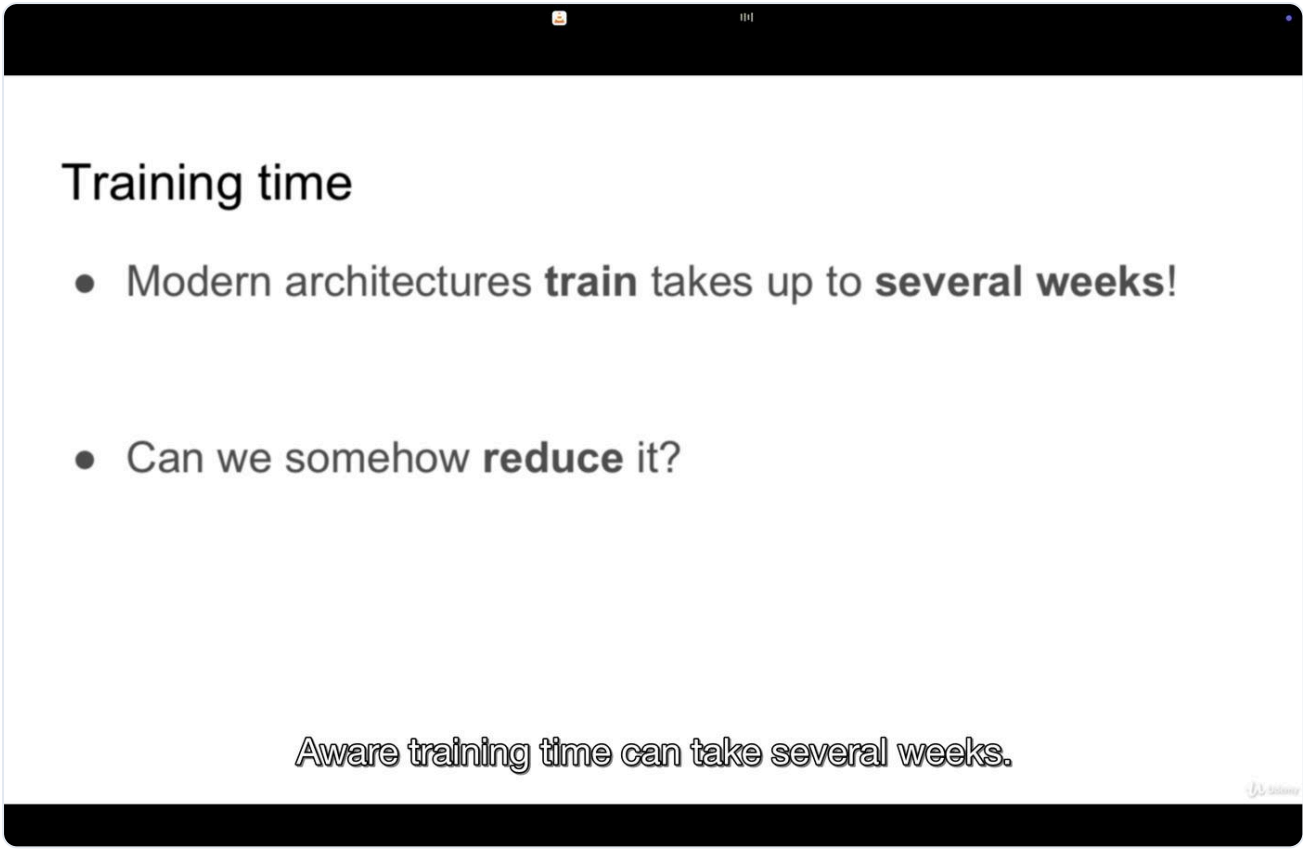


FIGURE 22.2 | Training Time

Aware training time can take several weeks. You win with multiple GPUs the one way to think of speeding up our training is data normalization if normalizing the data points can make their distributions more suitable for optimization the network will find their dependencies in the data faster usually normalization is performed in the way where we make our data to be zero centered that is with a zero mean and also with a standard deviation equal to one in order to do it. We should first pre compute the mean and standard deviation for each feature before training. After that we should first subtract the pre computed mean and divide by the pre computed standard deviation.

Know that we add some small Epsilon to our standard deviation in order to avoid division by zero.

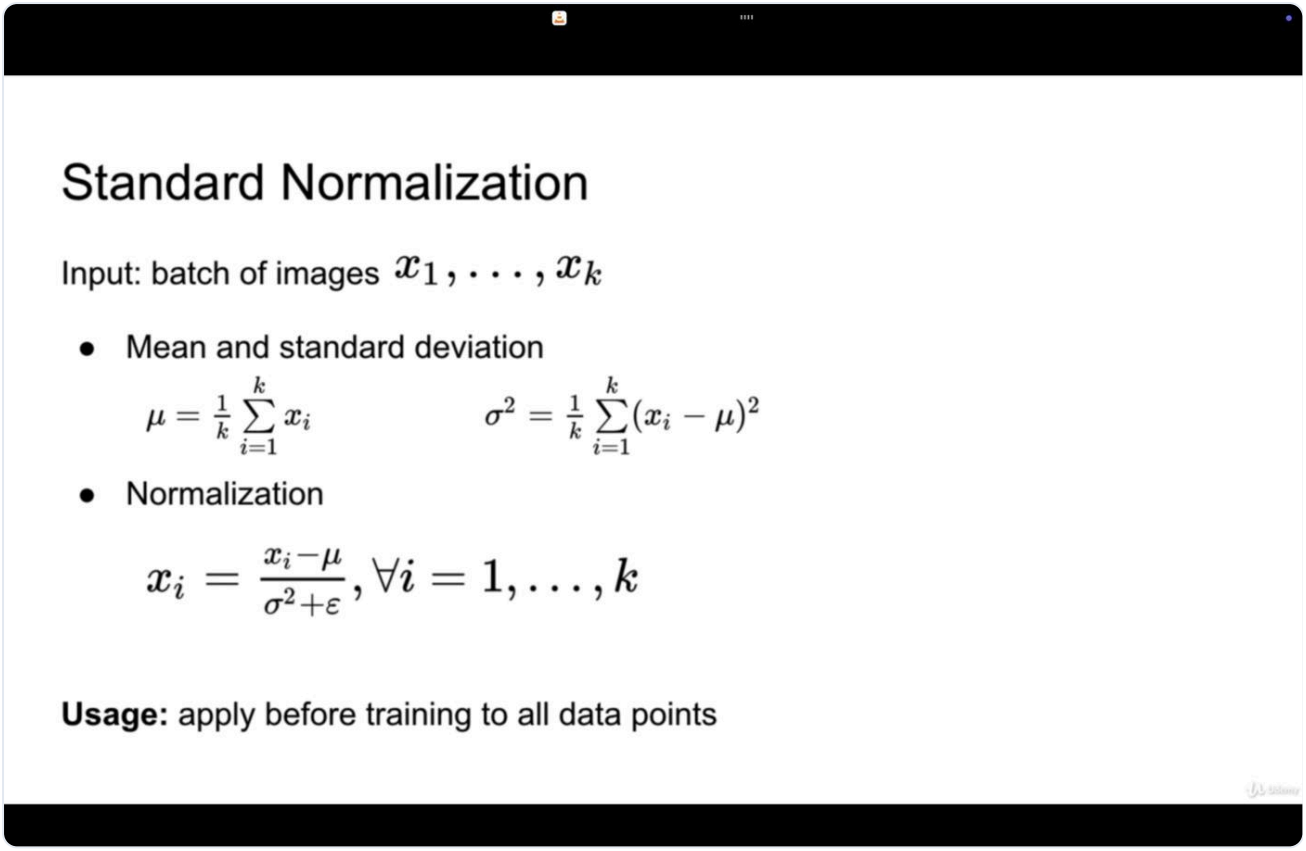


FIGURE 22.3 | Standard Normalization

These normalization is applied before training into all data point including train and test object however we have another problem that I described to you in the section with non-linear narratives in the convolution of module if you haven't seen it. Please do. It is really important to know and many other courses avoid this information so we have sigmoid non linearity function. What is the problem with it.

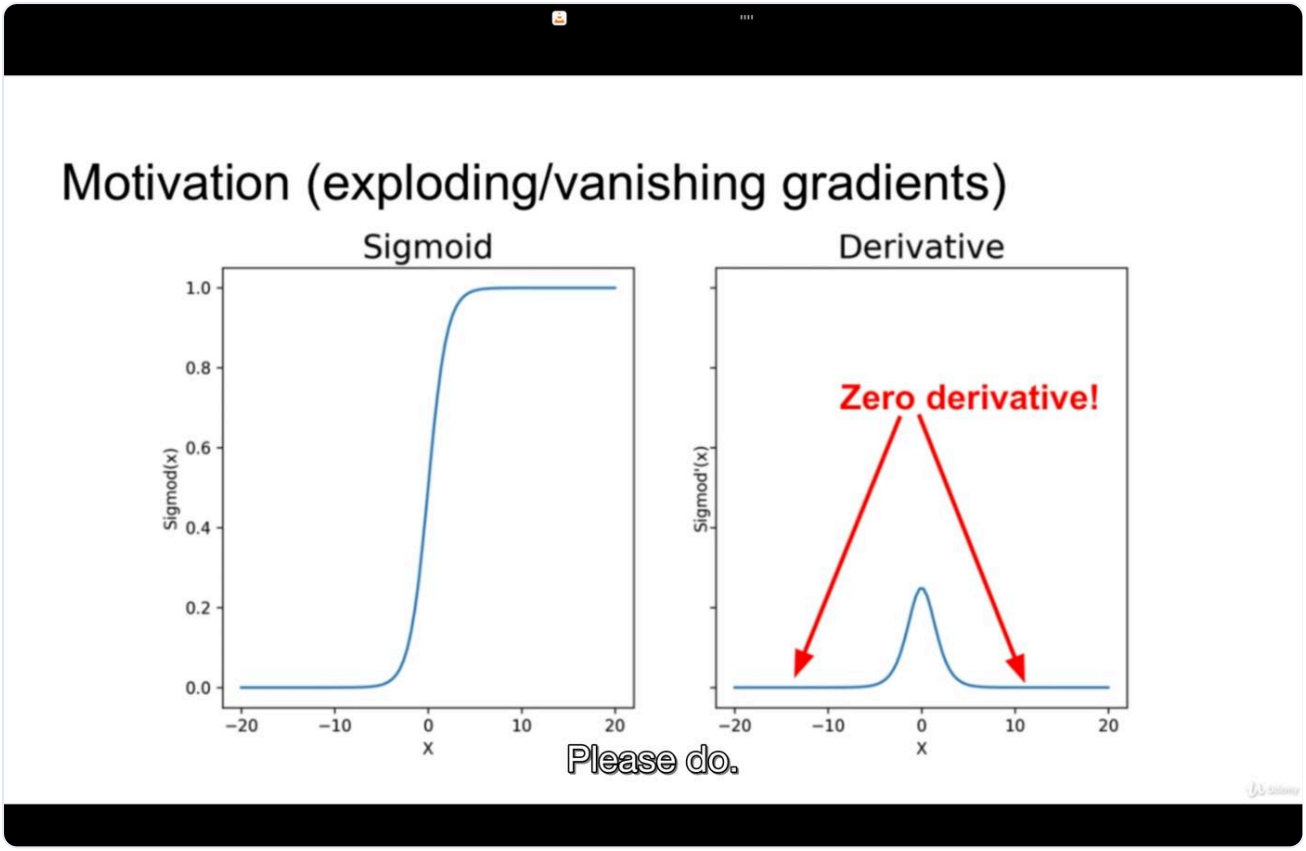


FIGURE 22.4

It is its gradient that is equal to zero almost everywhere except of the small region Iran 0 for us. It means that when our data after some layer will not be around 0 our gradient will vanish. It will be zero and our weights will receive no update at all and we will learn nothing from our data. Okay what can we do with it in 2015. Researchers proposed in union modernization technique called Baseline realisation.

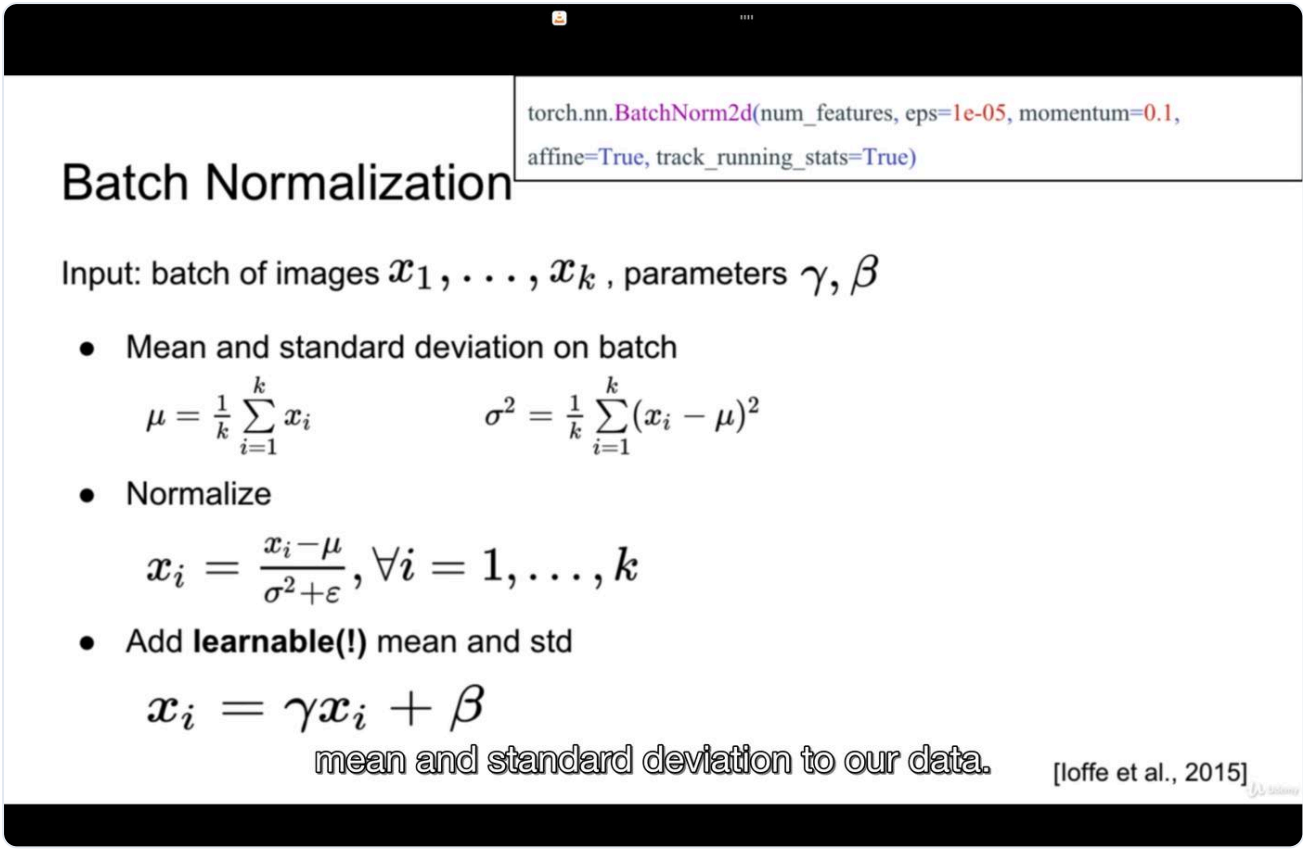


FIGURE 22.5 | Batch Normalization

Their idea behind it is simple Let us create a layer that will normalize our data and will set the learnable mean and standard deviation to our data. So how does it work. We again as in standard normalization compute mean and standard deviation then we subtract the mean and divide by standard deviation plus some small epsilon after that we multiply our normalized data to the new learnable standard deviation and add me you learn and boom mean value.

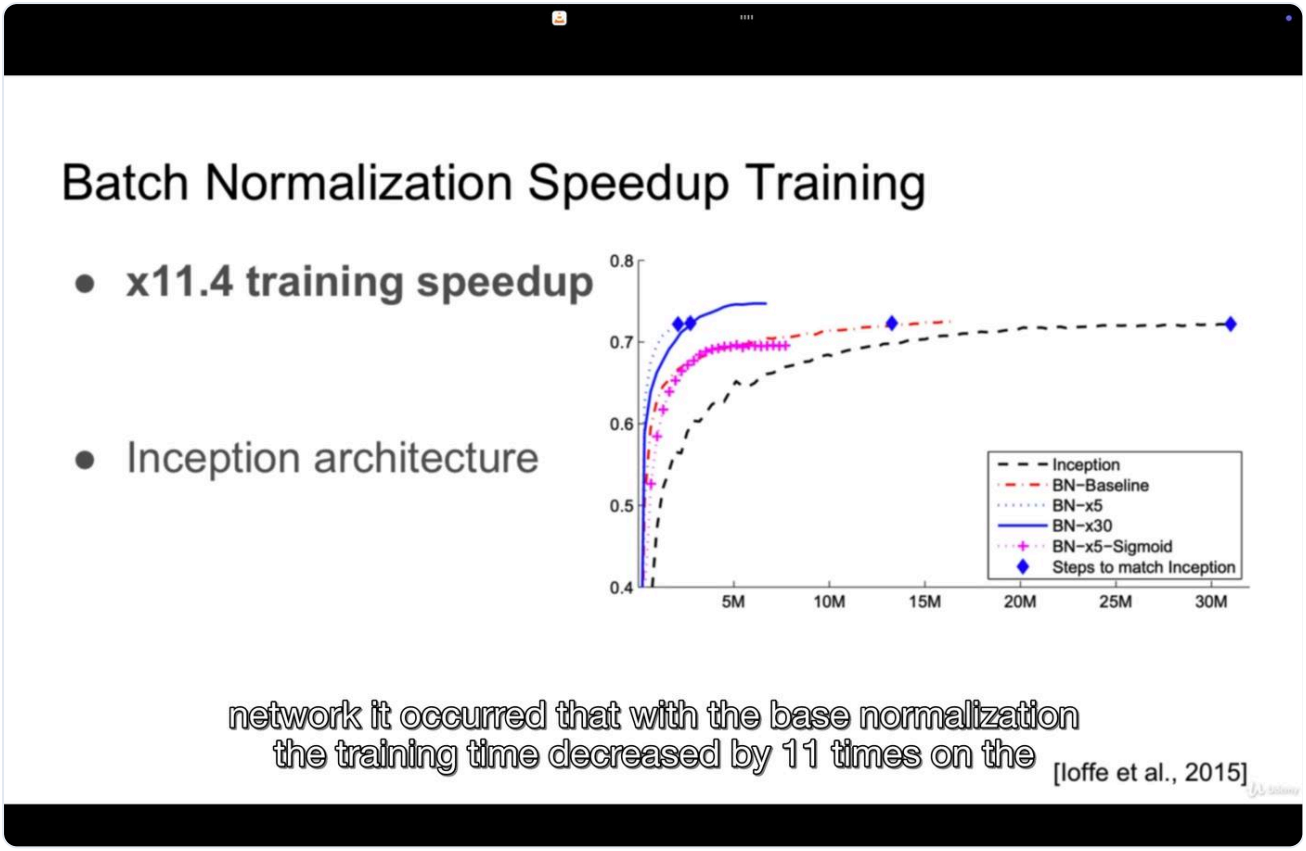


FIGURE 22.6 | Batch Normalization Speedup Training

Pay attention these parameters are learnable which means they are also optimized with gradient descent also know that this is a layer which can be inserted before each linear or convolution or layer in the network it occurred that with the batch normalization the training time decreased by 11 times on the Inception architecture which we will study in the next module.

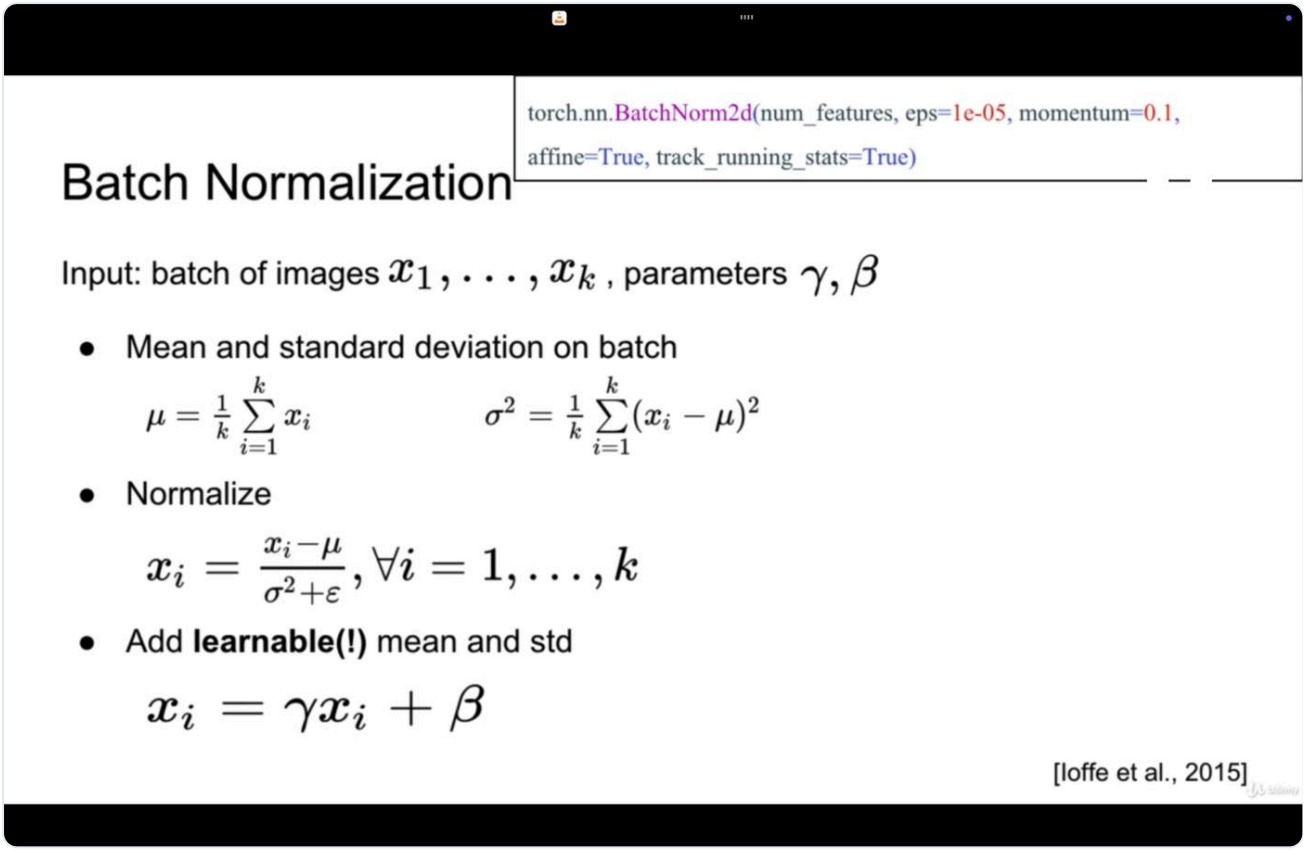


FIGURE 22.7 | Batch Normalization

Just imagine inserting several batch normalization layers can speed up our training by eleven times.

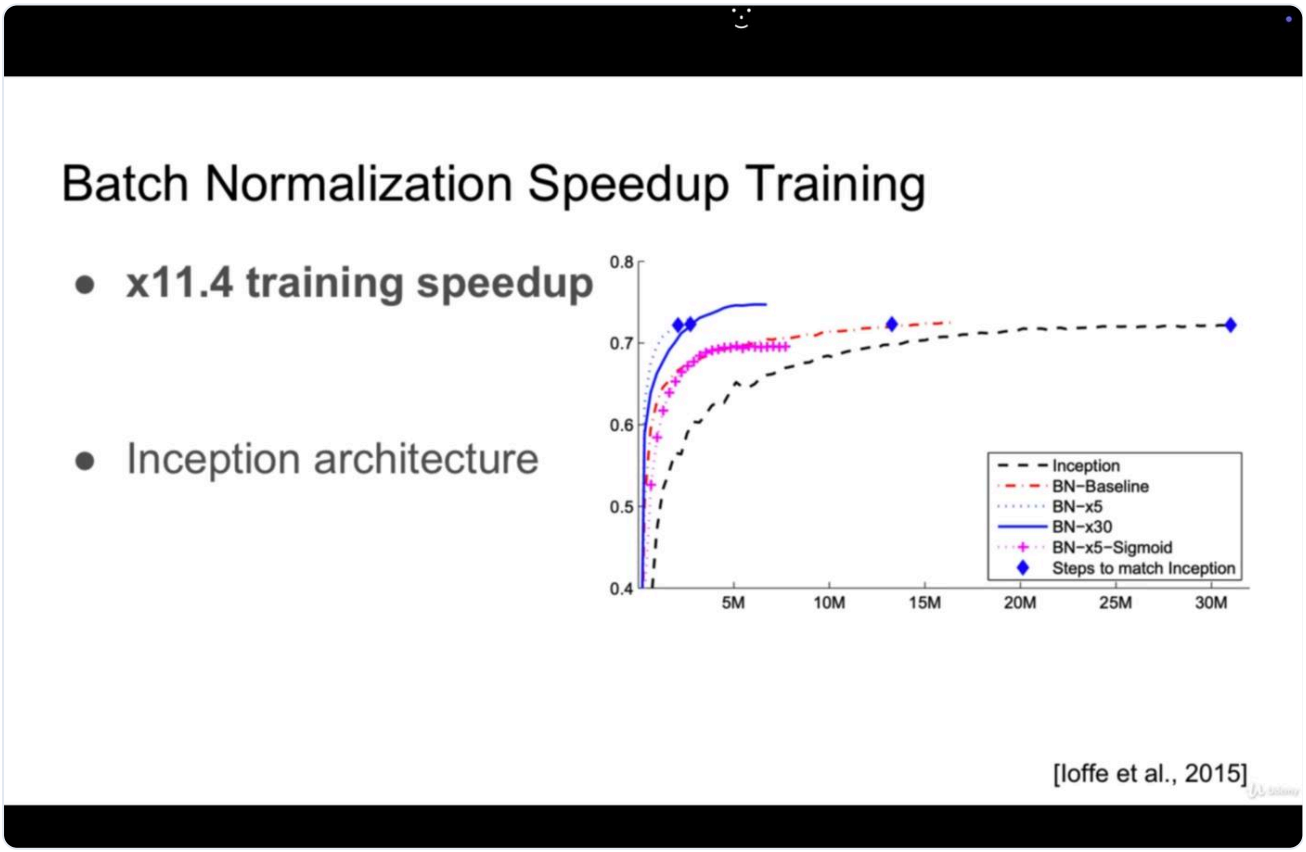


FIGURE 22.8 | Batch Normalization Speedup Training

It's amazing. There are many other organization techniques such as batch and medication like layer normalization is this normalization and so on.

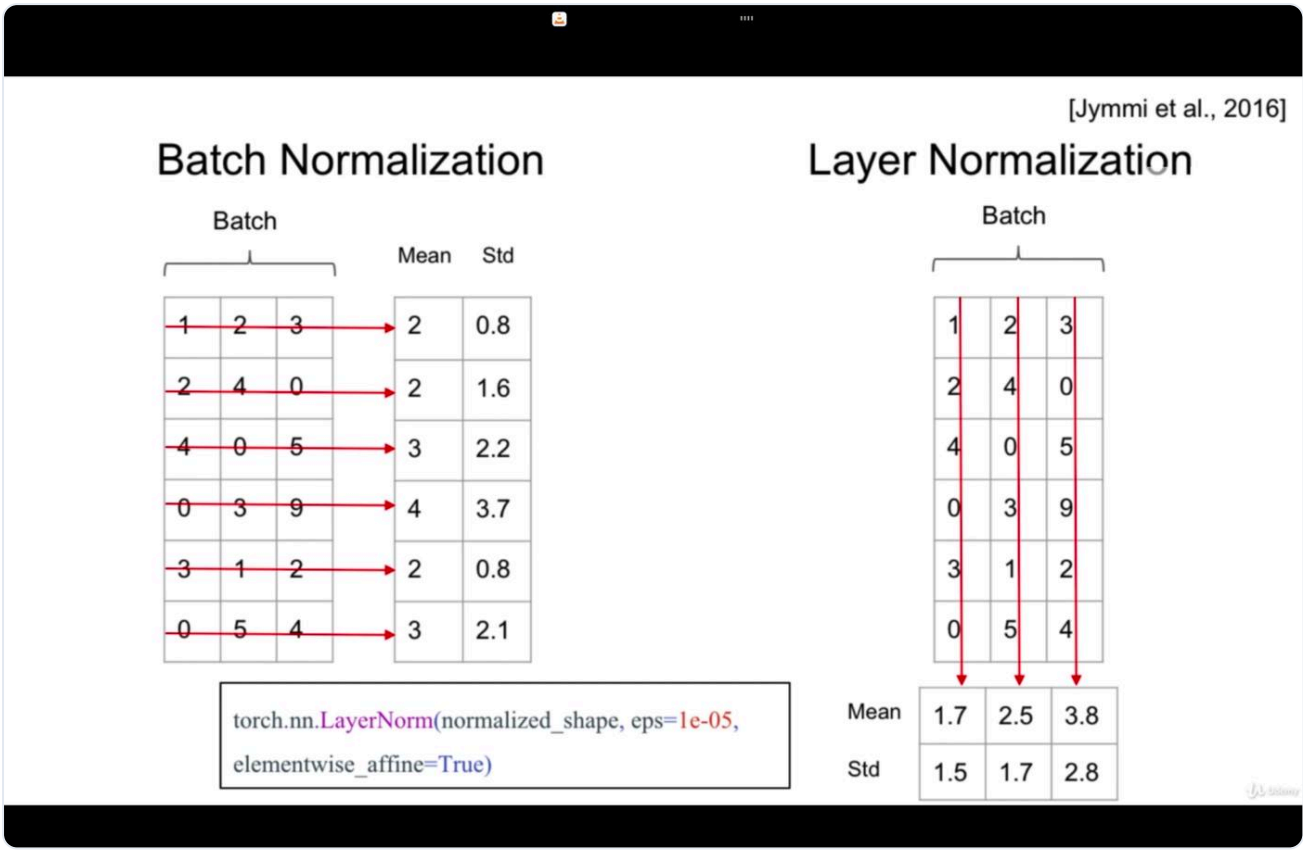


FIGURE 22.9 | Batch Normalization

Let us compare the batch normalization with layer normalization in case of batch normalization the mean and standard deviation is computed for each feature separately and we obtain the mean of a concrete feature in the batch along with the standard deviation for this concrete feature in case of layer normalization the mean and standard deviation are computed not for the feature separately but for the objects.

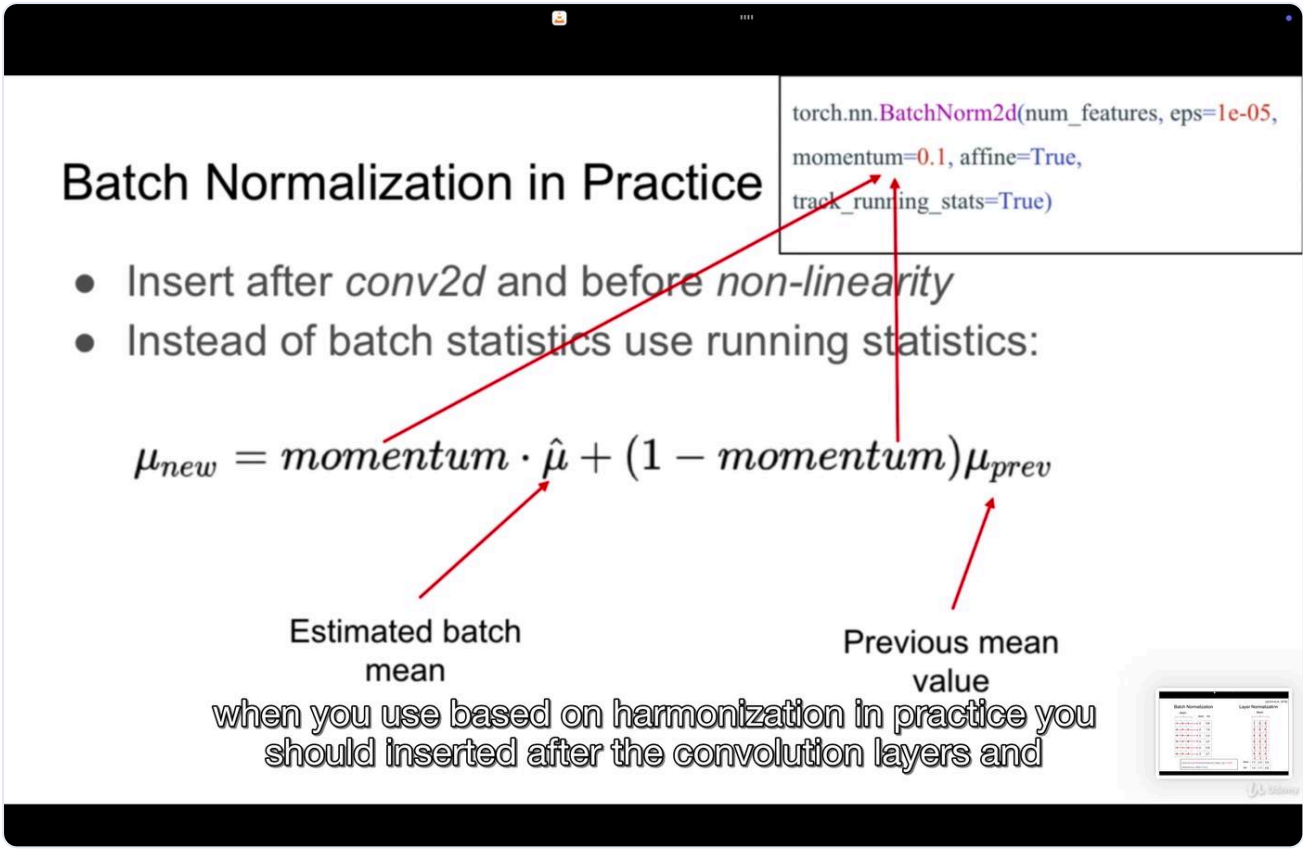


FIGURE 22.10 | Batch Normalization in Practice

So for each object we compute its mean and its standard deviation among all features as a result for each object we will have a single mean and standard deviation layer normalization can be used for images but usually people prefer to use based on memorization layer normalization is more popular for tax processing and natural language processing when you use based on harmonization in practice you should inserted after the convolution layers and before non linearity such as a ReLU or sigmoid or any other of course you can insert it in another way. For example often on linearity and sometimes it can be better instead of computing statistics such as mean and standard deviation for a bunch. Usually people use Iranian statistics which is computed with the use of momentum parameter which is available in PyTorch so that is all for normalization.

Next time we will discuss weight initialization. Thank you for watching and see in the next video.

LECTURE 23

Data Augmentation

Hello my friend. In this video we'll talk about data augmentation a good technique that allows you to extend your dataset and to improve your quality.

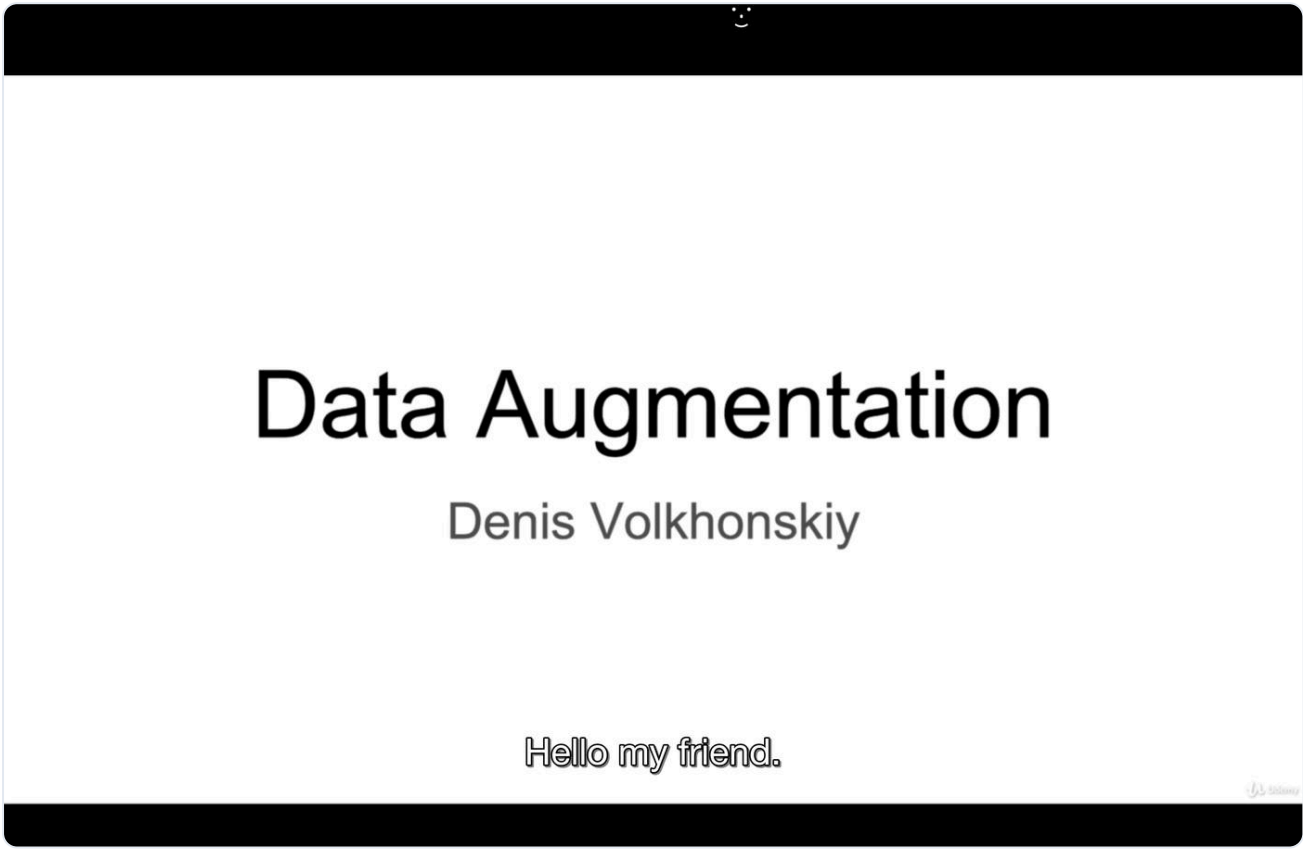


FIGURE 23.1 | Data Augmentation

So what are the problems of convolutional neural networks.

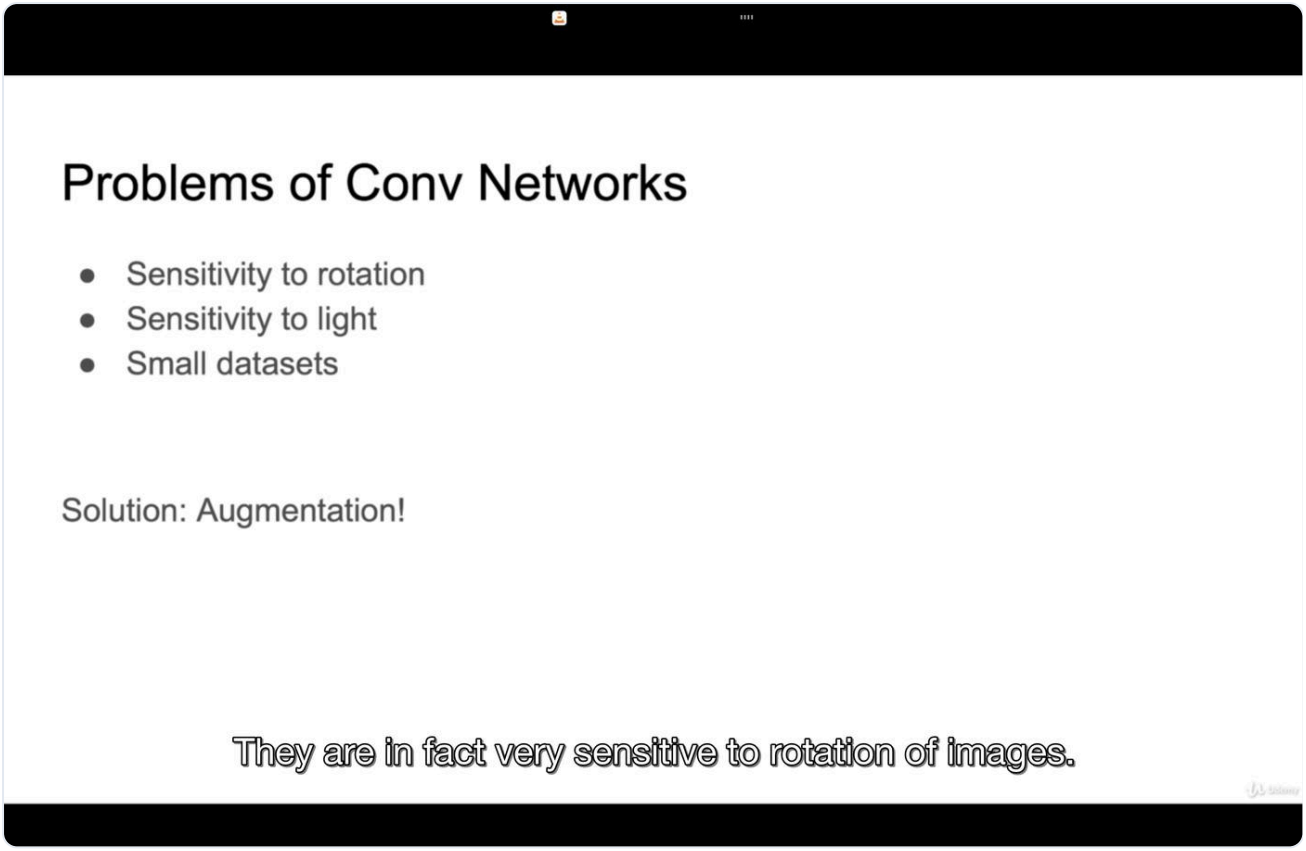


FIGURE 23.2 | Problems of Conv Networks

They are in fact very sensitive to rotation of images. So if we have good prediction for one image and if we rotated a little bit that quality may decrease. It is also sensitive to light. If we trained our network only on white images and then make the images dark then the quality again may decrease.

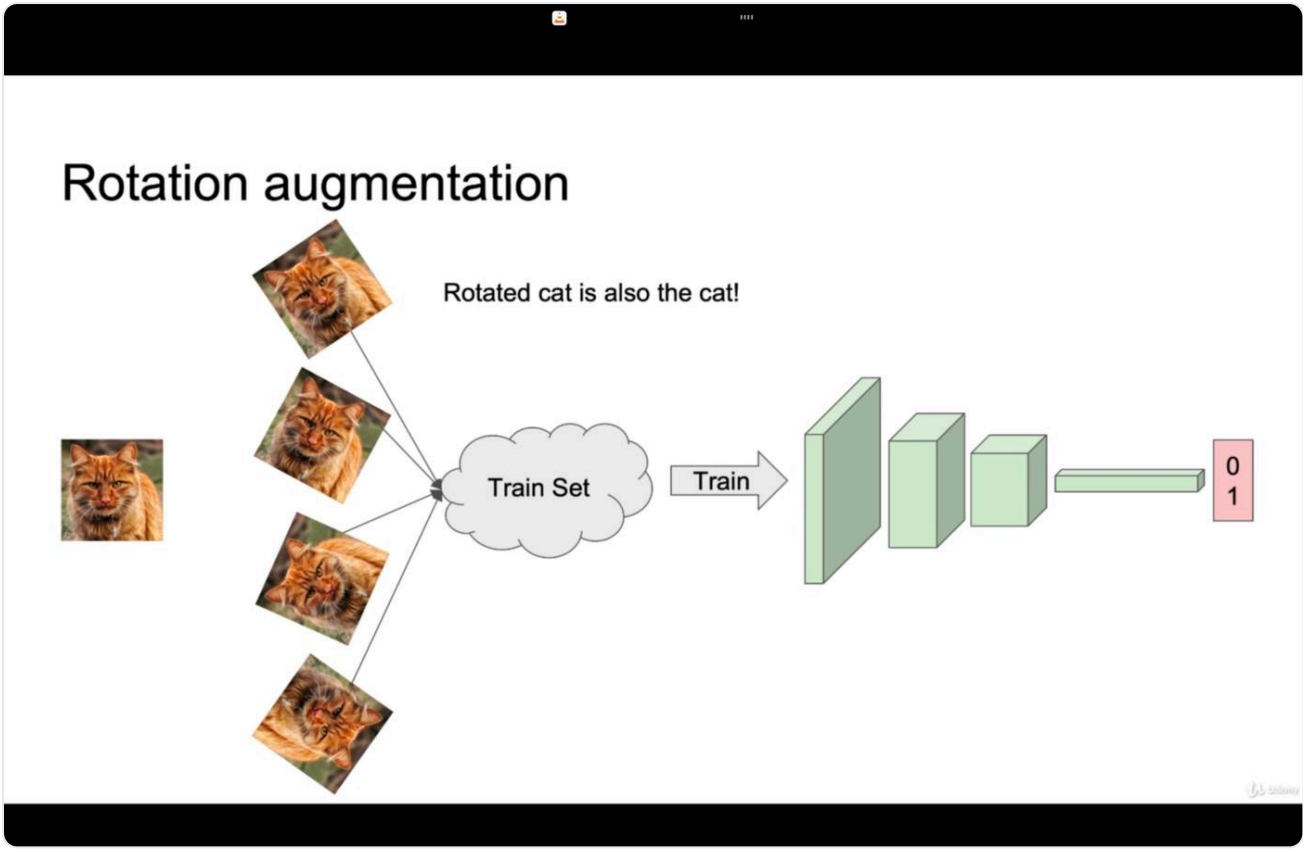


FIGURE 23.3 | Rotation Augmentation

And also there is a problem of small data sets. For example if you if you have very few amount of data so it's hard to train the neural network and obtain good quality and the solution for these is data augmentation. The first method of augmentation is rotation. So if we have an image of cat we can see that a rotated cat is still a cat we can rotate our training images randomly and add to our training set and also use them for training.

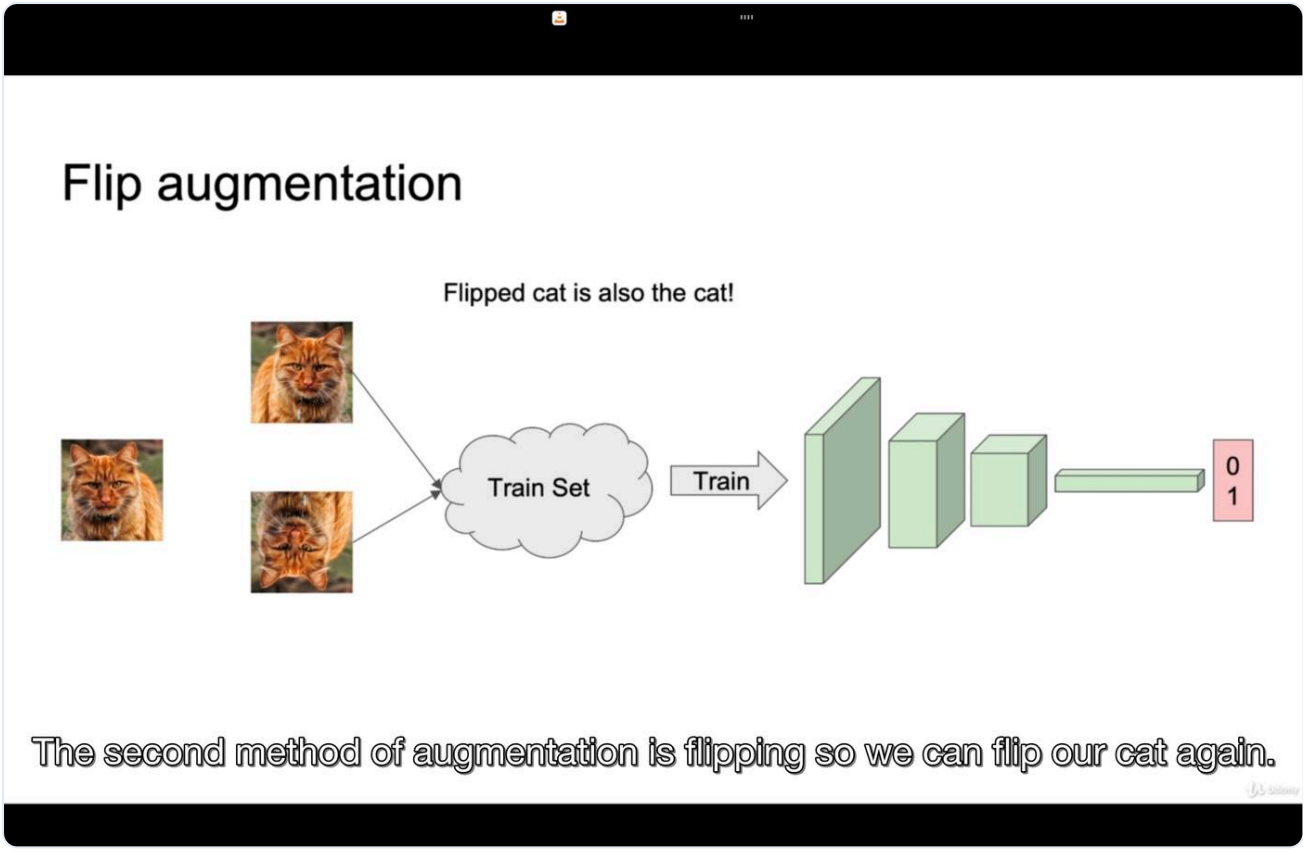


FIGURE 23.4 | Flip Augmentation

The second method of augmentation is flipping so we can flip our cat again.

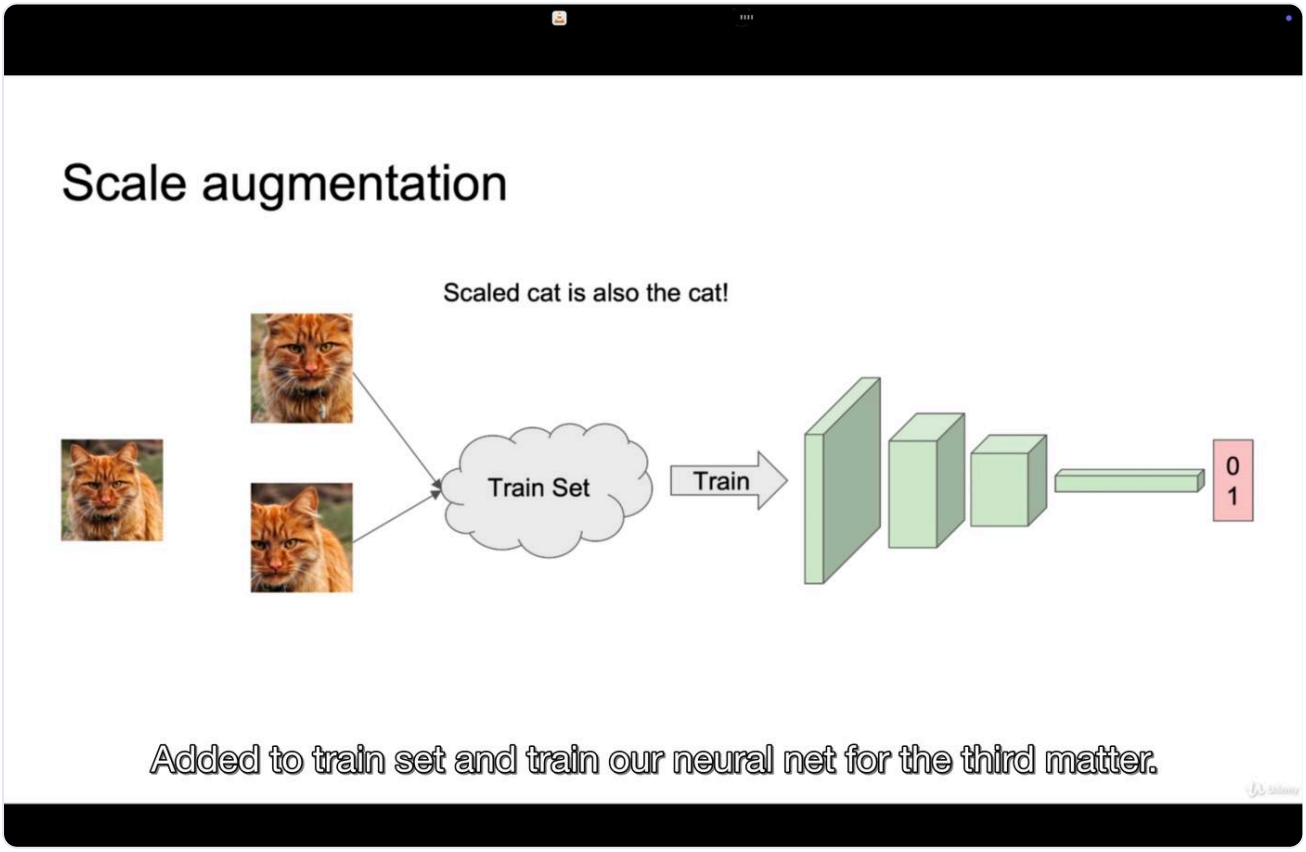


FIGURE 23.5

Added to train set and train our neural net for the third matter.

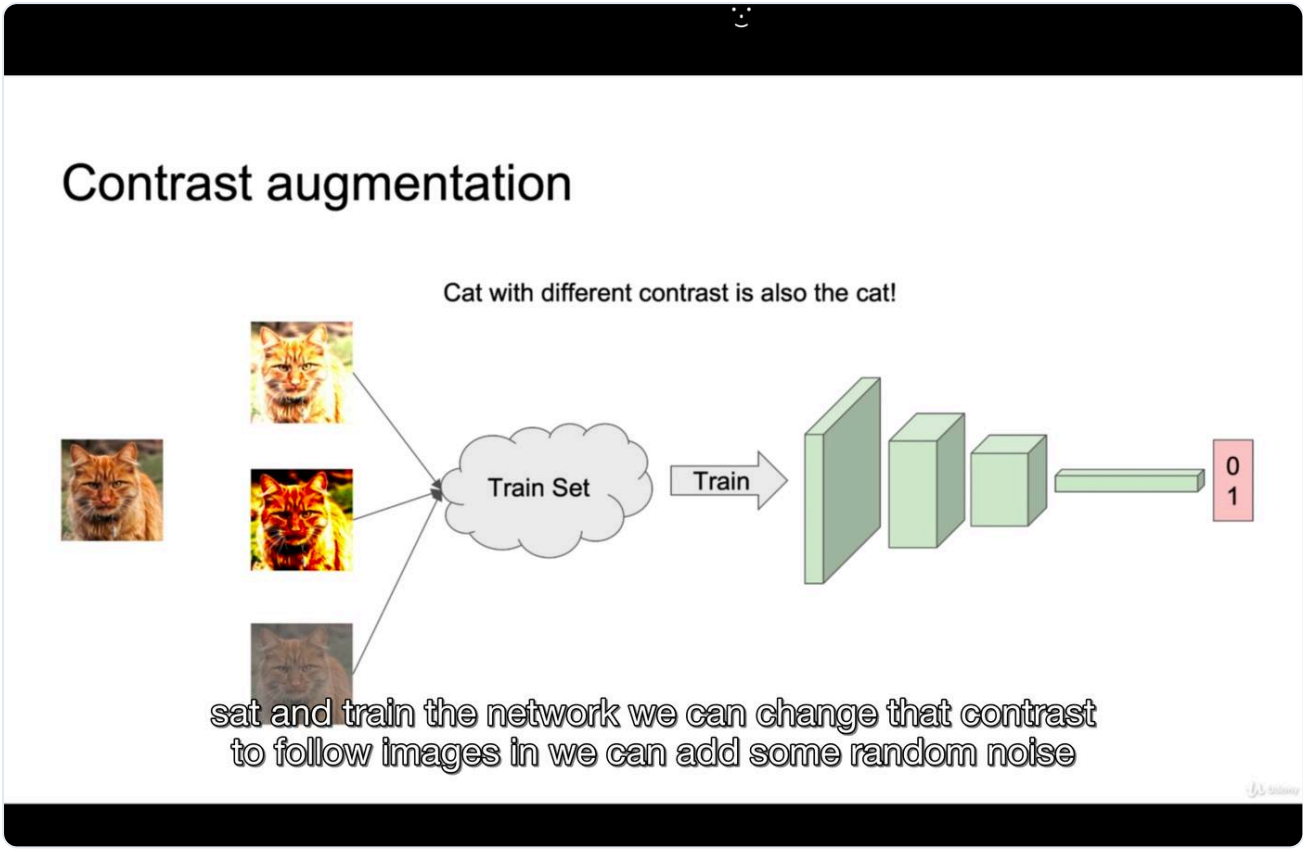


FIGURE 23.6 | Contrast Augmentation

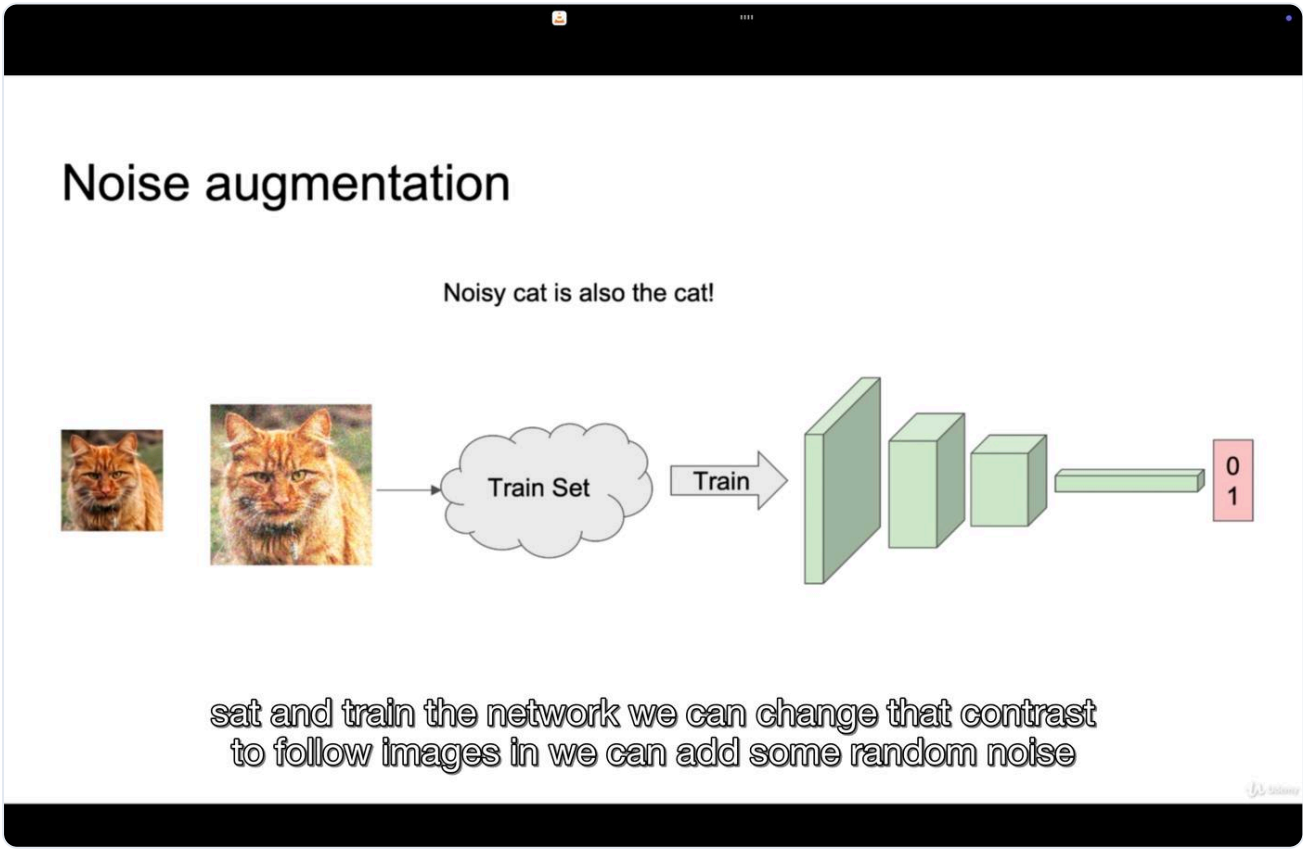


FIGURE 23.7 | Noise Augmentation

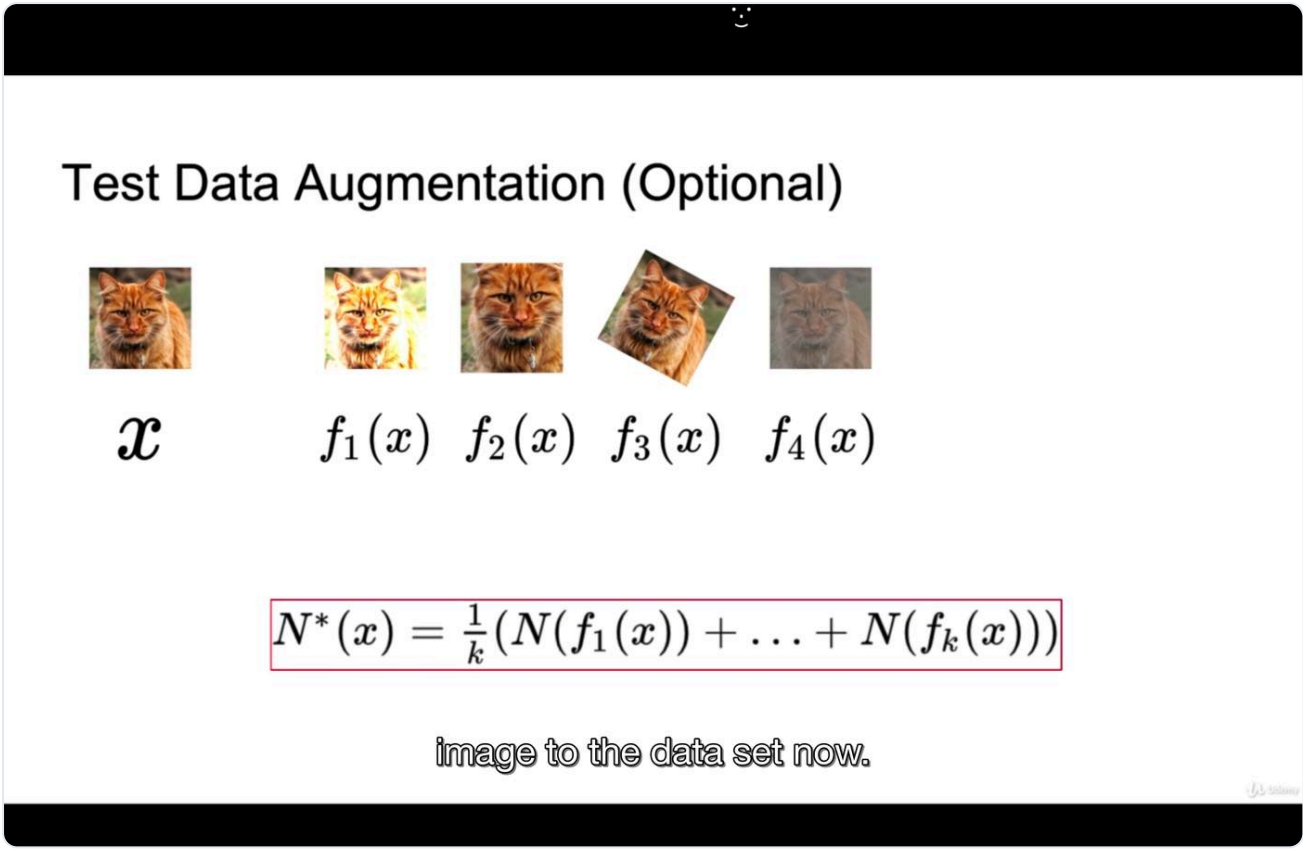


FIGURE 23.8 | Test Data Augmentation (Optional)

This scaling we can use their subset or the random subset of image and also again added to the training set and train the network we can change that contrast to follow images in we can add some random noise for example from the normal distribution or from uniform distribution to the image and add again this image to the dataset now. In fact this method can increase the size of your dataset up to several times. But now what have we do in case of testing. So there is no one good answer. One possible solution is not to use augmentation on testing. The second possible solution is to use some linear combination of different augmentation. So we take one test image apply contrast change in a apply scaling apply rotation apply blurring and then use ever each of these predictions.

The quality may vary from one task to another so you should try. But believe me the data augmentation is a very good technique that allows you to increase the size of the dataset and increase the quality a lot. Thank you for watching and see you in the next video.

LECTURE 24

MNIST Dataset

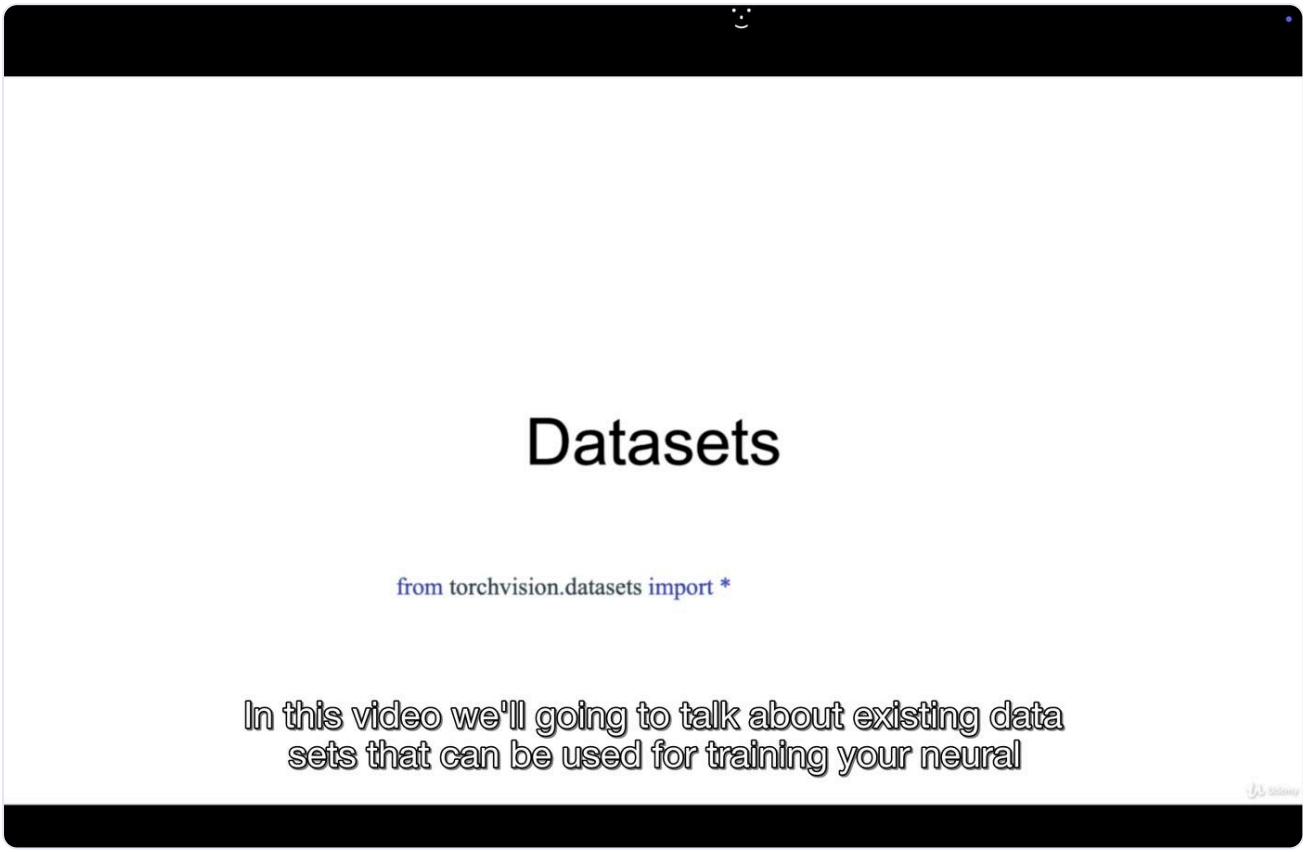


FIGURE 24.1

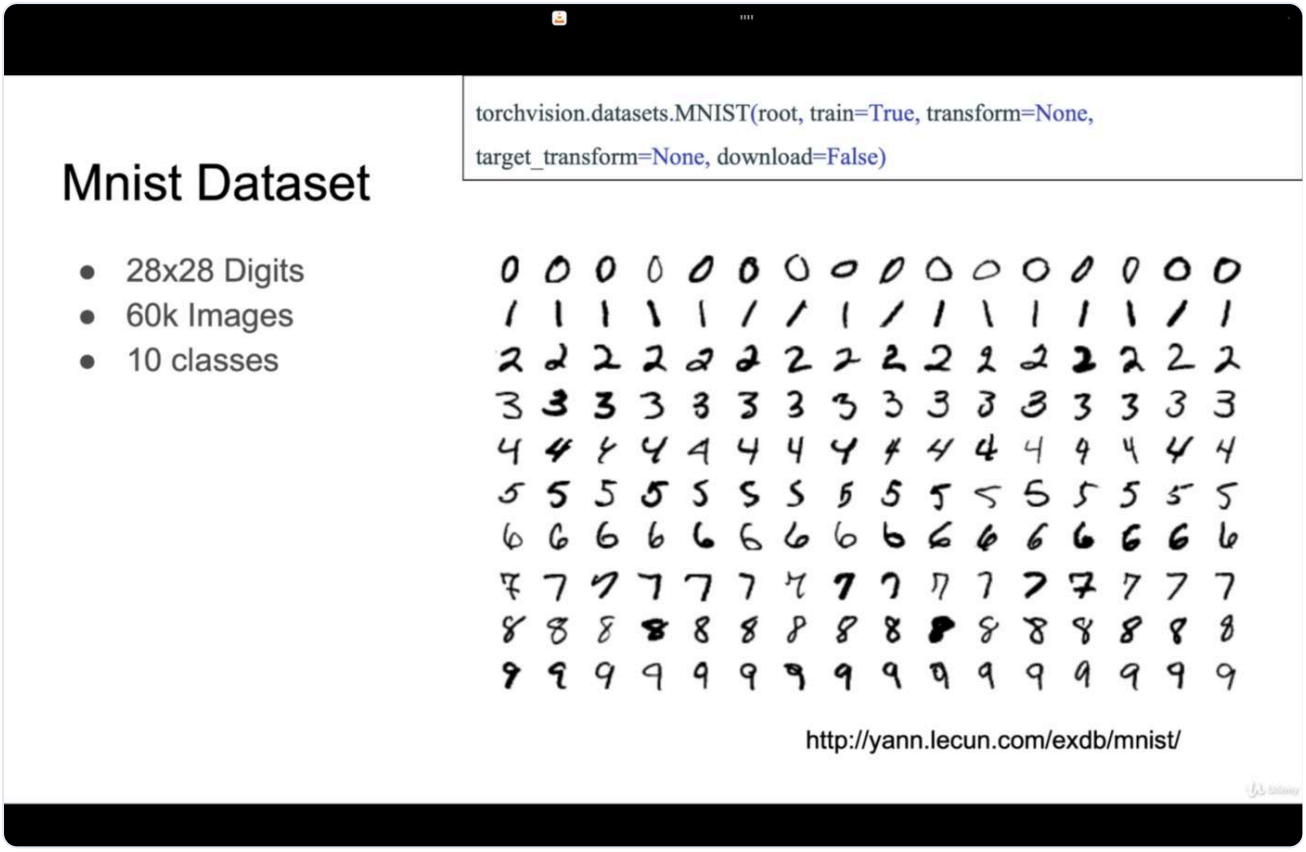


FIGURE 24.2 | MNIST Dataset

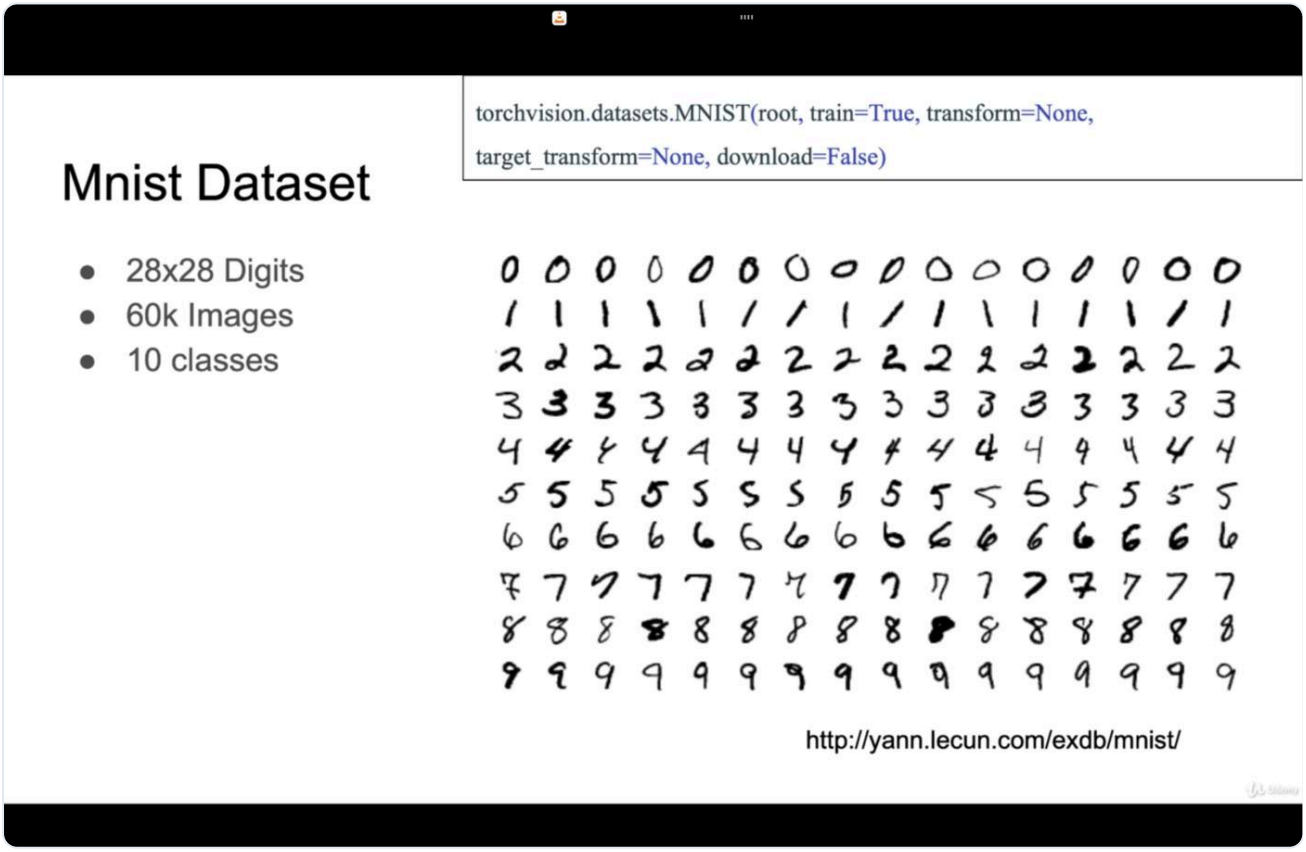


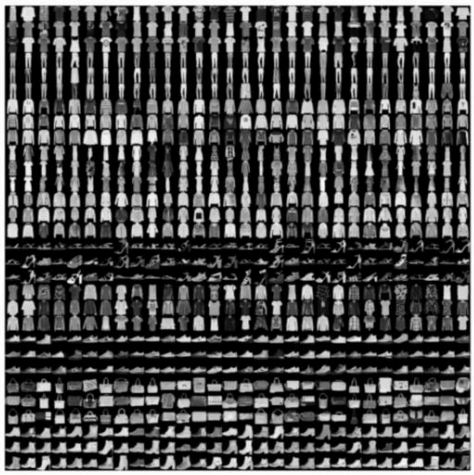
FIGURE 24.3 | MNIST Dataset

Fashion Mnist Dataset

- 28x28 Images
- 60k Images
- 10 classes

Alternative to MNIST

```
torchvision.datasets.FashionMNIST(root, train=True,
transform=None, target_transform=None, download=False)
```



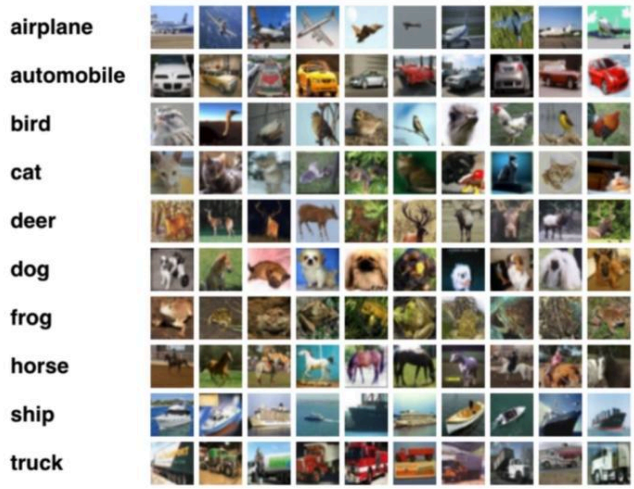
https://github.com/zalandoresearch/fashion-mnist

FIGURE 24.4

CIFAR-10 Dataset

- 60000 images
- 10 classes
- 6000 image per class
- Size: 32x32

```
torchvision.datasets.CIFAR10(root, train=True, transform=None,
target_transform=None, download=False)
```



https://www.cs.toronto.edu/~kriz/cifar.html

FIGURE 24.5

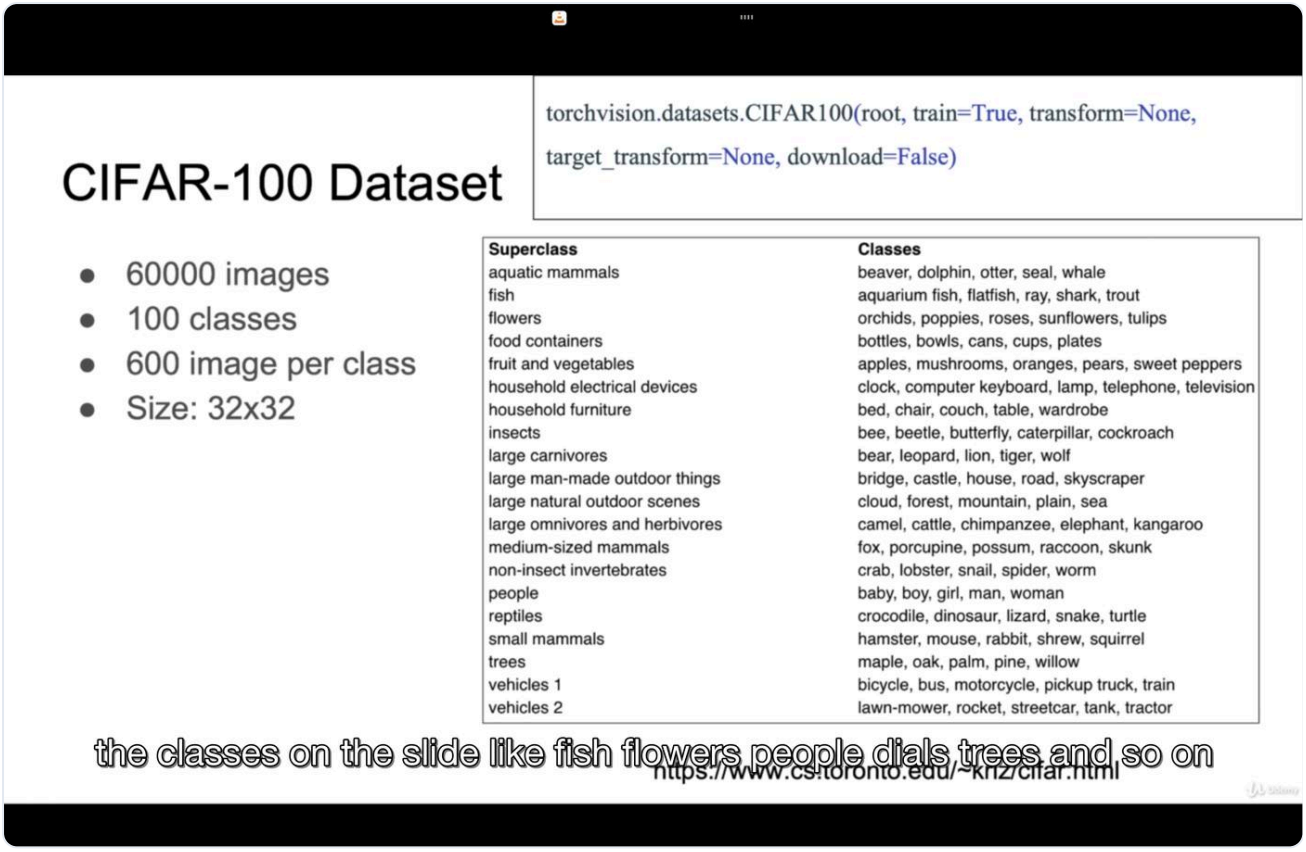


FIGURE 24.6

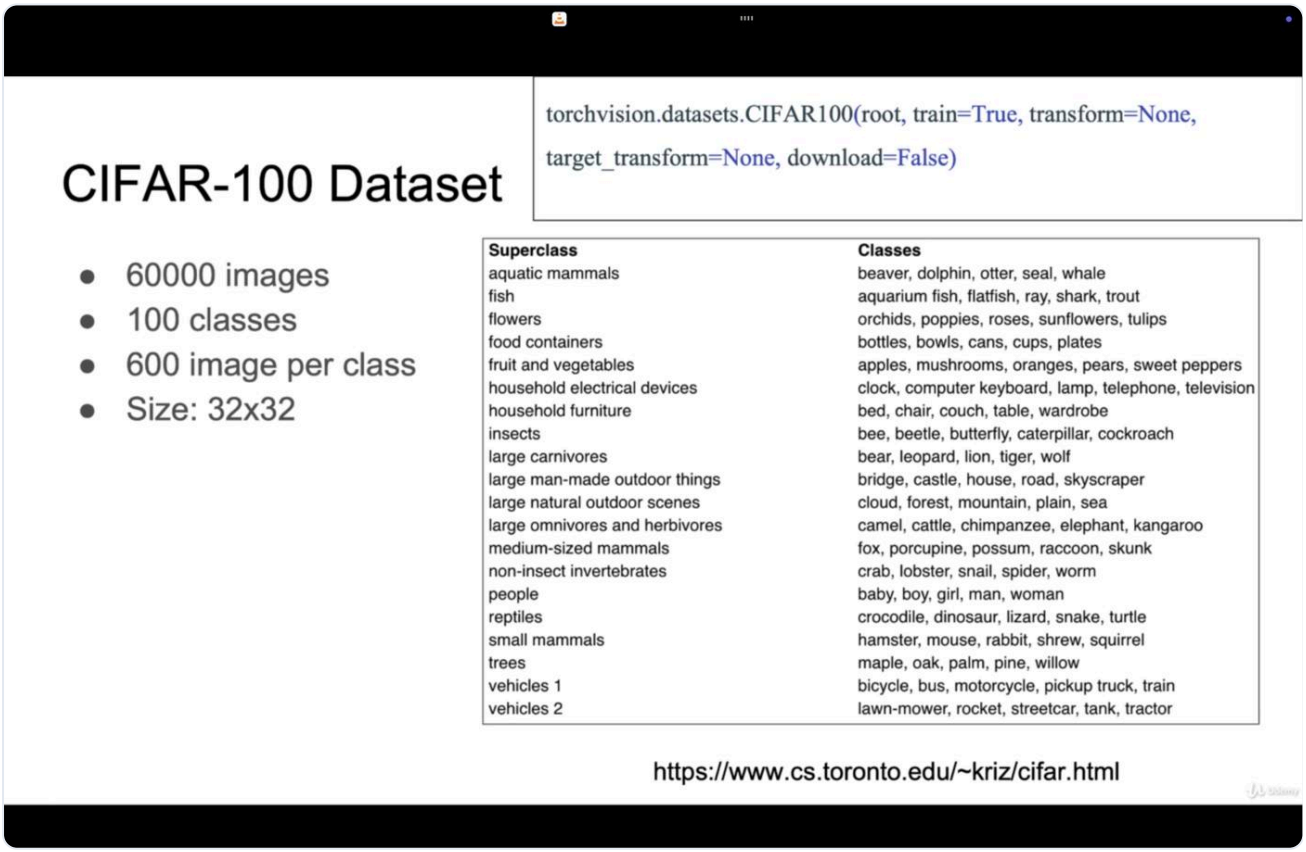


FIGURE 24.7

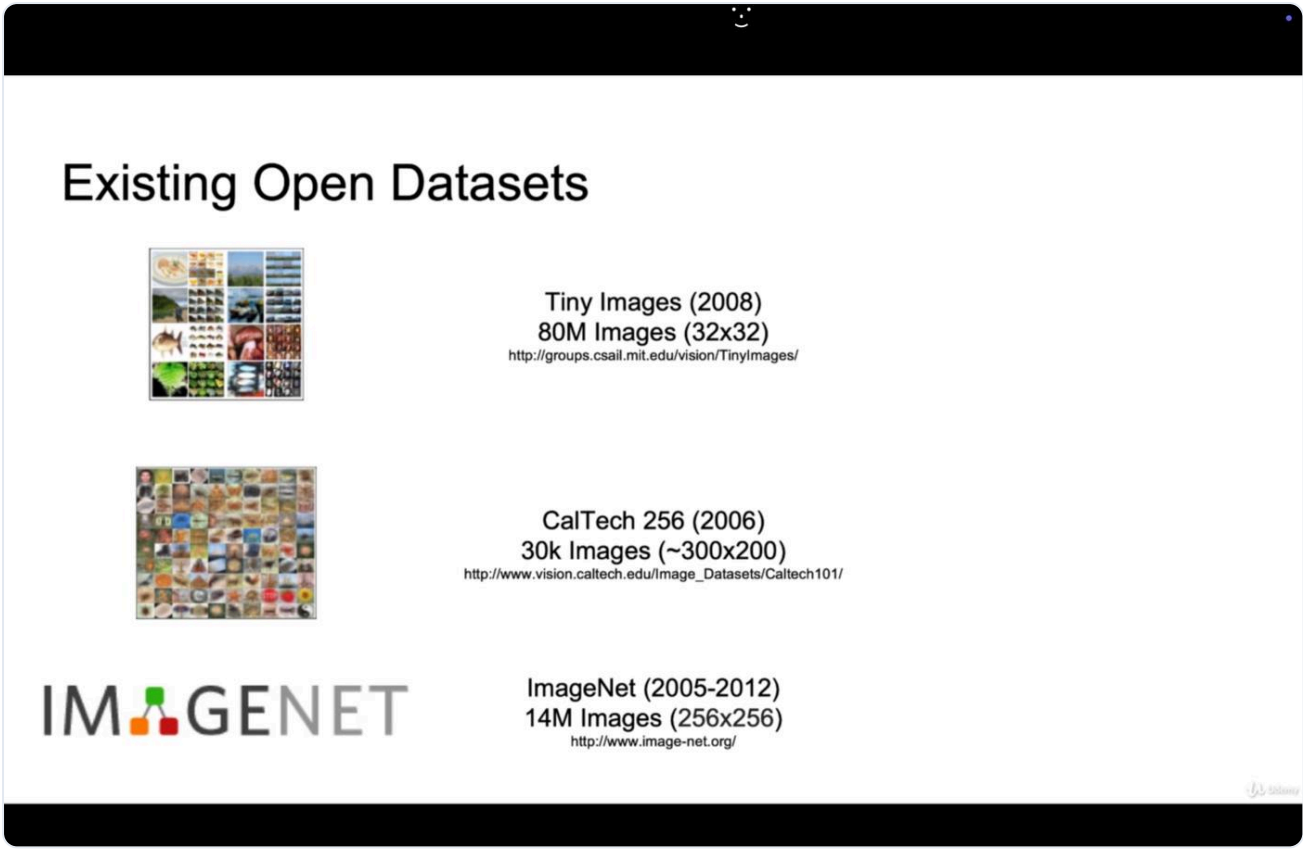


FIGURE 24.8

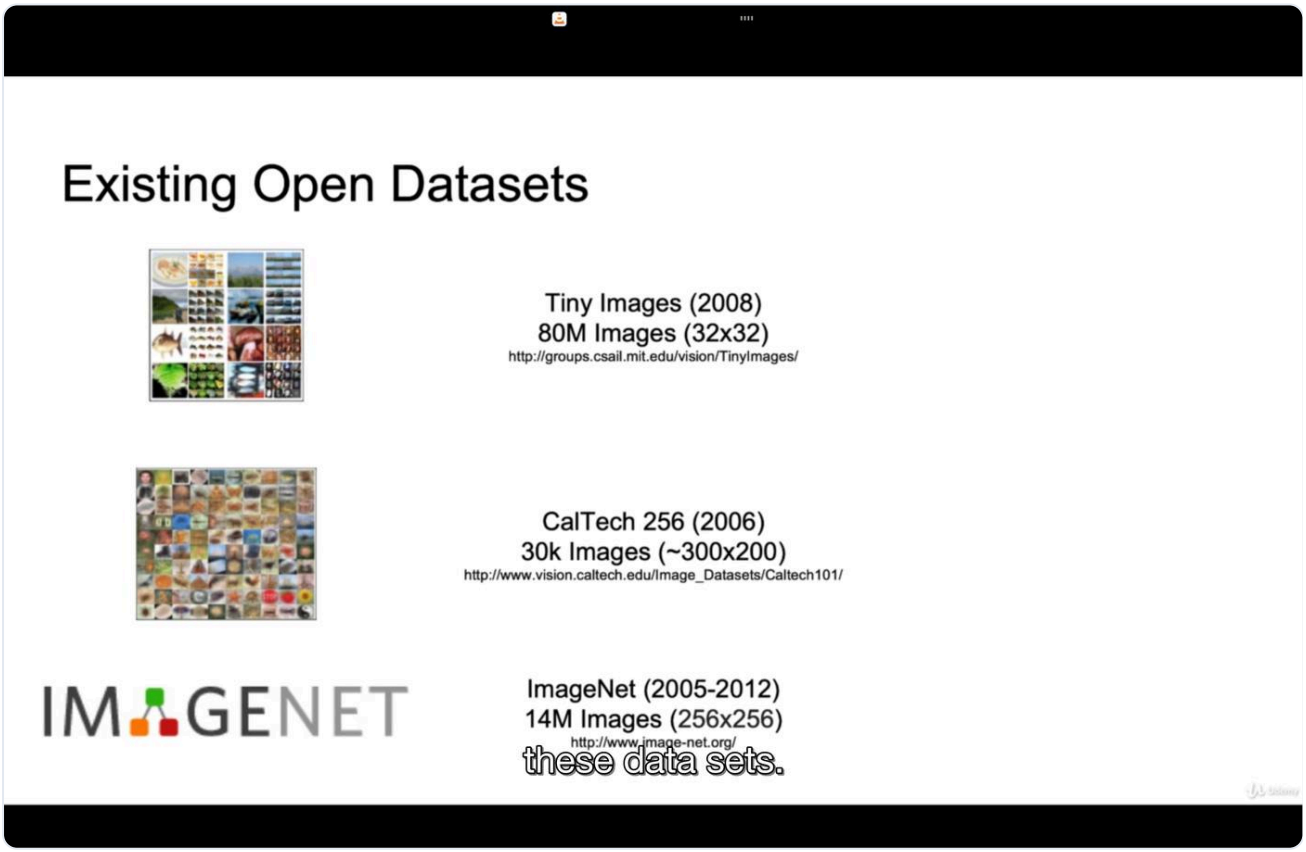


FIGURE 24.9

In this video we'll going to talk about existing data sets that can be used for training your neural network. The first and the most popular dataset is MNIST dataset. It consists of images of size 28 to 28 and all images are grayscale and there are 10 classes from 0 to 9. Totally there are sixty thousand four images of ten classes. As I said know that the quality of the network should be very big on these dataset so the accuracy of for example key nearest neighbours algorithm on these dataset is 96 percent. So this is rather bad dataset for testing because it's very easy for this reason and now the dataset which is called Fashion-MNIST was proposed.

There are again sixty thousand images of again size twenty eight twenty eight of 10 classes but now there are no digits but different types of quartz. And this is rather good alternative to e at the bottom of the slide you can see the link to this dataset and again all these data sets you can download with torch Regional Library the next famous dataset that we used in our practical videos is CIFAR-10 it consists of sixty thousand color images of ten classes like airplane Auto Mobile bird etc. and there are six thousand image per class and all of them have size 32 to 32 another dataset which is very similar to previous one is CIFAR-100 it again consists of sixty thousand images of. But now of one hundred classes there are six hundred images per class of size 32 to 32 and you can see the classes on the slide like fish flowers people dials trees and so on there are also a good set of big data sets.

First is tiny images that was proposed in 2008 and it has 80 millions of images of size 32 to 32 there exist a Caltech dataset which has around 300 to 200 size images. Thirty thousand images and the last one is the most popular is ImageNet which consist of 14 millions of images of big size two hundred and fifty six to two hundred and fifty six. And what I suggest you to do if for example you have a set of cats and a set of dogs and you want to build a classifier that distinguish cats and dogs try to use some additional data that is labeled from these data sets. So you are able to just download the ImageNet and take all cats and old dogs from there and add to your data dataset and the increasing of that your dataset will improve your quality.

That is all for this reader. Thank you for watching and see you in the next.

LECTURE 25

Modern Convolutional Architectures

In this video we'll talk about modern convolutional architectures.

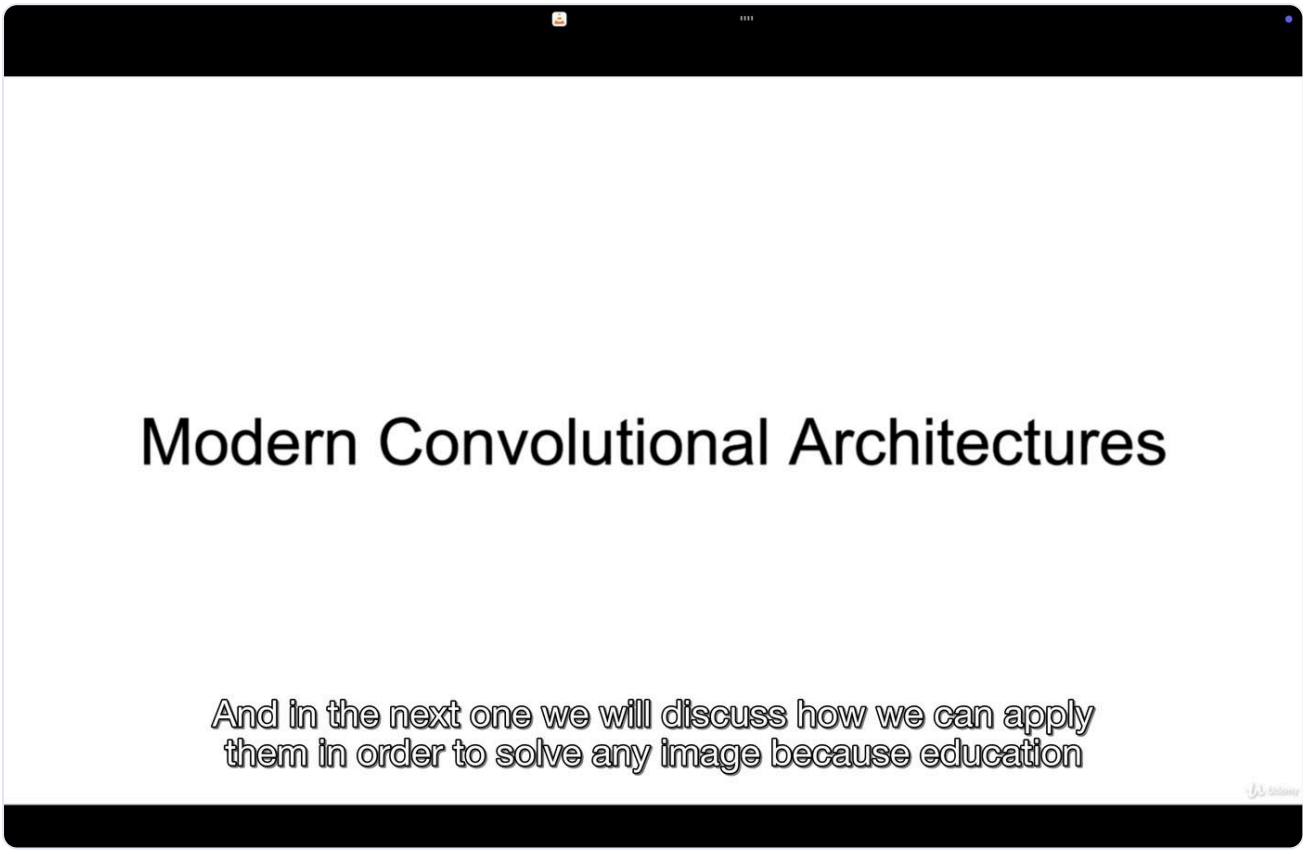


FIGURE 25.1 | Modern Convolutional Architectures

And in the next one we will discuss how we can apply them in order to solve any image classification problem.

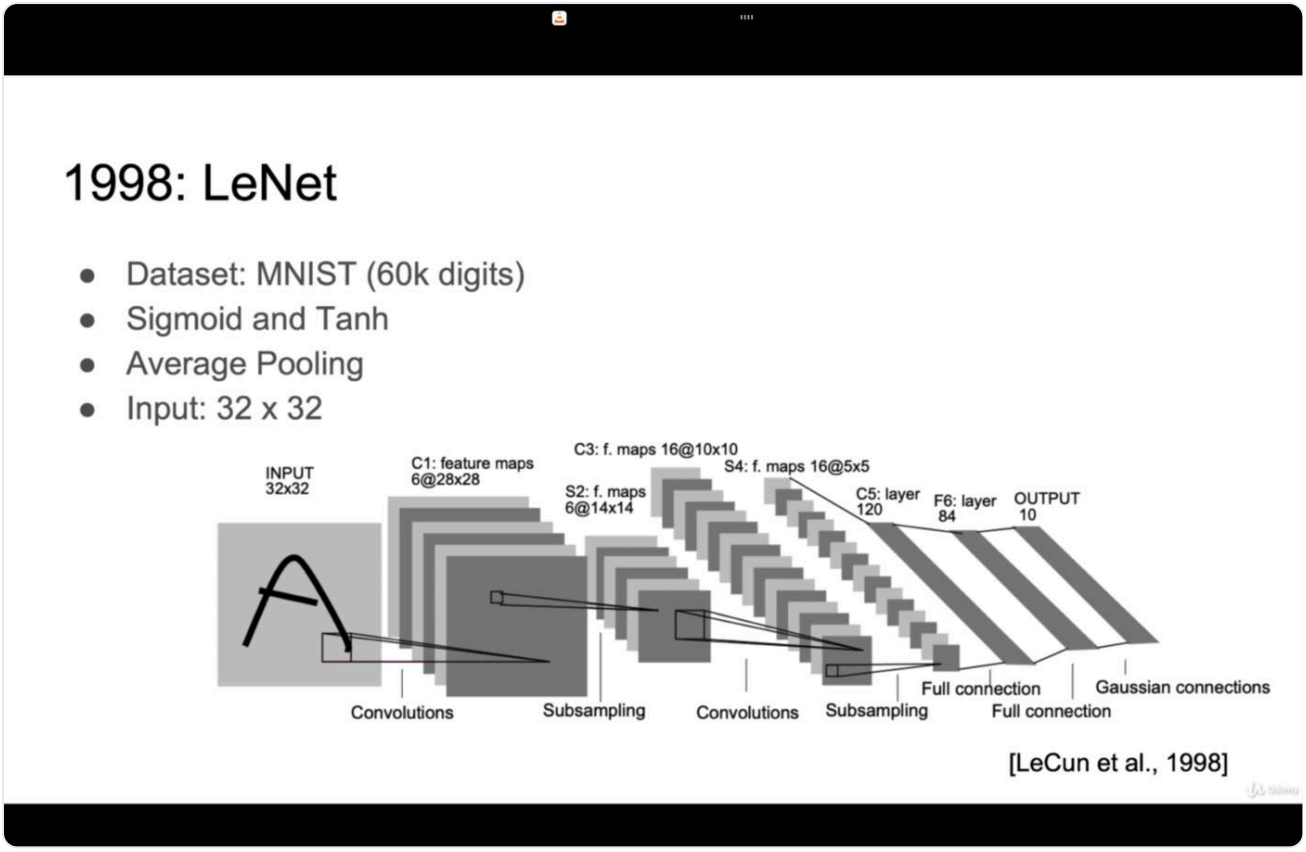


FIGURE 25.2

The first model that was proposed in 1998 was LeNet.

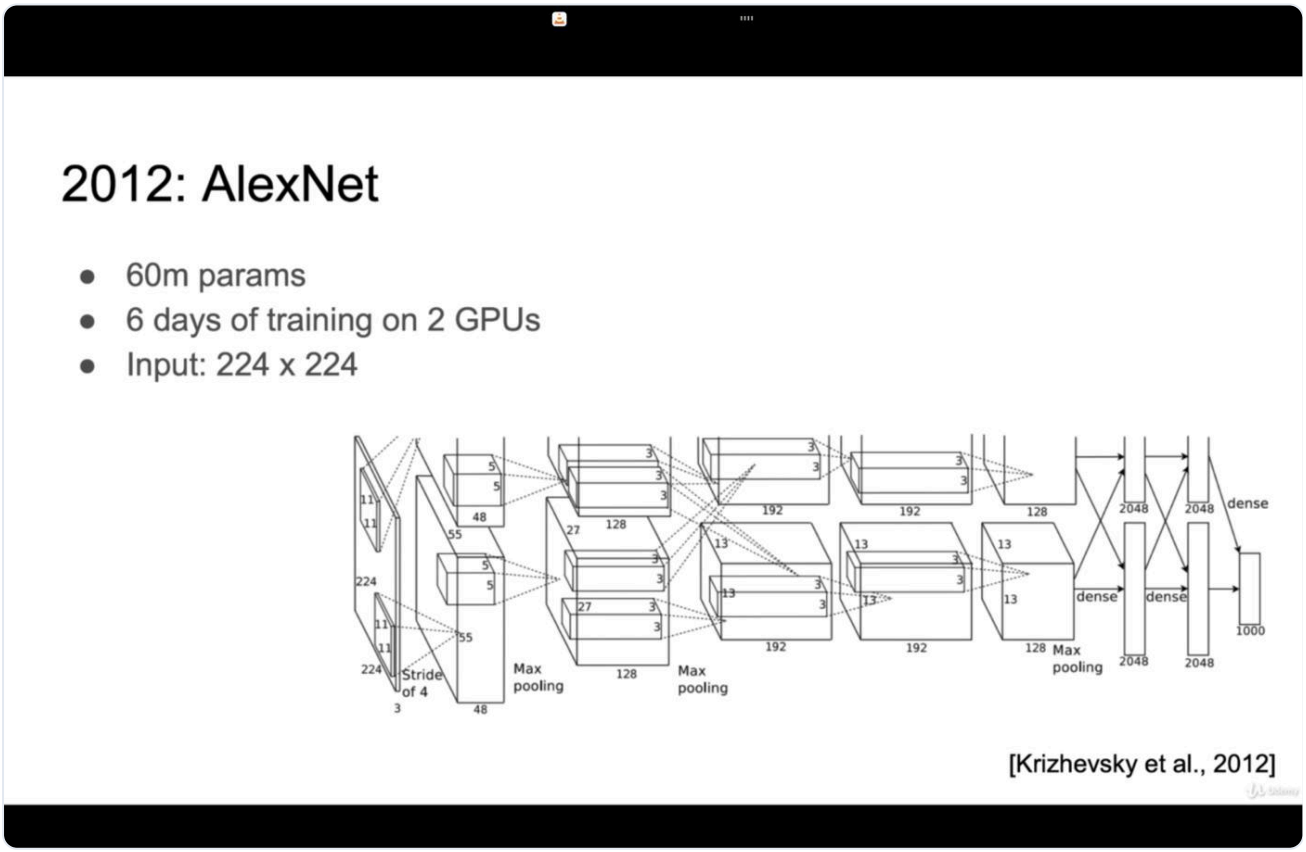


FIGURE 25.3

It was designed for the MNIST dataset and consist of convolutional layers, average pooling, sigmoid, and hyperbolic tangent nonlinearities and had the input of 32 to 32 and the output was 10 classes AlexNet was proposed in 2012 and had 60 million parameters.

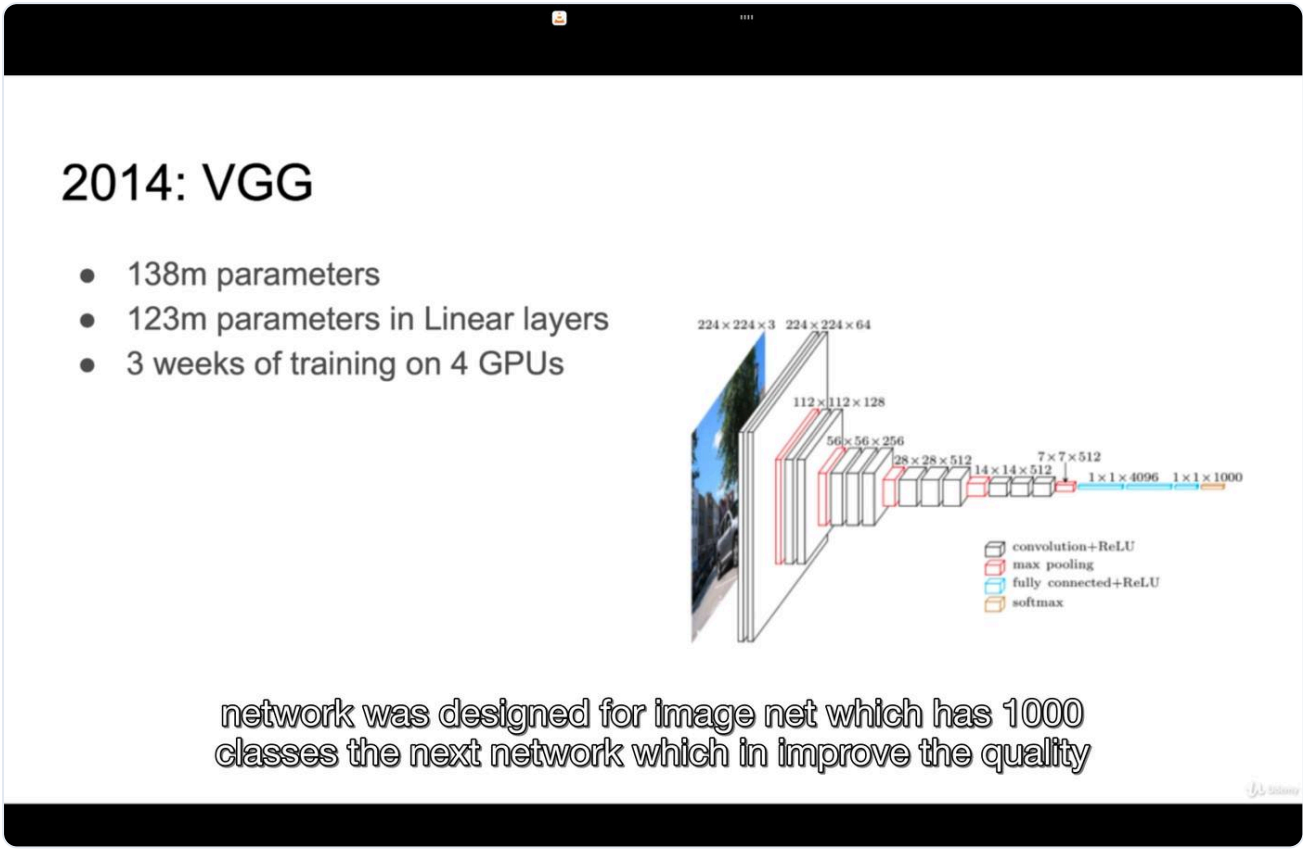


FIGURE 25.4

And it took up to six days of training to train this network on two GPUs the input to this network was 224 x 224 and the output had 1,000 classes because this network was designed for ImageNet which has 1000 classes the next network which in improve the quality of the previous one was VGG-16 which consist of 138 million parameters and most of these parameters was in linear layers. 123 million parameters it took up to three weeks of training on four GPUs. How can we solve the problem of different scale.

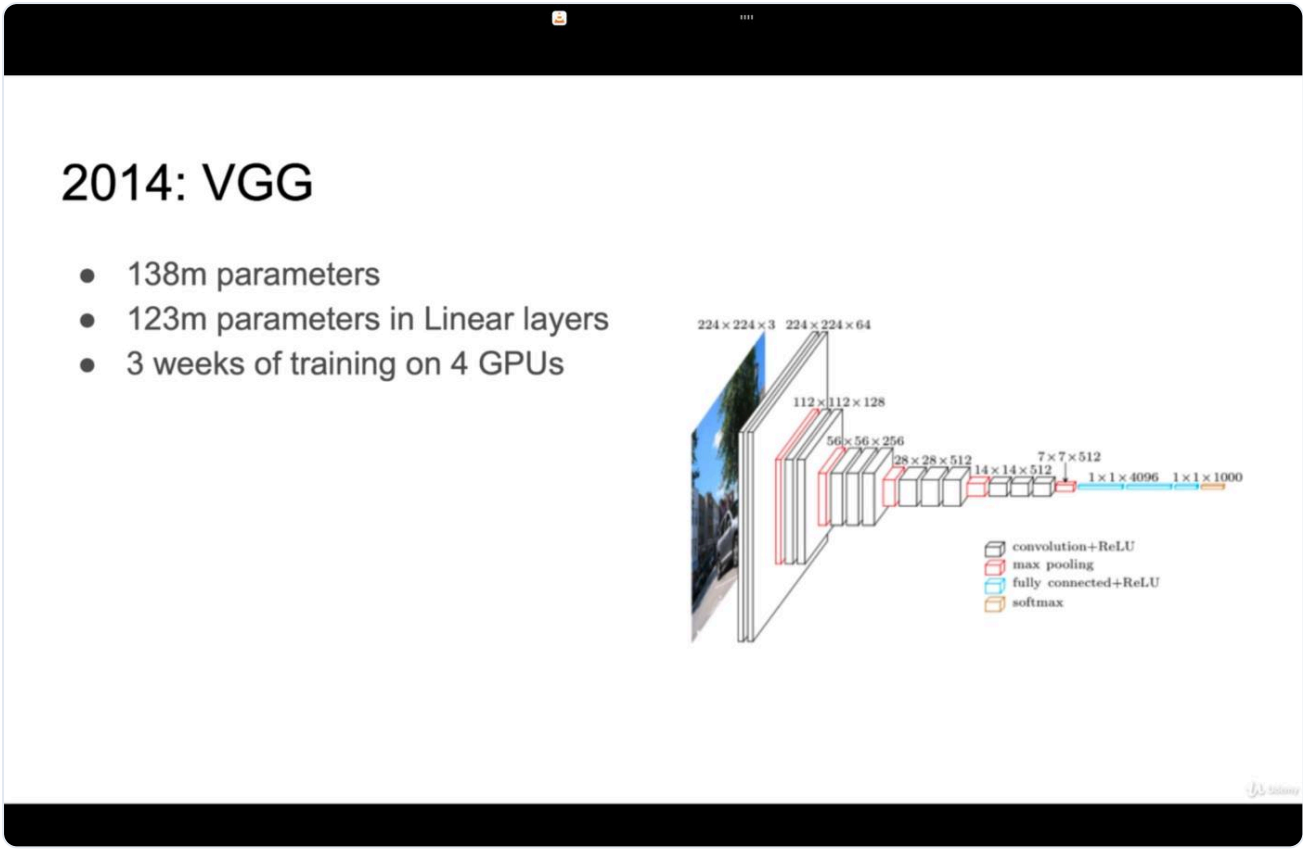


FIGURE 25.5

Look at these three images on each of these image. There is a dog but the scale is different. And on the first image the dog is on the most part of the photo and on the last image the dog taking only small part of the photo.

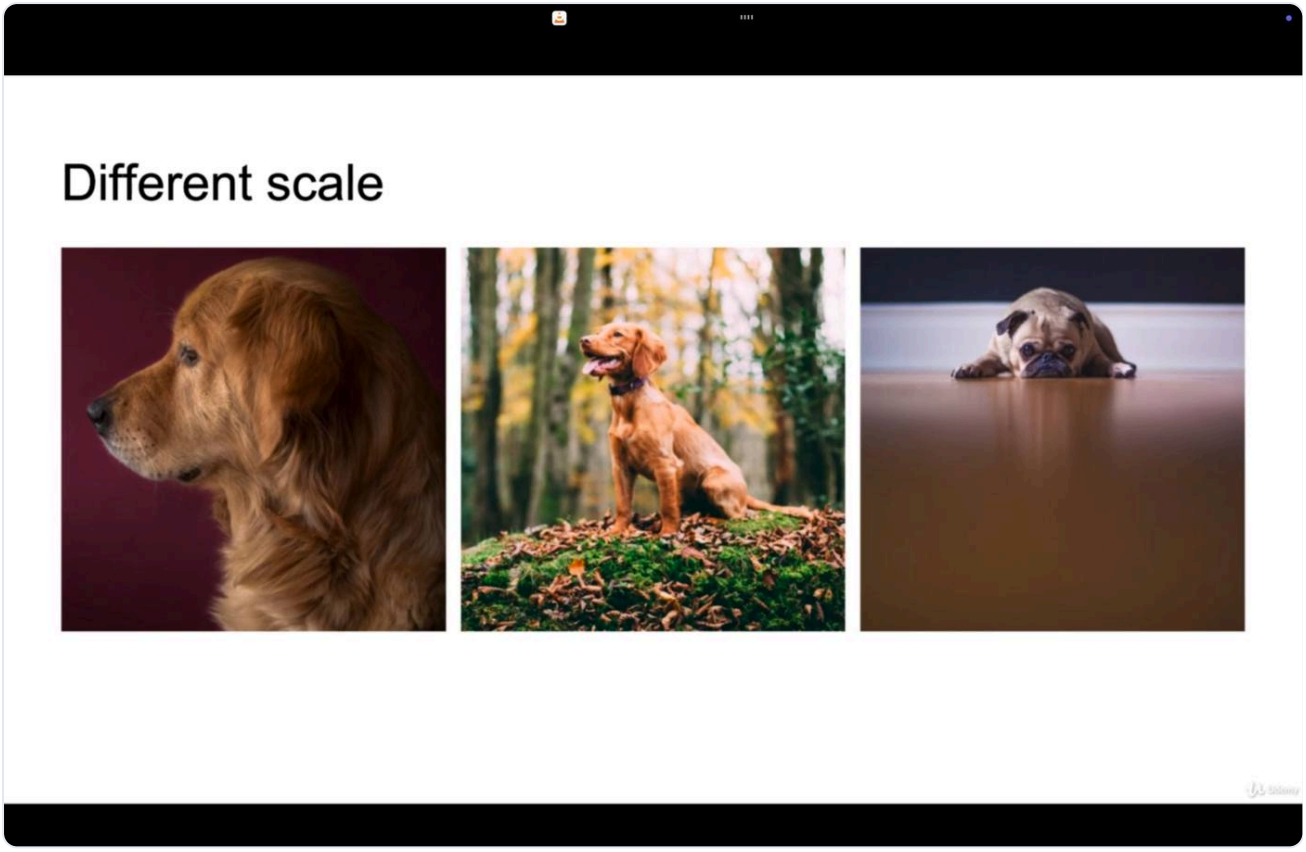


FIGURE 25.6 | Different Scale

In 2014, the Inception network was proposed to solve this problem. It consists of so-called Inception modules that new version of Inception modules is the following.

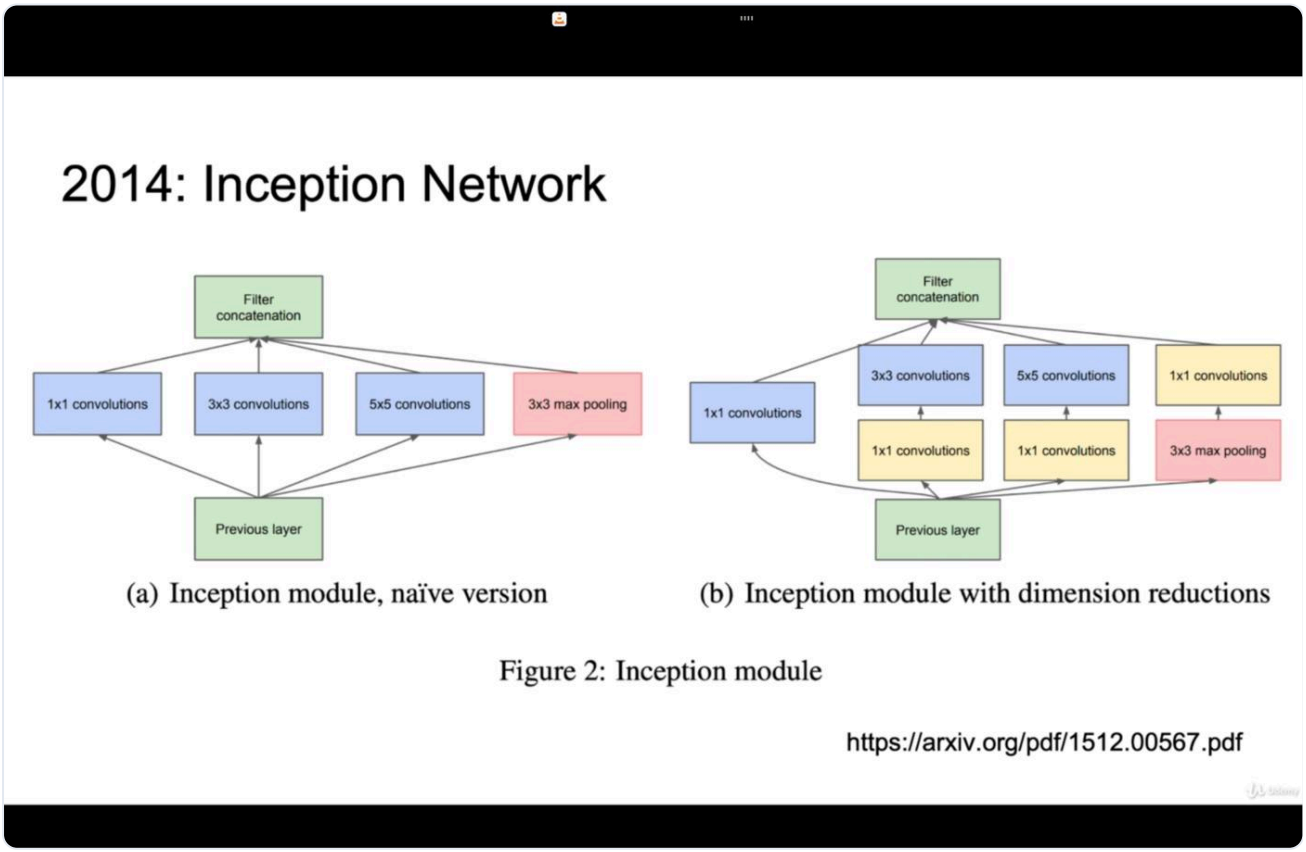


FIGURE 25.7

We have some previous layer the output of the previous layer. Or it can be the input of the network and the idea is simple Let's apply convolutions with different kernel sizes to handle objects at different scales, such as 1x1, 3x3, and 5x5 convolutions and also let's use max pooling and after we use these different types of convolution We can concatenate output of these convolution into one result. This is their naive version of Inception module in the same paper they proposed dimension reduction with the inception module.

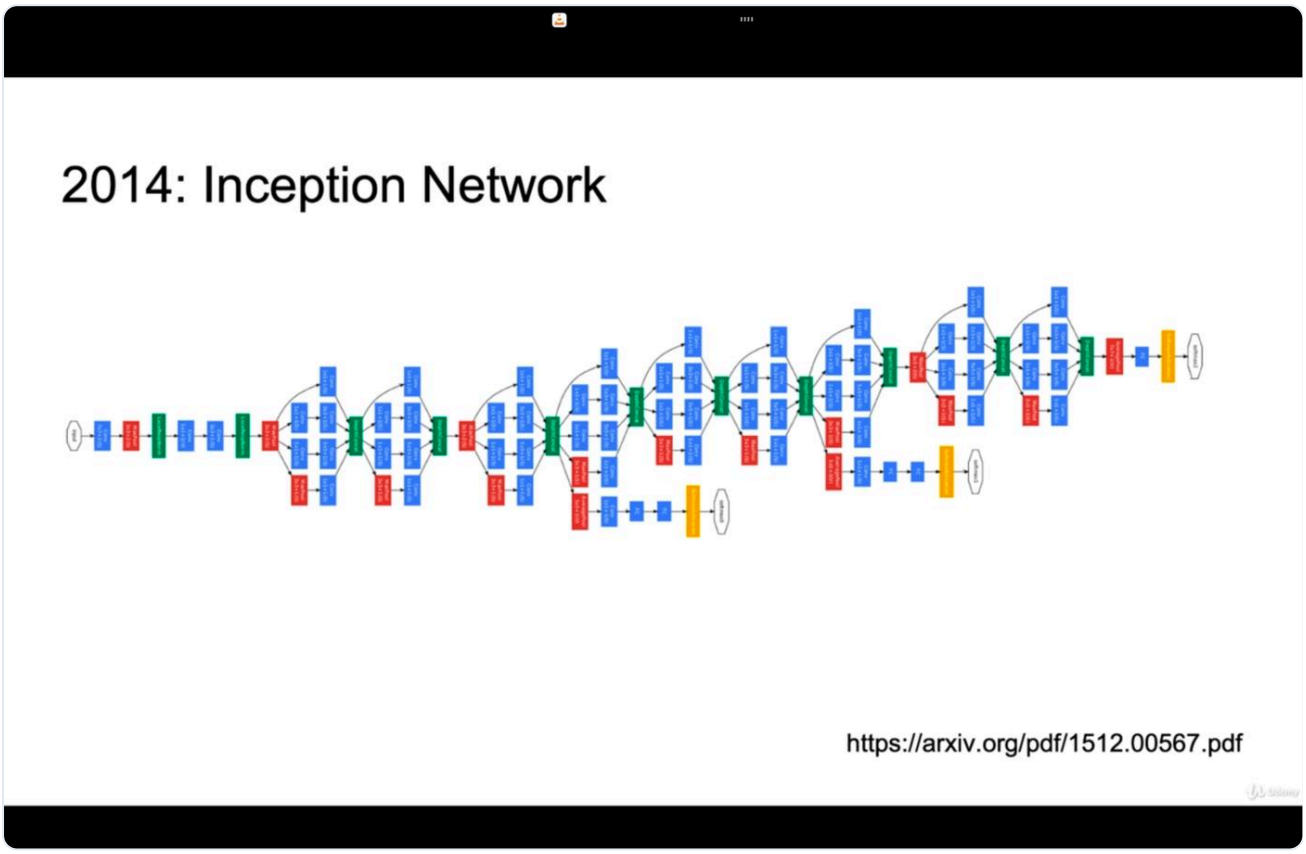


FIGURE 25.8

What is it. In fact we leave the same convolution as in any fashion but to add 1x1 convolution which is yellow on the right image 1 2 1 convolution allows us to have the same output size as an input size but to reduce the number of filters it means that we easily can decrease the number of parameters in 3x3 convolution and 5 to 5 convolution and they found that 1x1 convolution is useful after max pooling and before concatenating the feature maps. These branches together form the Inception network.

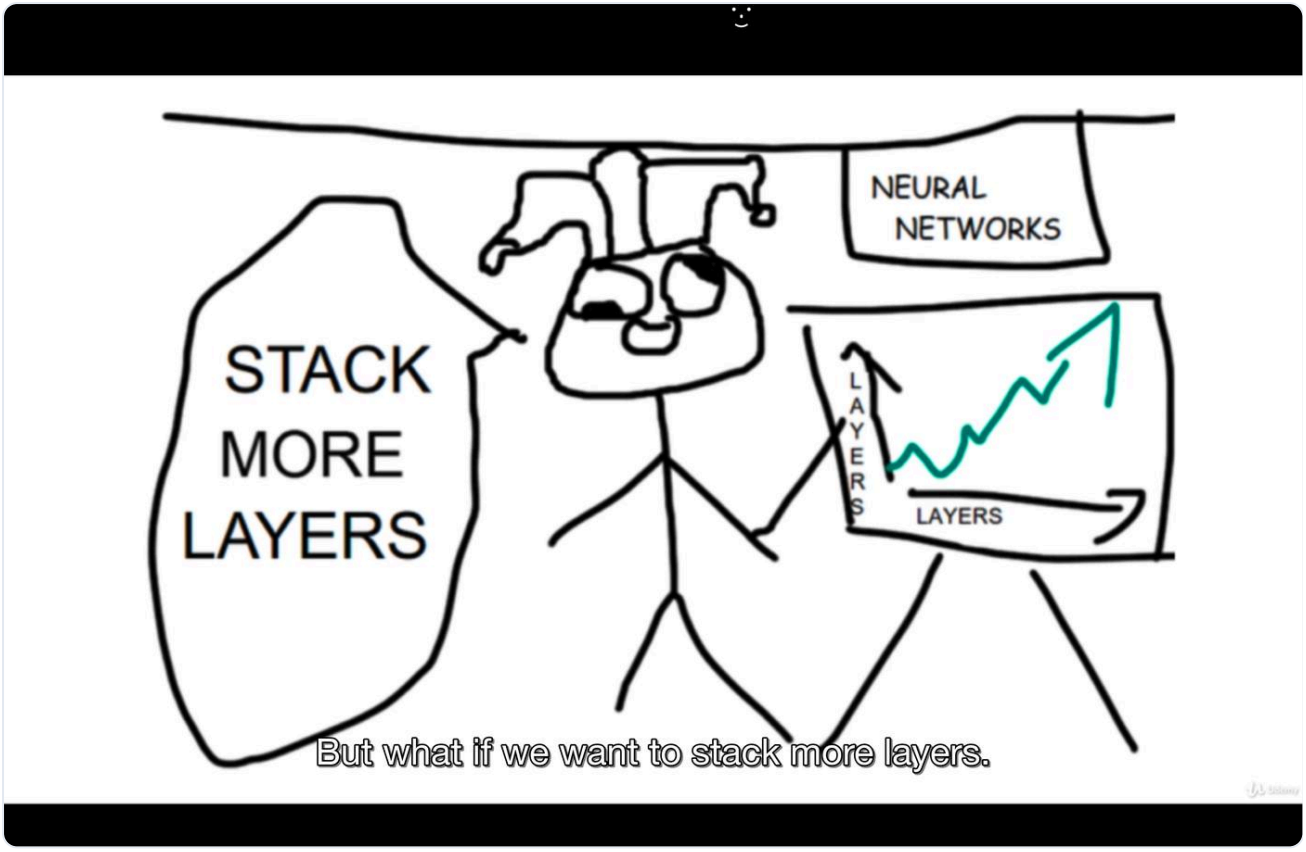


FIGURE 25.9

But what if we want to stack more layers. What do you want for example to use one hundred and fifty layers.

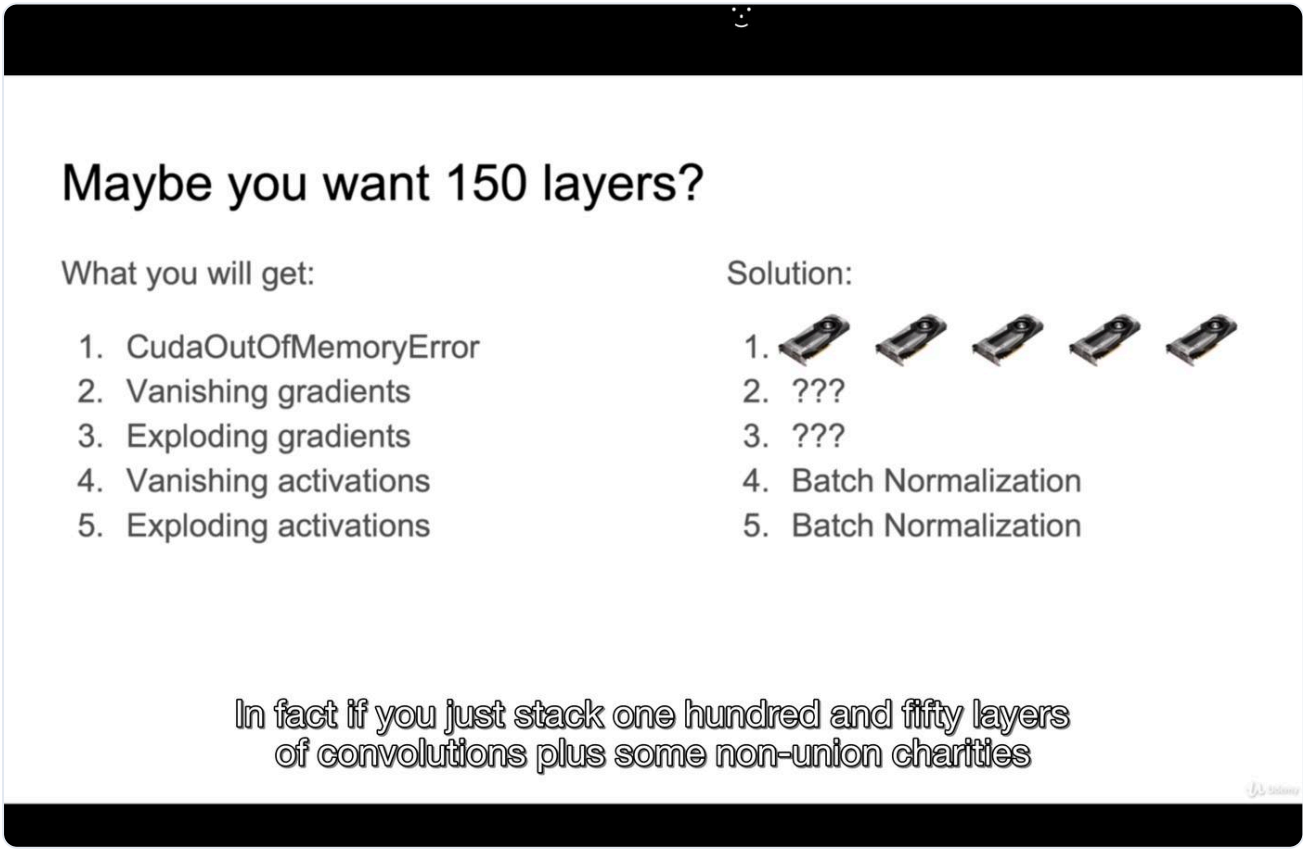


FIGURE 25.10 | What You Will Get: Solution:

Maybe you want 150 layers?

What you will get:

1. CudaOutOfMemoryError
2. Vanishing gradients
3. Exploding gradients
4. Vanishing activations
5. Exploding activations

Solution:

1. 
2. ???
3. ???
4. Batch Normalization
5. Batch Normalization

to the input.

FIGURE 25.11 | What You Will Get: Solution:

2015: ResNet

- 152 layers!!!
- Skip connections (aka residual blocks)

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$$

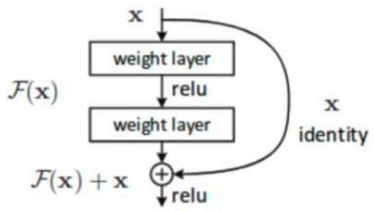


Figure 2. Residual learning: a building block.

What did it imagine.

https://arxiv.org/pdf/1512.03385.pdf

FIGURE 25.12

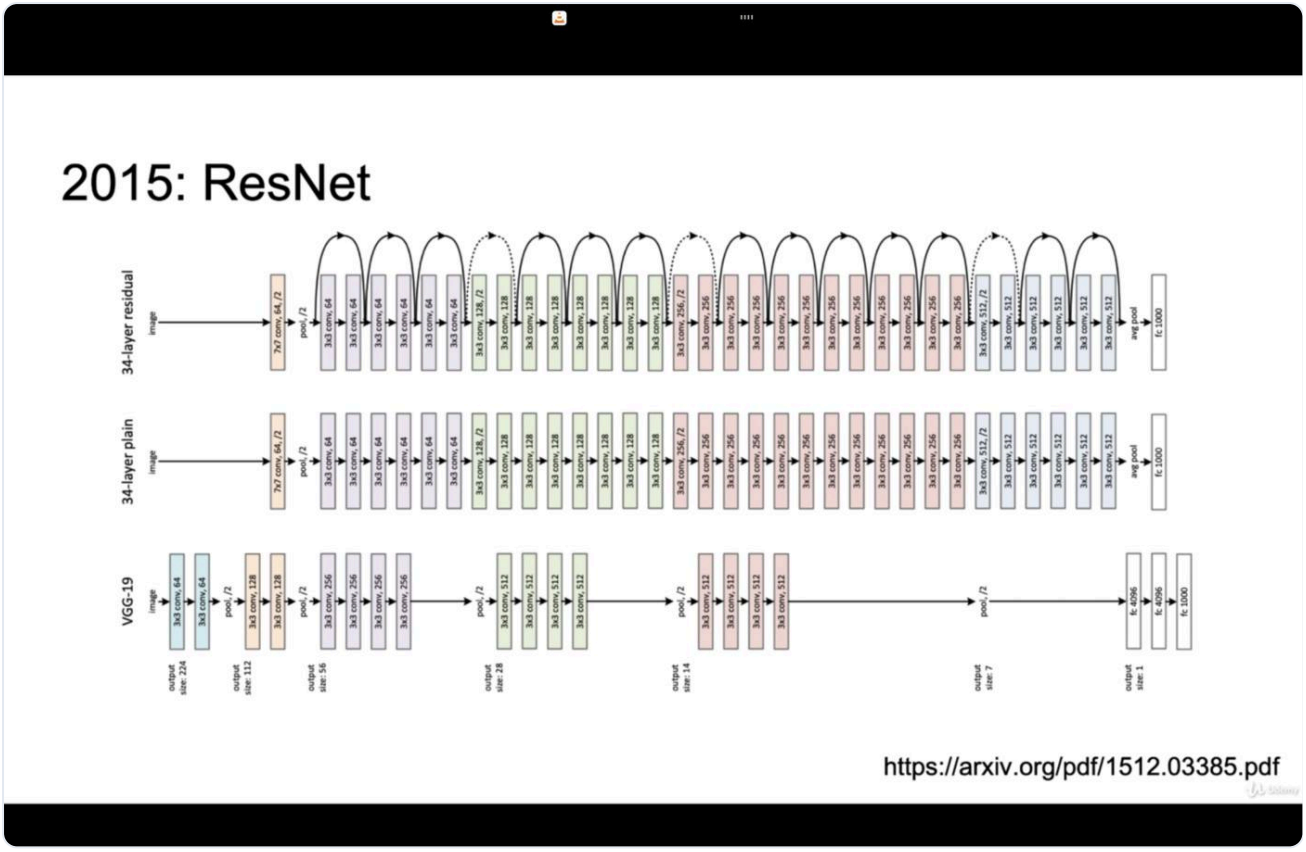


FIGURE 25.13

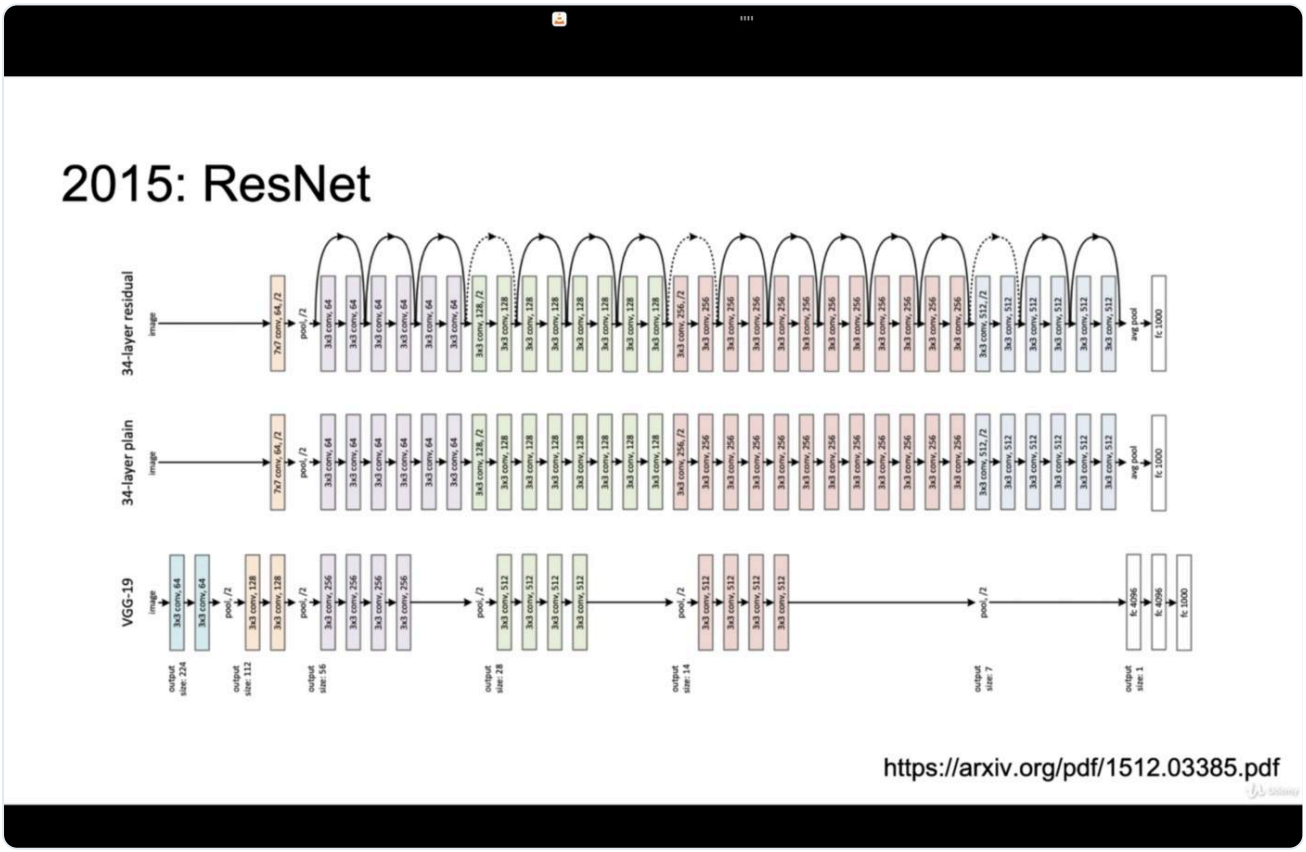


FIGURE 25.14

In fact if you just stack one hundred and fifty layers of convolutions plus some non-union charities you will obtain five problems. The first problem is quarter out of memory error. This problem can be solved using multiple GPUs. And let's imagine we can get another additional resources. Then we will have to problem with gradients which is when you shouldn't gradients and exploding gradients and will have vanishing activations and explode in activation the last two can be sold to US batch normalization. As we described in the previous section but vanishing gradients and exploding gradients of course not only because of activations because of non-linear edges but also because the number of layers and in fact it is very hard for gradient to go through the head of the network to the tail of the network to the input.

It is hard and usually it becomes zero to the input. And in fact if you just stack one hundred and fifty layers your network will not learn at all. But in 2015 people propose so-called ResNet architecture. It consisted of one hundred and fifty two layers and used skip connections also known as residual blocks. What did it imagine. Look at the right image. Imagine you have some X image. Then we use some weight layer which is convolution layer we can use it again. But at the right part we use X and we add this X to the output of the convolution without any modification. So the formula is on the left where f is that convolution and x is just our input. In fact this helps a lot.

Was building such a deep neural network and it occured that such a network allowed to outperform human on ImageNet dataset. That's how it looked like in fact here is thirty four layer residual network. But in practice they use one hundred and fifty two layers and we will also use it in practice. So the bottom network is VGG-19 which we look at. Few flights a go then thirty four layer network without residual connections and 34 layer network with the residual connections that is all for modern architectures. We'll look at them in details and we'll study how to use them in practice. Thank you and see you in the next video.

LECTURE 26

Transfer Learning

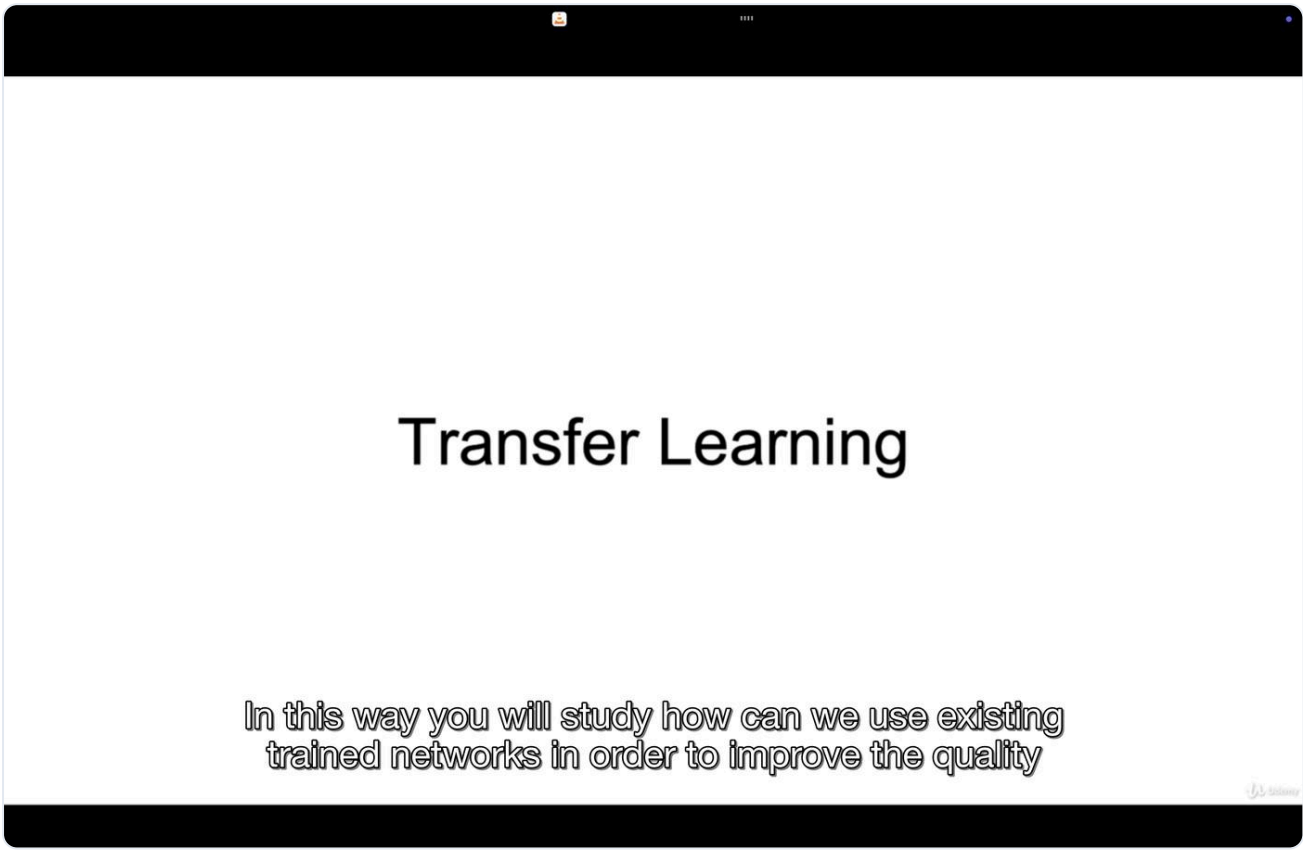


FIGURE 26.1 | Transfer Learning

In this way you will study how can we use existing trained networks in order to improve the quality of our problem that we solve.

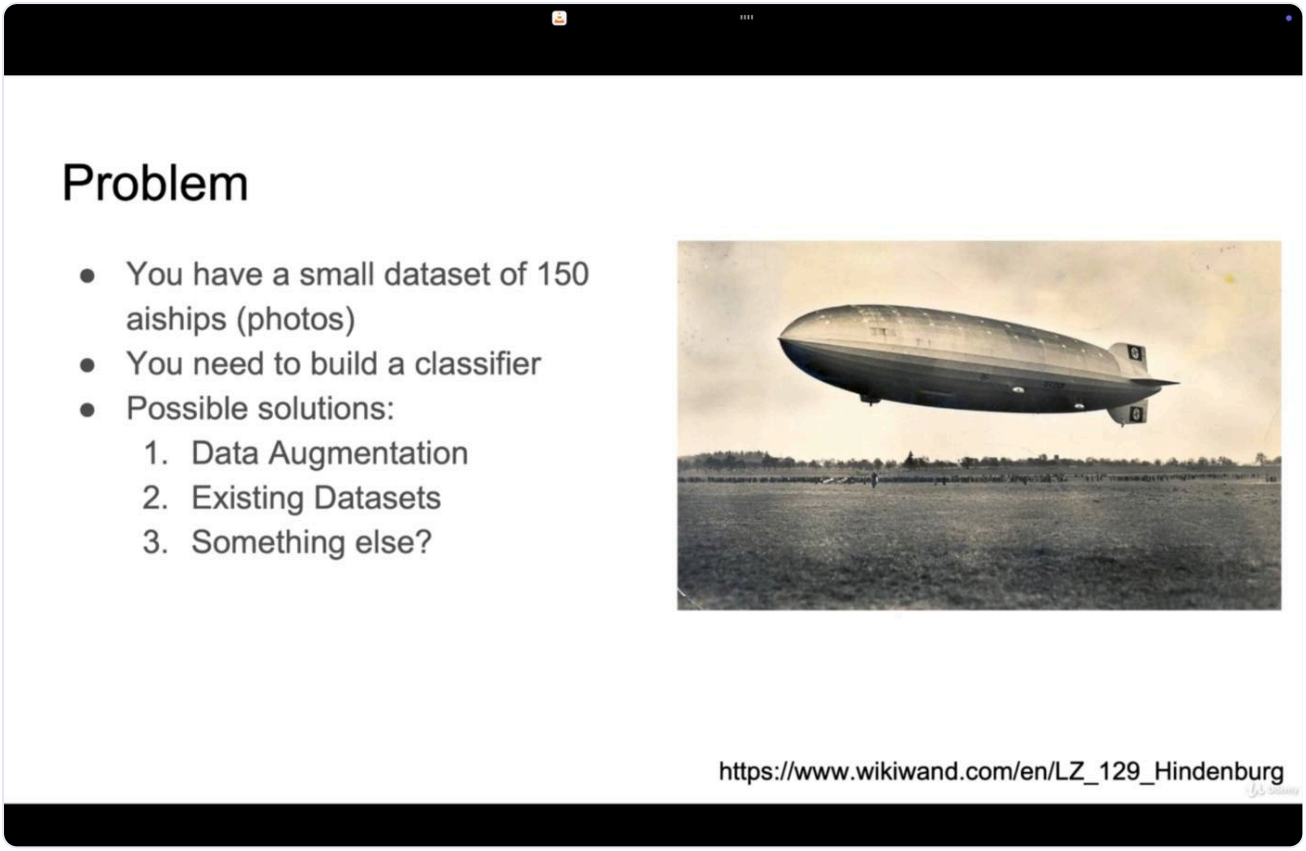


FIGURE 26.2

Let's imagine the forward you have a small dataset of one hundred and fifty airships of photos or fellowships and you want to build a classifier which classifiers for example different types of these airships. What possible solutions can we use. First is data augmentation and it is okay but it will be not enough because again we'll have too few images in our training set. We possibly can use some existing data sets but it is also not enough because in existing data sets I guess there are not so much airships. There are a lot of cats dogs birds etc. but not airships. Okay. Can we do something else. And in fact yes.

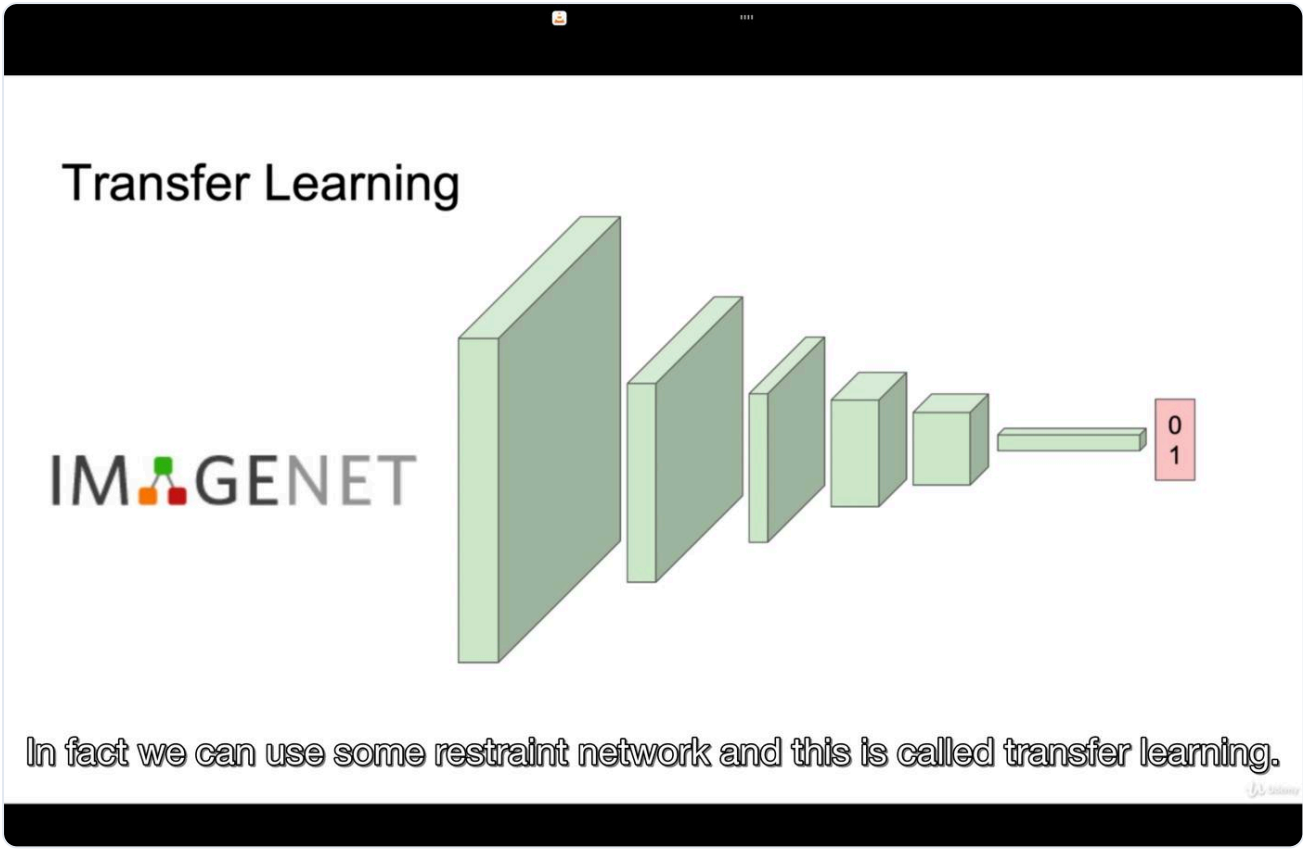


FIGURE 26.3 | Transfer Learning

In fact we can use some ResNet and this is called transfer learning. Imagine you have a big network that was trained on ImageNet. For example if it may be ResNet ResNet which consist of one hundred and fifty two layers which we studied in the previous video it was trained on the ImageNet which consists of 14 millions of images and 1000 classes.

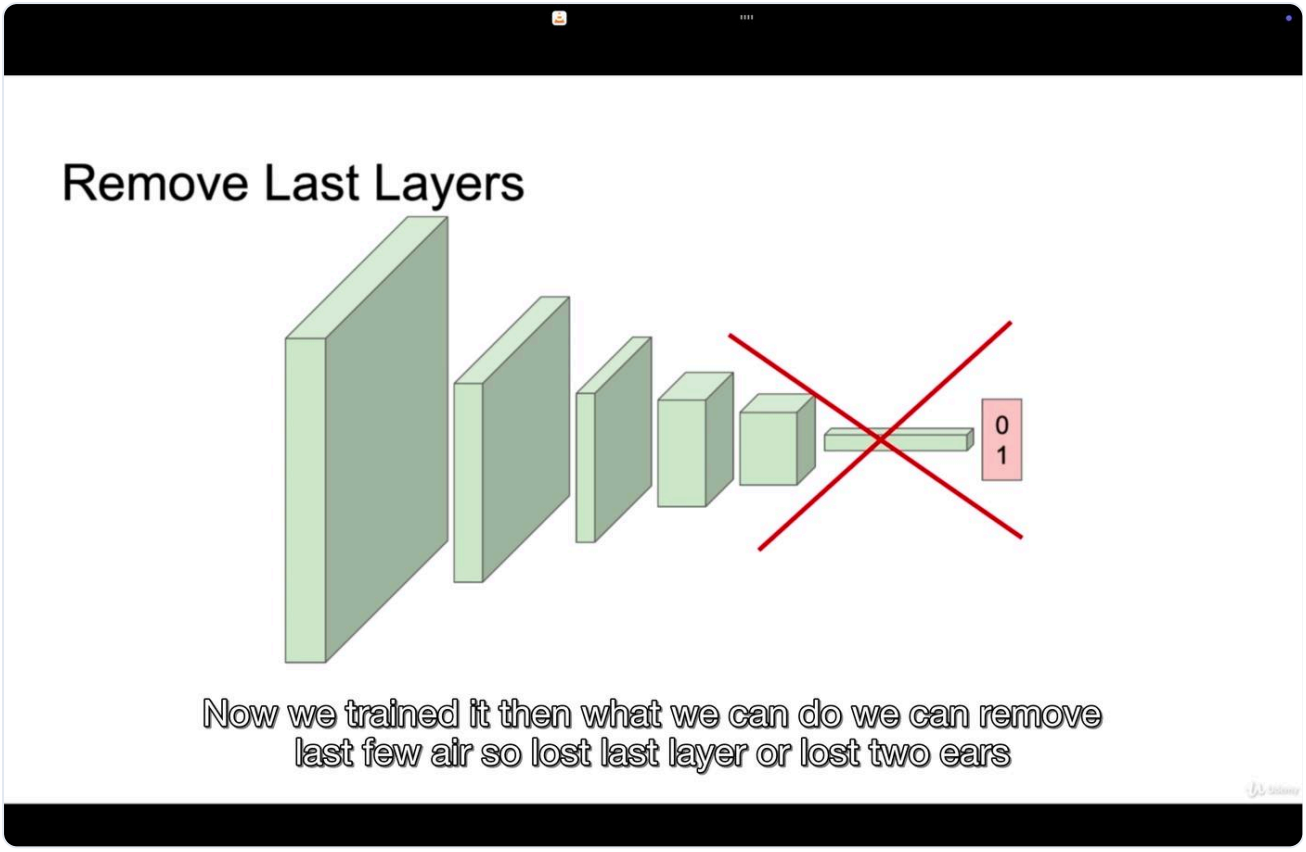


FIGURE 26.4 | Remove Last Layers

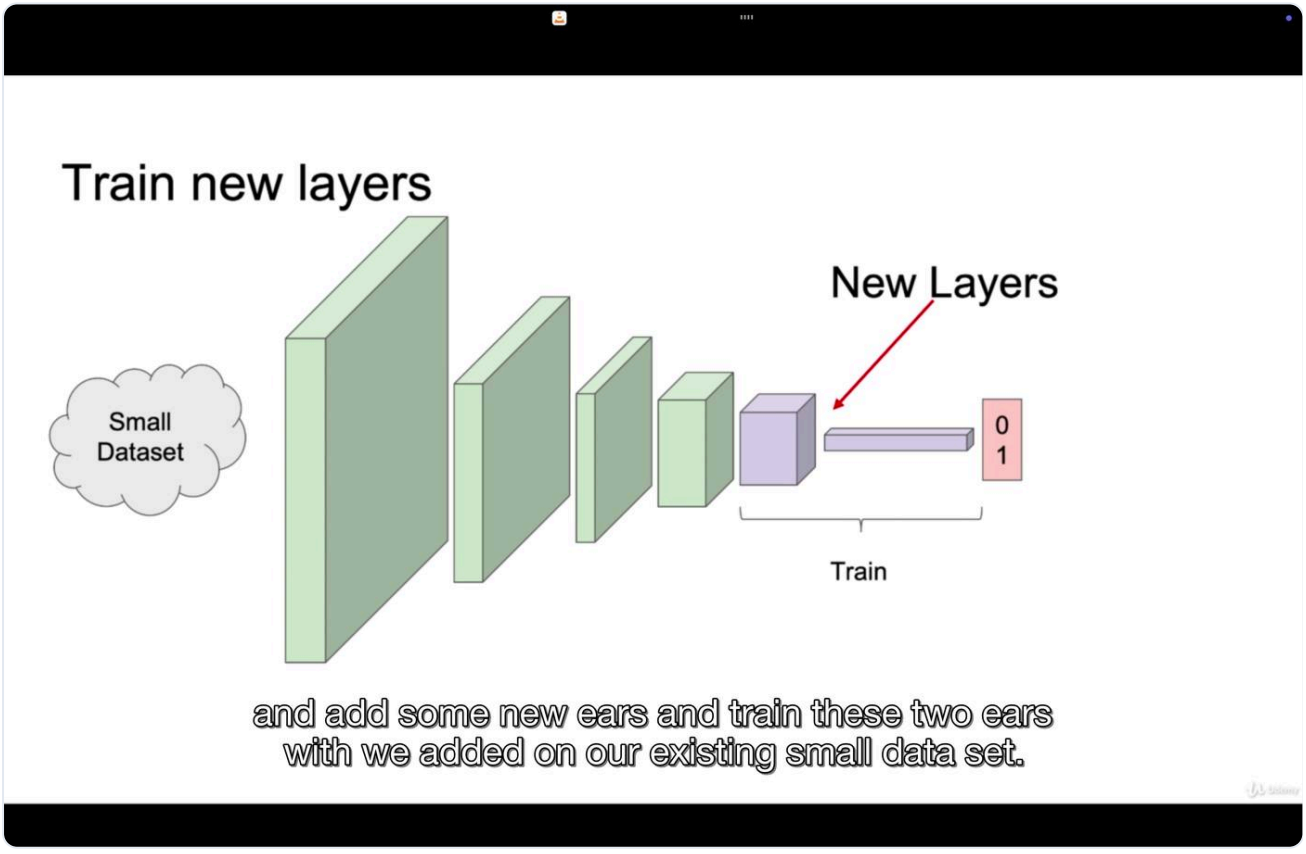


FIGURE 26.5

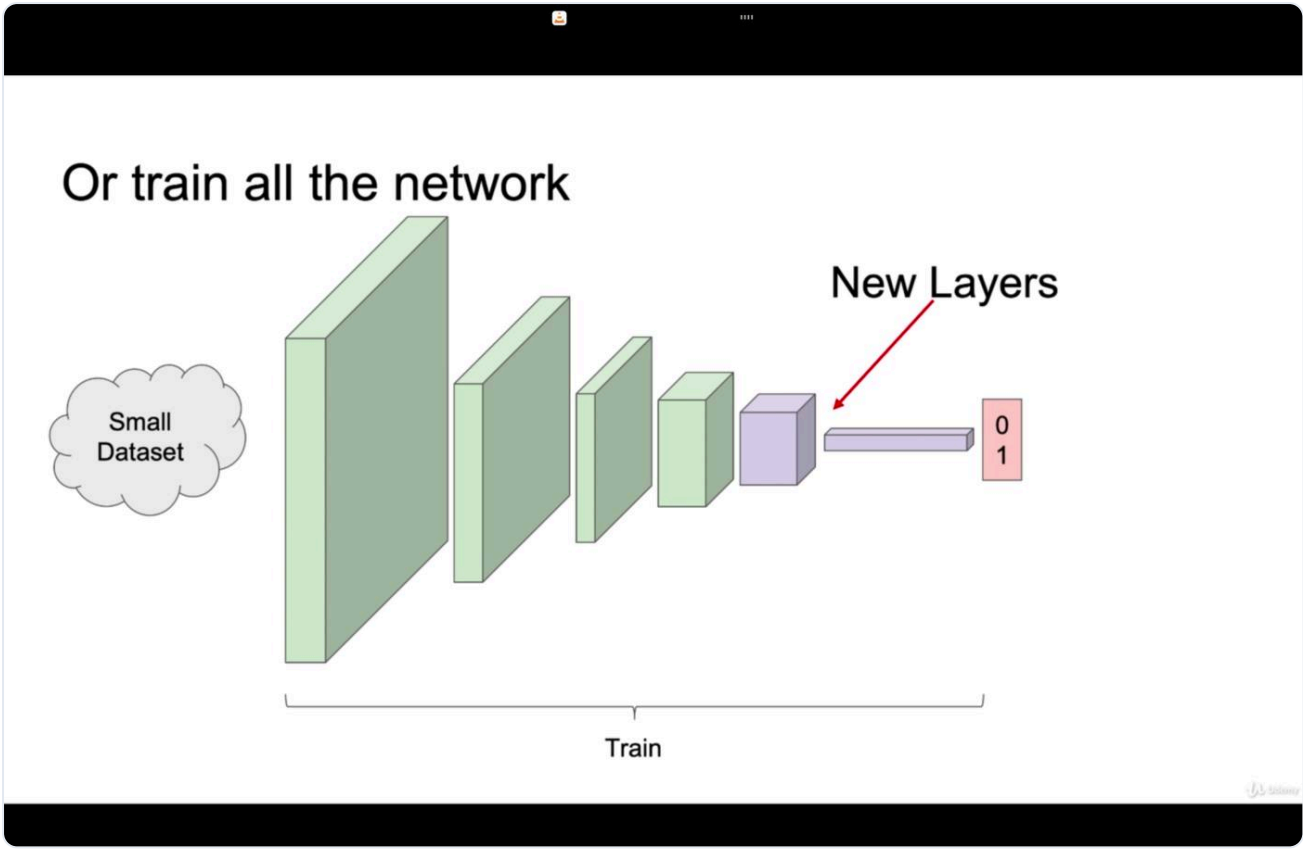


FIGURE 26.6 | Or Train All the Network

Now we trained it then what we can do we can remove last few layers so lost last layer or lost two ears and add some new ears and train these two ears with we added on our existing small dataset. And here we can of course apply the data augmentation look. Now we need to train not the deep network but just small one and the amount of data that we have usually is enough to train one or two layers. But as a result we will have a big deep network. And usually it works because first layers of the convolution network allows to obtain very nice features that are appropriate to any problem another possible solution is to train all the network not just new layers.

It depends on the problem of course and then on the amount of data that you have in your training set. That is all for transfer learning. Let's look how we can use it in practice. Thank you for watching and see you in the next video.

LECTURE 27

Data Sets and Dataset

See the notebook here

Hello my dear friend and welcome to The Broad Recognition section in this section we will solve practical problem of classifying different types of boards. Imagine you have a breach or with everywhere and you want to set that camera and you want to build an algorithm that will classify all boards that are going under the breach. For this reason you need to build some convolution or KYC fires for our goal we will use the dataset that I took from Kaggle which is the biggest machine learning competition. HORST You can download this data manually by the link or use cargo library that allows you to download data sets so you can just install gave a library and then write Google data sets.

Download it's the name of the dataset. What types of commission and minus B means path where to download the data. Let's import the necessary libraries in this video we will load the data. So now again as in previous practical videos we define their transformation that we will apply to all our images. Now we have not two but three transformations. First transformation is resize in each image to the site. Two hundred and twenty four to two hundred and twenty four. This is necessary because different images has different sizes and usually they are big like 1000 two 1000 and it is really hard to process such big images with neural network. And the second reason all the images should have the same size in order to use for neural network training and prediction otherwise will have different output sizes and can drain neural network at all.

The second transformation is to start to tensor which transforms the image to PyTorch dancer and the third is normalization. When we subtract mean and divide by standard deviation making our mean equal to zero and standard deviation equal to one. Now our images are allocated in the Data folder and if we look at it at less dot slash data you will see different folders containing images. For example Greece sheep and you can see a set of images. If we have the full data structure we can use to watch region class image folder. If you look at it documentation each acquires that all your images are allocated in the separate folders like doc and get so we load the dataset by just writing treason data sets.

Image folder and apply our transformation. After that we should divide our dataset to the train set and death said we can use for this reason torch dot you to store data dot random split and pass dataset batch size and their size which has just zero point seven. From our dataset and zero point three from our dataset. And after that we can define two loaders train loader and test loader as we did in previous practical videos with with predefined batch size shuffling the data and a number of processes that will load our native her memory is equal to two. Okay let's run this cell the length of our dataset is 1000 four hundred and seventy six. It means that it is a very small dataset.

And believe me it is really hard to train any convolution who they are and let us work on these dataset totally we have nine classes and it means that each class will have very few sets of images the classes our Crees sheep very bored frayed bored Gandalf and others since we have very few images in our train said we should do something with it and what I propose to do is to use first method is data augmentation which we will consider in the next media and also transfer learning. So let us use some pre train network that we again will consider in the next videos. Now let's visualize the data. Here you can see the set of images and there are some different types of shapes okay.

So that's all for this video. Thank you. And see in the next video.

LECTURE 1

Horizontal Flip and Random

In this video we will make data augmentation. So let's again import all our is in PyTorch. It is very convenient to make data augmentation everything we need. We can define in that transformation. So what we need here. The first transfer is resizing to size. Two hundred and twenty four to two hundred and twenty four. After that we can define some random transformation from transforms module of thoughts visual library. Let's look what available transformations do we have center crop crawler G10. This is not a random random crop random grayscale or random horizontal flip. Random resize rotation. So this is a set of friendly transformations that we can apply to our images.

Let's apply for example random rotation. If we look at its signature then it has the main parameter degrees. And here we should write degrees equal to minus 30 to 30. It means that we will rotate randomly our image to the angle which will be also random from the minus 30 to 30 degrees. Now let's try to do it and visualize the result. So okay we did it and here you can see rotated images all these images were rotated randomly. Now let's look what we can do more transforms random horizontal flip let's add random horizontal flip and just forms random vertical flip and let's look at the result more to gain the data said around cells. And here you can see horizontal flip.

Now it's vertical flip again vertical flip and some of them are flipped horizontally. Good. Now let's look what we can do more transforms dot random grayscale. So now you can see that some random images become grayscale and in fact this is all for data augmentation. So it is really easy to apply it and you should always apply it in practice when you have not only when you have small amount of data but only when you have big amount of data. Use the data augmentation. It is really easy. Thank you for watching and see you in the next video.

LECTURE 29

Fully Connected

In the previous video we studied how to make data I'll be stationed in PyTorch in this video. We will learn how we can reuse existing pre trained neural networks like Alex Nat Rose Ned or any other. So let's do it. First thing that we need to do is to import torch vision library. After that we have to create the network torch vision dot models dot raise net 18 for example. It's the shortest version of or isn't it. And there is one parameter which is called pre trained default default by default. It is false. Let's set it to true retrain decode the two. And these will be our network or gate. Let's look at it. This is it in fact we just loaded there is net 18 and it is pre trained it already has trained wait in order to look at the network and by torture we can just type net.

And here this. Here it here will have all the layers. And now what we should do you can see that this is sequential basis more called 2D conf 2D which known point 2D and so on. And at the end it has averaged pooling plus for the Connect layer. What we need now is to cut these fully connected layer off and that add new one new fully connected player. And after that train this one fully connected we're using our board's dataset. Turn off all the gradients of our network. We need these because we don't want to train again all these 18 layers before fully connected layer. For this reason let's do the following For Bahram in in NAND DOD parameters for RAM DOD wires red equal to False.

Here is it. Okay. We turned off the updating of all the layers. Now we want to set the following fully connected. No net dot fully connected FC is equal is equal to. And then not linear. And the input neurons will be five hundred and twelve. And the output neurons will be nine because now we have nine classes. Now we can check our net again. So we see it has a lot of convolutions much norms and so on. And the last layer is linear. Five hundred and twelve features and now number of features is nine. We have nine classes. Good. Let's check the output shape. We have an error because we didn't move our network to quarter. Let's do it. Net is equal to the net dot. Okay good.

The output of our network is 16 to nine. Sixteen is the badge size and nine is a number of output neurons which is equal to the number of output classes. Let's lower their criterion and optimizer and now we'll have two functions first of them is getting quality on the desk set and the second one is training. Function it gets the Net as a parameter and number of books for train so let's train our network for 5 a box and look at the quality so our train was finished and the final accuracy on test images is 70 percent which I think rather good taking into account the fact that we have only one thousand and five hundred images. Thank you for watching and see you in the next video.